

---

# RT-Comops Backend

Cahier de conception

Architecture Multi-Tenant — ConceptionKernel V3

---

Cahier de conception .



# Table des matières

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Introduction Générale</b>                                                | <b>1</b>  |
| <b>Vue Generale du Projet</b>                                               | <b>7</b>  |
| <b>1 kernel-core par Toulepi Jordan</b>                                     | <b>12</b> |
| 1.1 Description du core . . . . .                                           | 12        |
| 1.2 Rôle principal . . . . .                                                | 12        |
| 1.3 Rôles secondaires . . . . .                                             | 12        |
| Diagramme de classes — kernel-core . . . . .                                | 15        |
| Diagramme des cas d'utilisation — kernel-core . . . . .                     | 17        |
| 1.3.1 DS-K-01 : Provisionnement d'un nouveau tenant . . . . .               | 18        |
| DS-K-01 — Provisionnement logique d'un nouveau tenant . . . . .             | 19        |
| DS-K-02 — Relais Outbox vers Kafka . . . . .                                | 20        |
| DS-K-03 — Résolution réactive du contexte tenant . . . . .                  | 21        |
| DS-K-04 — Écriture de la piste d'audit (pattern aspect) . . . . .           | 22        |
| DS-K-05 — Initialisation d'une ClientApplication bootstrap . . . . .        | 23        |
| DS-K-06 — Authentification et garde-fous d'une requête protégée . . . . .   | 24        |
| DS-K-07 — Suspension d'un tenant . . . . .                                  | 25        |
| DS-K-08 — Supervision de l'outbox via health check . . . . .                | 26        |
| <b>2 common-core</b>                                                        | <b>27</b> |
| 2.1 Description du core . . . . .                                           | 27        |
| 2.2 Rôle principal . . . . .                                                | 27        |
| 2.3 Rôles secondaires . . . . .                                             | 27        |
| Diagramme de classes — common-core . . . . .                                | 29        |
| Diagramme des cas d'utilisation — common-core . . . . .                     | 31        |
| DS-C-01 — Attachement d'une adresse polymorphique . . . . .                 | 32        |
| 2.3.1 DS-C-02 : Résolution d'une PartyRef . . . . .                         | 33        |
| DS-C-02 — Résolution d'une PartyRef inter-cores . . . . .                   | 34        |
| DS-C-03 — Pagination standardisée des résultats . . . . .                   | 35        |
| DS-C-04 — Propagation automatique des champs BaseEntity . . . . .           | 36        |
| DS-C-05 — Gestion du contact principal (promotion/rétrogradation) . . . . . | 37        |
| DS-C-06 — Construction d'une réponse d'erreur standardisée . . . . .        | 38        |
| <b>3 actor-core</b>                                                         | <b>39</b> |
| 3.1 Description du core . . . . .                                           | 39        |
| 3.2 Rôle principal . . . . .                                                | 39        |

|          |                                                                                              |           |
|----------|----------------------------------------------------------------------------------------------|-----------|
| 3.3      | Rôles secondaires . . . . .                                                                  | 39        |
|          | Diagramme de classes — actor-core . . . . .                                                  | 41        |
|          | Diagramme des cas d'utilisation — actor-core . . . . .                                       | 43        |
|          | DS-A-01 — Création d'un acteur . . . . .                                                     | 44        |
|          | DS-A-02 — Suppression douce d'un acteur . . . . .                                            | 45        |
|          | DS-A-03 — Rattachement d'un acteur à une organisation . . . . .                              | 46        |
|          | DS-A-04 — Mise à jour du profil détaillé d'un acteur . . . . .                               | 47        |
|          | DS-A-05 — Réactivation d'un BusinessActor par auto-service . . . . .                         | 48        |
|          | DS-A-06 — Approbation d'un BusinessActor par l'administration . . . . .                      | 49        |
| <b>4</b> | <b>auth-core</b>                                                                             | <b>50</b> |
| 4.1      | Description du core . . . . .                                                                | 50        |
| 4.2      | Rôle principal . . . . .                                                                     | 51        |
| 4.3      | Rôles secondaires . . . . .                                                                  | 51        |
|          | Diagramme de classes — auth-core . . . . .                                                   | 54        |
| 4.4      | Flow Postman de référence . . . . .                                                          | 55        |
|          | Diagramme des cas d'utilisation — auth-core . . . . .                                        | 56        |
|          | DS-AU-01 — Création d'un compte utilisateur par un administrateur IAM . . . . .              | 57        |
|          | DS-AU-02 — Authentification locale tenant-scoped d'un utilisateur . . . . .                  | 58        |
|          | DS-AU-03 — Résolution des permissions et émission du JWT RS256 final . . . . .               | 59        |
|          | DS-AU-07 — Découverte globale des contextes de login . . . . .                               | 60        |
|          | DS-AU-08 — Sélection explicite d'un contexte tenant / organisation . . . . .                 | 61        |
|          | DS-AU-09 — Identification email-first et inscription contextuelle . . . . .                  | 62        |
|          | DS-AU-10 — Réinitialisation de mot de passe avec livraison SMTP ou<br>PREVIEW_ONLY . . . . . | 63        |
|          | DS-AU-11 — Vérification d'email après authentification . . . . .                             | 64        |
|          | DS-AU-04 — Consultation du profil utilisateur connecté (/users/me) . . . . .                 | 65        |
|          | DS-AU-05 — Mise à jour du plan utilisateur . . . . .                                         | 66        |
|          | DS-AU-06 — Mise à jour de l'onboarding utilisateur . . . . .                                 | 67        |
| <b>5</b> | <b>roles-core (Tsakem Irving, Tchoyi Wilson)</b>                                             | <b>68</b> |
| 5.1      | Description du core . . . . .                                                                | 68        |
| 5.2      | Rôle principal . . . . .                                                                     | 68        |
| 5.3      | Rôles secondaires . . . . .                                                                  | 68        |
|          | Diagramme de classes — roles-core . . . . .                                                  | 70        |
|          | Diagramme des cas d'utilisation — roles-core . . . . .                                       | 72        |
|          | DS-R-01 — Création d'un rôle avec ses codes de permission . . . . .                          | 73        |
|          | DS-R-02 — Affectation d'un rôle à un utilisateur avec portée . . . . .                       | 74        |
|          | DS-R-03 — Résolution des permissions avec cache Redis optionnel . . . . .                    | 75        |
|          | DS-R-04 — Construction des autorités scopeées à partir d'une affectation . . . . .           | 76        |
|          | DS-R-05 — Invalidation du cache après affectation . . . . .                                  | 77        |
|          | DS-R-06 — Provisionnement des templates réservé à administration-core . . . . .              | 78        |
| <b>6</b> | <b>organization-core</b>                                                                     | <b>79</b> |

|          |                                                                                                |            |
|----------|------------------------------------------------------------------------------------------------|------------|
| 6.1      | Description du core . . . . .                                                                  | 79         |
| 6.2      | Rôle principal . . . . .                                                                       | 80         |
| 6.3      | Rôles secondaires . . . . .                                                                    | 80         |
|          | Diagramme de classes — organization-core . . . . .                                             | 82         |
|          | Diagramme des cas d'utilisation — organization-core . . . . .                                  | 84         |
|          | DS-O-01 — Création d'une organisation avec agence siège automatique . . . . .                  | 85         |
|          | DS-O-02 — Création d'une agence opérationnelle . . . . .                                       | 86         |
|          | DS-O-03 — Mise à jour des horaires d'ouverture d'une agence . . . . .                          | 87         |
|          | DS-O-04 — Lien fonctionnel entre un acteur et une organisation . . . . .                       | 88         |
|          | DS-O-05 — Suspension d'une organisation et de ses agences . . . . .                            | 89         |
|          | DS-O-06 — Ajout d'un point d'intérêt géographique à une agence . . . . .                       | 90         |
| <b>7</b> | <b>tp-core — Gestion des Tiers</b>                                                             | <b>92</b>  |
| 7.1      | Description du core . . . . .                                                                  | 92         |
| 7.2      | Rôle principal . . . . .                                                                       | 92         |
| 7.3      | Rôles secondaires . . . . .                                                                    | 93         |
|          | Diagramme de classes — tp-core . . . . .                                                       | 94         |
|          | Diagramme des cas d'utilisation — tp-core . . . . .                                            | 96         |
|          | DS-TP-01 — Création d'un tiers avec rôle client . . . . .                                      | 97         |
|          | DS-TP-02 — Qualification commerciale d'un prospect . . . . .                                   | 98         |
|          | DS-TP-03 — Conversion d'un prospect en client . . . . .                                        | 99         |
|          | DS-TP-04 — Enregistrement d'une interaction commerciale . . . . .                              | 100        |
|          | DS-TP-05 — Ajout d'un compte bancaire à un tiers . . . . .                                     | 101        |
|          | DS-TP-06 — Recherche plein-texte de tiers (Elasticsearch / fallback PostgreSQL) . . . . .      | 102        |
| <b>8</b> | <b>settings-core</b>                                                                           | <b>103</b> |
| 8.1      | Description du core . . . . .                                                                  | 103        |
| 8.2      | Rôle principal . . . . .                                                                       | 103        |
| 8.3      | Rôles secondaires . . . . .                                                                    | 103        |
|          | Diagramme de classes — settings-core . . . . .                                                 | 105        |
|          | Diagramme des cas d'utilisation — settings-core . . . . .                                      | 107        |
|          | DS-S-01 — Génération d'un numéro documentaire avec fallback agence<br>→ organisation . . . . . | 108        |
|          | DS-S-02 — Lecture des paramètres métier effectifs . . . . .                                    | 109        |
|          | DS-S-03 — Mise à jour des options générales d'une organisation . . . . .                       | 110        |
|          | DS-S-04 — Upsert d'une séquence documentaire d'agence . . . . .                                | 111        |
|          | DS-S-05 — Lecture de la politique opérationnelle d'agence avec repli<br>organisation . . . . . | 112        |
|          | DS-S-06 — Mise à jour d'une politique opérationnelle d'agence . . . . .                        | 113        |
| <b>9</b> | <b>product-core — Catalogue Produits</b>                                                       | <b>114</b> |
| 9.1      | Description du core . . . . .                                                                  | 114        |
| 9.2      | Rôle principal . . . . .                                                                       | 114        |

|                                                                           |            |
|---------------------------------------------------------------------------|------------|
| 9.3 Rôles secondaires . . . . .                                           | 114        |
| Diagramme de classes — product-core . . . . .                             | 116        |
| Diagramme des cas d'utilisation — product-core . . . . .                  | 118        |
| DS-P-01 — Création d'un produit avec variante par défaut . . . . .        | 119        |
| DS-P-02 — Définition d'un prix pour une variante . . . . .                | 120        |
| DS-P-03 — Création d'un lot de fabrication . . . . .                      | 121        |
| DS-P-04 — Résolution du prix effectif d'une variante à une date . . . . . | 122        |
| DS-P-05 — Mise à jour de l'index Elasticsearch après modification produit | 123        |
| DS-P-06 — Ajout d'une traduction multi-langue à un produit . . . . .      | 124        |
| <b>10 inventory-core — Gestion des Stocks</b>                             | <b>125</b> |
| 10.1 Description du core . . . . .                                        | 125        |
| 10.2 Rôle Principal . . . . .                                             | 125        |
| 10.3 Rôles Secondaires . . . . .                                          | 126        |
| Diagramme de classe métier — inventory-core . . . . .                     | 128        |
| Diagramme des cas d'utilisation — inventory-core . . . . .                | 130        |
| DS-I-01 — Enregistrer puis valider un mouvement de stock . . . . .        | 131        |
| DS-I-02 — Dispatcher directement le stock d'une commande confirmée        | 132        |
| DS-I-03 — Créer puis valider une session d'inventaire physique . . . . .  | 133        |
| DS-I-04 — Enregistrer une transformation de produits . . . . .            | 134        |
| DS-I-05 — Créer puis compléter un transfert inter-entrepôts . . . . .     | 135        |
| DS-I-06 — Calculer le solde consolidé d'un produit . . . . .              | 136        |
| <b>11 resource-core — Ressources Matérielles</b>                          | <b>137</b> |
| 11.1 Description du core . . . . .                                        | 137        |
| 11.2 Rôle principal . . . . .                                             | 137        |
| 11.3 Rôles secondaires . . . . .                                          | 137        |
| Diagramme de classes — resource-core . . . . .                            | 139        |
| Diagramme des cas d'utilisation — resource-core . . . . .                 | 141        |
| DS-RE-01 — Enregistrement d'une ressource matérielle . . . . .            | 142        |
| DS-RE-02 — Affectation d'une ressource à un acteur . . . . .              | 143        |
| DS-RE-03 — Envoi d'une ressource en maintenance . . . . .                 | 144        |
| DS-RE-04 — Retour de maintenance vers l'état opérationnel . . . . .       | 145        |
| DS-RE-05 — Mise à jour de la localisation GPS d'une ressource . . . . .   | 146        |
| DS-RE-06 — Mise au rebut irréversible d'une ressource . . . . .           | 147        |
| <b>12 sales-core — Commandes de Vente</b>                                 | <b>149</b> |
| 12.1 Description du core . . . . .                                        | 149        |
| 12.2 Rôle principal . . . . .                                             | 149        |
| 12.3 Rôles secondaires . . . . .                                          | 149        |
| Diagramme de classes — sales-core . . . . .                               | 151        |
| Diagramme des cas d'utilisation — sales-core . . . . .                    | 153        |
| DS-SA-01 — Création d'une commande en brouillon . . . . .                 | 154        |
| DS-SA-02 — Confirmation avec dispatch direct du stock . . . . .           | 155        |

|                                                                                                                |            |
|----------------------------------------------------------------------------------------------------------------|------------|
| DS-SA-03 — Annulation d'une commande restée en brouillon . . . . .                                             | 156        |
| DS-SA-04 — Mise à disposition d'un snapshot confirmé pour facturation<br>explicite . . . . .                   | 157        |
| DS-SA-05 — Ajout d'une ligne article à une commande en brouillon . .                                           | 158        |
| DS-SA-06 — Consultation paginée et filtrée du carnet de commandes .                                            | 159        |
| <b>13 accounting-core — Comptabilité</b>                                                                       | <b>160</b> |
| 13.1 Description du core . . . . .                                                                             | 160        |
| 13.2 Rôle principal . . . . .                                                                                  | 160        |
| 13.3 Rôles secondaires . . . . .                                                                               | 160        |
| Diagramme de classes — accounting-core . . . . .                                                               | 162        |
| Diagramme des cas d'utilisation — accounting-core . . . . .                                                    | 164        |
| DS-AC-01 — Créer une facture canonique à partir d'une commande<br>confirmée . . . . .                          | 165        |
| DS-AC-02 — Valider explicitement une facture canonique . . . . .                                               | 166        |
| DS-AC-03 — Appliquer explicitement un règlement sur une facture postée                                         | 167        |
| DS-AC-04 — Construire une projection <code>invoice-accounting</code> depuis<br>une facture canonique . . . . . | 168        |
| DS-AC-05 — Enregistrer un posting comptable de caisse . . . . .                                                | 169        |
| DS-AC-06 — Poster un brouillard comptable en écriture manuelle . . .                                           | 170        |
| <b>14 billing-core — Facturation commerciale legacy</b>                                                        | <b>171</b> |
| 14.1 Description du core . . . . .                                                                             | 171        |
| 14.2 Rôle principal . . . . .                                                                                  | 171        |
| 14.3 Rôles secondaires . . . . .                                                                               | 171        |
| Diagramme de classes — billing-core . . . . .                                                                  | 174        |
| Diagramme des cas d'utilisation — billing-core . . . . .                                                       | 175        |
| DS-BI-01 — Création d'un document commercial legacy . . . . .                                                  | 176        |
| DS-BI-02 — Mise à jour d'un document avec remplacement de ses lignes                                           | 177        |
| DS-BI-03 — Consultation enrichie d'un document commercial . . . . .                                            | 178        |
| DS-BI-04 — Transition explicite d'un bon de livraison vers <code>FULFILLED</code>                              | 179        |
| DS-BI-05 — Enregistrement d'un paiement legacy . . . . .                                                       | 180        |
| DS-BI-06 — Consultation des paiements filtrés par facture legacy . . .                                         | 181        |
| 14.4 Surface API effectivement exposée . . . . .                                                               | 182        |
| <b>15 treasury-core — Trésorerie</b>                                                                           | <b>183</b> |
| 15.1 Description du core . . . . .                                                                             | 183        |
| 15.2 Rôle principal . . . . .                                                                                  | 183        |
| 15.3 Rôles secondaires . . . . .                                                                               | 183        |
| Diagramme de classes — treasury-core . . . . .                                                                 | 185        |
| Diagramme des cas d'utilisation — treasury-core . . . . .                                                      | 187        |
| DS-TR-01 — Enregistrement d'un compte bancaire . . . . .                                                       | 188        |
| DS-TR-02 — Enregistrement explicite d'un règlement bancaire de facture                                         | 189        |
| DS-TR-03 — Import d'un relevé bancaire . . . . .                                                               | 190        |

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| DS-TR-04 — Appariement d'une transaction bancaire à une ligne de relevé                   | 191        |
| DS-TR-05 — Émission d'un chèque avec création de la transaction en<br>attente . . . . .   | 192        |
| DS-TR-06 — Clôture définitive d'un rapprochement bancaire . . . . .                       | 193        |
| <b>16 cashier-core — Caisse opérationnelle</b>                                            | <b>194</b> |
| 16.1 Description du core . . . . .                                                        | 194        |
| 16.2 Rôle principal . . . . .                                                             | 194        |
| 16.3 Rôles secondaires . . . . .                                                          | 194        |
| Diagramme de classes — cashier-core . . . . .                                             | 196        |
| Diagramme des cas d'utilisation — cashier-core . . . . .                                  | 197        |
| DS-CA-01 — Création d'une caisse avec compte comptable et wallet<br>d'ouverture . . . . . | 198        |
| DS-CA-02 — Ouverture d'une session de caisse avec posting comptable                       | 199        |
| DS-CA-03 — Encaissement d'un bill local avec mouvement et posting<br>de caisse . . . . .  | 200        |
| DS-CA-04 — Import explicite d'une facture comptable comme bill lié .                      | 201        |
| DS-CA-05 — Synchronisation explicite d'un bill lié vers la comptabilité                   | 202        |
| DS-CA-06 — Clôture d'une session avec traitement des écarts . . . . .                     | 203        |
| 16.4 Surface API effectivement exposée . . . . .                                          | 204        |
| <b>17 administration-core — Gouvernance et Administration</b>                             | <b>205</b> |
| 17.1 Description du core . . . . .                                                        | 205        |
| 17.2 Rôle principal . . . . .                                                             | 205        |
| 17.3 Rôles secondaires . . . . .                                                          | 205        |
| Diagramme de classes — administration-core . . . . .                                      | 207        |
| Diagramme des cas d'utilisation — administration-core . . . . .                           | 209        |
| DS-AD-01 — Approbation d'un BusinessActor . . . . .                                       | 210        |
| DS-AD-02 — Mise à jour des options de plateforme avec audit . . . . .                     | 211        |
| DS-AD-03 — Consultation filtrée de la piste d'audit . . . . .                             | 212        |
| DS-AD-04 — Rejet d'une demande de gouvernance avec notification . .                       | 213        |
| DS-AD-05 — Supervision de l'état du système via le port de management                     | 214        |
| DS-AD-06 — Approbation d'une organisation soumise à validation . . .                      | 215        |
| <b>18 file-core — Gestion des Fichiers</b>                                                | <b>216</b> |
| 18.1 Description du core . . . . .                                                        | 216        |
| 18.2 Rôle principal . . . . .                                                             | 216        |
| 18.3 Rôles secondaires . . . . .                                                          | 216        |
| Diagramme de classes — file-core . . . . .                                                | 218        |
| Diagramme des cas d'utilisation — file-core . . . . .                                     | 220        |
| DS-F-01 — Téléversement d'un fichier et audit système . . . . .                           | 221        |
| DS-F-02 — Téléchargement d'un fichier stocké . . . . .                                    | 222        |
| DS-F-03 — Rattachement d'un document à une cible métier . . . . .                         | 223        |
| DS-F-04 — Consultation des documents d'une cible . . . . .                                | 224        |



|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| DS-F-05 — Mise à jour d'une politique de gouvernance documentaire . . . . . | 225        |
| DS-F-06 — Revue d'un document lié avec expiration éventuelle . . . . .      | 226        |
| <b>19 hrm-core</b>                                                          | <b>227</b> |
| 19.1 Description du core . . . . .                                          | 227        |
| 19.2 Rôle principal . . . . .                                               | 227        |
| 19.3 Sous-domaines couverts . . . . .                                       | 227        |
| 19.4 Intégrations inter-cores . . . . .                                     | 228        |
| <b>20 blockchain-core</b>                                                   | <b>230</b> |
| 20.1 Description du core . . . . .                                          | 230        |
| 20.2 Rôle principal . . . . .                                               | 230        |
| 20.3 Capacités exposées . . . . .                                           | 230        |
| 20.4 Persistance et modèle . . . . .                                        | 231        |
| 20.5 Ancrage transversal du projet . . . . .                                | 231        |
| 20.6 Sécurité . . . . .                                                     | 232        |
| 20.7 Flow Postman de référence . . . . .                                    | 232        |
| <b>21 Synthèse opérationnelle</b>                                           | <b>233</b> |
| 21.1 Rôle dans la lecture projet . . . . .                                  | 233        |
| 21.2 Corpus documentaire backend . . . . .                                  | 233        |
| 21.3 Readiness production . . . . .                                         | 234        |
| 21.4 Contrat de déploiement . . . . .                                       | 234        |
| 21.5 Observabilité et exploitation . . . . .                                | 235        |
| 21.6 Capacité, charge et scaling . . . . .                                  | 235        |
| 21.7 Résilience et runbooks . . . . .                                       | 236        |
| 21.8 Gouvernance des intégrations . . . . .                                 | 236        |
| 21.9 Guide de connexion d'un backend externe . . . . .                      | 237        |
| 21.10 Roadmap encadrée . . . . .                                            | 238        |
| <b>Conclusion Générale</b>                                                  | <b>239</b> |



# Introduction Générale

---

## Objet du document

Ce cahier d'analyse décrit l'architecture technique et fonctionnelle de la plateforme **RT-Comops Backend**. Il constitue le document de référence pour la conception, le développement et l'évolution du système.

Pour chaque module fonctionnel — appelé *core* dans ce document — une analyse complète est fournie, comprenant : la description détaillée, les rôles primaire et secondaires, le diagramme de classes, le diagramme des cas d'utilisation, ainsi qu'un ensemble de diagrammes de séquences couvrant les interactions techniques les plus significatives.

## Périmètre fonctionnel

La plateforme vise à couvrir l'intégralité du cycle opérationnel d'une entreprise, articulé autour des domaines suivants :

- **Gestion des identités** — acteurs, comptes utilisateurs, rôles et permissions
- **Structure organisationnelle** — organisations, agences, tiers, paramétrage
- **Catalogue et stocks** — produits, variantes, lots, mouvements de stock, transformations
- **Ressources matérielles** — équipements, véhicules, affectations
- **Commercial** — commandes de vente, pièces commerciales, paiements legacy et cycle de facturation
- **Finance** — comptabilité générale, billing, trésorerie, encaissement caisse et rapprochement bancaire
- **Ressources humaines** — employés, contrats, congés, paie, recrutement, onboarding, formation, frais, missions, médical et KPI RH
- **Transverse** — multi-tenancy, audit immuable, publication d'événements, fichiers et ancrage blockchain

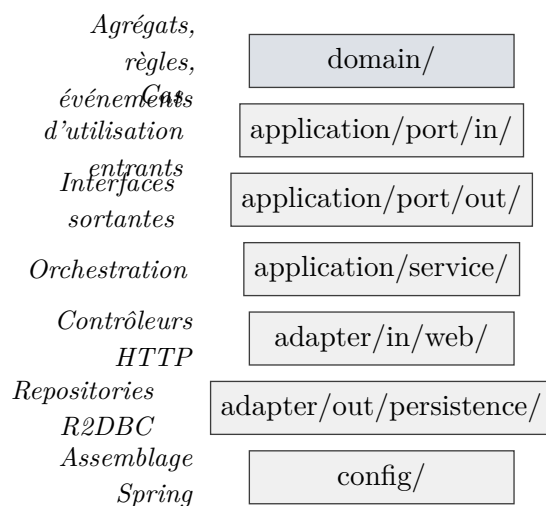
## Vision architecturale

### Monolithe modulaire hexagonal

Le système est conçu comme un **monolithe modulaire** à architecture hexagonale, organisé en vingt cores fonctionnels plus un module de bootstrap. Cette approche garantit :

- une isolation stricte des domaines métier sans couplage fort,
- une testabilité maximale à chaque niveau,
- une extractibilité vers des microservices sans refactorisation du modèle domaine.

Chaque core suit la structure interne suivante :



### Modèle canonique de base

Le modèle conceptuel canonique de la plateforme est centré sur les agrégats Actor, BusinessActor, Organization, Agency, ThirdParty, Address, Contact, ainsi que sur les structures de rattachement (OrganizationActor, AgencyAffiliation, OrganizationDomain, AgencyDomain).

Ce modèle sert de référence pour l'ensemble du dépôt. Les diagrammes de cores déclinent cette base selon deux angles complémentaires :

- **vision conceptuelle métier** — nommage canonique des attributs (`shortName`, `longName`, `taxNumber`, `businessRegistrationNumber`, etc.) ;
- **vision technique d'exécution** — apparition ponctuelle de champs de portée comme `tenantId`, d'horodatages techniques ou de mécanismes de routage lorsque cela est utile pour expliquer le runtime.

En conséquence, un champ de portée technique ne doit jamais être interprété comme le centre de gravité du domaine métier ; la sémantique fonctionnelle reste portée par le modèle canonique.

## Contexte multi-tenant et portées d'administration

Le système gère le cloisonnement des données selon quatre niveaux de portée :

| Niveau              | Rôle                      | Isolation                                  | Portée           |
|---------------------|---------------------------|--------------------------------------------|------------------|
| <b>SYSTEM</b>       | Administrateur plateforme | Logique globale partagée                   | Tous les tenants |
| <b>TENANT</b>       | Administrateur tenant     | Filtrage logique par <code>tenantId</code> | Un tenant        |
| <b>ORGANIZATION</b> | Unité métier              | Filtrage logique dans le tenant            | Une organisation |
| <b>AGENCY</b>       | Unité opérationnelle      | Filtrage logique dans l'organisation       | Une agence       |

Dans l'état actuel du backend, l'isolation multi-tenant n'est pas portée par un schéma PostgreSQL dédié par tenant. Elle repose sur :

- l'injection du `tenantId` courant par le `TenantWebFilter` dans le contexte réactif ;
- la persistance systématique de `tenant_id` dans les tables métier ;
- les contrôles de sécurité, de quota et d'entitlements évalués sur les portées `TENANT`, `ORGANIZATION` et `AGENCY`.

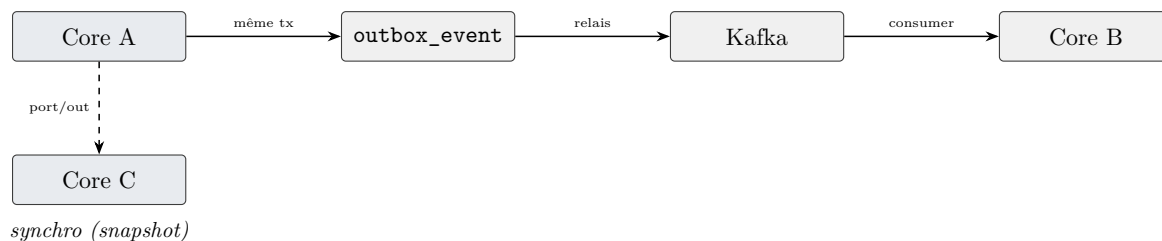
Cette isolation reste une contrainte transverse d'exécution ; elle ne remplace pas la modélisation métier de base `BusinessActor` → `Organization` → `Agency`.

## Communication inter-cores

Les cores communiquent selon trois modalités complémentaires :

1. **Ports et adaptateurs (synchrone)** — un core définit une interface sortante (*port/out*), implémentée par un adaptateur du core cible. Le couplage est limité à des identifiants et des instantanés (*snapshots*) dénormalisés, jamais à des objets domaine directs.
2. **Outbox transactionnel + Kafka (asynchrone)** — lors d'une opération, l'événement domaine est écrit dans la table `kernel.outbox_event` au sein de la même transaction que l'agrégat. Un relais de fond le publie ensuite vers Kafka (`iwm.events.business`). Les cores abonnés consomment et mettent à jour leurs projections locales.

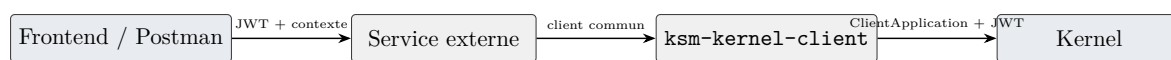
3. **Communication inter-services explicite et gardée** — lorsqu'un core métier déclenche volontairement une opération dans un autre core métier du monolithe, l'appel reste synchrone mais passe par une garde transverse. Le kernel vérifie alors l'entitlement runtime du service cible et consomme son quota d'organisation avant d'autoriser l'opération.



## Services externes spécialisés

En complément du monolithe modulaire, le workspace `KSM_services` porte des services externes spécialisés : `billing-service`, `banking-service`, `accounting-service` et `cashier-service`. Ces services ne remplacent pas le kernel et ne deviennent pas propriétaires des vérités transverses.

Le contrat d'intégration repose sur le module commun `ksm-kernel-client`. Celui-ci centralise l'appel HTTP vers le kernel, injecte l'identité machine du backend consommateur (`X-Client-Id`, `X-API-Key`), propage le contexte `X-Tenant-Id`, `X-Organization-Id` et relaie le bearer token utilisateur lorsqu'un endpoint métier requiert un contexte RBAC.

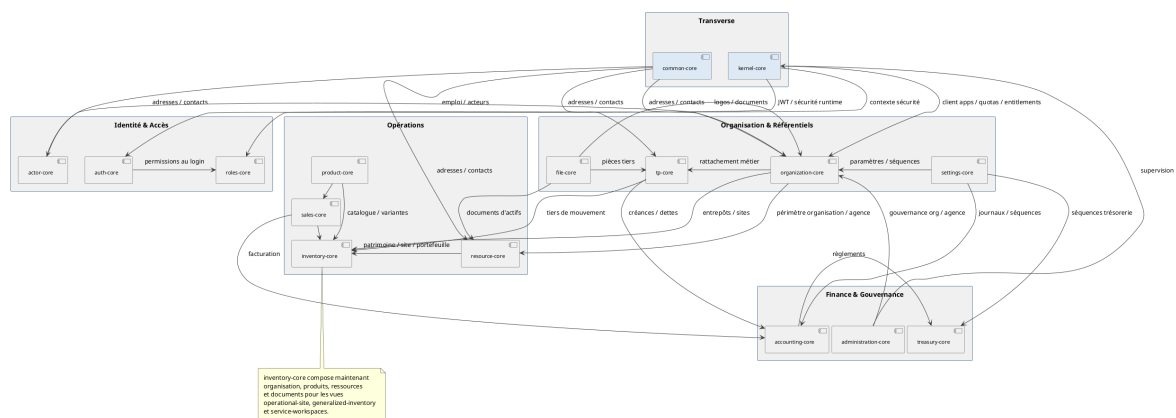


Chaque service externe doit posséder sa propre `ClientApplication` (`billing-backend`, `banking-backend`, `accounting-backend`, `cashier-backend`) et ne doit pas réutiliser durablement le client bootstrap. Lorsque la base ne contient aucun administrateur, un bootstrap admin initial est nécessaire avant de créer ces identités machine.

## Pile technologique

| Composant             | Version | Rôle                                      |
|-----------------------|---------|-------------------------------------------|
| Spring Boot / WebFlux | 4.0.5   | Serveur réactif HTTP                      |
| Java                  | 21      | Langage                                   |
| PostgreSQL            | 18.3    | Base relationnelle principale             |
| R2DBC                 | —       | Driver réactif PostgreSQL                 |
| Liquibase             | —       | Migrations de schéma                      |
| Apache Kafka          | 8.2.0   | Bus d'événements en mode KRaft local      |
| Redis                 | 8.6.2   | Cache permissions, quotas et verrous      |
| Elasticsearch         | 9.3.3   | Projections de recherche                  |
| Prometheus + Grafana  | —       | Métriques et supervision                  |
| Nginx                 | 1.29.8  | Passerelle HTTP, rate limiting et Swagger |

## Cartographie des cores



## Conventions du document

### Diagrammes de classes

- Les classes concrètes sont représentées par un rectangle à trois compartiments (nom, attributs, opérations).
- Les interfaces sont indiquées en tirets avec le stéréotype «interface».
- Les classes abstraites apparaissent en italique.
- La multiplicité est notée près des extrémités de relation.

### Diagrammes de séquences

- Les lignes de vie verticales représentent les composants du système.
- Les messages synchrones sont représentés par une flèche pleine.
- Les messages asynchrones (événements) sont représentés par une flèche en tirets.
- Les fragments combinés (`alt`, `opt`, `loop`) délimitent les blocs conditionnels.

### Diagrammes de cas d'utilisation

- Les acteurs humains sont représentés par un pictogramme.
- Les systèmes ou acteurs externes sont représentés par un rectangle.
- Les relations «include» et «extend» suivent la notation UML standard.



# Vue Generale du Projet

---

## Lecture rapide

**RT-Comops Backend** est le kernel applicatif du projet. Il porte les domaines transverses, les droits, les organisations, les abonnements, les quotas, les evenements, les fichiers, les fonctions metier principales et les contrats d'integration des services externes. Lire ce cahier revient a lire la structure generale du projet : ce que le systeme fait, comment il est decoupe, comment il s'execute et quelles regles encadrent son evolution.

Le projet n'est pas organise comme une collection d'APIs isolees. Il est organise comme un **monolithe modulaire hexagonal** : les modules metier sont dans un meme backend deployable, mais ils gardent des frontieres internes strictes. Chaque core expose ses cas d'utilisation, protege son domaine, persiste ses donnees par PostgreSQL reactif et communique avec les autres cores par ports, snapshots ou evenements outbox.

## Ce que couvre le projet

Le backend couvre six grands ensembles fonctionnels :

- **Kernel et socle transverse** : multi-tenancy logique, audit, outbox, ClientApplications, quotas, observabilite et contrats d'execution.
- **Identite et acces** : acteurs, comptes utilisateurs, login, OIDC central, JWT RS256, OTP, MFA, roles, permissions et scopes d'administration.
- **Organisation et catalogue** : organisations, agences, tiers, abonnements de services, plans, parametres, produits, categories, prix et stocks.
- **Operations metier** : ventes, inventaire, ressources materiels, fichiers, administration et projections transverses d'exploitation.
- **Finance et encaissement** : billing commercial, comptabilite, tresorerie, comptes de caisse, transferts, depots, retraits et lien explicite entre encaissement et comptabilite.
- **Extensions fortes** : ressources humaines completes et blockchain interne pour wallets, signatures, blocs, ancrage documentaire et ancrage d'evenements metier.

## Decoupage des depots et documents

Le workspace est lu comme un ensemble coherent :

- `iwm-backend/RT-comops-*` contient l'implementation backend par modules.
- `iwm-backend/docs/` contient les documents d'exploitation, de gouvernance, de deploiement, de readiness, de performance et d'integration.
- `docs-analyse/` est le cahier de reference qui consolide la vue fonctionnelle, technique et operationnelle du backend.
- `KSM_services/` contient les services specialises externes qui consomment le kernel via `ksm-kernel-client`.
- Les collections Postman versionnees dans `iwm-backend/docs/postman/` decrivent les flows exploitables de reference, notamment auth/IAM/OIDC et blockchain.

## Architecture en une page

- **Runtime applicatif** : Spring Boot WebFlux, Java 21, R2DBC et PostgreSQL.
- **Source de verite** : PostgreSQL reste le store principal ; Redis et Elasticsearch sont des accelerateurs, jamais des sources de verite.
- **Evenements** : les evenements metier passent par une outbox transactionnelle, puis Kafka.
- **Recherche et projections** : Elasticsearch porte des projections de lecture et de recherche.
- **Capacite** : Nginx protege le trafic en frontal ; Redis porte les quotas backend et organisationnels.
- **Observabilite** : Actuator, Prometheus et Grafana exposent le health, les metriques, l'outbox, Kafka, Redis, Elasticsearch et les projections.
- **Deploiement** : Docker Compose couvre local, preprod et prod-like avec secrets montes par fichiers et validation runtime.

## Pour les equipes qui integrent le projet

Une equipe qui veut integrer RT-Comops dans son propre produit doit comprendre trois niveaux distincts :

- **Le kernel** : il reste proprietaire des tenants, utilisateurs, organisations, roles, abonnements, quotas, services autorises, evenements et verites metier principales.
- **Le backend consommateur** : il represente la plateforme ou le service externe qui appelle le kernel. Il doit etre declare comme `ClientApplication`, posseder son propre secret, et etre limite aux services dont il a reellement besoin.

- **Le frontend consommateur** : il ne doit jamais transporter l'**X-Api-Key**. Il appelle son backend métier, qui relaie ensuite les appels vers le kernel avec son identité serveur.

Le parcours d'intégration standard est le suivant :

1. créer ou choisir le tenant cible ;
2. créer une **ClientApplication** pour le backend consommateur ;
3. limiter cette application aux **allowedServices** nécessaires ;
4. stocker le secret de cette application dans le gestionnaire de secrets du backend consommateur ;
5. connecter le backend consommateur avec **ksm-kernel-client** ou avec un client HTTP équivalent ;
6. propager à chaque appel le tenant, l'organisation, l'utilisateur et les headers serveur requis ;
7. vérifier que l'organisation cible est abonnée aux services appelés ;
8. traiter les refus fonctionnels du kernel comme des signaux de contrat : service non autorisé, organisation non abonnée, quota dépasse ou permission utilisateur insuffisante.

Un backend externe ne doit pas être traité comme un simple client HTTP. Il est une application consommatrice gouvernée par le kernel. La connexion est valide seulement si l'identité machine, le tenant, l'organisation, le service, le quota et le JWT utilisateur sont cohérents.

## Contrat minimal backend vers kernel

Tout backend qui se connecte au kernel doit envoyer :

- **X-Client-Id** : identifiant stable de la **ClientApplication** ;
- **X-Api-Key** : secret serveur de cette application ;
- **X-Tenant-Id** : tenant cible de l'appel ;
- **Authorization: Bearer <accessToken>** : requis pour les actions utilisateur ;
- **X-Organization-Id** : requis pour les services métier scopes organisation.

Le kernel contrôle ensuite, dans l'ordre : l'identité de la **ClientApplication**, le service autorisé, le quota backend, l'abonnement de l'organisation, le quota organisationnel et les permissions utilisateur. Ainsi, un intégrateur sait où chercher lorsqu'un appel est refusé :

- **CLIENT\_APPLICATION\_SERVICE\_NOT\_ALLOWED** : service non autorisé pour le backend consommateur ;
- **TENANT\_REQUEST\_QUOTA\_EXCEEDED** : quota plateforme atteint ;

- `ORGANIZATION_CONTEXT_REQUIRED` : contexte organisation manquant ;
- `ORGANIZATION_SERVICE_NOT_SUBSCRIBED` : organisation non abonnee au service ;
- `ORGANIZATION_SERVICE_QUOTA_EXCEEDED` : quota organisationnel atteint ;
- refus RBAC : permissions utilisateur insuffisantes.

## Regles qui structurent tout le projet

1. **Pas de logique metier hors core proprietaire.** Un core ne manipule pas les objets domaine vivants d'un autre core.
2. **Tout appel protege porte un contexte explicite.** Les headers `X-Tenant-Id`, `X-Client-Id`, `X-API-Key`, `X-Organization-Id` et le bearer utilisateur sont utilises selon le type d'appel.
3. **Les services externes ne contournent jamais le kernel.** Ils passent par une `ClientApplication` dediee et par `ksm-kernel-client`.
4. **Les abonnements et quotas sont executoires.** Un service n'est consommable par une organisation que si l'abonnement et le quota le permettent.
5. **L'outbox est le mecanisme canonique de propagation.** Les synchronisations implicites entre modules sont evitees.
6. **La production demande un durcissement explicite.** Le socle est pret pour le developpement professionnel et le durcissement preprod, mais pas encore pour un rollout production sans restriction.

## Etat courant

Le socle est gele comme reference d'analyse : les cores actifs sont documentes, les principaux flows d'exploitation sont versionnes, les services externes sont encadres par le contrat backend-to-kernel, et le depot backend est allége des artefacts generes.

La lecture operationnelle est la suivante :

- **Developpement** : autorise sur le socle actuel.
- **Preproduction** : autorisee apres validation des secrets, des variables runtime et des scripts de deployment.
- **Production** : conditionnee par l'industrialisation des secrets, les tests de charge, les runbooks, les drills de resilience, les probes et les limites de ressources.

## Comment lire le reste du document

Les chapitres suivants se lisent comme une decomposition du projet :

- les premiers chapitres expliquent le kernel, le common model, les identites, l'authentification et les roles ;

- les chapitres centraux decrivent les domaines metier : organisation, tiers, settings, produits, inventaire, ressources, ventes et finance ;
- les chapitres specialises couvrent caisse, administration, fichiers, RH et blockchain ;
- la synthese operationnelle reprend les documents de `iwm-backend/docs` pour relier le modele aux conditions reelles de deploiement et d'exploitation ;
- la conclusion rappelle les invariants transverses a ne pas casser.

# 1 kernel-core par Toulepi Jordan

---

## 1.1 Description du core

Le **kernel-core** constitue la colonne vertébrale technique de la plateforme. Il concentre les mécanismes transverses communs à tous les autres cores : contexte multi-tenant, authentification applicative des backends consommateurs, validation des JWT d'accès RS256, traçabilité immuable, publication fiable des événements métier, gouvernance des **ClientApplications**, quotas backend et exposition du jeu de clés publiques (JWKS).

Toute requête HTTP protégée traverse ce core avant d'atteindre un module métier. Le **TenantWebFilter** et la chaîne de sécurité construisent le contexte courant (**tenantId**, **organizationId**, **agencyId**, **userId**, **actorId**) à partir des credentials de l'application cliente et, lorsqu'il existe, du JWT portant le contexte utilisateur. Les accès métier sont ensuite bornés par trois couches successives : **ClientApplication** autorisée sur le service visé, abonnement de l'organisation au service métier si le module est abonnable, puis permissions utilisateur résolues sur le bon scope. Les communications synchrones explicites entre services métier du monolithe passent désormais par une garde transverse dédiée, qui vérifie l'accès au service cible et consomme son quota d'organisation avant de laisser une opération inter-service aboutir.

## 1.2 Rôle principal

**Rôle principal** : garantir l'exécution sûre des APIs du kernel en combinant l'isolation multi-tenant, l'authentification des *ClientApplications*, la validation des JWT RS256, la traçabilité immuable et la publication transactionnelle des événements de domaine.

## 1.3 Rôles secondaires

### 1. Provisionnement logique des tenants

Création des enregistrements de tenant, activation du contexte de sécurité

associé, exécution des migrations globales du backend et mise à disposition du tenant dans le registre logique courant. Le modèle réel ne repose pas sur un schéma PostgreSQL dédié par tenant.

## 2. Propagation logique du contexte tenant

Le tenant est résolu logiquement depuis le contexte de requête et injecté dans toute la chaîne réactive. Les cores métier restent découplés de cette résolution et portent eux-mêmes les contrôles de cohérence `tenantId + organizationId + agencyId`.

## 3. Propagation du contexte de requête

Le `TenantWebFilter` et le convertisseur de sécurité lisent les headers `X-Tenant-Id`, `X-Organization-Id`, `X-Agency-Id`, `X-Client-Id` et le bearer token éventuel, puis injectent ce contexte dans toute la chaîne réactive.

## 4. Authentification applicative par `ClientApplication`

Toute API métier protégée exige l'identité du backend appelant via `X-Client-Id` et `X-API-Key`. Les secrets sont stockés hachés et chaque application cliente peut être créée, révoquée ou voir son secret pivoté sans affecter les autres intégrations.

## 5. Restriction par service des backends consommateurs

Chaque `ClientApplication` porte une liste `allowedServiceCodes`. Un backend consommateur ne peut appeler que les routes correspondant aux services explicitement autorisés (`COMMERCIAL`, `SALES`, `TREASURY`, etc.). Un écart produit un refus `CLIENT_APPLICATION_SERVICE_NOT_ALLOWED`. Les valeurs officielles du catalogue sont `ORGANIZATION`, `SETTINGS`, `COMMERCIAL`, `PRODUCT`, `INVENTORY`, `SALES`, `BILLING`, `ACCOUNTING`, `BANKING`, `TREASURY`, `CASHIER`, `RESOURCE`, `HRM` et `BLOCKCHAIN`.

## 6. Validation JWT RS256 et exposition du JWKS

Le kernel valide les JWT d'accès RS256, extrait les claims métier (`sub`, `tid`, `oid`, `aid`, `actor`) et expose le document `/.well-known/jwks.json` pour permettre aux plateformes externes de vérifier la signature avec la clé publique.

## 7. Outbox transactionnel

Toute écriture d'événement domaine est stockée dans `kernel.outbox_event` dans la même transaction que l'agrégat concerné. Un relais asynchrone publie ensuite ces événements vers Kafka, garantissant une livraison *at-least-once*.

## 8. Piste d'audit immuable

L'entité `AuditTrail` est en insertion seule : aucune mise à jour ni suppression n'est autorisée. Elle enregistre les opérations significatives avec l'acteur, l'horodatage, la cible et les éléments de contexte utiles au diagnostic.

## 9. Supervision et observabilité

Exposition des métriques Micrometer / Prometheus sur l'état de l'outbox, des

contrôles de santé et de l'état de la configuration de sécurité (*bootstrap client*, présence des clés JWT, etc.).

10. **Quota backend par tenant, client et service**

Le filtre de quotas Redis borne les appels selon la combinaison `tenantId + clientId + serviceCode + bucket`, avec un code `CORE` configurable pour les routes non mappées. Cette séparation évite qu'une plateforme consommatrice monopolise toute la capacité d'un tenant sur l'ensemble du kernel.

11. **Quota métier par organisation et service**

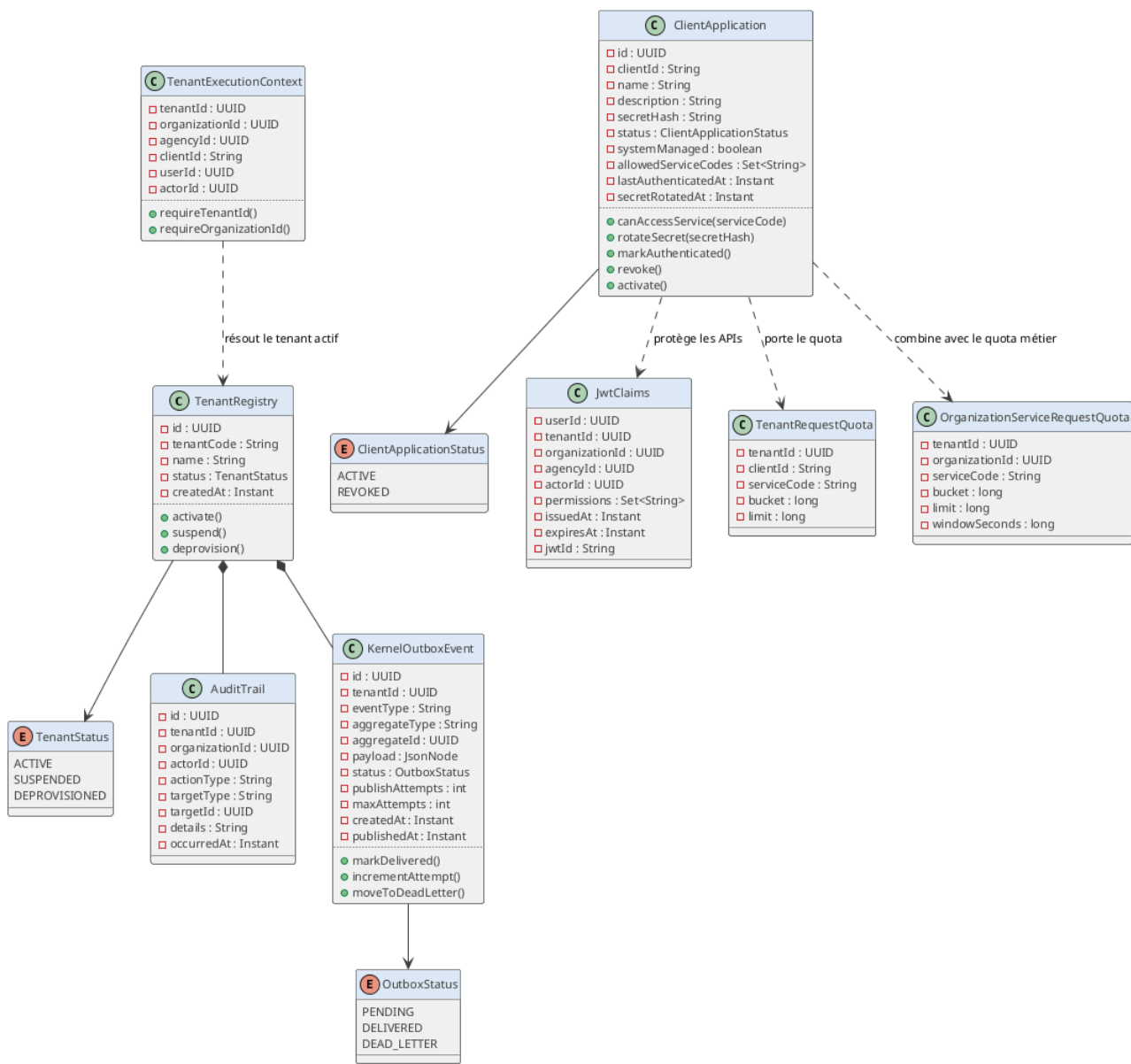
Sur les routes métier à scope organisationnel, un second filtre Redis borne les appels selon `tenantId + organizationId + serviceCode + bucket`. Les seuils proviennent directement de l'abonnement de service porté par l'organisation, ce qui permet de distinguer la protection d'infrastructure de la politique de fair-use/licensing.

12. **Garde de communication inter-services explicite**

Lorsqu'un core métier veut appeler explicitement un autre core métier dans le même monolithe (par exemple `cashier-core` vers `accounting-core`), le kernel applique la même politique d'entitlement runtime et la même consommation de quota que pour une requête HTTP dirigée vers le service cible. Une intégration interne peut donc être autorisée, refusée ou limitée sans propagation automatique implicite.



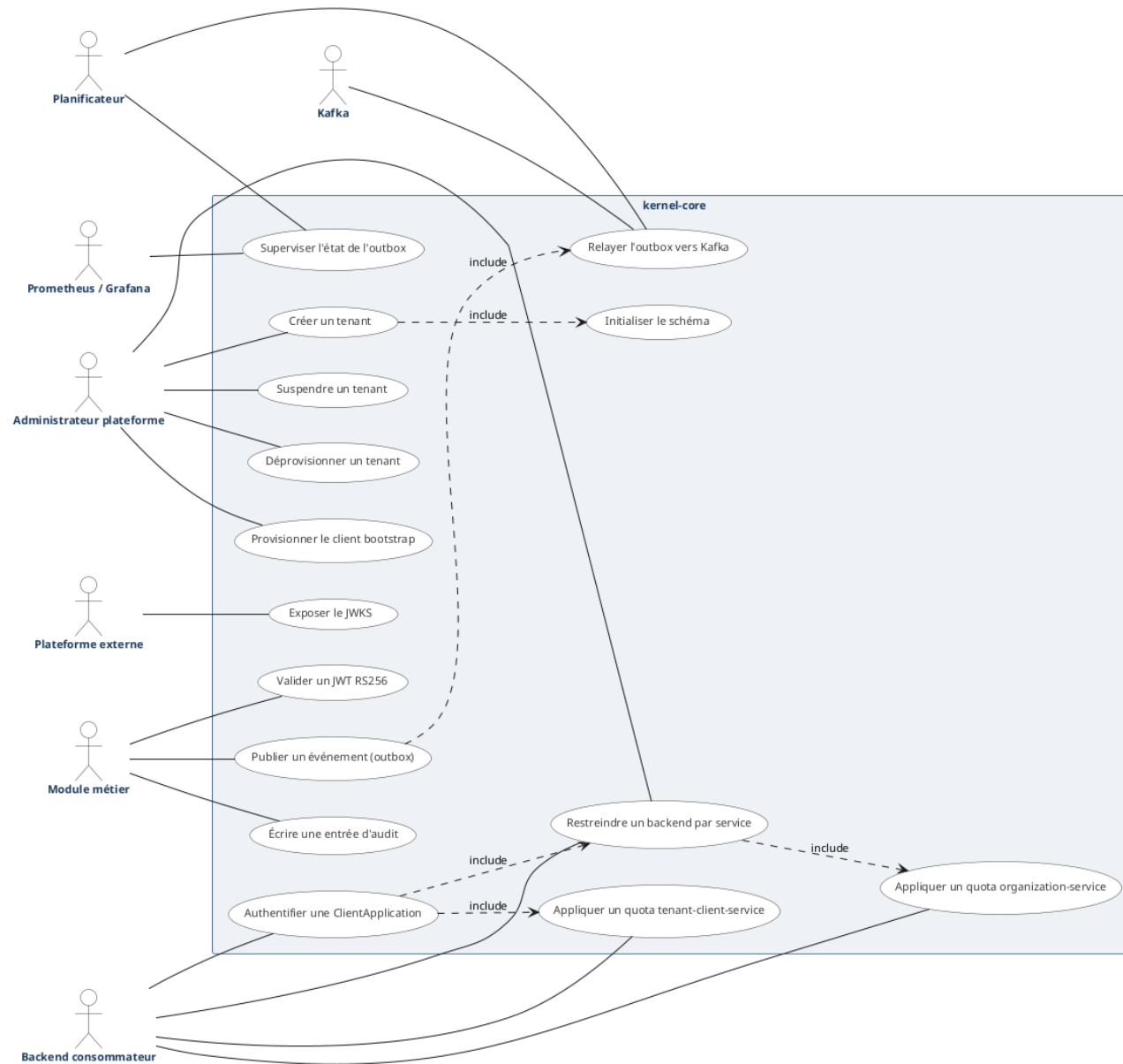
Diagramme de classes — kernel-core



**Invariants métier du kernel-core :**

- `tenantCode` est globalement unique.
- `status` d'un tenant ne peut régresser de `DEPROVISIONED`.
- `clientId` d'une `ClientApplication` est globalement unique.
- Une `ClientApplication` `REVOKED` ne peut pas être authentifiée.
- `allowedServiceCodes` d'une `ClientApplication` est non vide et borné au catalogue `PlatformServiceCode`.
- Les alias de services acceptés par le kernel sont normalisés avant persistance : par exemple `THIRD_PARTY` vers `COMMERCIAL`, `CAISSE` vers `CASHIER` et `COMPTA` vers `ACCOUNTING`.
- Les services externes doivent utiliser une `ClientApplication` dédiée ; le client bootstrap sert au provisionnement initial, pas à l'exploitation courante.
- Une requête sur un service métier organisationnel doit satisfaire à la fois le quota `tenant + client + service` et le quota `organization + service`.
- Une communication inter-services explicite vers un service cible n'est autorisée que si ce service est effectif pour l'organisation et que son quota n'est pas dépassé.
- `AuditTrail` est en insertion seule.
- `publishAttempts`  $\geq 0$  et `maxAttempts`  $> 0$ .
- Un événement `DEAD_LETTER` n'est jamais re-tenté automatiquement.

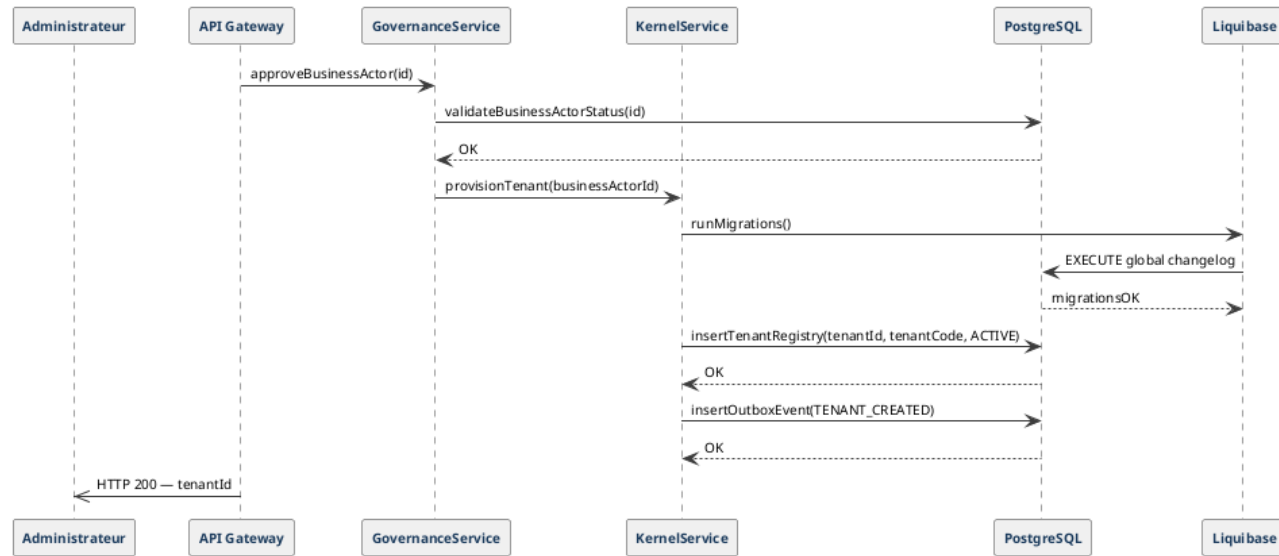
## Diagramme des cas d'utilisation — kernel-core



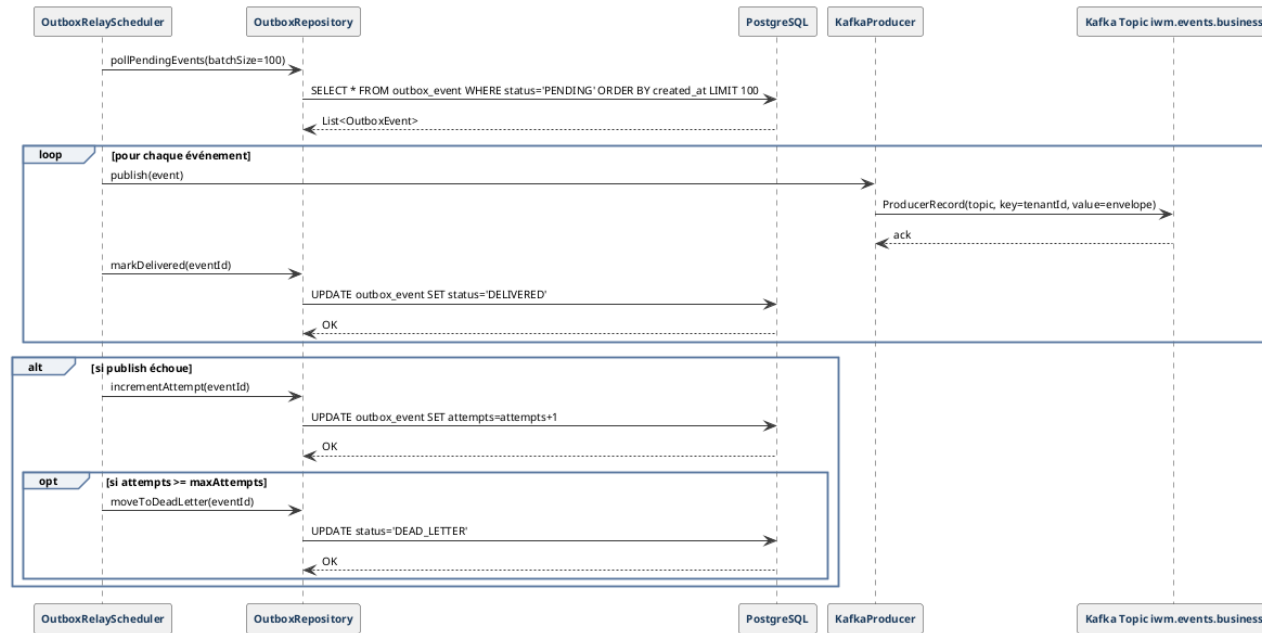
### 1.3.1 DS-K-01 : Provisionnement d'un nouveau tenant

Ce diagramme décrit la séquence complète d'activation logique d'un nouveau tenant, depuis la validation du **BusinessActor** jusqu'à la disponibilité opérationnelle du contexte tenant.

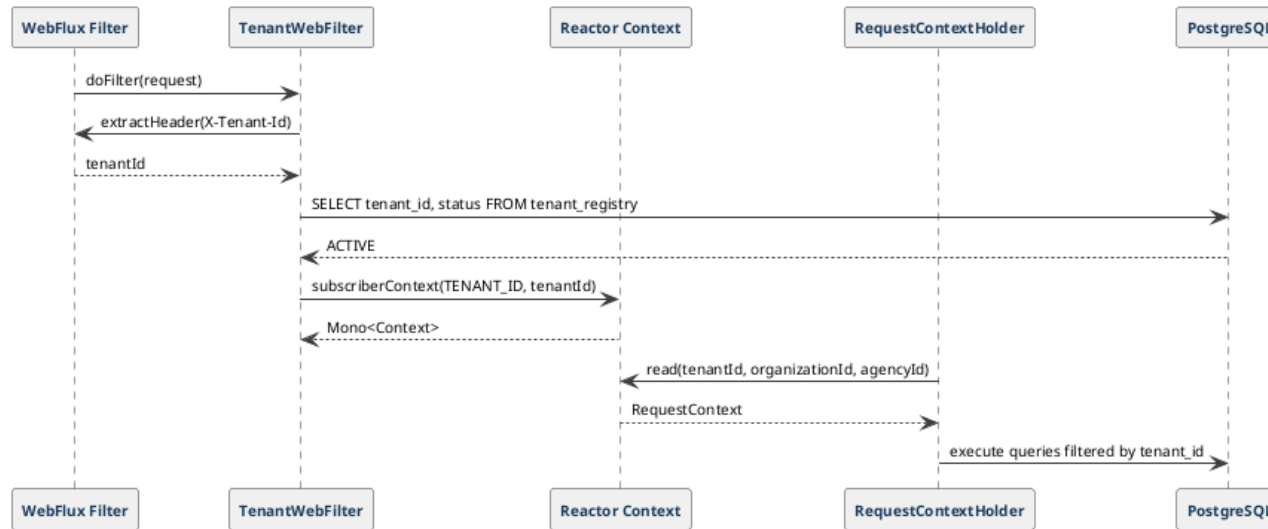
## DS-K-01 — Provisionnement logique d'un nouveau tenant



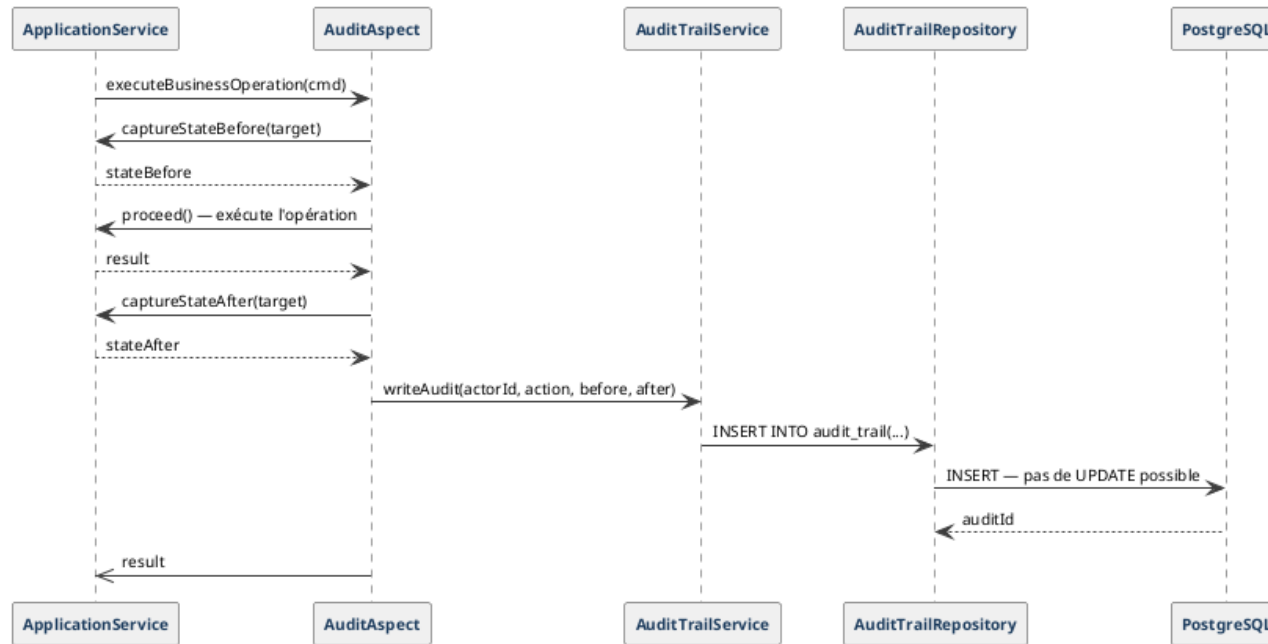
## DS-K-02 — Relais Outbox vers Kafka



## DS-K-03 — Résolution réactive du contexte tenant

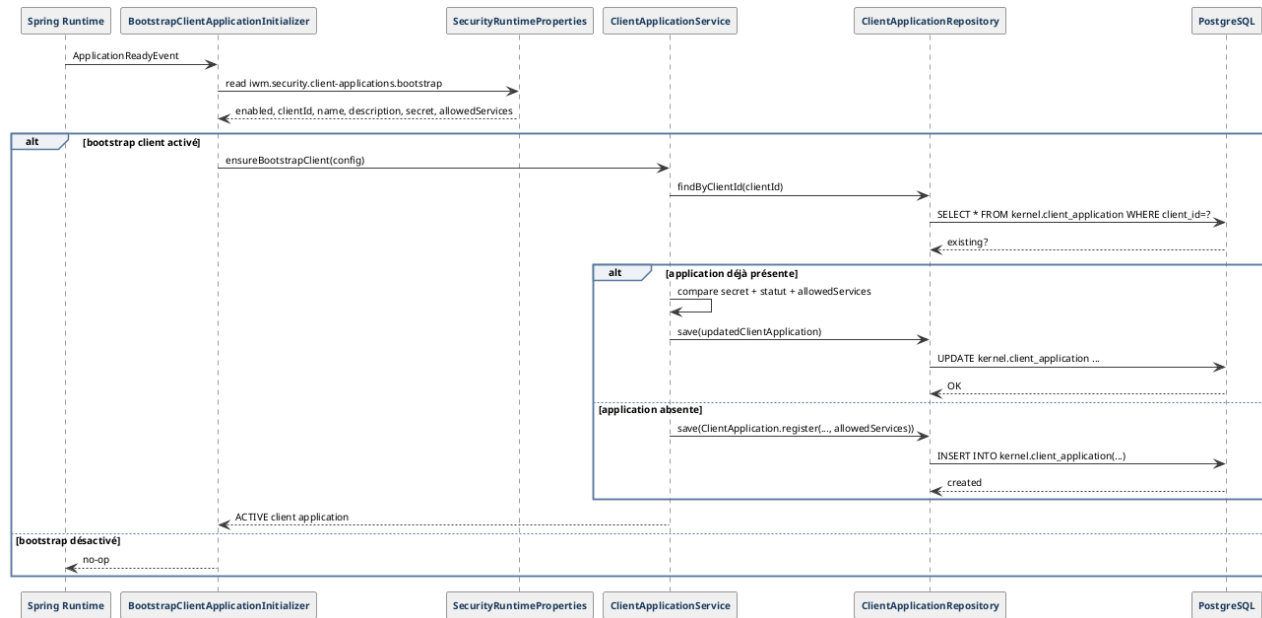


## DS-K-04 — Écriture de la piste d'audit (pattern aspect)

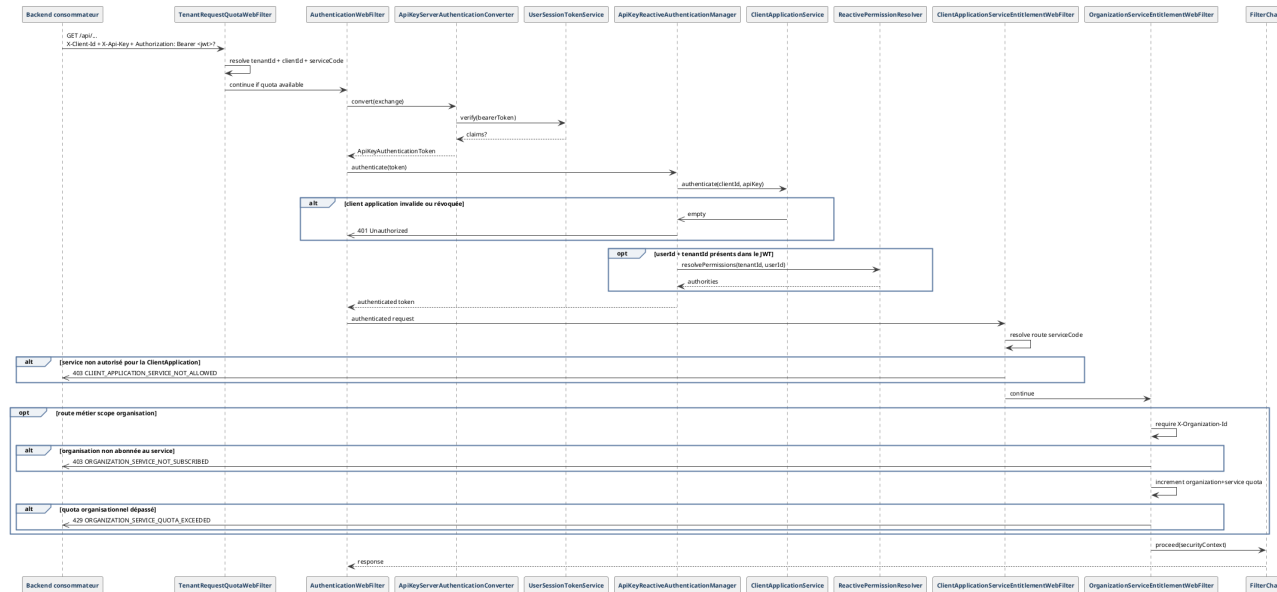




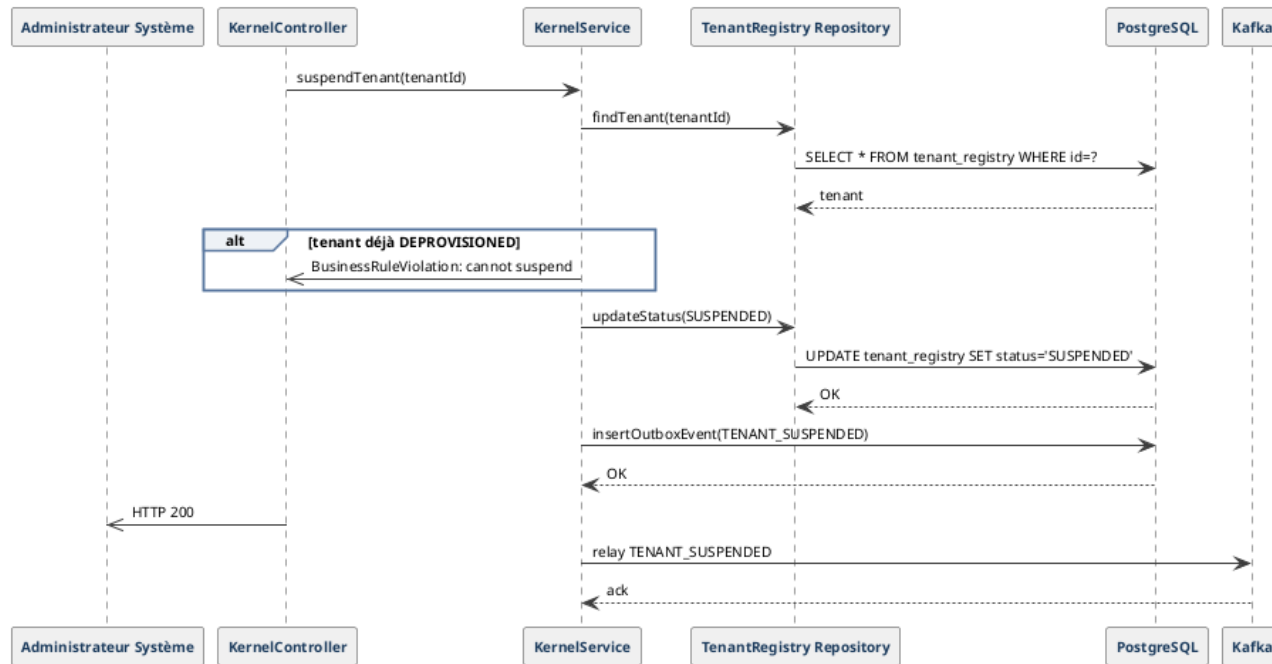
## DS-K-05 — Initialisation d'une ClientApplication bootstrap



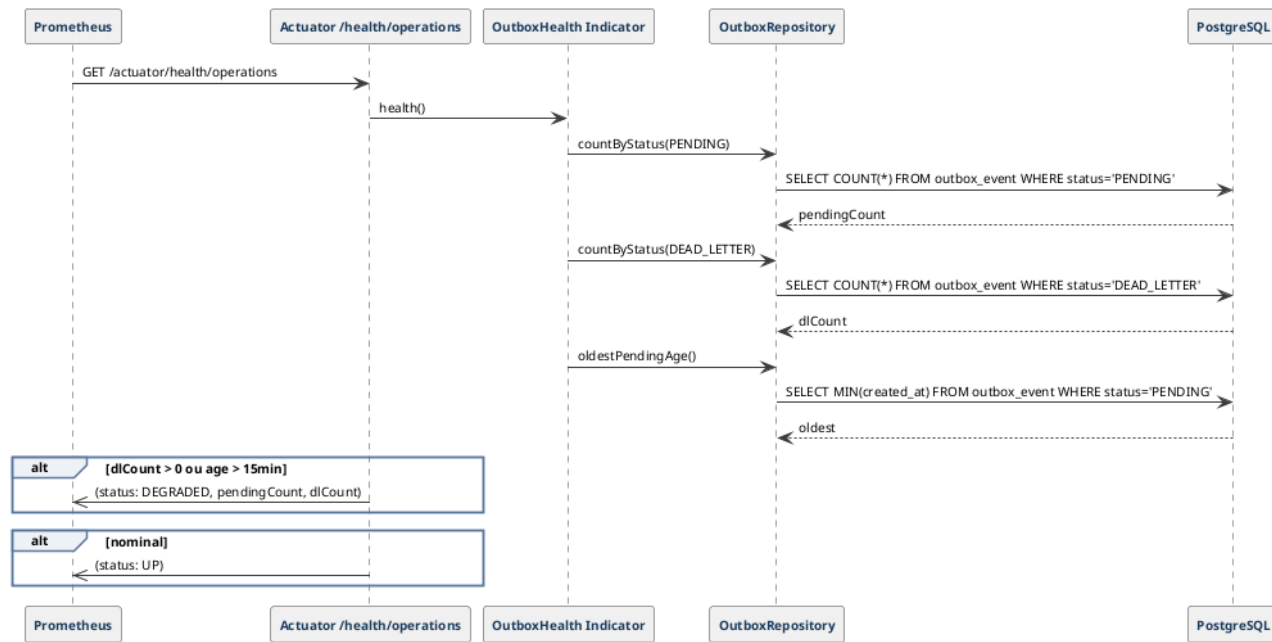
## DS-K-06 — Authentification et garde-fous d'une requête protégée



## DS-K-07 — Suspension d'un tenant



## DS-K-08 — Supervision de l'outbox via health check



## 2 common-core

---

### 2.1 Description du core

Le **common-core** est le socle transverse de types partagés de la plateforme. Il définit les contrats et abstractions réutilisés par les autres cores pour maintenir une cohérence structurelle. On y trouve les types de base du modèle canonique, les structures polymorphiques transverses (*Address*, *Contact*), les références légères inter-cores (*PartyRef*), l'énumération partagée **PlatformServiceCode**, ainsi que les enveloppes standard de réponse API.

Ce core porte aussi un sous-système transverse léger d'adresses et de contacts polymorphiques : cas d'usage applicatifs, adaptateurs de persistance R2DBC et adaptateurs WebFlux dédiés. Il ne pilote cependant aucun workflow métier propriétaire, ne publie pas d'événements métier propres, et reste cantonné aux capacités mutualisées réutilisables par les autres cores.

Dans l'état courant du backend, ces structures polymorphiques ne sont plus cantonnées à une simple bibliothèque de types. Elles sont effectivement consommées par **actor-core**, **organization-core**, **tp-core** et **resource-core** comme socle canonique des carnets d'adresses et de contacts attachés aux agrégats métiers.

Le diagramme de classes du présent chapitre expose volontairement les champs minimaux complets des structures polymorphiques **Address** et **Contact**, y compris les horodatages et les marqueurs de vérification, afin d'éviter toute ambiguïté entre héritage technique et minimum métier canonique.

### 2.2 Rôle principal

**Rôle principal** : Fournir un socle commun de types, d'abstractions de persistance et de contrats d'interface qui garantissent la cohérence structurelle entre tous les cores, sans créer de couplage fonctionnel ni de dépendances circulaires.

### 2.3 Rôles secondaires

#### 1. Entité de base partagée (**BaseEntity**)

Classe abstraite de domaine portant les champs transverses communs : identifiant UUID, identifiant de portée technique (**tenantId**), horodatages

de création et de mise à jour. Les détails R2DBC spécifiques restent hors du domaine.

## 2. Adresses polymorphiques (**Address**)

Modèle d'adresse réutilisable attachable à n'importe quelle cible **addressable**. Il couvre les champs canoniques de localisation : lignes d'adresse, ville, état, localité, pays, codes postaux, boîte postale, coordonnées GPS, et description libre.

## 3. Contacts polymorphiques (**Contact**)

Informations de contact attachables à toute cible **contactable** avec vérification d'e-mail, de téléphone, notion de favori, et canaux primaires/secondaires.

## 4. Cas d'usage transverses d'adresses et de contacts

Le core expose les opérations standardisées de création, consultation et suppression d'adresses et de contacts polymorphiques, sans imposer de logique métier spécifique au core consommateur.

## 5. Socle réellement absorbé par les autres cores

Les structures **Address** et **Contact** sont réutilisées par les contrôleurs imbriqués des acteurs, organisations, agences, tiers et ressources. Le common-core devient ainsi un socle d'attachement transverse effectif, et non plus seulement un paquet utilitaire partagé.

## 6. Référence légère inter-cores (**PartyRef**)

Pointeur minimal composé de **partyType** et **partyId**, permettant à un core de référencer un acteur ou une organisation sans imposer de jointure croisée ni de dépendance forte entre domaines.

## 7. Catalogue partagé des services plateforme

L'énumération **PlatformServiceCode** fournit un langage commun pour les modules souscrits, les entitlements du kernel et les quotas de service. Le catalogue courant contient **ORGANIZATION**, **SETTINGS**, **COMMERCIAL**, **PRODUCT**, **INVENTORY**, **SALES**, **BILLING**, **ACCOUNTING**, **BANKING**, **TREASURY**, **CASHIER**, **RESOURCE**, **HRM** et **BLOCKCHAIN**. Il porte aussi les aliases legacy contrôlés (**THIRD\_PARTY**, **COMPTA**, **CAISSE**, **TRESORERIE**, etc.) et les dépendances entre services.

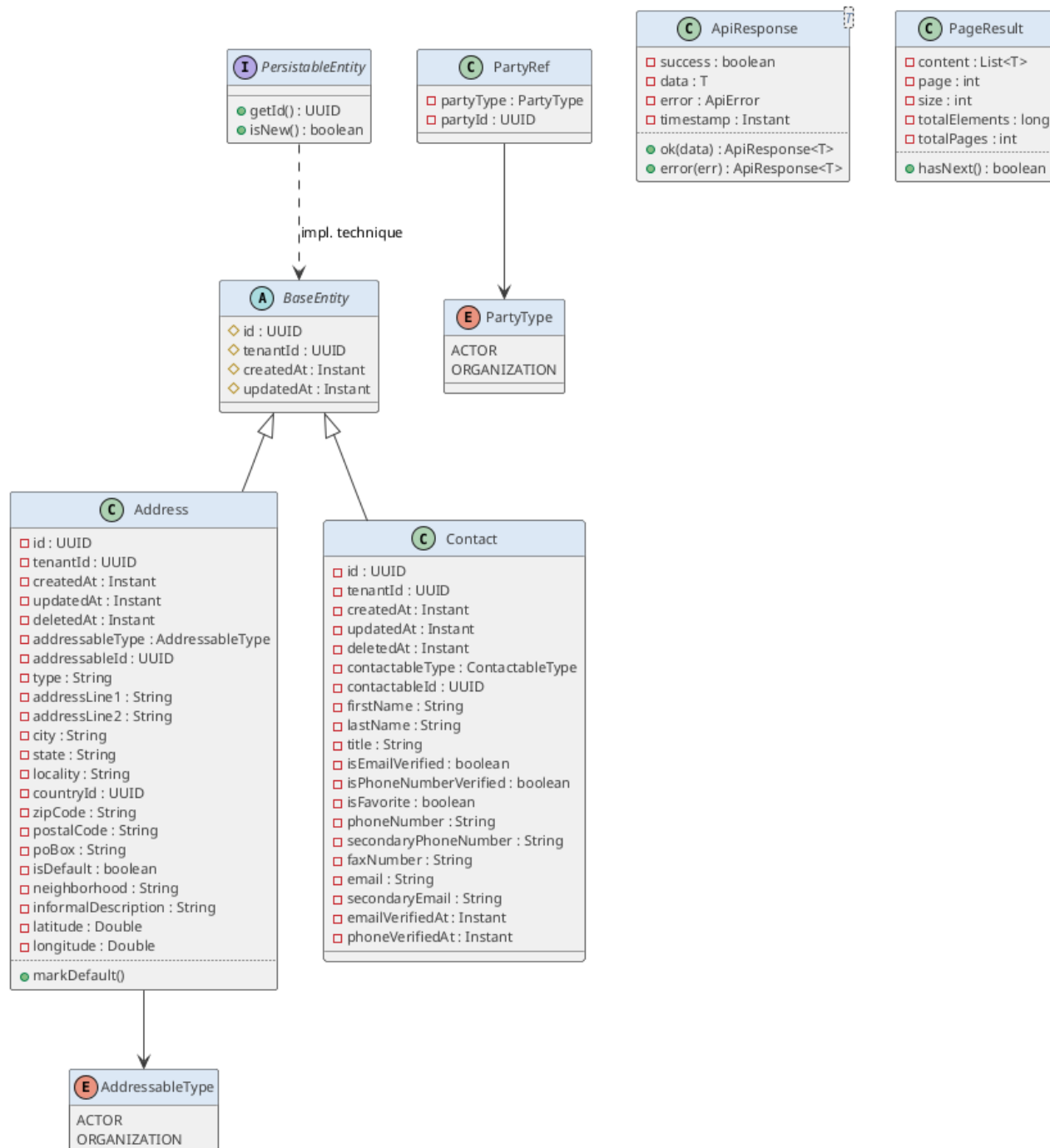
## 8. Enveloppes de réponse API

Types génériques **ApiResponse<T>** et **PageResult<T>** assurant un format de réponse JSON uniforme sur l'ensemble de l'API publique.

## 9. Contrat de persistance technique

L'interface **PersistableEntity**, logée côté adaptateur de persistance, permet aux entités techniques R2DBC de partager un contrat d'identification sans devenir une abstraction métier.

## Diagramme de classes — common-core

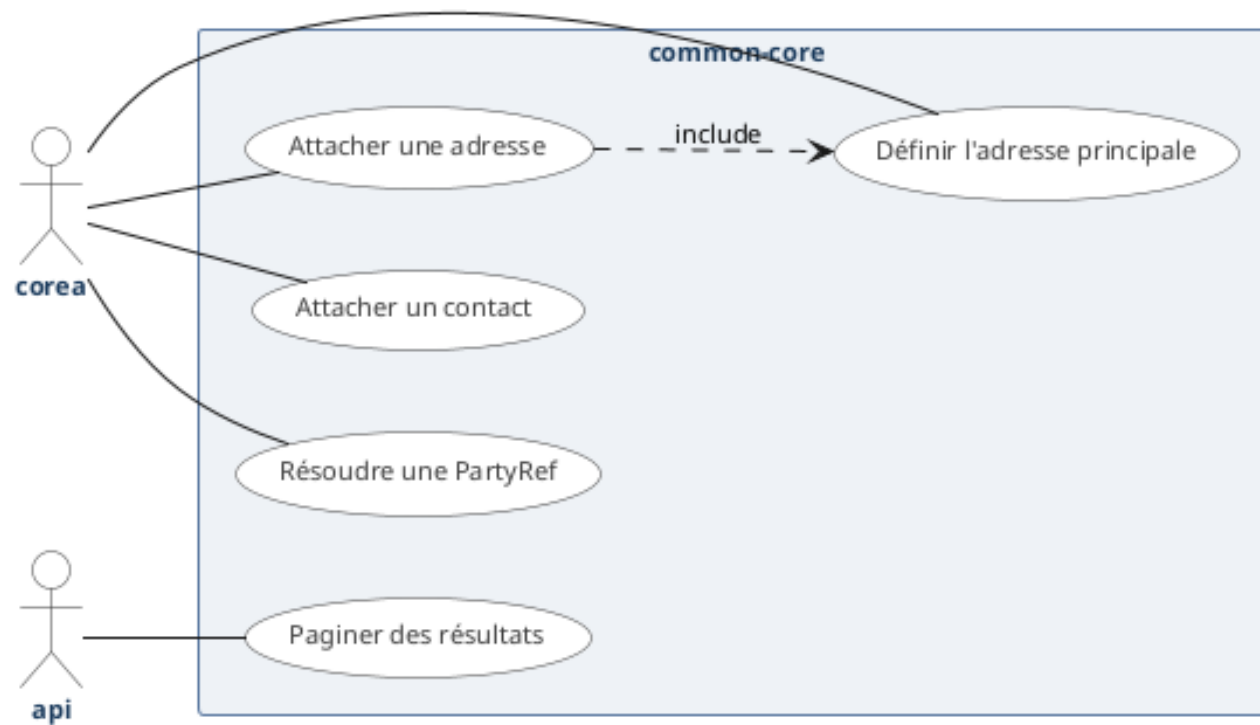


**Invariants du common-core :**

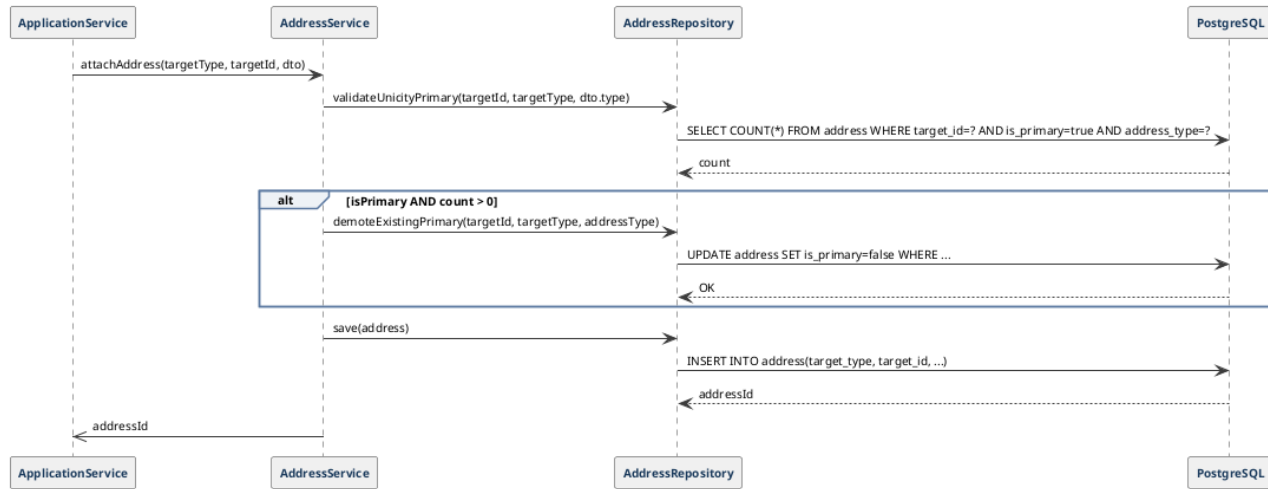
- Une seule adresse par défaut par combinaison (`addressableId`, `addressableType`, `addressType`).
- `PartyRef.partyType` est limité aux valeurs `ACTOR` et `ORGANIZATION`.
- `tenantId` reste obligatoire sur les structures polymorphiques d'exécution.
- Un code de service inconnu ne peut pas être utilisé dans une `ClientApplication` ou une souscription d'organisation.
- Les dépendances requises restent portées par le catalogue partagé : `INVENTORY` requiert `PRODUCT`, `SALES` requiert `COMMERCIAL + PRODUCT`, `TREASURY` requiert `ACCOUNTING + BANKING`, et `CASHIER` requiert `ACCOUNTING`.
- `PersistableEntity` est une abstraction technique et ne porte aucune règle métier.



## Diagramme des cas d'utilisation — common-core



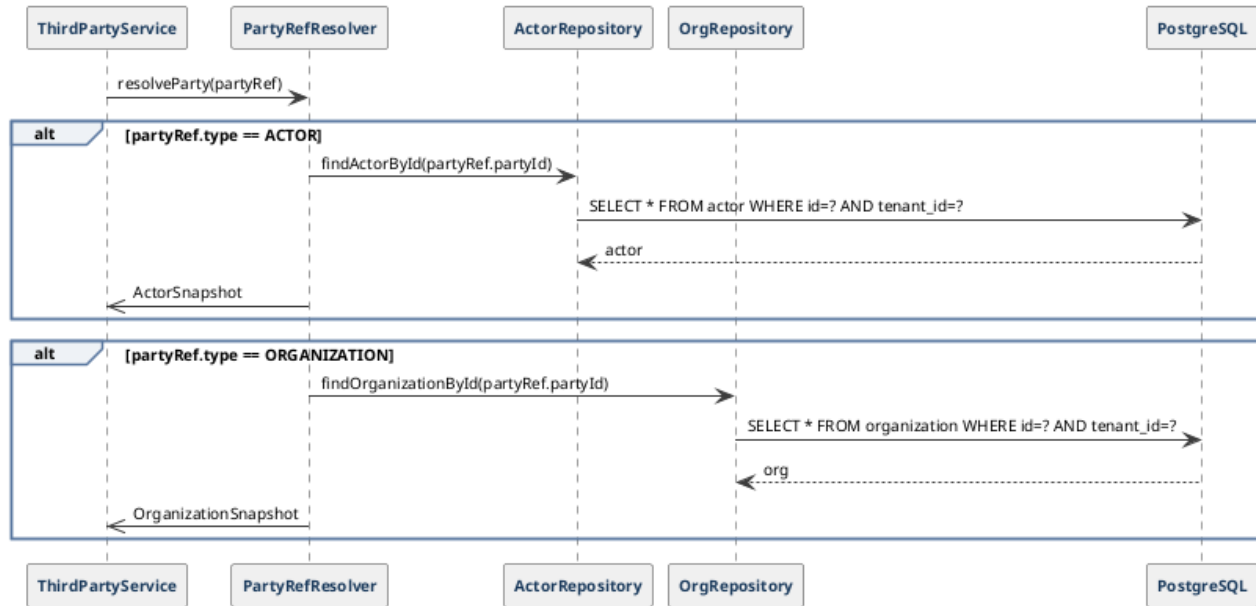
## DS-C-01 — Attachement d'une adresse polymorphique



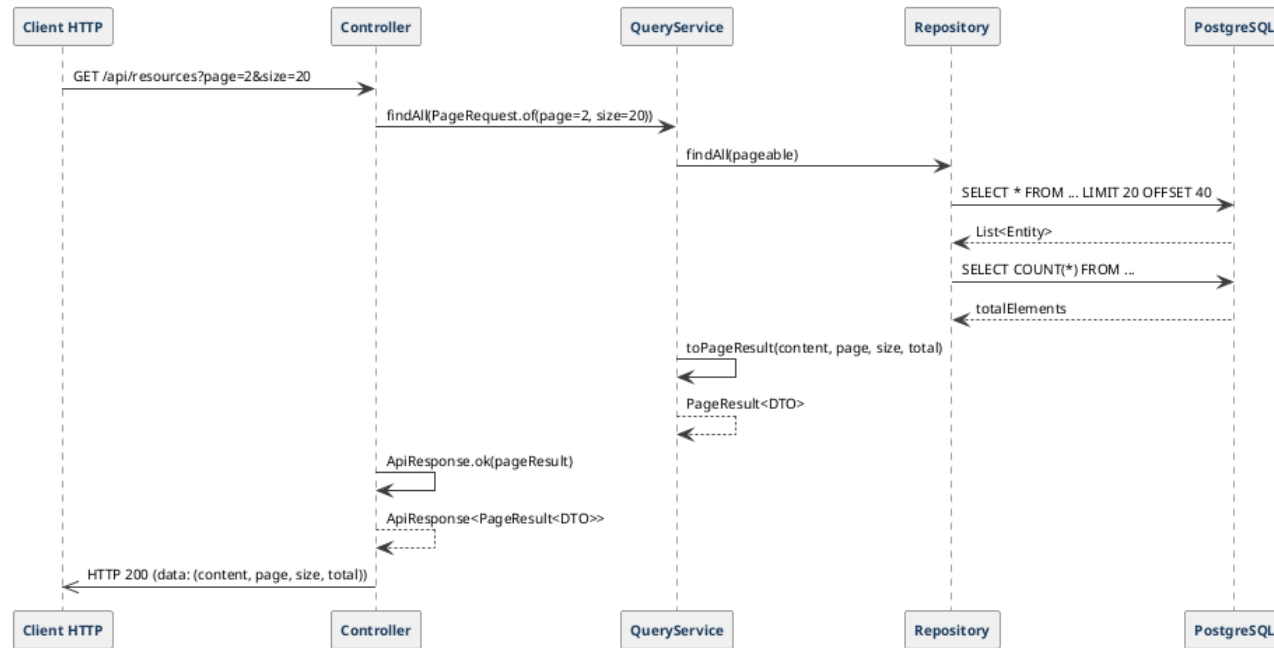
### 2.3.1 DS-C-02 : Résolution d'une PartyRef

Ce mécanisme est utilisé par le tp-core pour résoudre dynamiquement la cible canonique (*Actor* ou *Organization*) associée à un tiers.

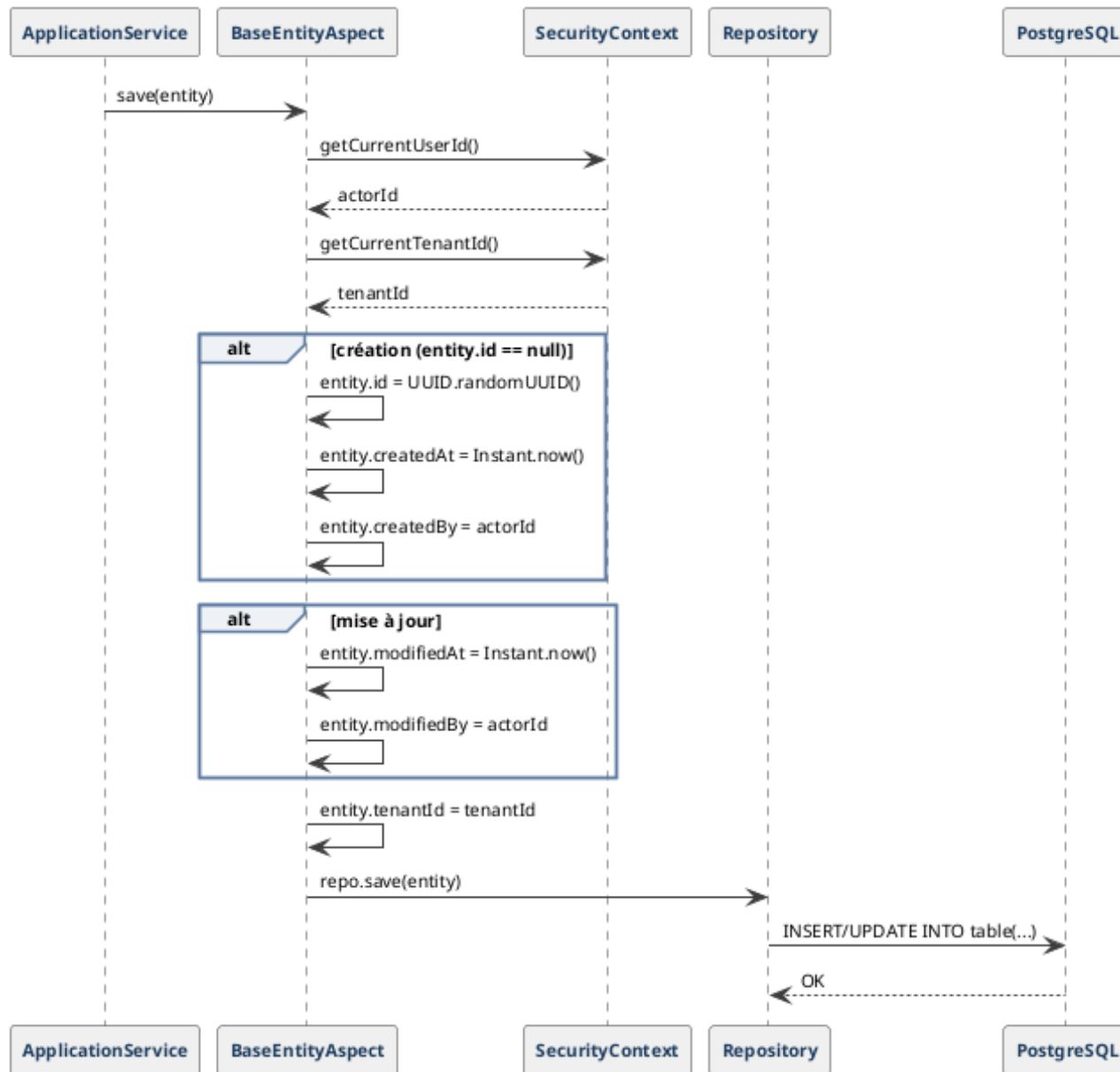
## DS-C-02 — Résolution d'une PartyRef inter-cores



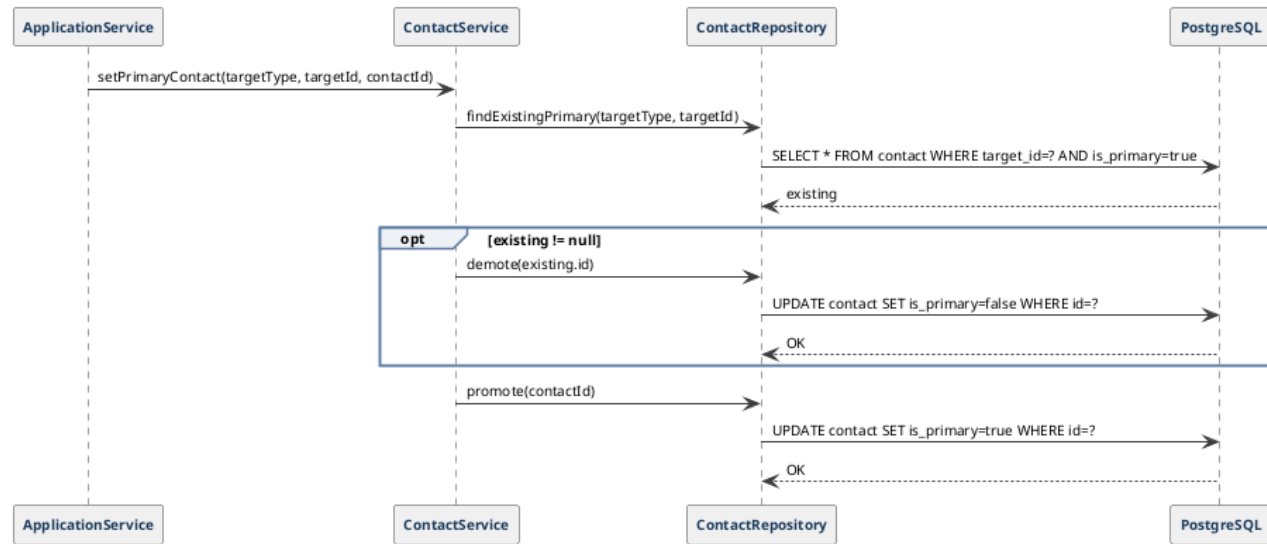
## DS-C-03 — Pagination standardisée des résultats



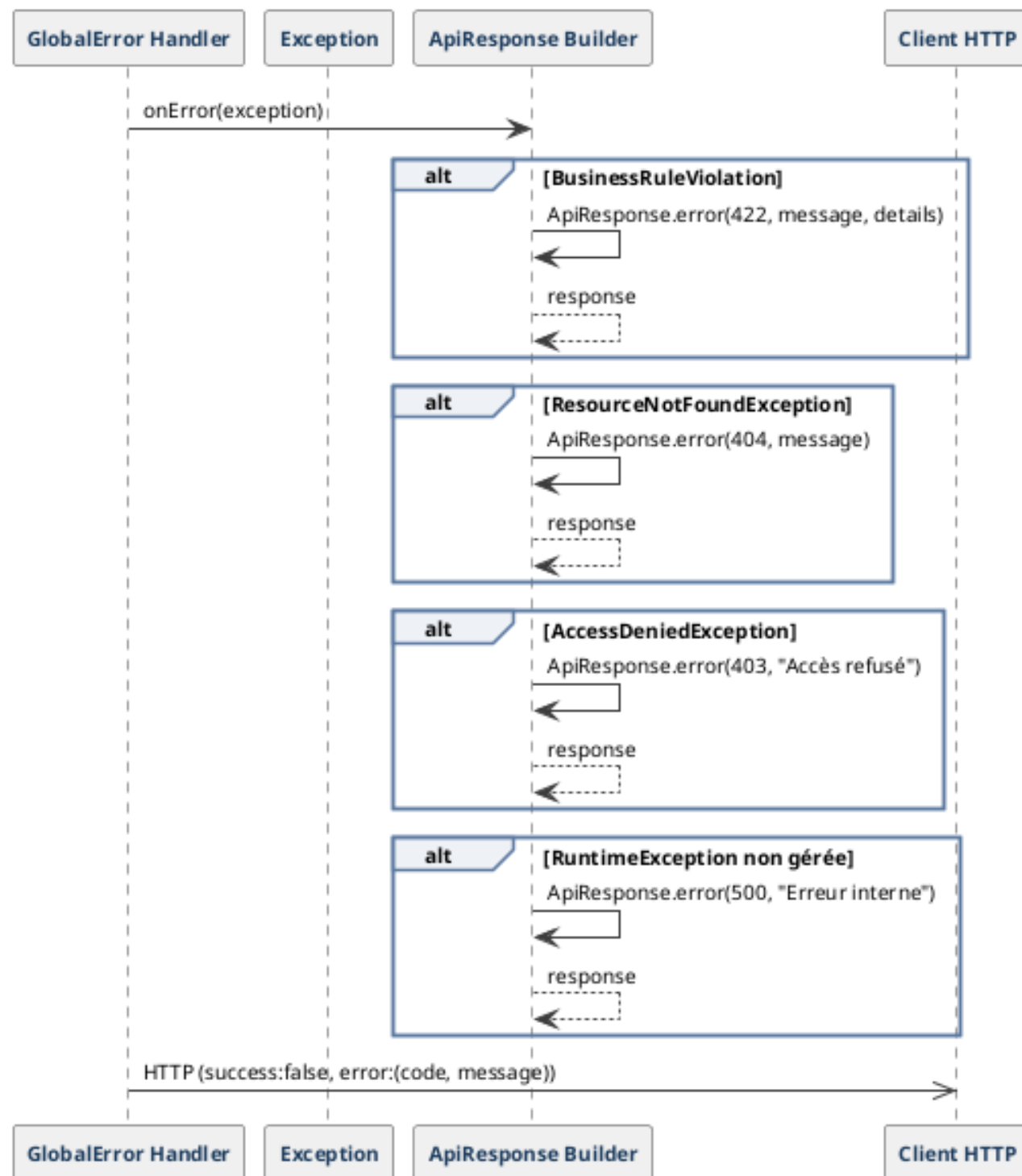
## DS-C-04 — Propagation automatique des champs BaseEntity



## DS-C-05 — Gestion du contact principal (promotion/rétrogradation)



## DS-C-06 — Construction d'une réponse d'erreur standardisée





## 3 actor-core

---

### 3.1 Description du core

L'**actor-core** gère les identités humaines de la plateforme, c'est-à-dire les personnes physiques manipulées avant toute projection métier spécialisée. Un *Actor* représente une identité nominative enrichie (coordonnées, description, genre, nationalité, profession, biographie, adresses et contacts). Le contrat canonique *BusinessActor* complète cet acteur pour les cas de gouvernance de propriétaire, fondateur ou responsable métier ; dans l'implémentation actuelle, il est porté par *BusinessActorProfile*.

Ce core est le point d'entrée du cycle de vie des personnes physiques dans la plateforme. Il précède et conditionne la création d'un compte utilisateur (auth-core) et toute affectation de rôle (roles-core). Il ne gère ni l'authentification ni les autorisations.

La vue d'analyse présente explicitement le minimum canonique des attributs **Actor** et **BusinessActor** : identité personnelle, liens d'adresses et de contacts, ainsi que les marqueurs métier `isIndividual`, `isAvailable`, `isVerified` et `isActive`.

### 3.2 Rôle principal

**Rôle principal** : Gérer le cycle de vie complet des identités humaines (création, mise à jour, suppression douce, réactivation) et porter le profil gouverné des *BusinessActors*, sans empiéter sur l'authentification ni sur les spécialisations commerciales aval.

### 3.3 Rôles secondaires

#### 1. Gestion des *BusinessActors*

Le contrat *BusinessActor*, implémenté par le *BusinessActorProfile*, porte les informations de gouvernance d'un acteur fondateur ou propriétaire : code, type, rôle, qualifications, moyens de paiement, disponibilité, vérification et statut de gouvernance. Un *BusinessActor* peut posséder plusieurs organisations.

#### 2. Profil personnel canonique

L'acteur concentre les attributs personnels de référence : prénom, nom, nom

complet, téléphone, e-mail, description, photo, nationalité, date de naissance, profession et biographie.

**3. Adresses et contacts associés**

Les identités humaines peuvent être enrichies par des collections d'adresses et de contacts référencés via le common-core.

**4. Suppression douce (*soft delete*)**

Les acteurs ne sont jamais supprimés physiquement. La suppression douce préserve l'historique des opérations associées tout en retirant l'identité des flux actifs.

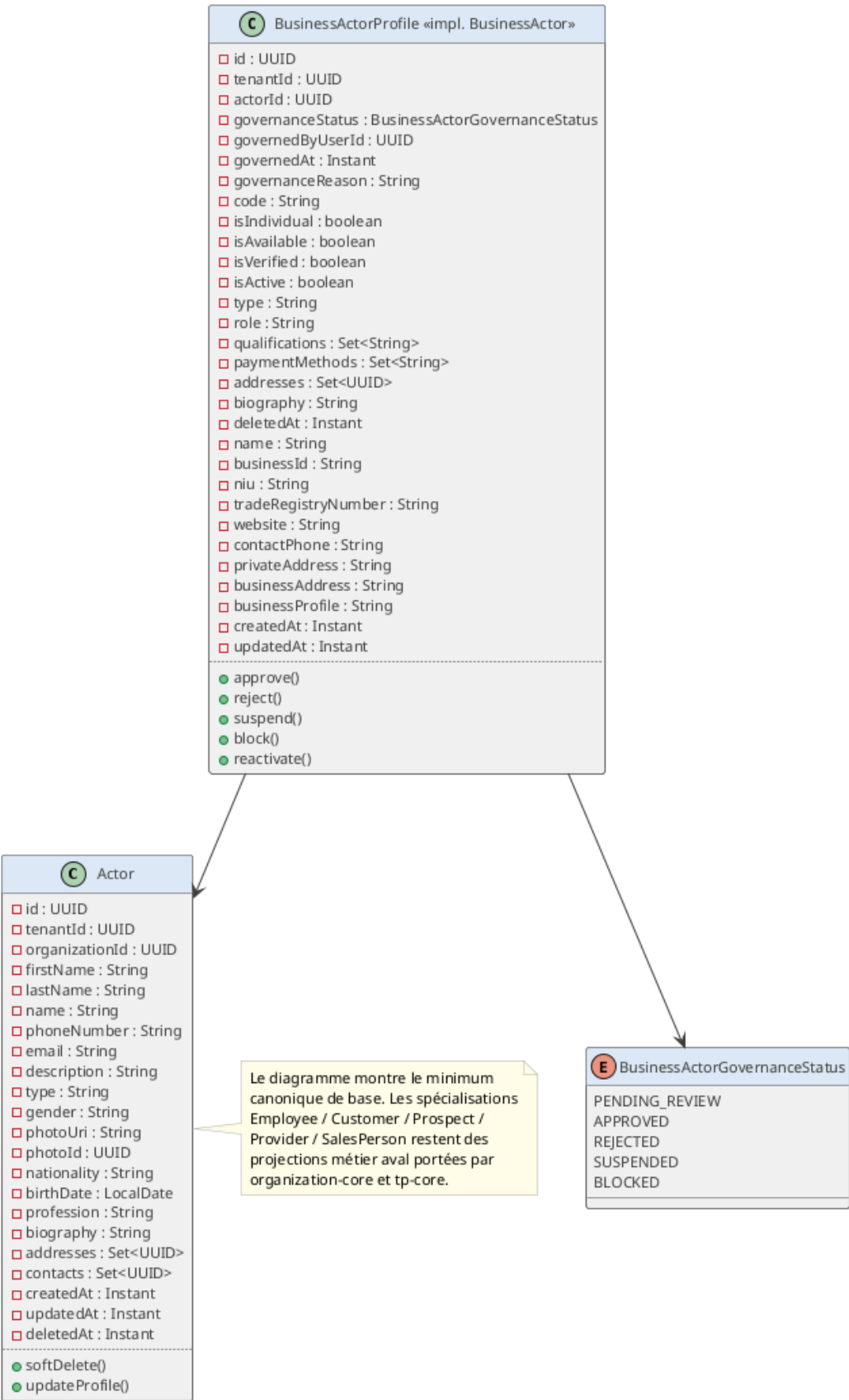
**5. Source de vérité pour les références humaines**

Tout autre core qui doit référencer une personne physique (par exemple le core tiers ou le core organisation) utilise l'identifiant d'acteur et non une duplication de l'objet complet.

**6. Spécialisations aval**

Les notions d'employé, client, prospect, fournisseur ou commercial restent des spécialisations conceptuelles du diagramme canonique, mais leur orchestration opérationnelle est portée par les cores métier spécialisés (**organization-core** et **tp-core**).

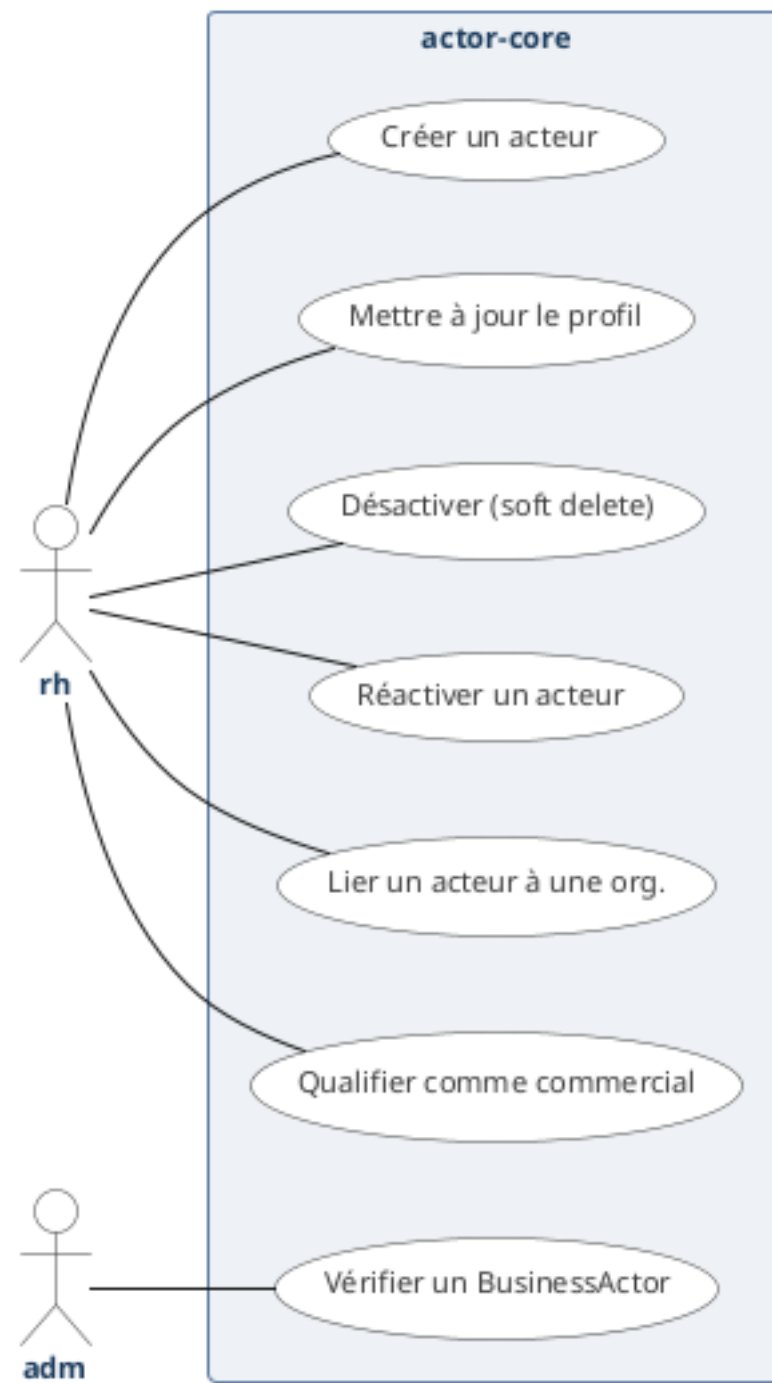
Diagramme de classes — actor-core



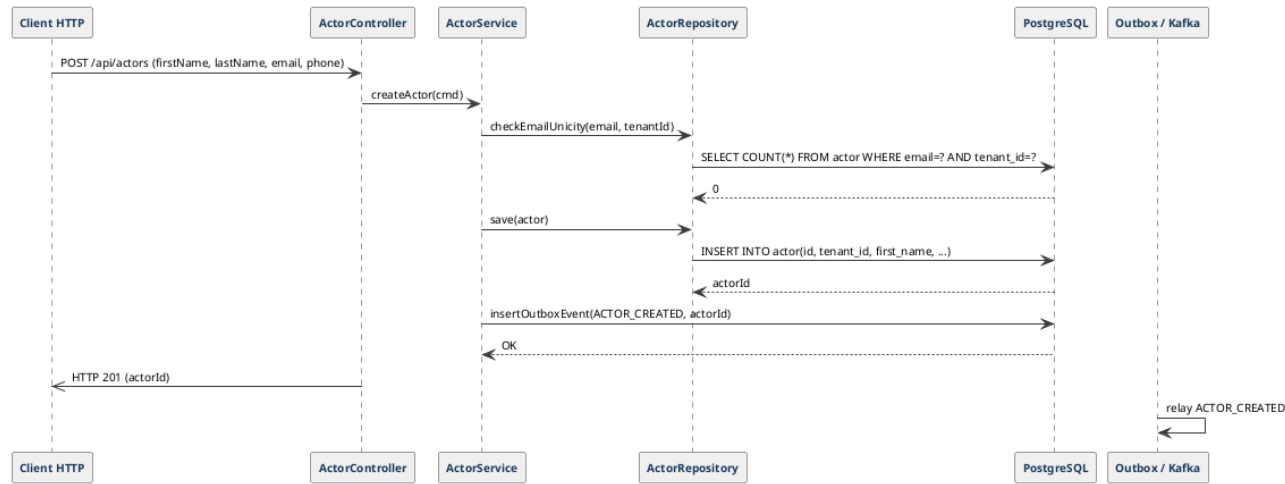
**Invariants du actor-core :**

- L'e-mail d'un Actor est unique par tenant (si renseigné).
- Un *BusinessActor* référence exactement un *Actor*.
- Un *BusinessActor* BLOCKED ne peut pas être réactivé par auto-service.
- Un Actor supprimé (soft delete) ne peut pas être rattaché à de nouveaux liens organisationnels actifs.

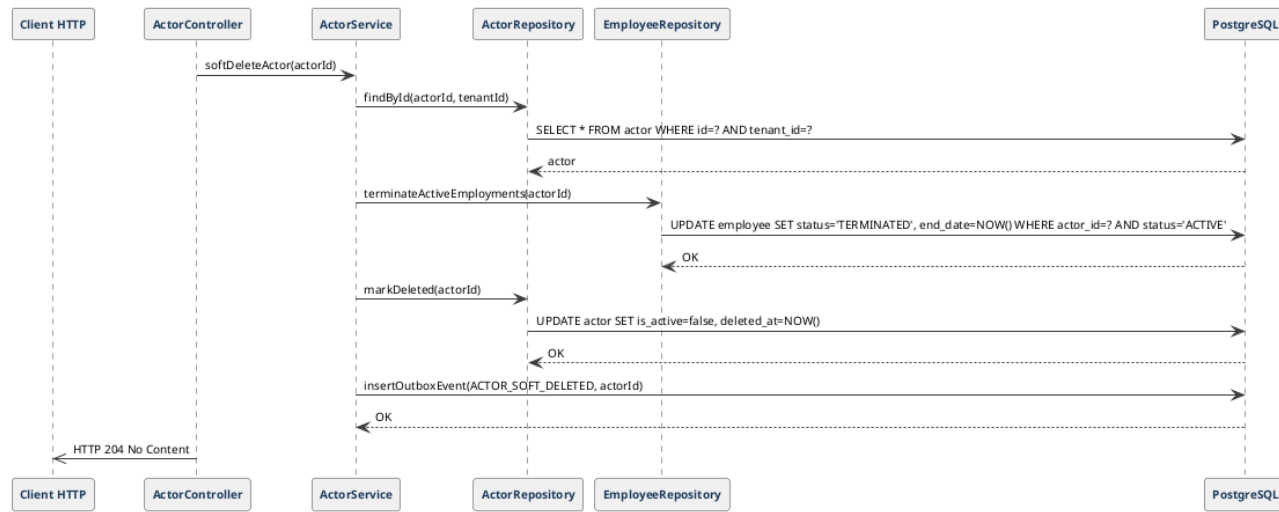
## Diagramme des cas d'utilisation — actor-core



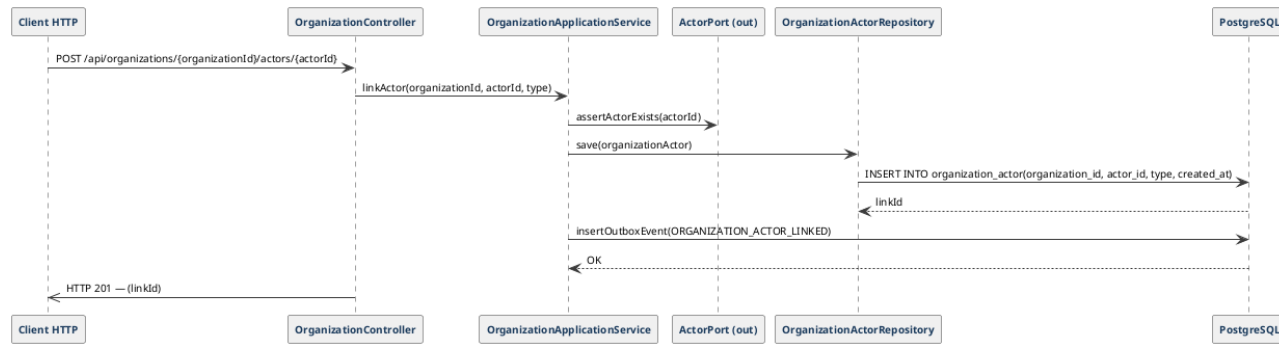
## DS-A-01 — Création d'un acteur



## DS-A-02 — Suppression douce d'un acteur

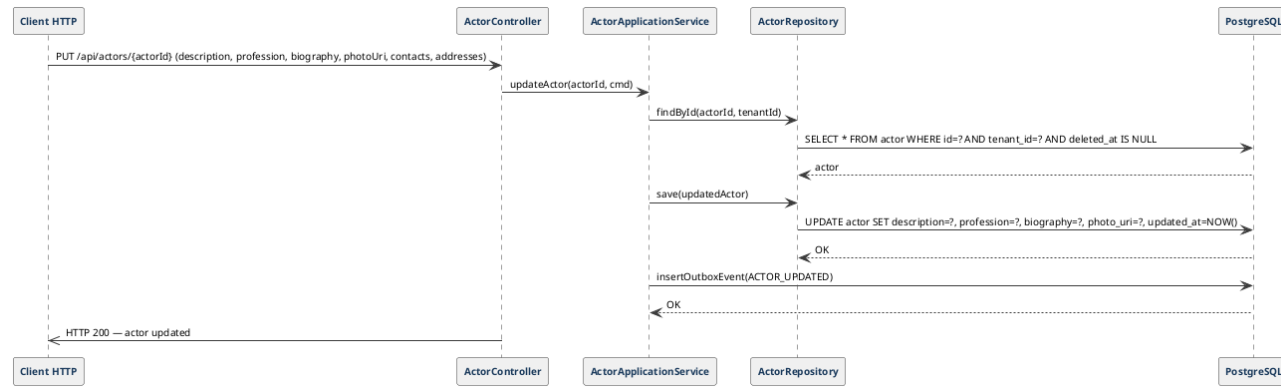


## DS-A-03 — Rattachement d'un acteur à une organisation

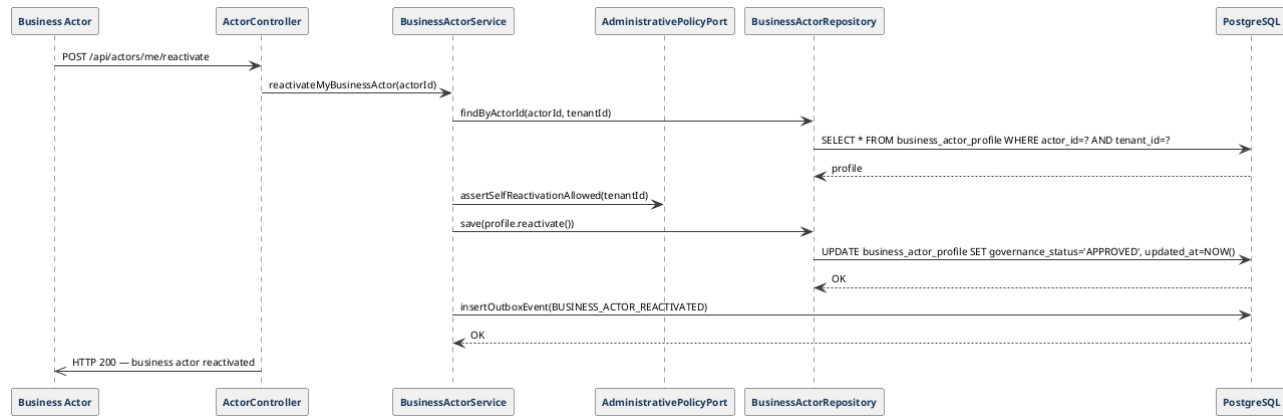




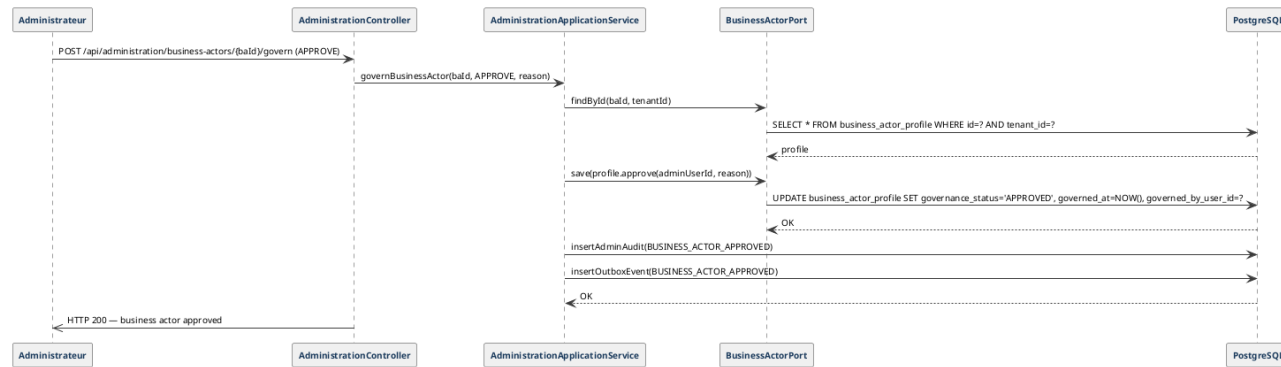
## DS-A-04 — Mise à jour du profil détaillé d'un acteur



## DS-A-05 — Réactivation d'un BusinessActor par auto-service



## DS-A-06 — Approbation d'un BusinessActor par l'administration



## 4 auth-core

---

### 4.1 Description du core

L'**auth-core** est le module responsable des comptes utilisateurs et de l'authentification locale. Il sépare clairement l'identité humaine (**Actor**, gérée par actor-core) du compte technique (**UserAccount**) utilisé pour se connecter et porter un contexte utilisateur dans les appels au kernel.

Le flux HTTP exposé est désormais en variantes complémentaires couvrant le cycle complet IAM attendu :

- le flux historique `POST /api/auth/login`, qui reste **tenant-scoped** et suppose un `X-Tenant-Id` déjà connu ;
- le flux **email-first** `POST /api/auth/identify`, qui détermine si le principal doit poursuivre vers un `SIGN_IN_PASSWORD` ou vers un `SIGN_UP` ;
- le nouveau flux moderne `POST /api/auth/discover-contexts` puis `POST /api/auth/select-context`, qui permet un **login global par principal/password** avant sélection du contexte tenant/organisation ;
- le flux de **self-service de compte** : `discover-sign-up-contexts`, `sign-up`, `forgot-password`, `password-reset/issue`, `reset-password`, `email-verification/request` et `email-verification/confirm`.
- les flux de **challenge et durcissement** : `captcha`, `captcha/verify`, `otp`, `otp/verify`, `phone-verification/request`, `phone-verification/confirm`, `mfa/enable`, `mfa/confirm`, `mfa/disable` et `login/mfa/confirm`.
- le provider **OIDC/OAuth2 central** : `/.well-known/openid-configuration`, `/.well-known/oauth-authorization-server`, `/oauth2/token`, `/oauth2/userinfo`, `/oauth2/introspect` et `/.well-known/jwks.json`.

Dans tous les cas, la signature RS256 des JWT et l'exposition du JWKS restent portées par le kernel-core ; auth-core orchestre l'émission, la sélection de contexte, les challenges de sécurité et l'exposition OIDC après authentification réussie et résolution des permissions. Le core projette aussi les **organisations accessibles** par l'utilisateur ainsi que les **services actifs** de chacune d'elles dans les réponses de login et de profil.

**Important** : le backend courant ne possède pas encore de catalogue métier des tenants avec nom d'affichage. Le flux de découverte de contexte retourne donc des

`tenantId` techniques, accompagnés des organisations accessibles dans chacun de ces contextes.

## 4.2 Rôle principal

**Rôle principal** : gérer le cycle de vie des comptes `UserAccount`, assurer l'authentification locale dans un contexte tenant donné ou via une découverte globale de contextes, puis retourner un JWT d'accès signé contenant l'identité utilisateur, le contexte sélectionné et les permissions résolues.

## 4.3 Rôles secondaires

### 1. Création contrôlée des comptes

L'endpoint `POST /api/auth/register` est réservé aux appels authentifiés portés par une `ClientApplication` et un utilisateur disposant des permissions IAM adéquates. Le module normalise les identifiants et hache le mot de passe avant persistance.

### 2. Authentification locale

Le backend supporte deux parcours :

- `/api/auth/login` : résolution d'un compte par `principal` dans un tenant donné, suivie de la vérification du mot de passe via `BCryptPasswordEncoder` ;
- `/api/auth/discover-contexts` : résolution globale de tous les comptes correspondant au `principal`, filtrage par mot de passe valide, puis émission d'un jeton court de sélection de contexte.

### 3. Sélection de contexte tenant/organisation

L'endpoint `/api/auth/select-context` consomme le jeton court renvoyé par `discover-contexts`, valide le `contextId` choisi et, si nécessaire, l'appartenance à l'organisation demandée. Il émet ensuite le JWT final portant le `tenantId` et éventuellement le `organizationId` sélectionnés.

### 4. Émission d'un JWT d'accès RS256

Après authentification, `auth-core` résout les permissions du user, puis délègue au `UserSessionTokenService` du `kernel-core` l'émission du bearer token RS256.

### 5. Auto-service utilisateur

Les endpoints `GET /api/users/me`, `PUT /api/users/me/plan` et `PUT /api/users/me/onboarding` permettent à l'utilisateur authentifié de consulter et ajuster son état applicatif. Le profil retourné contient aussi la liste `organizations[]` et, pour chaque organisation, les `services` effectivement

disponibles. `PUT /api/users/me/identity-onboarding` porte la mise à jour du profil d'identité demandé par l'onboarding prospect, freelance ou organisation.

## 6. Projection des entitlements organisationnels

Auth-core interroge l'organization-core pour agréger les organisations accessibles par l'utilisateur et les services effectifs de chacune. Cette projection permet au backend consommateur du kernel d'afficher ou masquer les modules métier selon l'abonnement réel de l'organisation.

## 7. Compatibilité ascendante

Le flux historique `/api/auth/login` est conservé pour les consommateurs qui connaissent déjà le `tenantId`. Le nouveau flux de découverte/sélection de contexte n'impose pas de migration brutale.

## 8. Journalisation applicative

Les opérations sensibles (`USER_REGISTERED`, `USER_SIGNED_UP`, `USER_LOGIN`, `USER_PASSWORD_RESET_REQUESTED`, `USER_PASSWORD_RESET_COMPLETED`, `USER_EMAIL_VERIFICATION_REQUESTED`, `USER_EMAIL_VERIFIED`, `USER_PLAN_UPDATED`, `USER_ONBOARDING_UPDATED`) sont inscrites dans l'audit système via le kernel-core.

## 9. Inscription contextuelle sans tenant explicite

Le backend permet désormais une inscription sans `tenantId` direct lorsque le frontend découvre d'abord les contextes d'inscription via `POST /api/auth/discover-sign-up-contexts` avec un `organizationCode`. Le résultat inclut un `selectionToken` signé et un ou plusieurs `contextId` réutilisables par `POST /api/auth/sign-up`.

## 10. Réinitialisation de mot de passe

Le parcours `forgot-password` résout tous les comptes correspondant à un principal, émet un jeton court de sélection, puis `password-reset/issue` produit un jeton signé de reset. Si un provider SMTP est configuré, le backend envoie réellement l'email ; sinon, il retombe explicitement en mode `PREVIEW_ONLY`.

## 11. Vérification d'adresse email

Le parcours `email-verification/request` émet un jeton signé associé à l'utilisateur connecté. `email-verification/confirm` valide ce jeton et renseigne `emailVerifiedAt` sur le `UserAccount`.

## 12. OTP, téléphone et MFA

Le module émet des challenges OTP utilisables pour email, SMS ou WhatsApp selon le canal demandé. Le téléphone peut être confirmé via `phone-verification/request` et `phone-verification/confirm`. L'activation MFA suit un flux en deux temps : émission d'un challenge, confirmation du code, puis exigence de `login/mfa/confirm` lors des connexions suivantes.

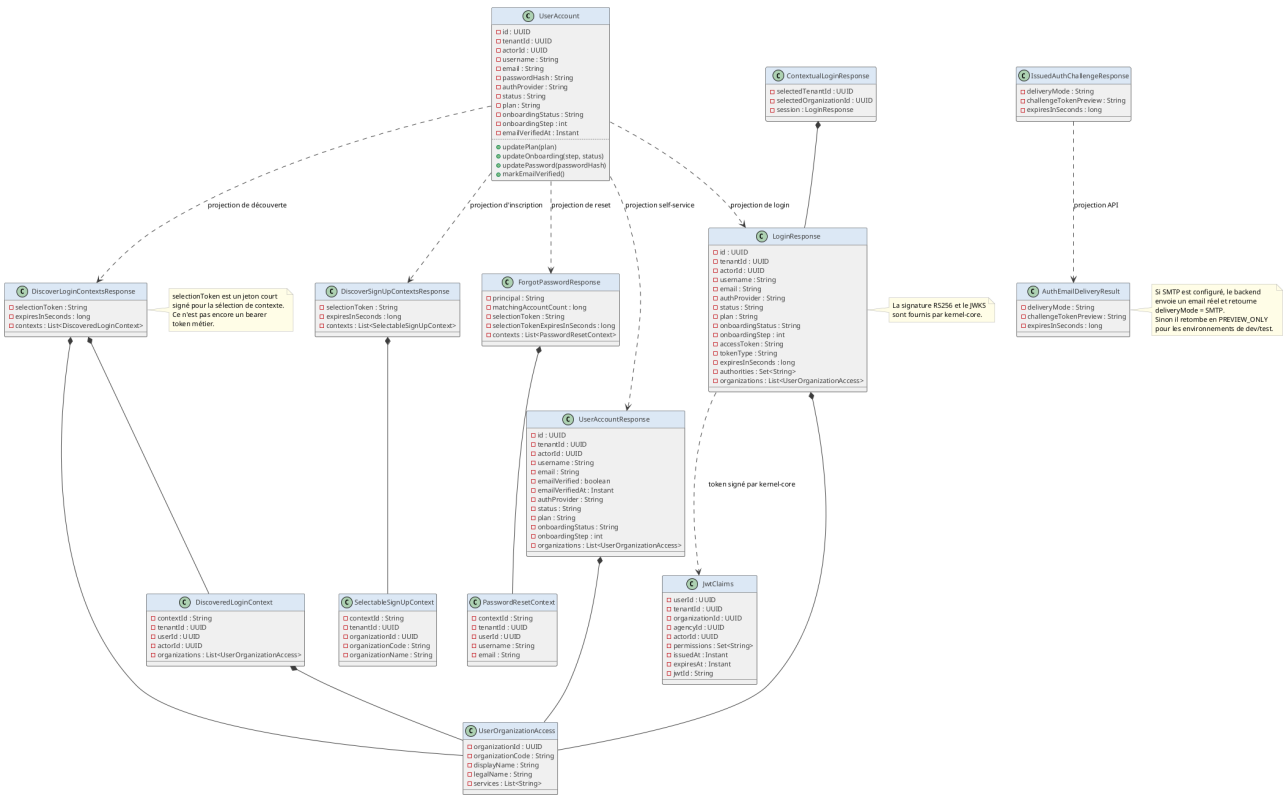
**13. Captcha et anti-abus**

`captcha` génère un challenge court et `captcha/verify` produit un jeton de vérification réutilisable par `sign-up`. L'inscription publique ne dépend donc pas d'une simple confiance frontend.

**14. Provider OIDC central**

Auth-core expose les métadonnées OIDC/OAuth2, accepte l'échange de jeton sur `/oauth2/token`, fournit `/oauth2/userinfo` et `/oauth2/introspect`, et s'appuie sur le JWKS RS256 du kernel. Le provider est centralisé pour les applications clientes du tenant, tout en conservant la sélection explicite du contexte métier.

Diagramme de classes — auth-core





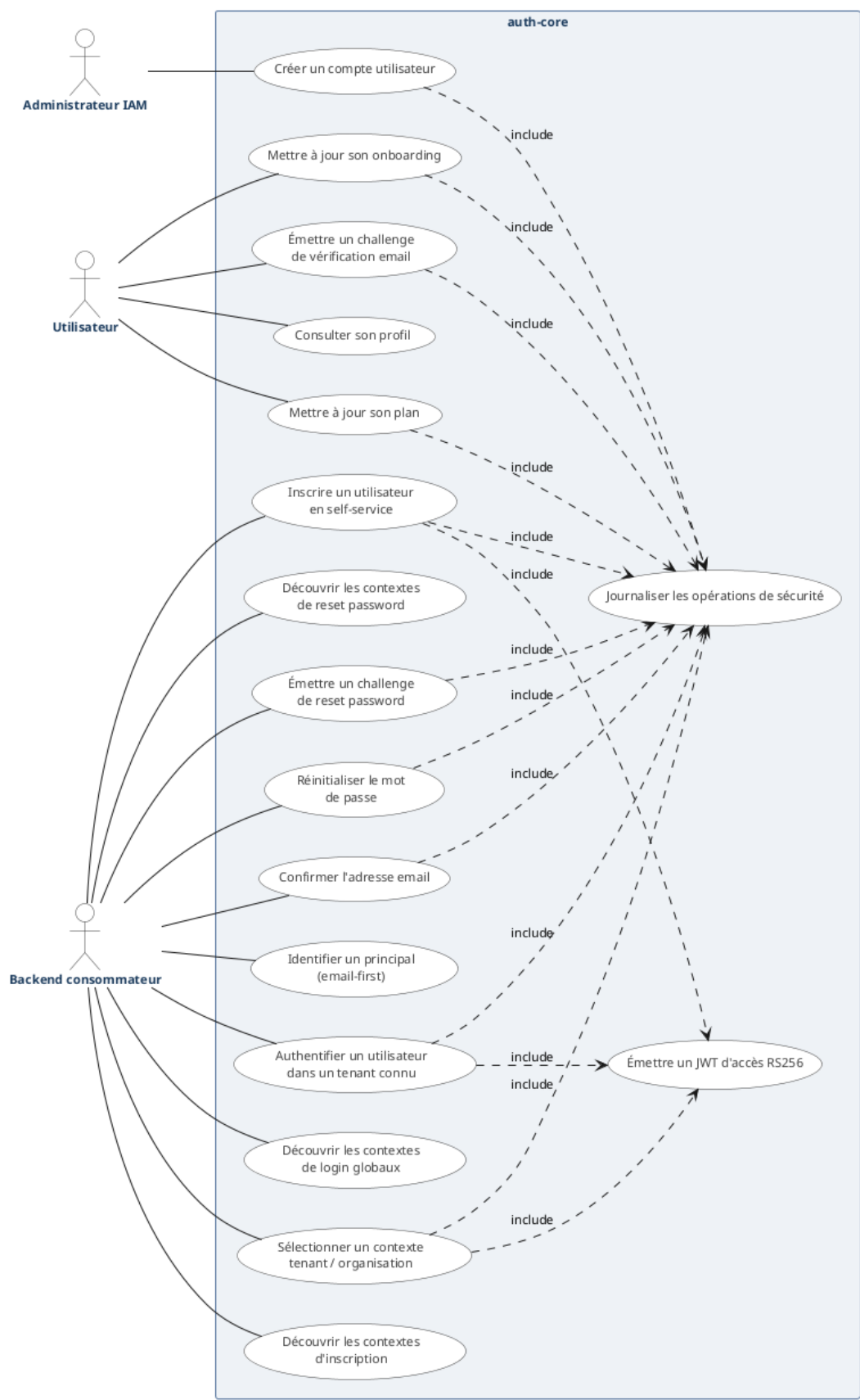
**Invariants du auth-core :**

- `username` est unique par tenant et normalisé en minuscule.
- `email` est unique par tenant et normalisé en minuscule.
- `passwordHash` est obligatoire pour le flux local courant.
- `authProvider` est obligatoire et décrit l'origine du compte.
- `plan` et `onboardingStatus` ne peuvent pas être vides.
- `onboardingStep`  $\geq 0$ .
- `emailVerifiedAt` nul signifie que l'adresse email n'a pas encore été confirmée.
- `phoneVerifiedAt` nul signifie que le téléphone n'a pas encore été confirmé.
- Un login MFA-enabled ne doit pas retourner de session finale avant validation du challenge MFA.
- Les OTP, captchas, jetons de reset, jetons de sélection et jetons MFA sont courts, signés et bornés dans le temps.
- Le jeton court de sélection de contexte est signé, de courte durée et ne constitue pas un bearer token métier.
- Le jeton court de sélection d'inscription et le jeton court de sélection de reset sont signés et temporaires.
- Le jeton de reset et le jeton de vérification email sont signés par le kernel-core via la même infrastructure JWT RS256.
- Le provider OIDC central ne remplace pas le RBAC métier ; il publie des tokens et informations utilisateur alignés avec le contexte sélectionné.
- Les réponses `login`, `select-context` et `/users/me` doivent refléter les organisations réellement accessibles et leurs services effectifs.

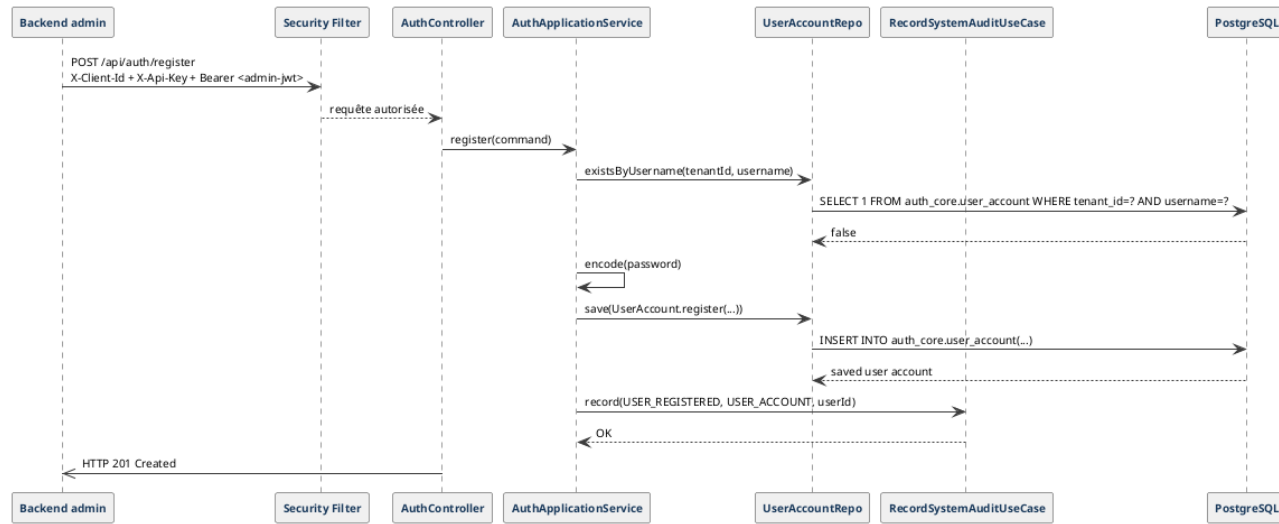
## 4.4 Flow Postman de référence

Le flow exploitable complet est versionné dans `iwm-backend/docs/postman/auth-core-full-iam`. Il couvre l'identification, le login tenant-scoped, la découverte et la sélection de contexte, l'inscription publique avec captcha, les OTP, la vérification email/téléphone, le MFA, le reset de mot de passe et les endpoints OIDC/OAuth2 centraux. Ce fichier fait partie de la référence fonctionnelle : une évolution des endpoints auth doit mettre à jour ce flow dans le même lot.

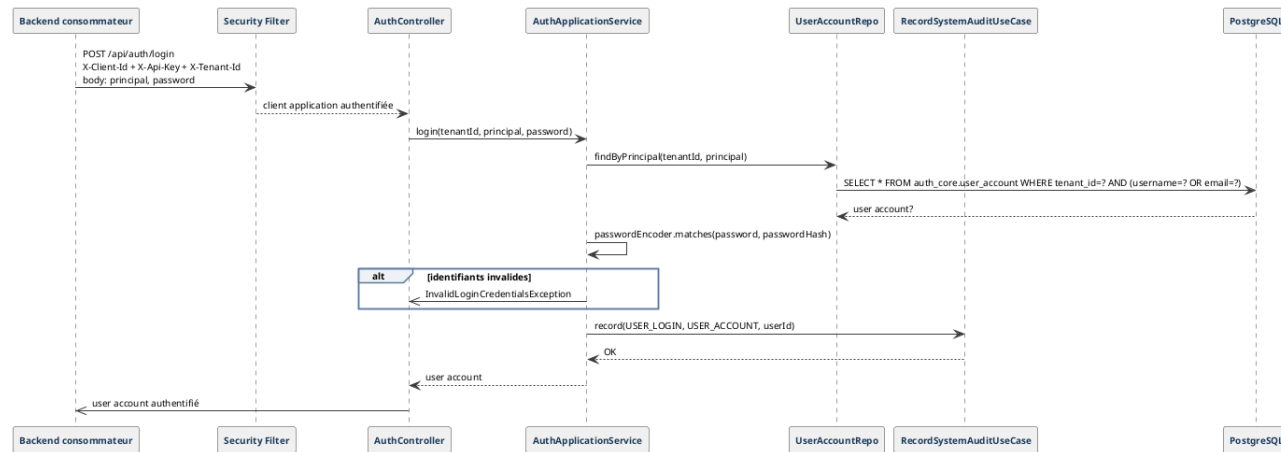
Diagramme des cas d'utilisation — auth-core



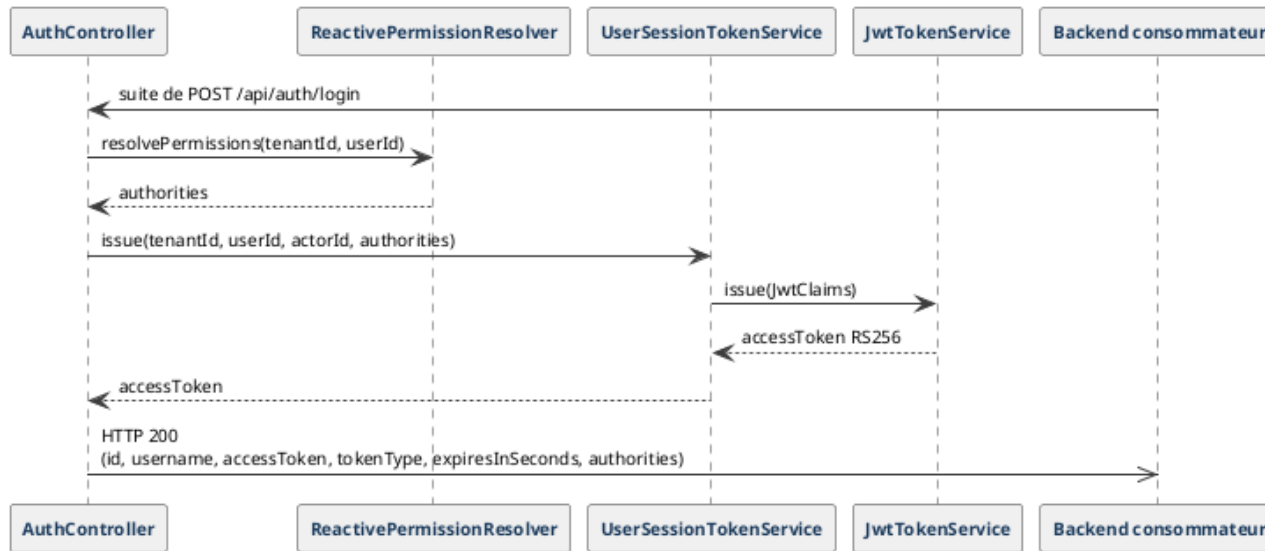
## DS-AU-01 — Création d'un compte utilisateur par un administrateur IAM



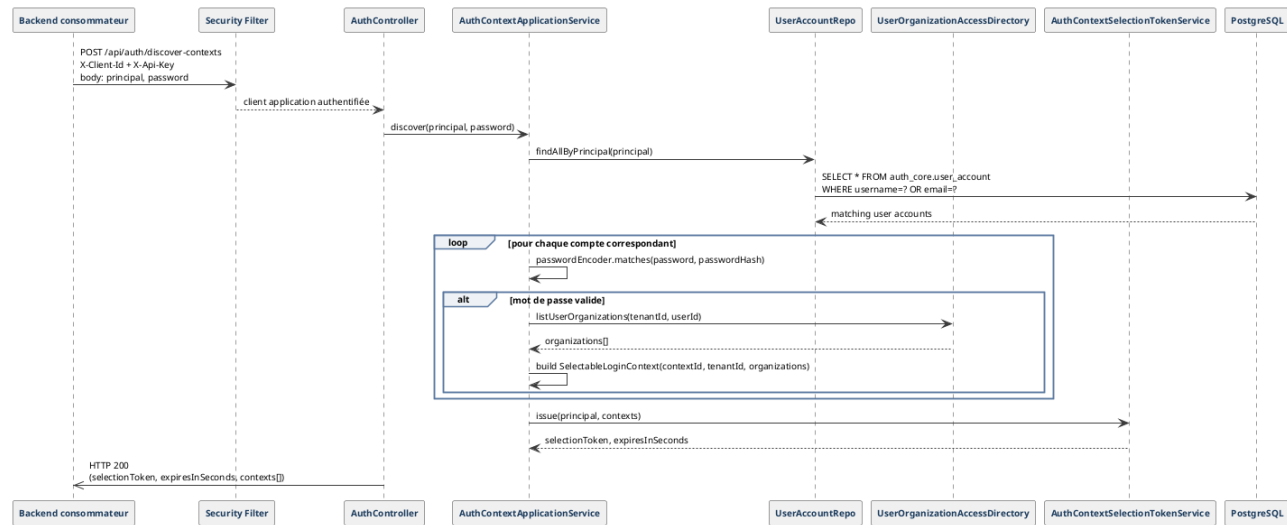
## DS-AU-02 — Authentification locale tenant-scoped d'un utilisateur



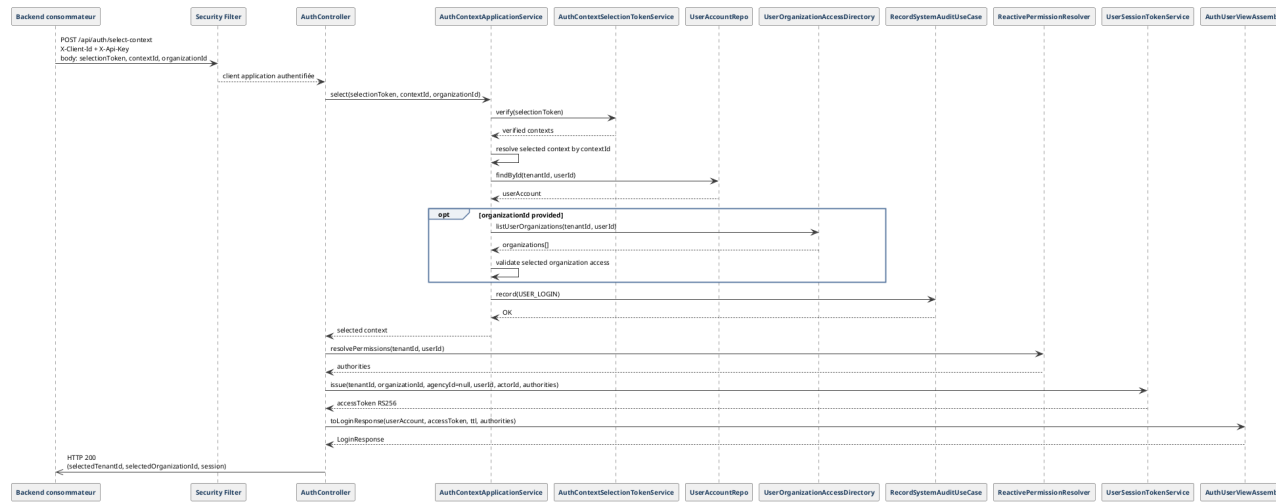
## DS-AU-03 — Résolution des permissions et émission du JWT RS256 final



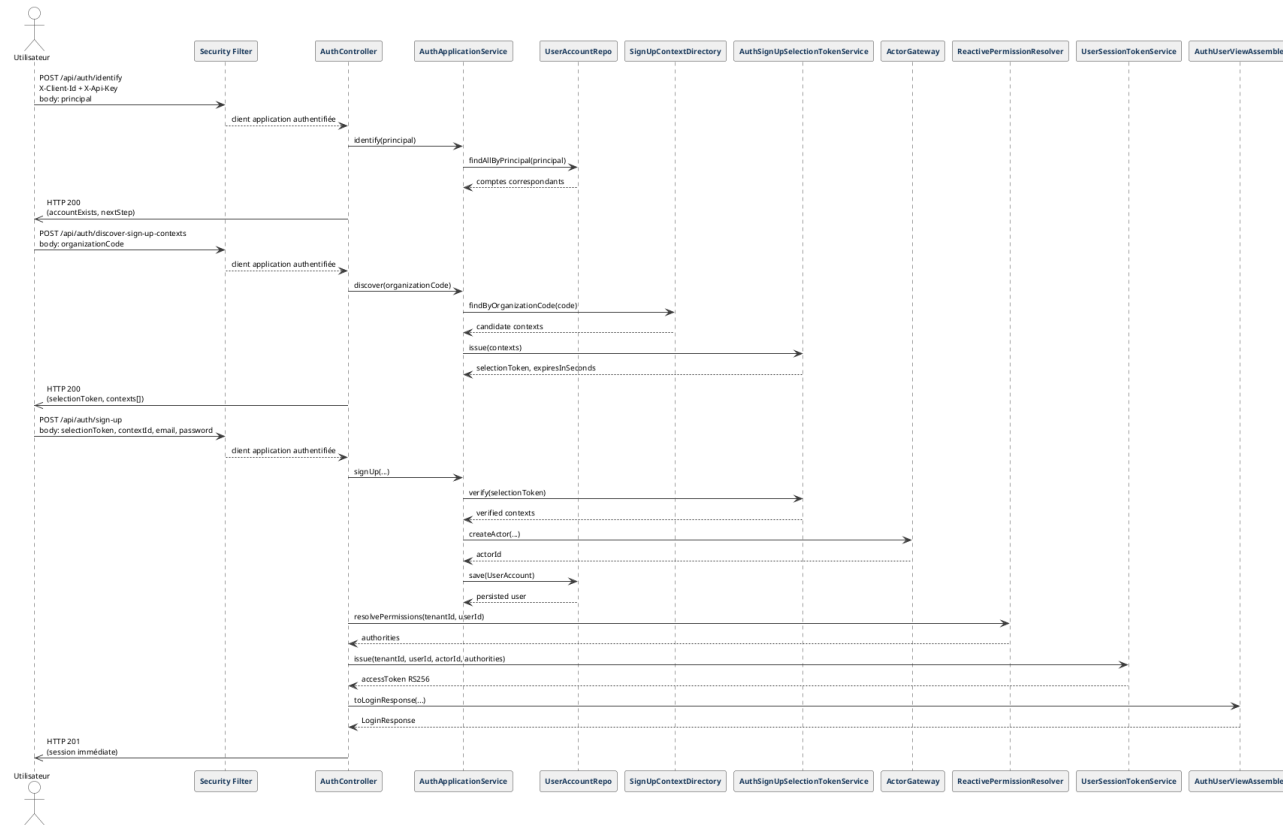
## DS-AU-07 — Découverte globale des contextes de login



## DS-AU-08 — Sélection explicite d'un contexte tenant / organisation

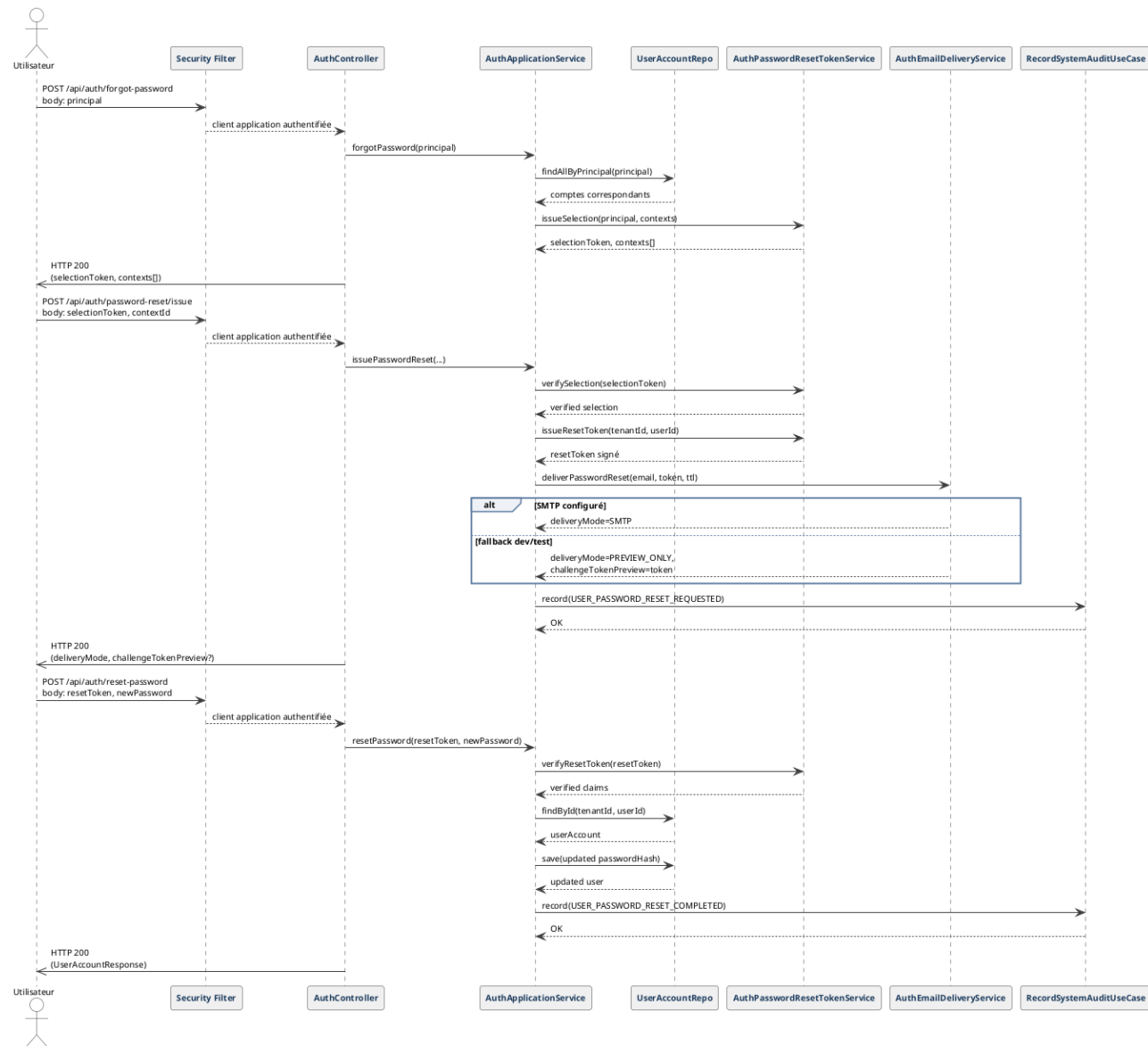


## DS-AU-09 — Identification email-first et inscription contextuelle

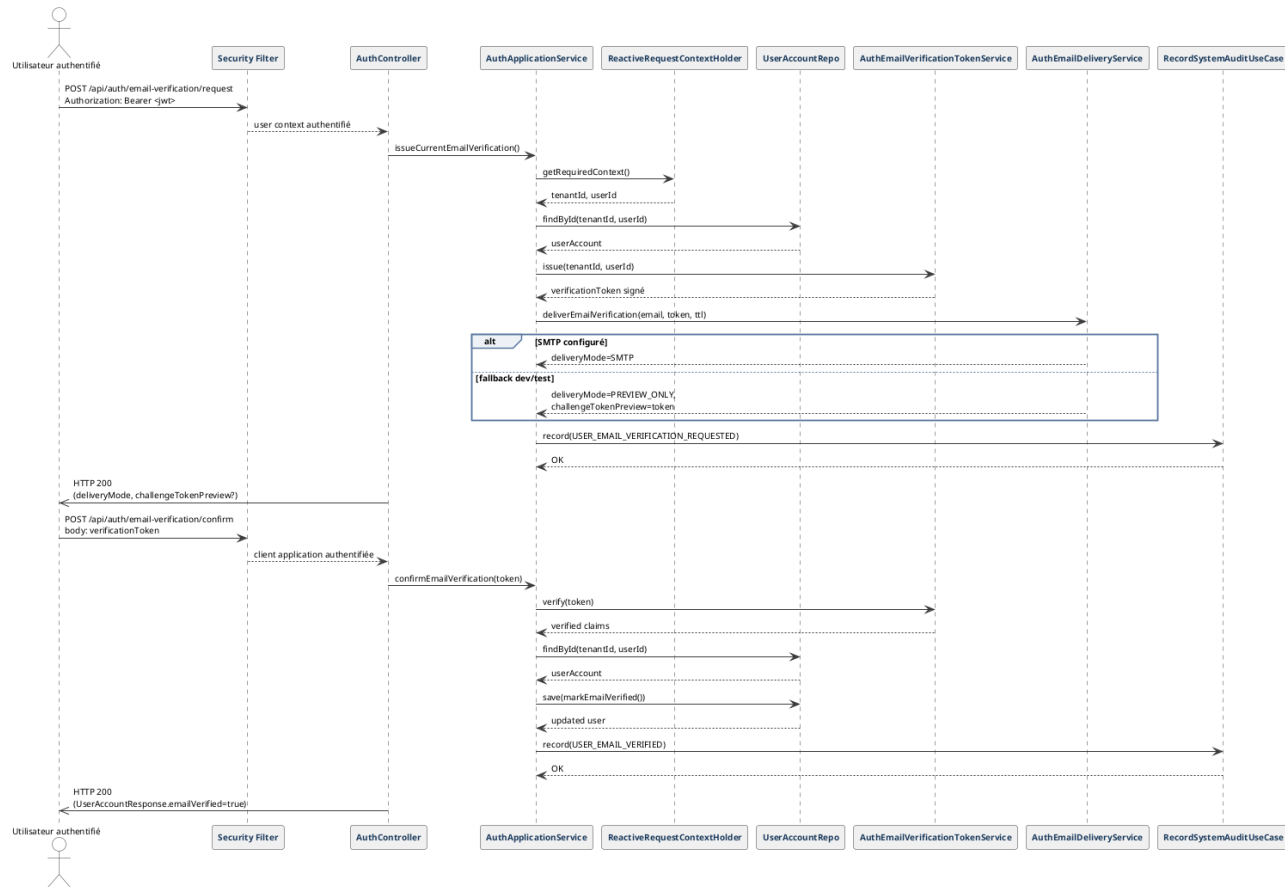




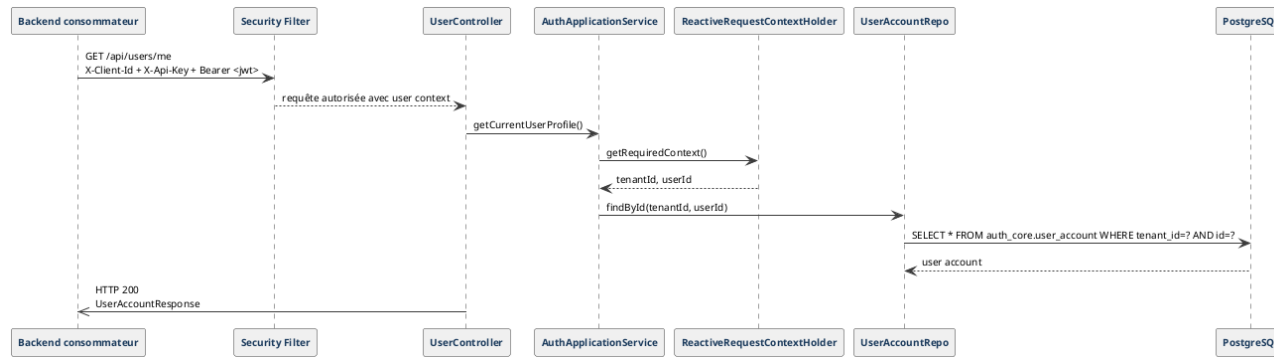
## DS-AU-10 — Réinitialisation de mot de passe avec livraison SMTP ou PREVIEW\_ONLY



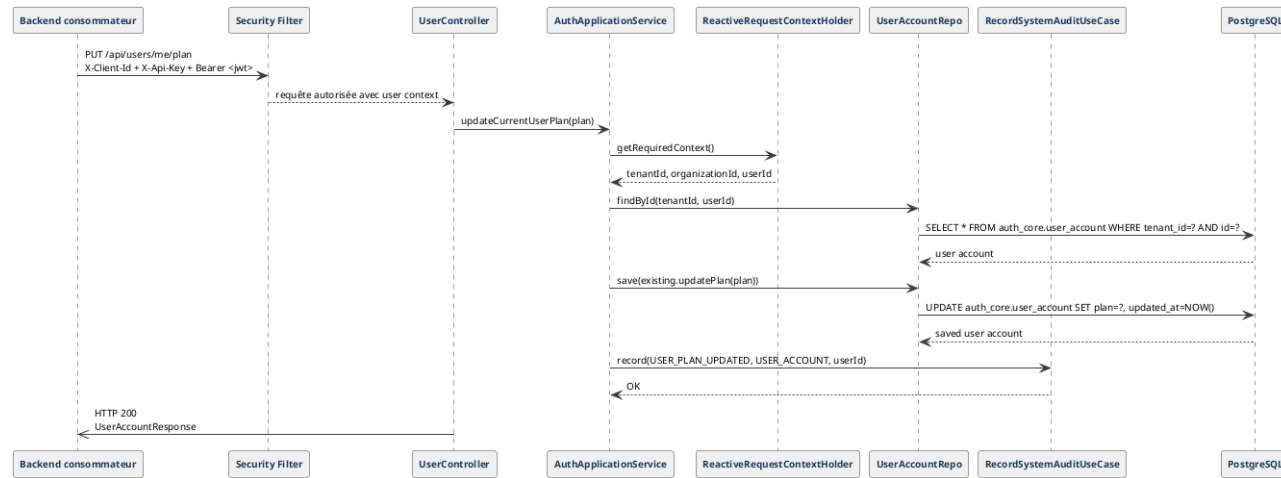
## DS-AU-11 — Vérification d'email après authentification



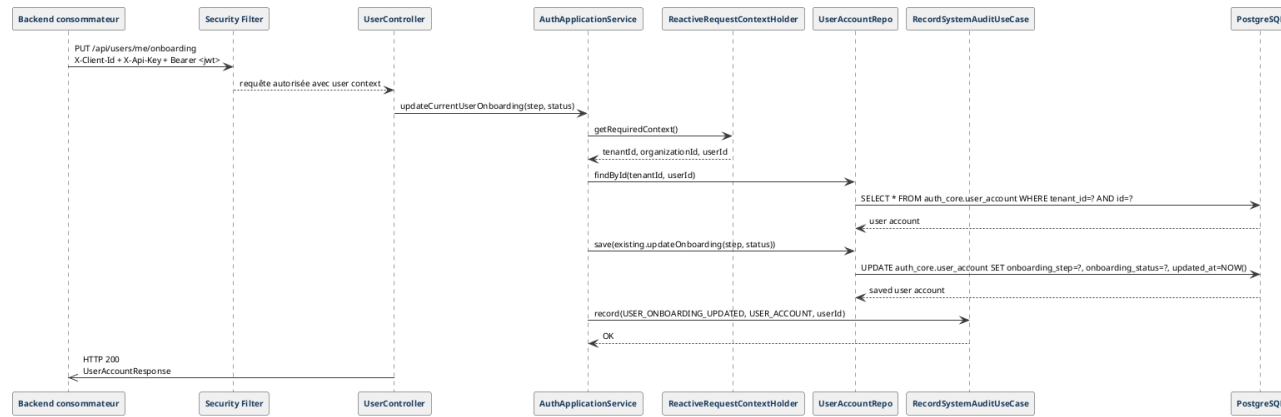
## DS-AU-04 — Consultation du profil utilisateur connecté (/users/me)



## DS-AU-05 — Mise à jour du plan utilisateur



## DS-AU-06 — Mise à jour de l'onboarding utilisateur



## 5 rôles-core (Tsakem Irving, Tchoyi Wilson)

---

### 5.1 Description du core

Le **roles-core** est le moteur RBAC réactif de la plateforme. Il ne gère pas l'authentification elle-même ; il porte plutôt la définition des rôles, les affectations utilisateur-rôle et la résolution des permissions effectives à l'exécution. Le modèle réel est plus léger que l'ancienne documentation : il ne maintient pas une entité **Permission** autonome ni une table de jointure **RolePermission**. Chaque **Role** persiste directement un ensemble ordonné de codes de permissions.

Le core supporte quatre niveaux de portée : **SYSTEM**, **TENANT**, **ORGANIZATION** et **AGENCY**. Une affectation produit ensuite des autorités scopeées comme **permission#TENANT** ou **permission#ORGANIZATION:<orgId>** utilisées par les politiques d'accès du kernel. La résolution peut s'appuyer sur un cache Redis optionnel ; sinon, elle se fait directement depuis les repositories **roles\_core**.

Le rôle-core expose aujourd'hui une API publique volontairement étroite : créer un rôle et affecter un rôle à un utilisateur. Les flux de gouvernance plus riches (catalogue de permissions, templates réservés, clonage, révocation, audit administratif) sont portés par **administration-core**, qui réutilise les mêmes repositories.

### 5.2 Rôle principal

**Rôle principal** : Définir des rôles composés de codes de permissions, enregistrer des affectations scopeées utilisateur-rôle, et résoudre les autorités effectives utilisées par le kernel pour l'autorisation runtime.

### 5.3 Rôles secondaires

#### 1. Création de rôles multi-portée

Un rôle est créé avec un **code**, un **name**, un **scopeType** et un ensemble non vide de permissions textuelles. Si la portée n'est pas fournie, elle vaut **TENANT**.

#### 2. Affectations scopeées

Une affectation associe un utilisateur à un rôle avec une portée **SYSTEM**, **TENANT**, **ORGANIZATION** ou **AGENCY**. Pour les deux dernières, un `scopeId` est requis.

### 3. Résolution des autorités effectives

Les permissions d'un rôle sont transformées en autorités runtime. Une affectation **SYSTEM** ou **TENANT** produit à la fois la permission brute et sa variante scopeée ; une affectation **ORGANIZATION** ou **AGENCY** ne produit que la variante suffixée.

### 4. Cache Redis optionnel

Quand `iwm.redis.permission-cache.enabled=true`, la résolution des permissions est mise en cache dans Redis sous une clé de la forme `iwm:permissions:<tenantId>:<userId>` avec un TTL configurable (15 minutes par défaut).

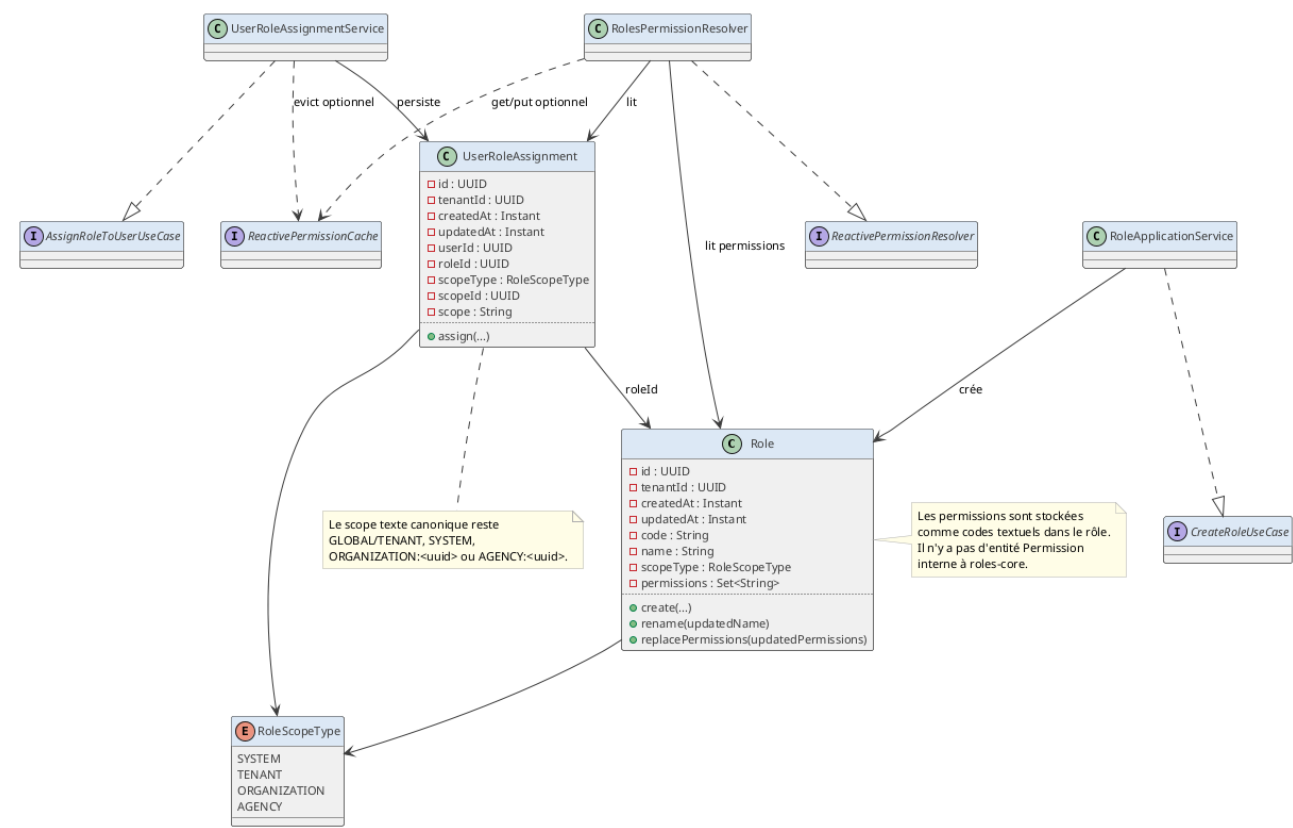
### 5. Invalidation locale du cache

Lorsqu'une affectation est créée via `UserRoleAssignmentService`, le cache de permissions de l'utilisateur est invalidé immédiatement si le composant `ReactivePermissionCache` est présent.

### 6. Socle RBAC réutilisé par `administration-core`

Les templates de rôles réservés, le clonage, la révocation et l'audit administratif ne sont plus des responsabilités directes du contrôleur `/api/roles` ; ils sont orchestrés plus haut par `administration-core` au-dessus des mêmes repositories.

Diagramme de classes — roles-core

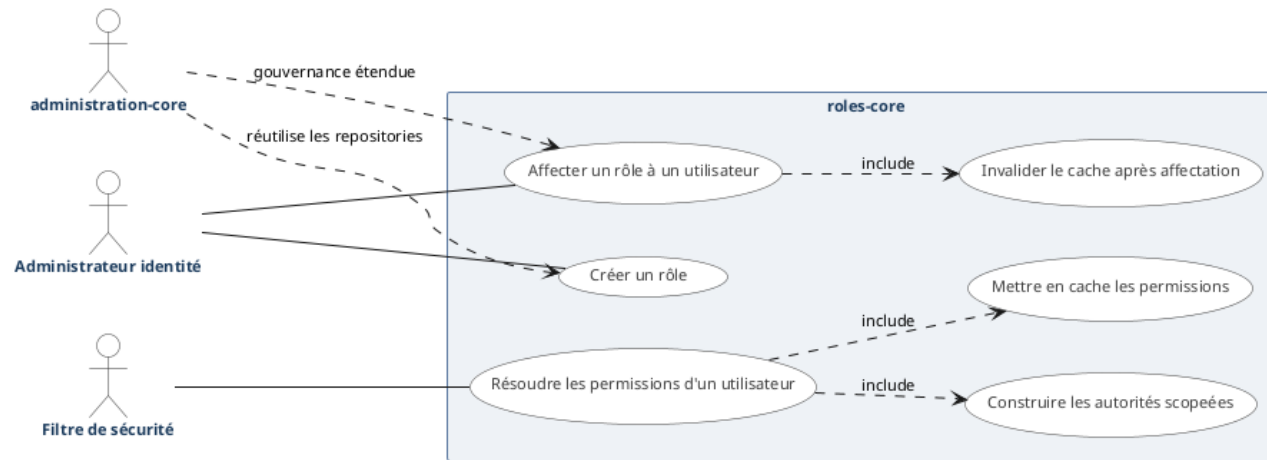




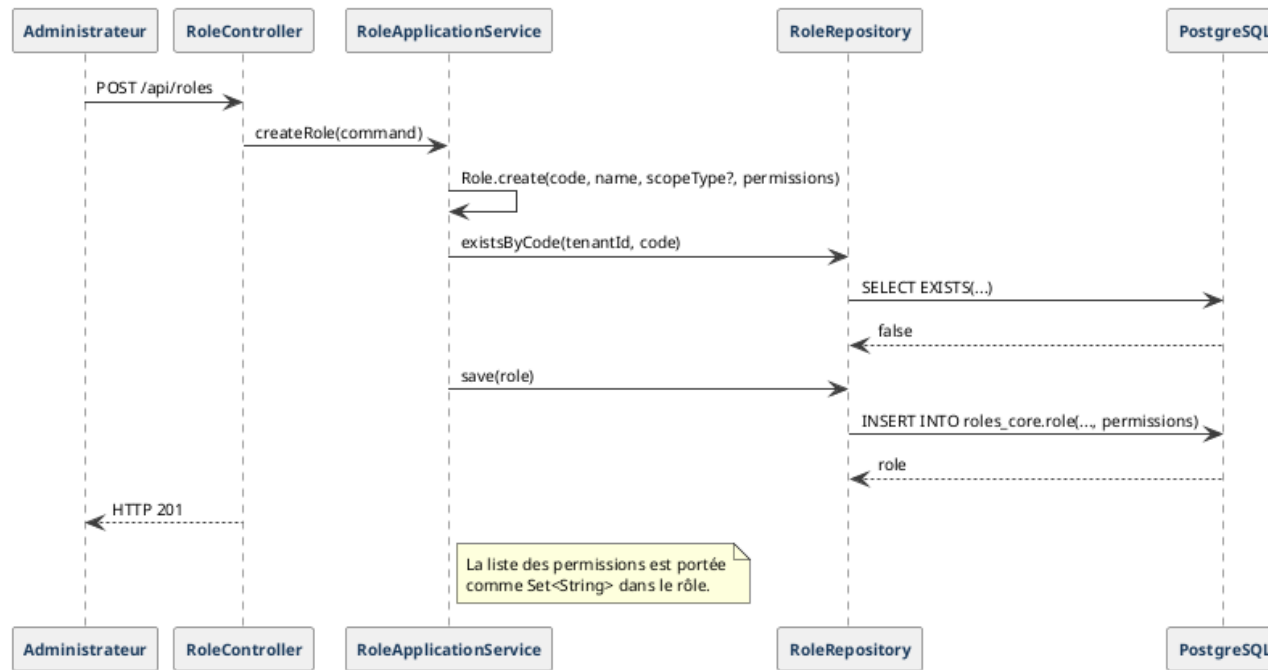
**Invariants du roles-core :**

- Le code d'un rôle est unique par tenant.
- Un Role doit toujours porter un ensemble non vide de permissions.
- `Role.scopeType` vaut `TENANT` par défaut si elle n'est pas fournie.
- `UserRoleAssignment.scopeType` vaut `TENANT` par défaut si la portée est vide.
- Un `scopeId` est interdit pour `SYSTEM` et `TENANT`, et obligatoire pour `ORGANIZATION` et `AGENCY`.
- La représentation textuelle `scope` est canonique : `SYSTEM`, `GLOBAL`, `ORGANIZATION:<uuid>` ou `AGENCY:<uuid>`.

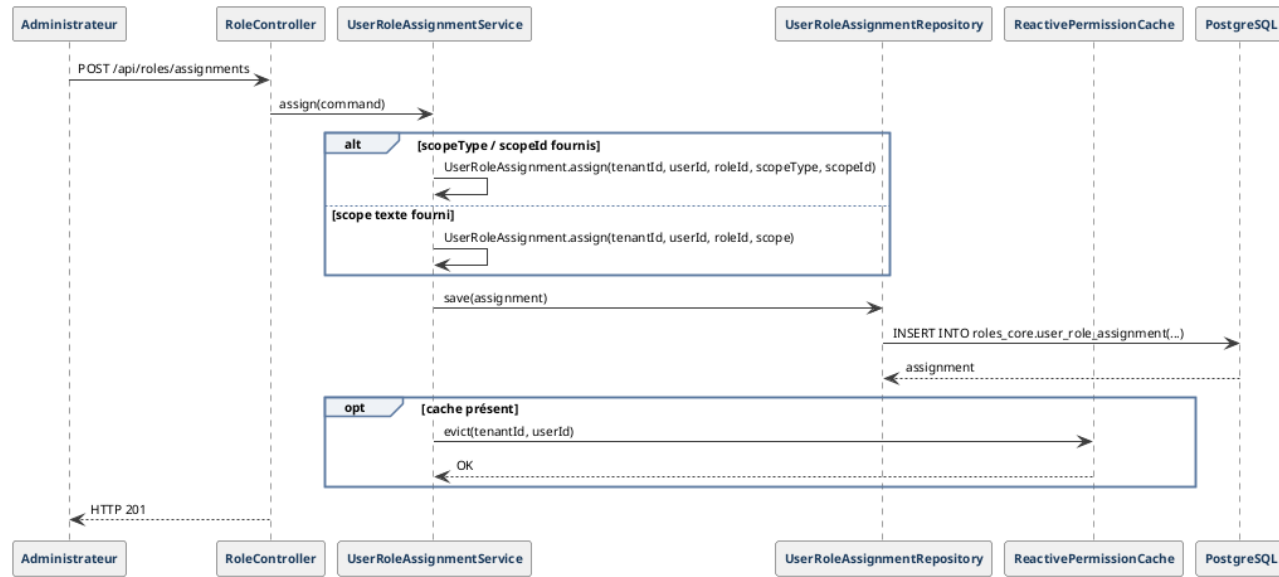
## Diagramme des cas d'utilisation — roles-core



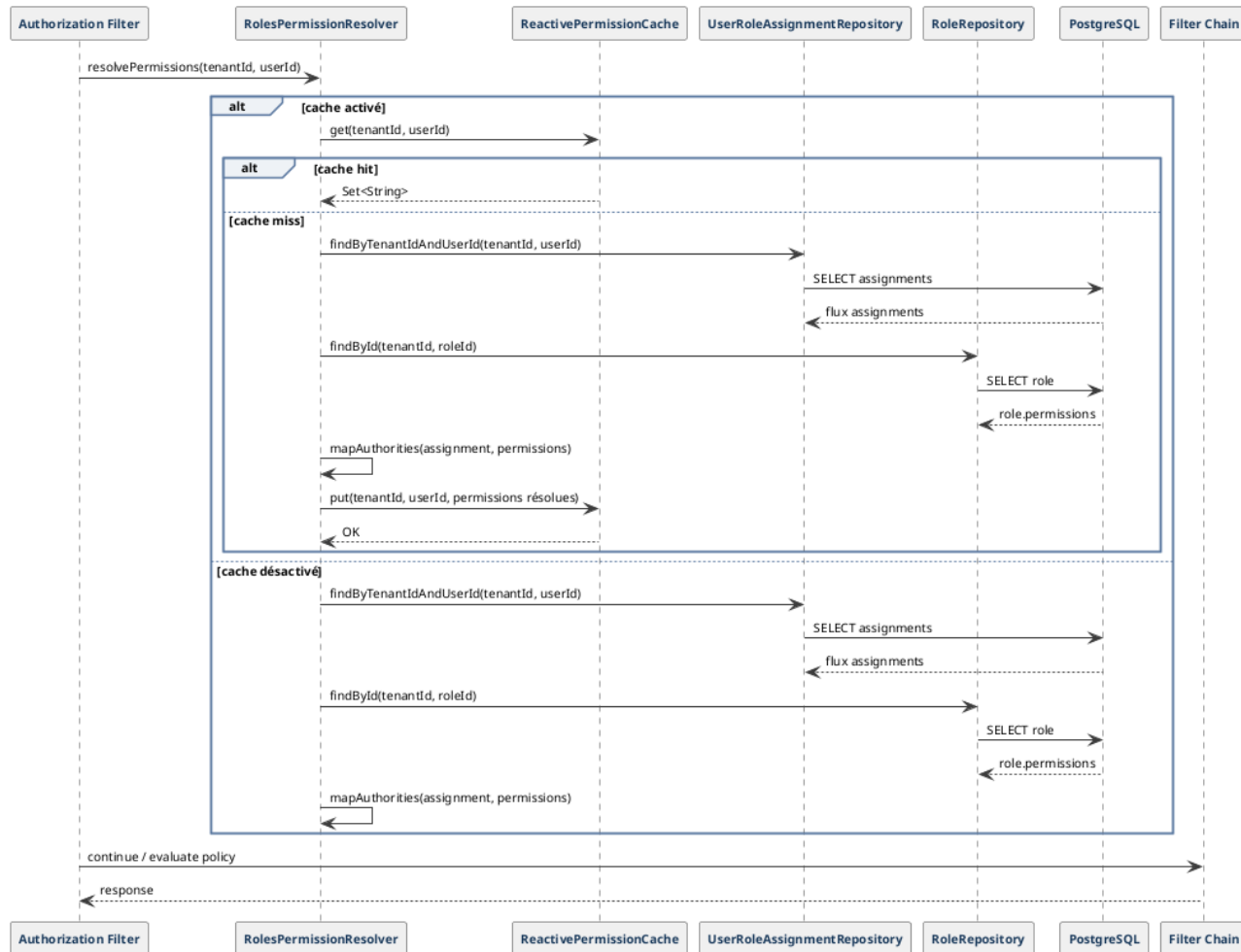
## DS-R-01 — Création d'un rôle avec ses codes de permission



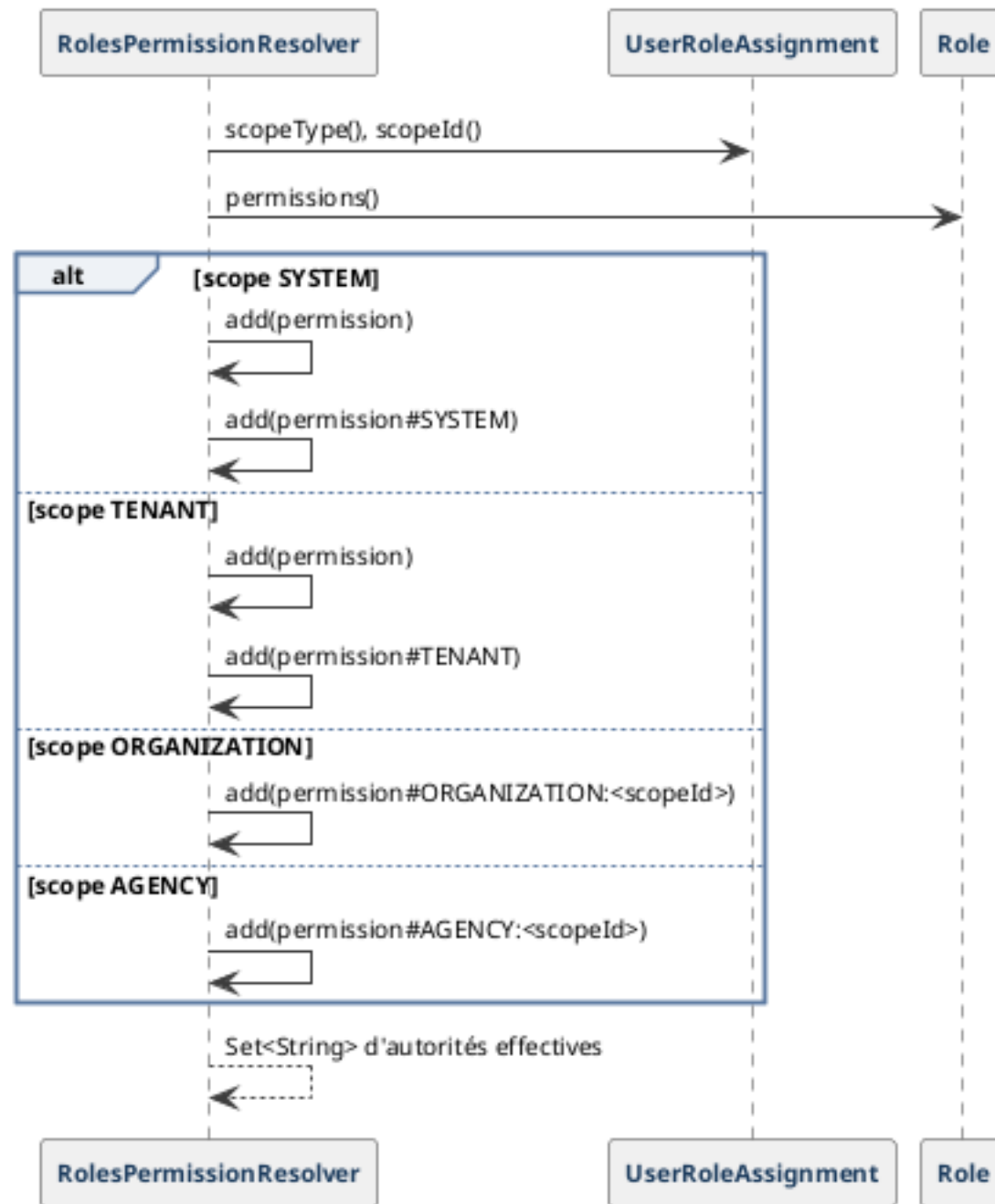
## DS-R-02 — Affectation d'un rôle à un utilisateur avec portée



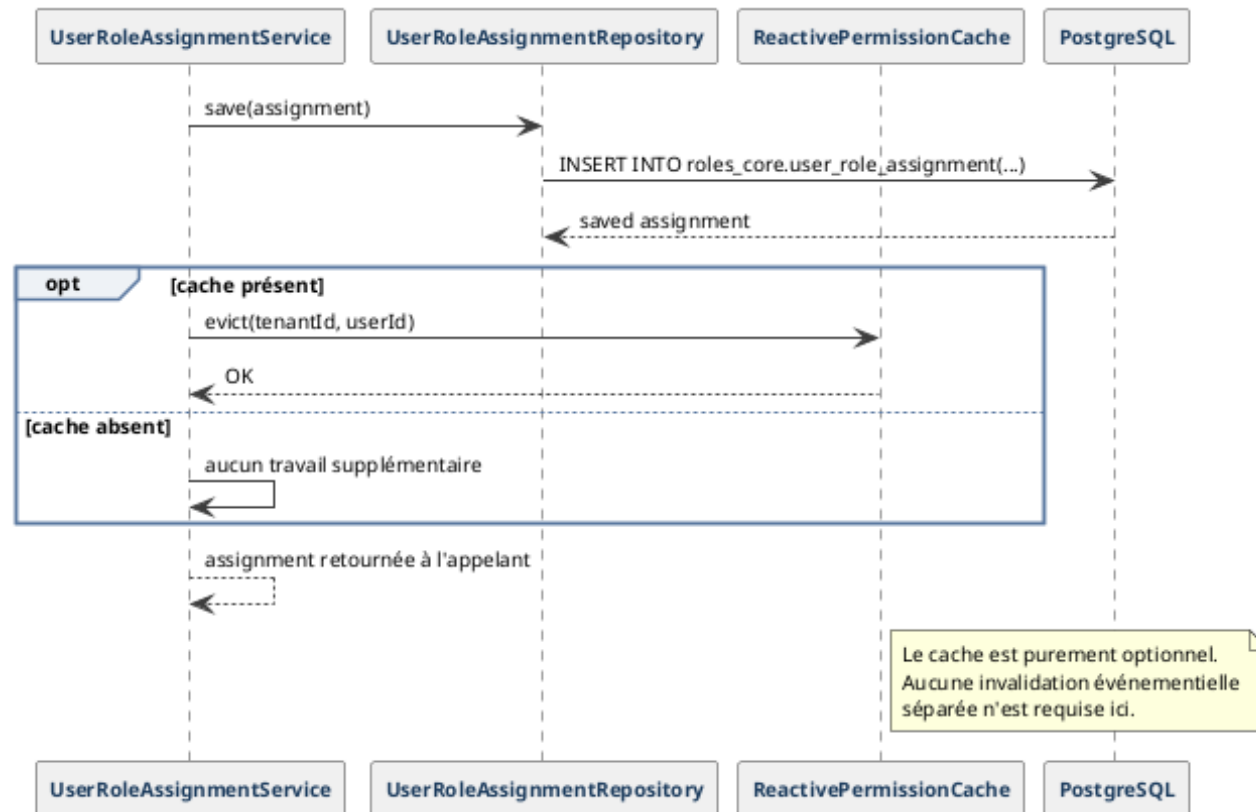
## DS-R-03 — Résolution des permissions avec cache Redis optionnel



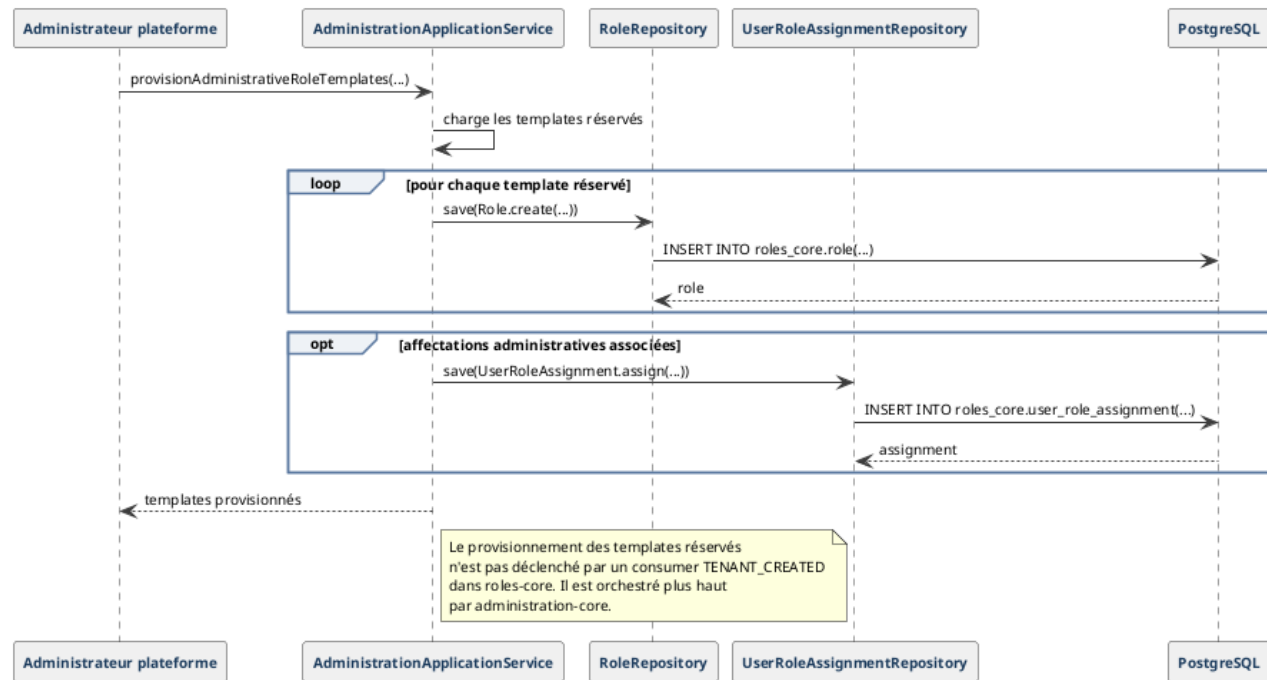
## DS-R-04 — Construction des autorités scopeées à partir d'une affectation



## DS-R-05 — Invalidation du cache après affectation



## DS-R-06 — Provisionnement des templates réservé à administration-core





## 6 organization-core

---

### 6.1 Description du core

L'**organization-core** modélise la structure organisationnelle des entités opérant sur la plateforme. Une *Organisation* est l'unité métier propriétaire des produits, des commandes et des stocks. Elle est composée d'une ou plusieurs *Agences* — des unités opérationnelles locales qui portent les horaires d'ouverture, les affiliations de personnel, les points d'intérêt géographiques, et désormais une hiérarchie physique interne d'espaces : bâtiments, étages, salles, postes ou sièges.

Ce core fournit également le modèle de domaine d'activité (*BusinessDomain*) qui classifie les organisations selon leur secteur, ainsi que la gestion des certifications, des activités proposées, et des liens fonctionnels entre les acteurs humains et les entités organisationnelles.

Il porte aussi le **catalogue de services plateforme** et les **abonnements de services par organisation**. Cette capacité permet de déclarer quels modules métier (COMMERCIAL, PRODUCT, INVENTORY, SALES, ACCOUNTING, TREASURY, RESOURCE) sont activés pour une organisation donnée, avec deux services obligatoires toujours effectifs : ORGANIZATION et SETTINGS. Chaque abonnement explicite porte en plus un quota propre `requestQuotaLimit` et une fenêtre `requestQuotaWindowSeconds`, utilisés par le kernel pour borner le trafic au niveau métier de l'organisation, y compris lorsque ce trafic est consommé par une communication explicite entre deux services du monolithe.

*Organization* et *Agency* portent les attributs métiers complets de présentation et de conformité : `shortName`, `longName`, `email`, `businessRegistrationNumber`, `taxNumber`, `capitalShare`, `legalForm`, coordonnées, et indicateurs de gouvernance.

Le diagramme de classes fait apparaître ce minimum canonique complet sans dépendre d'un héritage implicite ou d'un alias technique. Les champs de base de l'organisation et de l'agence y sont donc listés explicitement, même lorsqu'une implémentation les projette encore en compatibilité.

Les entités secondaires *BusinessDomain*, *Certification*, *ProposedActivity*, *OrganizationActor*, *AgencyAffiliation*, *OrganizationDomain* et *AgencyDomain* disposent d'une représentation explicite dans le code et dans le schéma PostgreSQL du backend, ainsi que de cas d'usage et d'endpoints dédiés de structure organisationnelle.

Le core porte en plus le sous-système *PhysicalSpace*, rattaché à une agence et organisé sous forme d'arbre parent/enfant. Il sert de support canonique à la hiérarchie physique d'un site opérationnel, à laquelle les ressources matérielles et les documents

peuvent être rattachés indirectement via *resource-core* et *file-core*.

La spécialisation conceptuelle *Employee* du modèle canonique reste portée opérationnellement par *EmployeeMembership*, afin de conserver dans un seul agrégat le lien d'appartenance à l'organisation ou à l'agence, le cycle de vie d'affectation et les rôles d'exécution associés.

## 6.2 Rôle principal

**Rôle principal :** Définir et maintenir la structure hiérarchique *Organisation* → *Agence*, gérer les liens fonctionnels entre acteurs et entités organisationnelles, et fournir aux autres cores une référence stable de l'identité et du contexte opérationnel de chaque unité.

## 6.3 Rôles secondaires

### 1. Gestion des agences opérationnelles

Chaque organisation possède une agence siège (*headquarter*) et peut avoir autant d'agences opérationnelles supplémentaires que nécessaire. L'agence est le niveau le plus fin d'exécution des opérations (stocks, horaires, affectations).

### 2. Horaires d'ouverture

Modèle d'horaires récurrents (*OpeningHoursRule*) et d'exceptions ponctuelles (*SpecialOpeningHours*) permettant de modéliser des calendriers complexes (jours fériés, plages horaires spécifiques).

### 3. Points d'intérêt géographique

Les agences peuvent déclarer des *PointOfInterest* : hubs logistiques, points de service, repères géographiques.

### 4. Hiérarchie physique native

Chaque agence peut définir un arbre d'espaces physiques (*PhysicalSpace*) pour modéliser bâtiments, étages, locaux, zones, postes ou sièges. Cette hiérarchie structure les vues opérationnelles, l'affectation d'actifs matériels et les rattachements documentaires.

### 5. Domaines d'activité et certifications

Classification sectorielle des organisations via une taxonomie hiérarchique (*BusinessDomain*), et traçabilité des certifications professionnelles.

### 6. Source de vérité organisationnelle

Tous les cores référençant une organisation ou une agence utilisent uniquement leurs identifiants. L'organization-core est la seule source de vérité pour la dénomination, le statut, les métadonnées légales et la structure hiérarchique.

**7. Gouvernance et cycle de vie**

Transitions de statut contrôlées pour les organisations et agences : création, activation, suspension, fermeture.

**8. Catalogue et abonnements de services**

L'organization-core expose un catalogue de services plateforme et persiste les abonnements effectifs par organisation. Il devient ainsi la source de vérité de l'activation des modules métier pour un tenant et une organisation donnés.

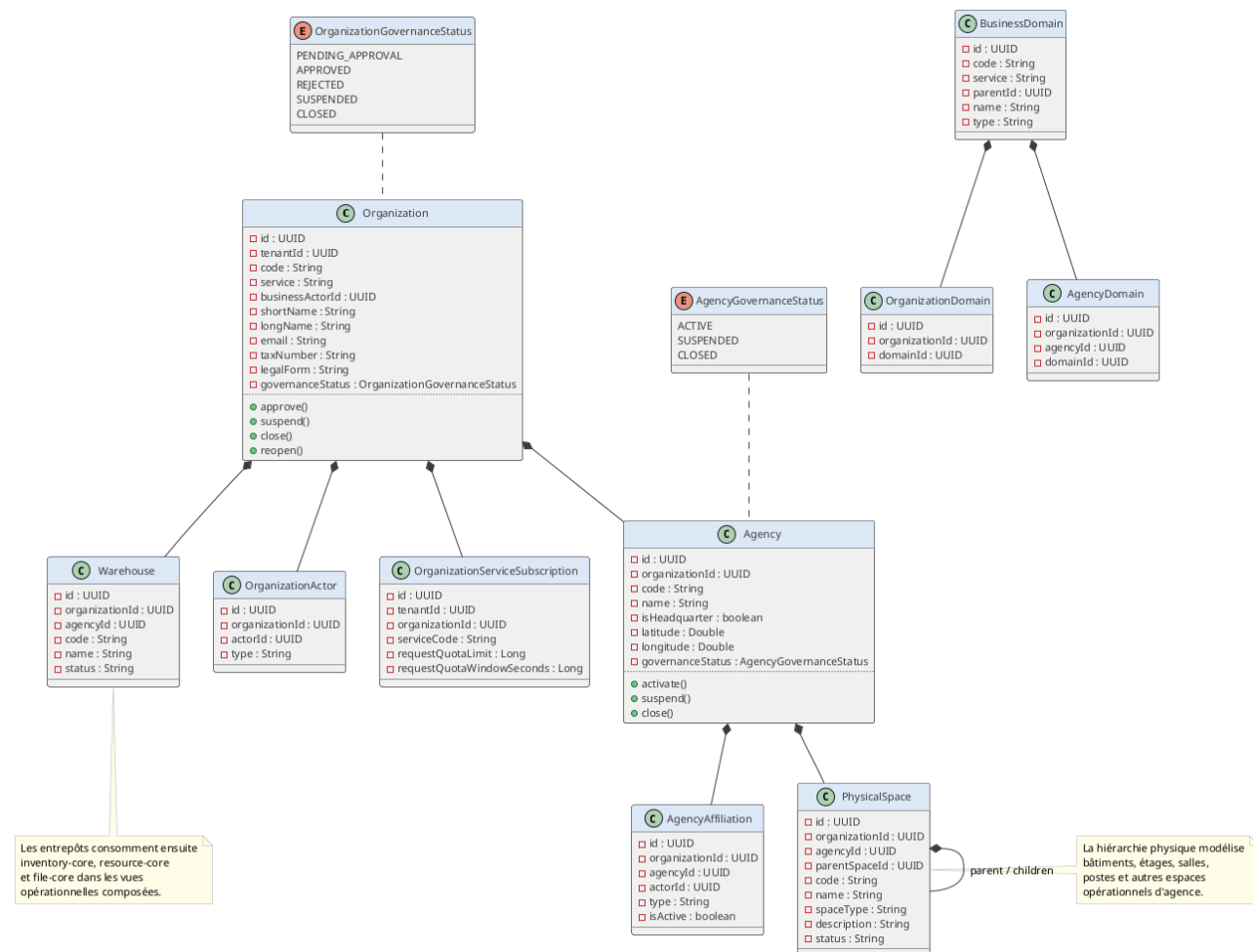
**9. Politique de quota par service d'organisation**

L'abonnement à un service ne se limite plus à un booléen *abonné / non abonné*. Il porte aussi la capacité de trafic attendue pour ce service, ce qui permet au kernel d'appliquer un quota d'usage cohérent avec l'offre ou le fair-use de l'organisation.

**10. Politique runtime réutilisable**

La résolution *effective / non effective* d'un service et de son quota alimente à la fois les filtres HTTP du kernel et les communications synchrones explicites entre cores métier. Une collaboration inter-service peut donc être autorisée ou refusée selon le service cible réellement disponible pour l'organisation courante.

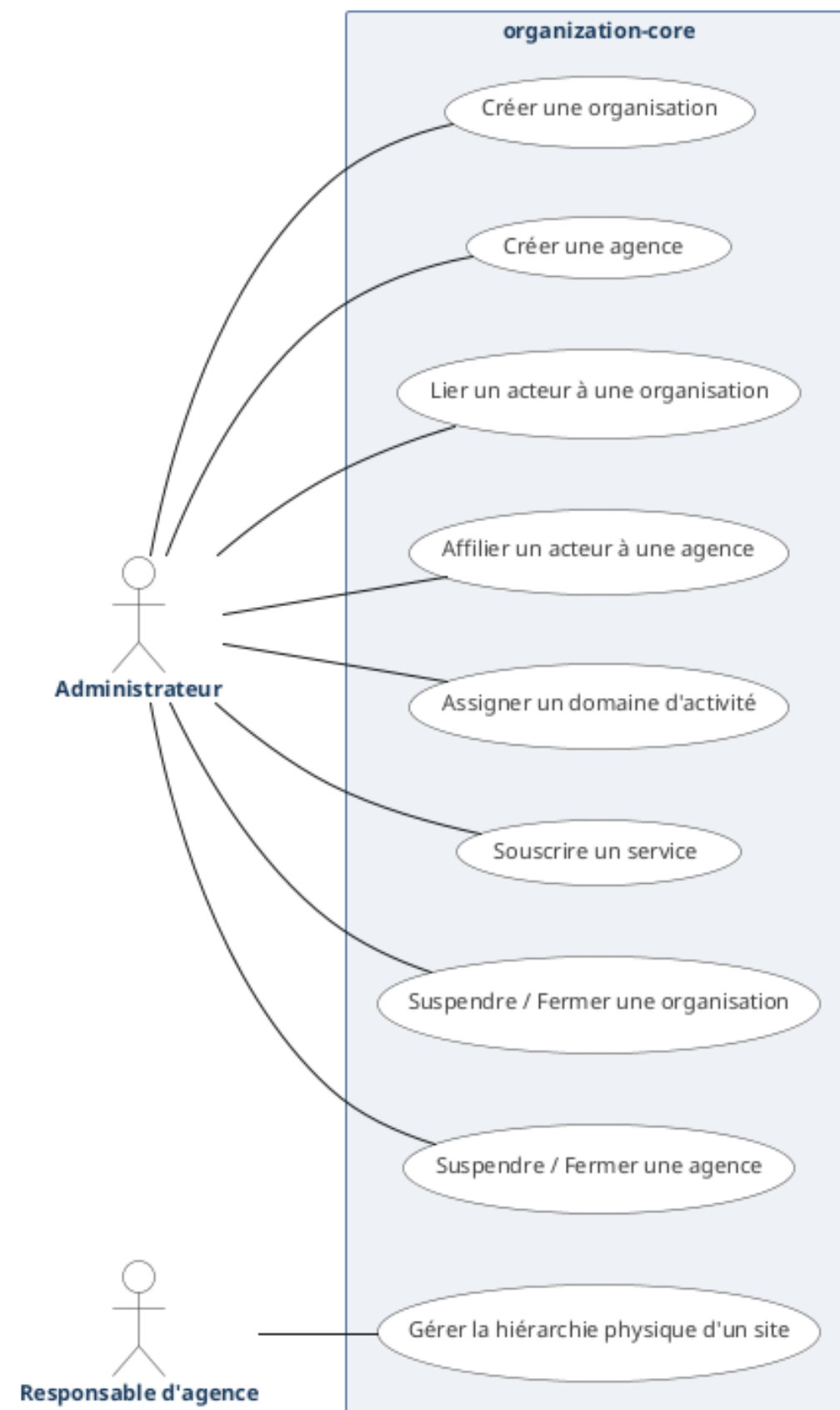
## Diagramme de classes — organization-core



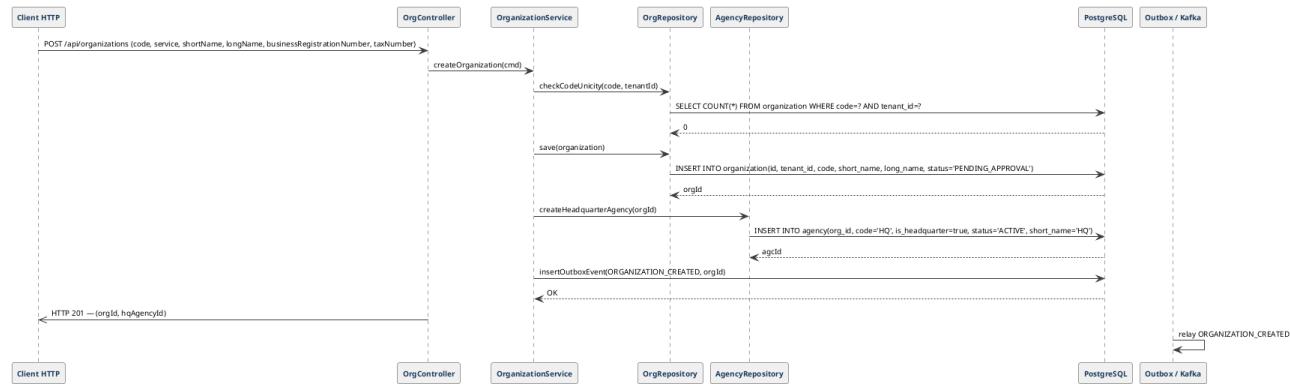
**Invariants du organization-core :**

- Le code d'une organisation est unique par tenant.
- Le code d'une agence est unique par organisation.
- Le code d'un espace physique est unique par couple (`organizationId`, `agencyId`).
- Exactement une agence siège (`isHeadquarter=true`) par organisation.
- Chaque couple (`organizationId`, `serviceCode`) est unique dans les abonnements de services.
- Tout abonnement explicite porte un `requestQuotaLimit > 0` et un `requestQuotaWindowSeconds > 0`.
- `ORGANIZATION` et `SETTINGS` restent toujours effectifs, même sans ligne d'abonnement explicite.
- Une organisation `CLOSED` ne peut être réactivée.
- Le gestionnaire d'une agence doit être un acteur actif du même périmètre organisationnel.
- Un espace physique parent et son enfant appartiennent toujours à la même agence.

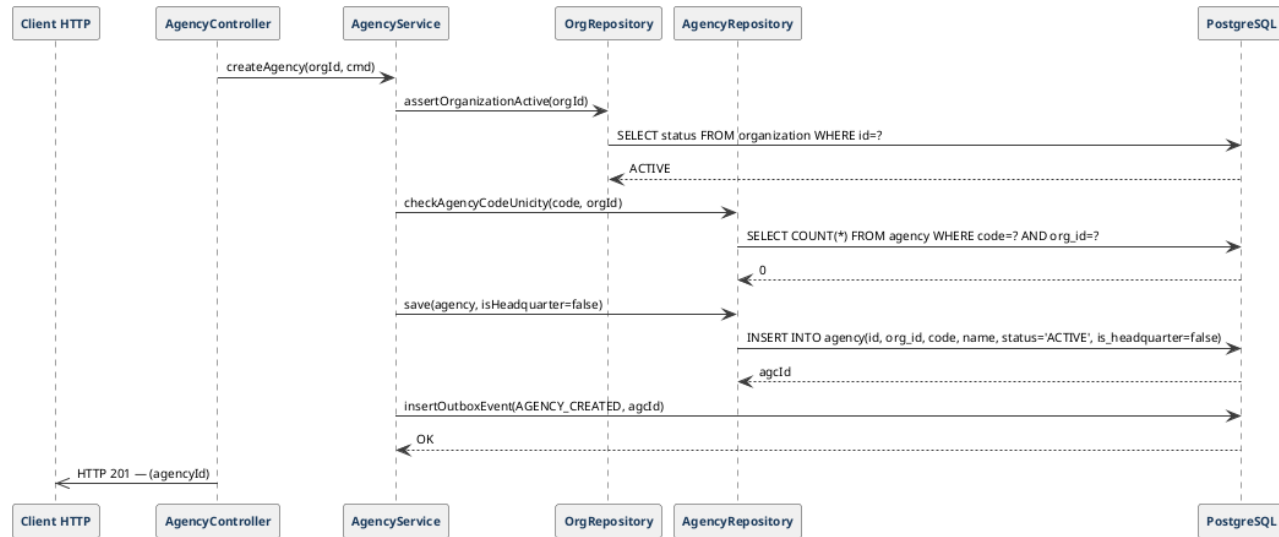
## Diagramme des cas d'utilisation — organization-core



## DS-O-01 — Création d'une organisation avec agence siège automatique

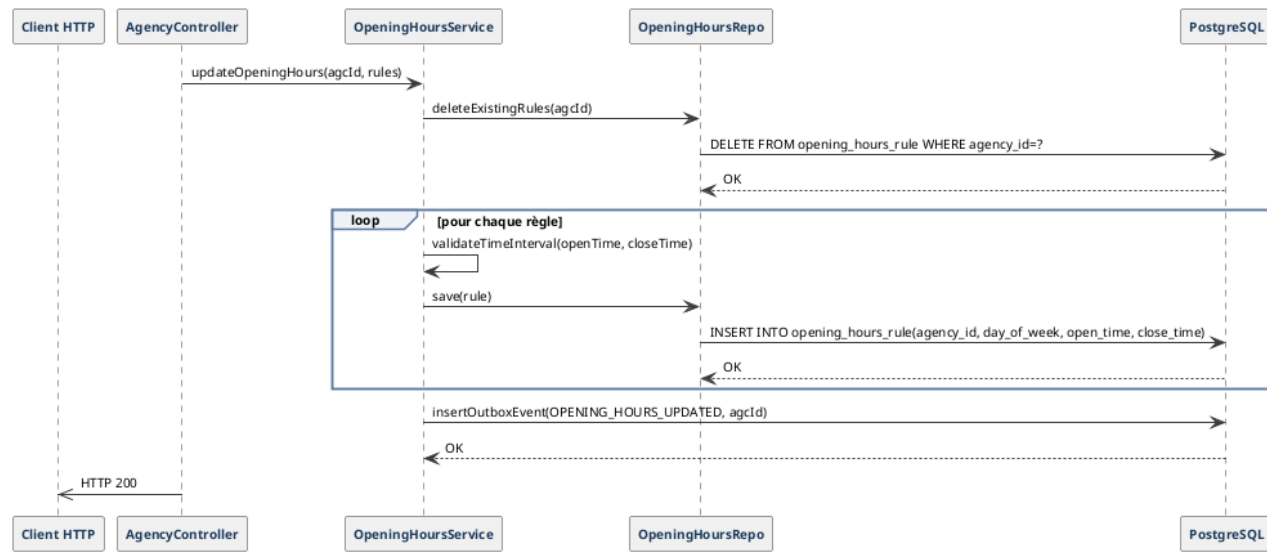


## DS-O-02 — Création d'une agence opérationnelle

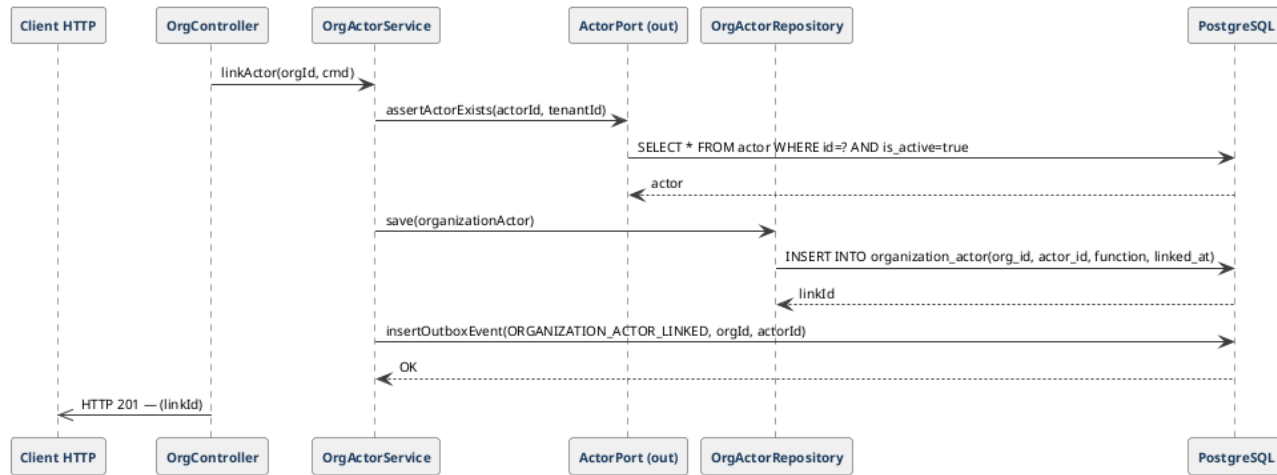




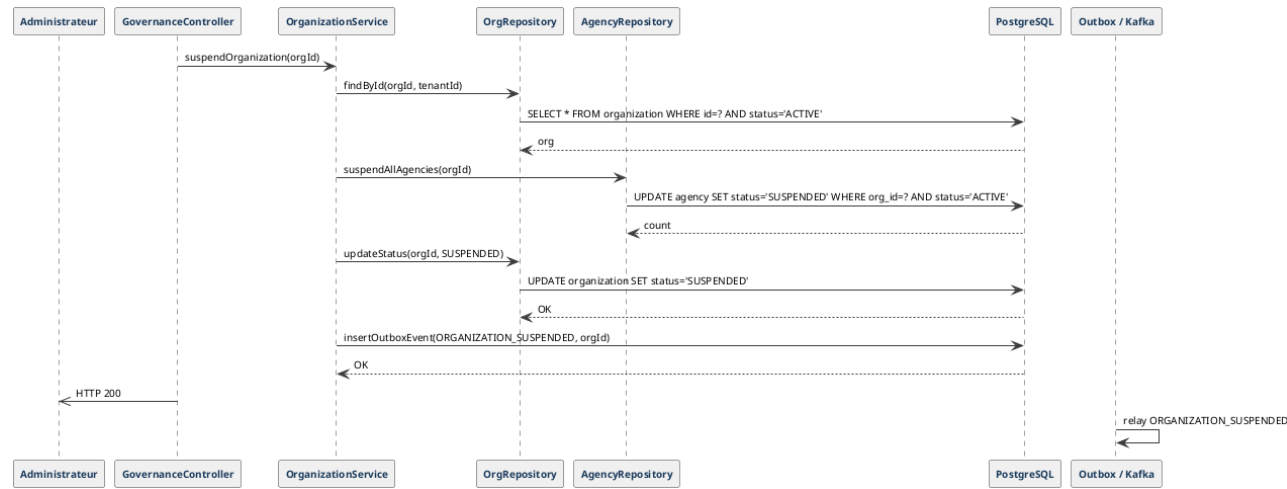
## DS-O-03 — Mise à jour des horaires d'ouverture d'une agence



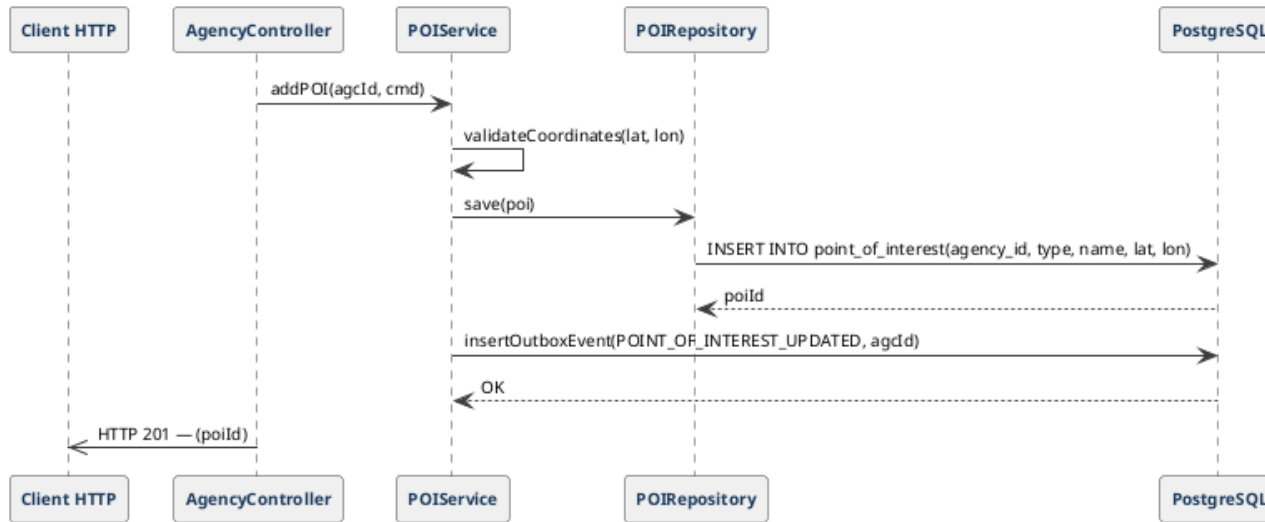
## DS-O-04 — Lien fonctionnel entre un acteur et une organisation



## DS-O-05 — Suspension d'une organisation et de ses agences



## DS-O-06 — Ajout d'un point d'intérêt géographique à une agence



**Extension transverse notable** La hiérarchie physique de **organization-core** est consommée en lecture par les vues composées de **inventory-core** (*operational-site, generalized-inventory, service-workspaces*) et en validation par **resource-core** pour permettre l'affectation ou la réservation d'une ressource matérielle à un **PHYSICAL\_SPACE**.

## 7 tp-core — Gestion des Tiers

---

### 7.1 Description du core

Le **tp-core** (Third-Parties Core) gère l'ensemble des entités externes avec lesquelles une organisation entretient des relations commerciales : clients, fournisseurs, agents commerciaux, et prospects. Un *ThirdParty* est une entité unifiée qui peut porter simultanément plusieurs qualifications (*rôles*) selon la nature de ses interactions avec l'organisation.

Ce core intègre également un module de gestion du cycle prospectif (*CRM léger*) : qualification, relances, interactions commerciales, et conversion en client. Les données de qualification avancées (comptes bancaires dédiés, score de qualification, segment de marché, comptes comptables, moyens de paiement autorisés) enrichissent le profil commercial des tiers.

*ThirdParty* est l'agrégat central du module. Les rôles **CUSTOMER**, **SUPPLIER**, **PROSPECT** et **SALES\_AGENT** sont implémentés comme des capacités portées par le même agrégat, complétées par des sous-modèles dédiés comme **ThirdPartyBankAccount** et **Interaction**.

La source de vérité interne du tiers reste portée par ses champs canoniques **code**, **name** et **accountingAccountNumbers**. Les alias historiques éventuellement exposés en bordure ne sont que des projections de compatibilité et ne gouvernent plus l'état métier de l'agrégat.

Le diagramme de classes de ce chapitre expose le minimum canonique complet du tier : identité, conformité, moyens de paiement, classification, fidélité, équilibres comptables, activation et suivi commercial.

L'entité **Interaction** formalise la trace métier persistable du suivi prospectif.

### 7.2 Rôle principal

**Rôle principal :** Centraliser les informations sur les entités externes (clients, fournisseurs, prospects, agents), gérer leur cycle de qualification commerciale, et fournir les références stables utilisées par les cores opérationnels (ventes,

comptabilité, trésorerie) pour identifier les contreparties.

## 7.3 Rôles secondaires

### 1. Multi-qualification d'un tiers

Un même tiers peut être simultanément client, fournisseur et agent commercial. Les rôles sont gérés comme une collection sur le tiers, sans duplication de données de base.

### 2. Qualification commerciale (CRM léger)

Segmentation, score de qualification, suivi de relance, dernière prise de contact, prochaine relance et statut de suivi permettent de piloter le cycle commercial sans dupliquer le tiers dans plusieurs tables métier.

### 3. Conversion prospect → client

Opération atomique qui marque le prospect comme converti et ajoute la qualification `CUSTOMER` au tiers sans perdre l'historique.

### 4. Gestion des comptes bancaires

Enregistrement des coordonnées bancaires (`ThirdPartyBankAccount`) pour les paiements et rapprochements, avec notion de compte principal.

### 5. Mapping comptable

Association d'un ou plusieurs comptes du plan comptable à chaque tiers pour l'imputation automatique lors de la facturation et des règlements.

### 6. Recherche plein-texte et statistiques

Projection Elasticsearch des tiers permettant une recherche par nom, code, rôle, segment, score, ou statut de relance, ainsi que le calcul de statistiques métiers (prospects convertis, tiers actifs, tiers bancarisés).

## Diagramme de classes — tp-core

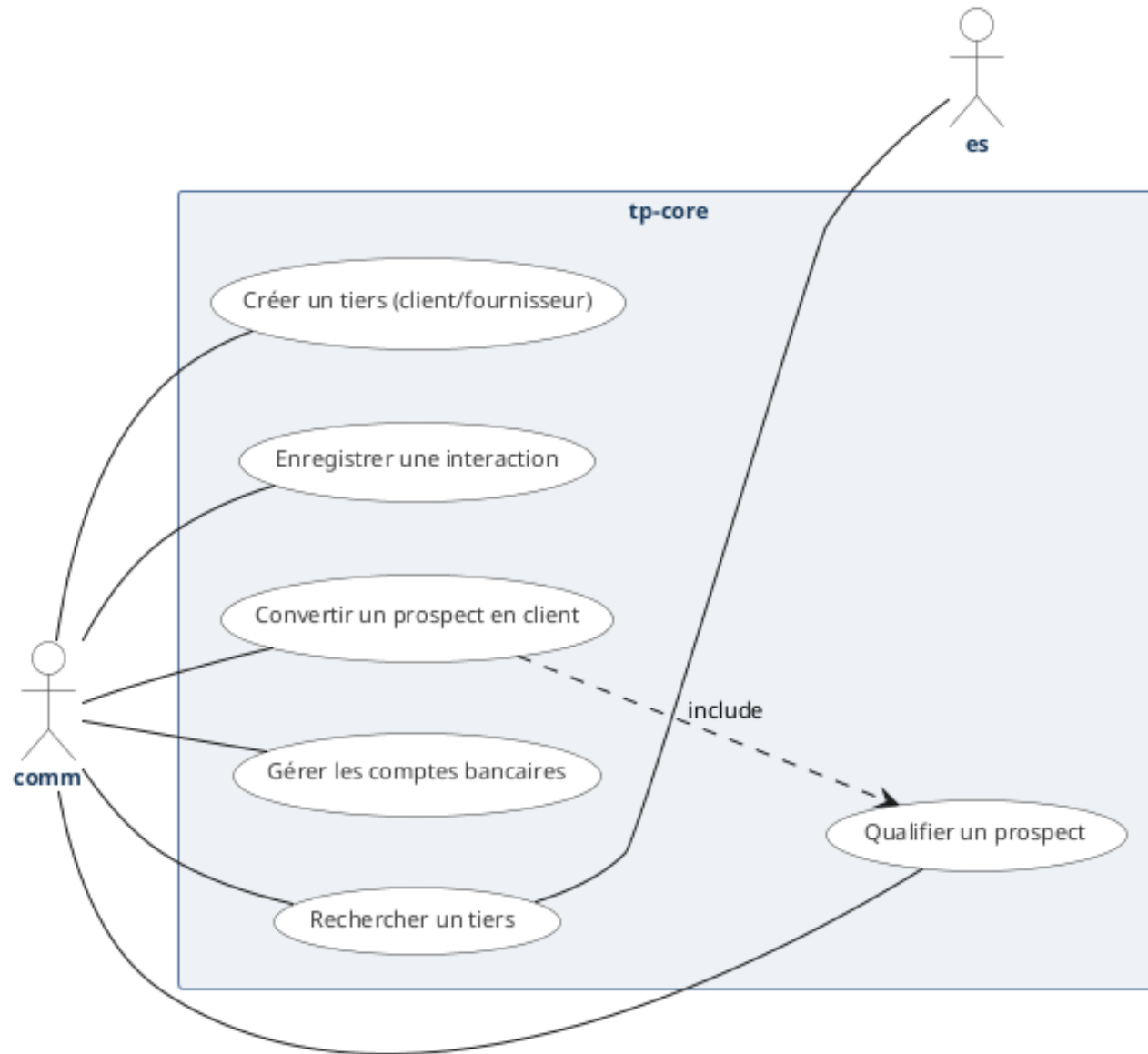




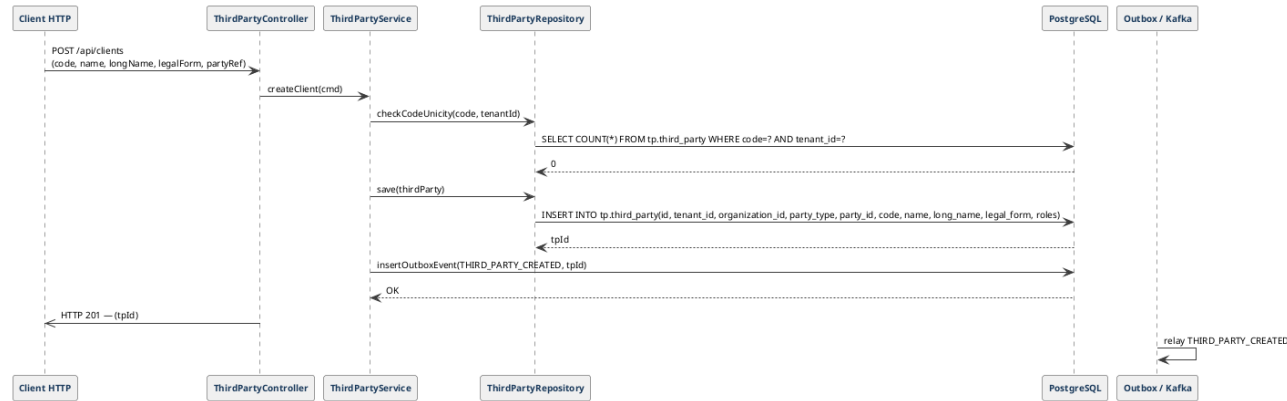
**Invariants du tp-core :**

- Le code d'un tiers est unique par tenant.
- Un seul compte bancaire principal par tiers.
- Un prospect converti devient un client sans perdre son historique d'interactions.
- Le score de qualification est borné et cohérent avec le segment retenu.
- Un tiers désactivé ne peut être référencé dans de nouvelles commandes.

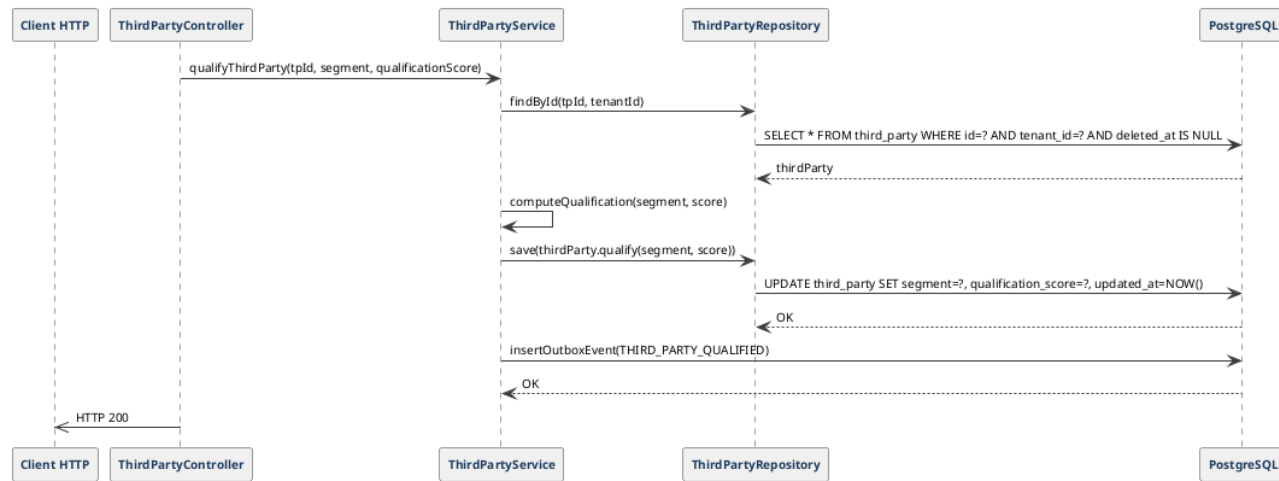
## Diagramme des cas d'utilisation — tp-core



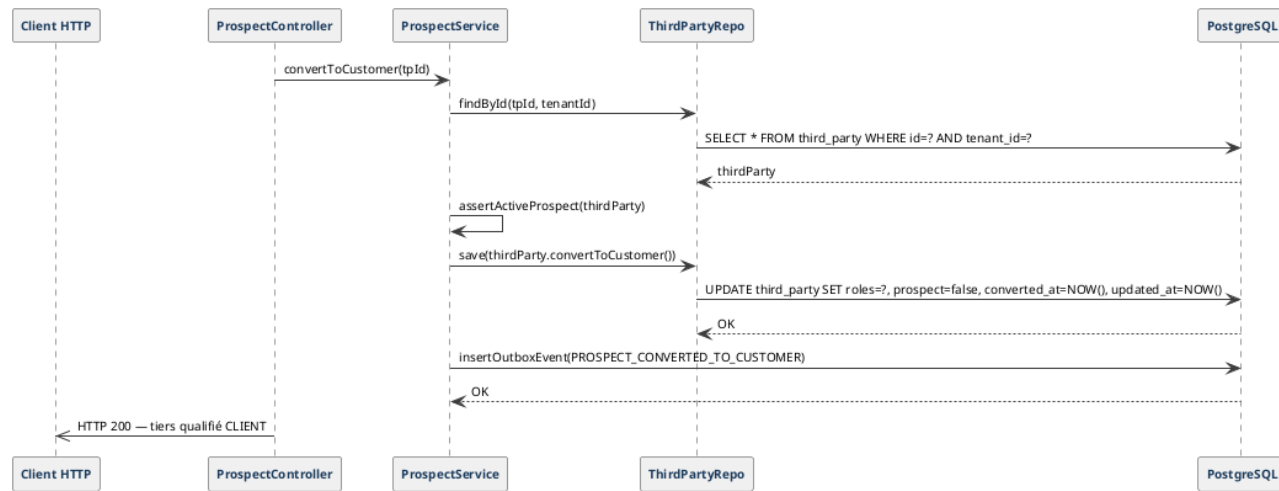
## DS-TP-01 — Création d'un tiers avec rôle client



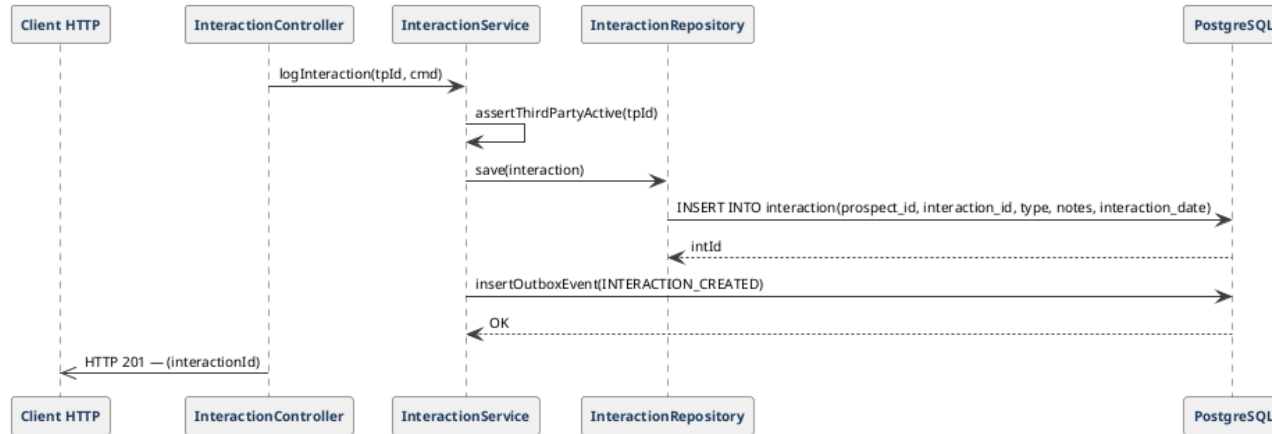
## DS-TP-02 — Qualification commerciale d'un prospect



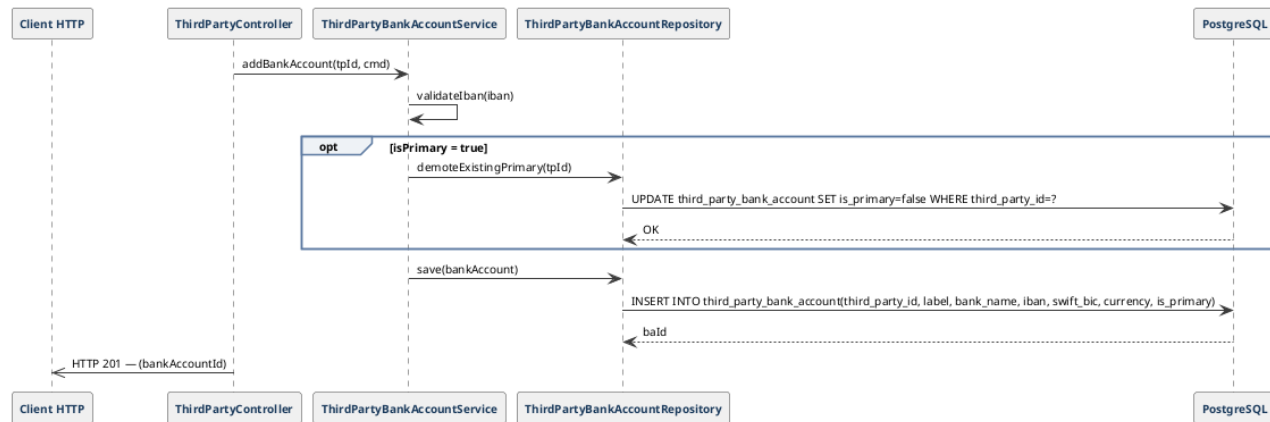
## DS-TP-03 — Conversion d'un prospect en client



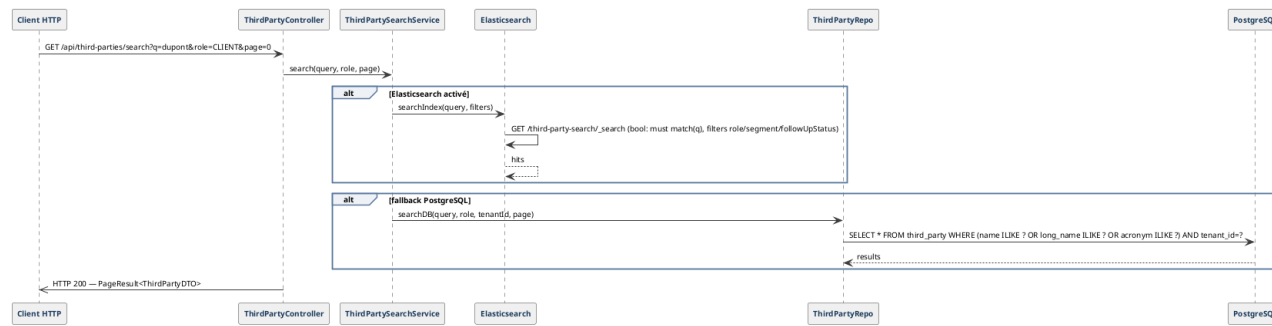
## DS-TP-04 — Enregistrement d'une interaction commerciale



## DS-TP-05 — Ajout d'un compte bancaire à un tiers



## DS-TP-06 — Recherche plein-texte de tiers (Elasticsearch / fallback PostgreSQL)





## 8 settings-core

---

### 8.1 Description du core

Le **settings-core** centralise les paramètres métier configurables et les séquences documentaires utilisées par les autres modules. Dans l'état courant du backend, il ne porte pas un moteur générique de numérotation fiscale avec verrou Redis, seuil d'épuisement et événements dédiés ; il expose plus simplement trois sous-domaines cohérents : **AppBusinessSettings**, **DocumentSequence** et **OperationalPolicyProfile**.

**AppBusinessSettings** porte les options métier usuelles d'une organisation ou d'une agence : devise par défaut, identité légale, paramètres d'impression, remises exceptionnelles, politiques d'approbation commerciale et alertes transverses. Lorsqu'une agence ne possède pas de réglage propre, la lecture retombe sur la configuration par défaut de l'organisation.

**DocumentSequence** fournit des séquences documentaires configurables par (**tenant**, **organization**, **agency?**, **documentType**) avec préfixe, suffixe, largeur de padding et prochain numéro. La génération d'un numéro consomme la séquence configurée pour l'agence si elle existe, sinon celle de l'organisation.

**OperationalPolicyProfile** complète enfin les réglages généraux avec un profil de politique opérationnelle centré sur les usages d'inventaire, de ressource et de gouvernance documentaire : approbations d'affectation, variance d'inventaire tolérée, seuils d'alerte de maintenance, nombre maximal campagnes ouvertes, et politique de documents obligatoires.

### 8.2 Rôle principal

**Rôle principal** : Fournir un référentiel de configuration métier par organisation et par agence, ainsi qu'un service simple de séquences documentaires réutilisé par les autres cores pour produire des références lisibles et cohérentes.

### 8.3 Rôles secondaires

#### 1. Paramètres métier généraux

Le core stocke les options de vente, de remise, d'impression et d'identité légale

dans `AppBusinessSettings`. Les valeurs par défaut sont créées à la première lecture si aucune ligne n'existe encore.

2. **Fallback organisation → agence**

La lecture des paramètres et des séquences supporte un repli vers le niveau organisation lorsque le niveau agence n'est pas configuré.

3. **Séquences documentaires configurables**

Une séquence est définie par `documentType`, `prefix`, `suffix`, `paddingWidth` et `nextNumber`. Le numéro courant est formaté puis la séquence est avancée d'une unité.

4. **Politique opérationnelle dédiée**

`OperationalPolicyProfile` couvre des règles plus fines que les options générales : approbation des affectations, checklist d'ouverture, tolérance de variance d'inventaire, seuil de faible utilisation, etc.

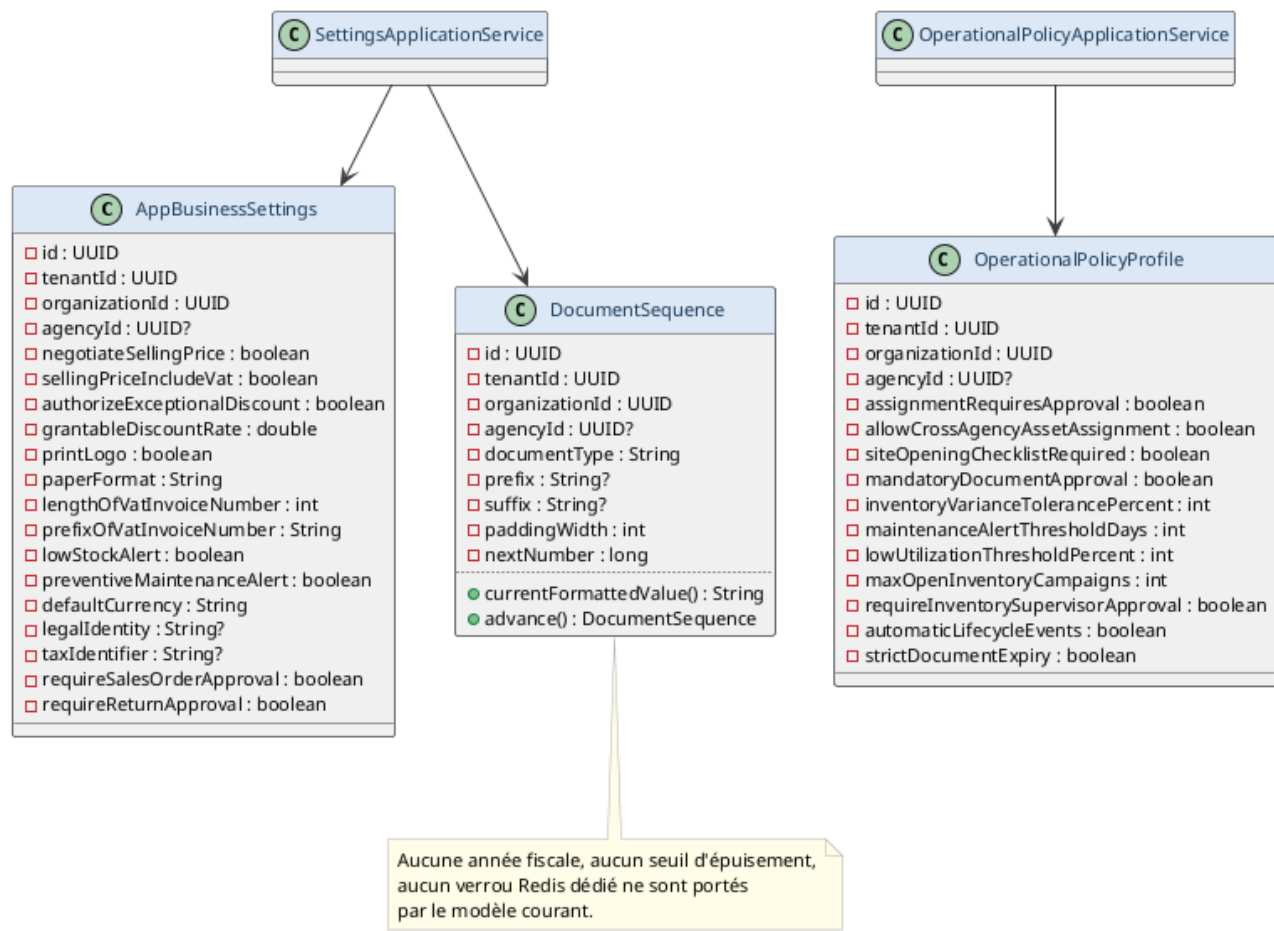
5. **API spécialisée par sous-domaine**

Les points d'entrée sont séparés entre options générales (`/api/general-options`), séquences documentaires (`/api/settings/document-sequences`) et politiques opérationnelles (`/api/settings/organizations/.../operational-policy`).

6. **Socle consommé par les autres cores**

Les commandes de vente, de facturation et d'inventaire consomment les séquences; les vues métier lisent les options générales; les modules `resource/inventory/document hub` peuvent s'appuyer sur les politiques opérationnelles pour leur gouvernance locale.

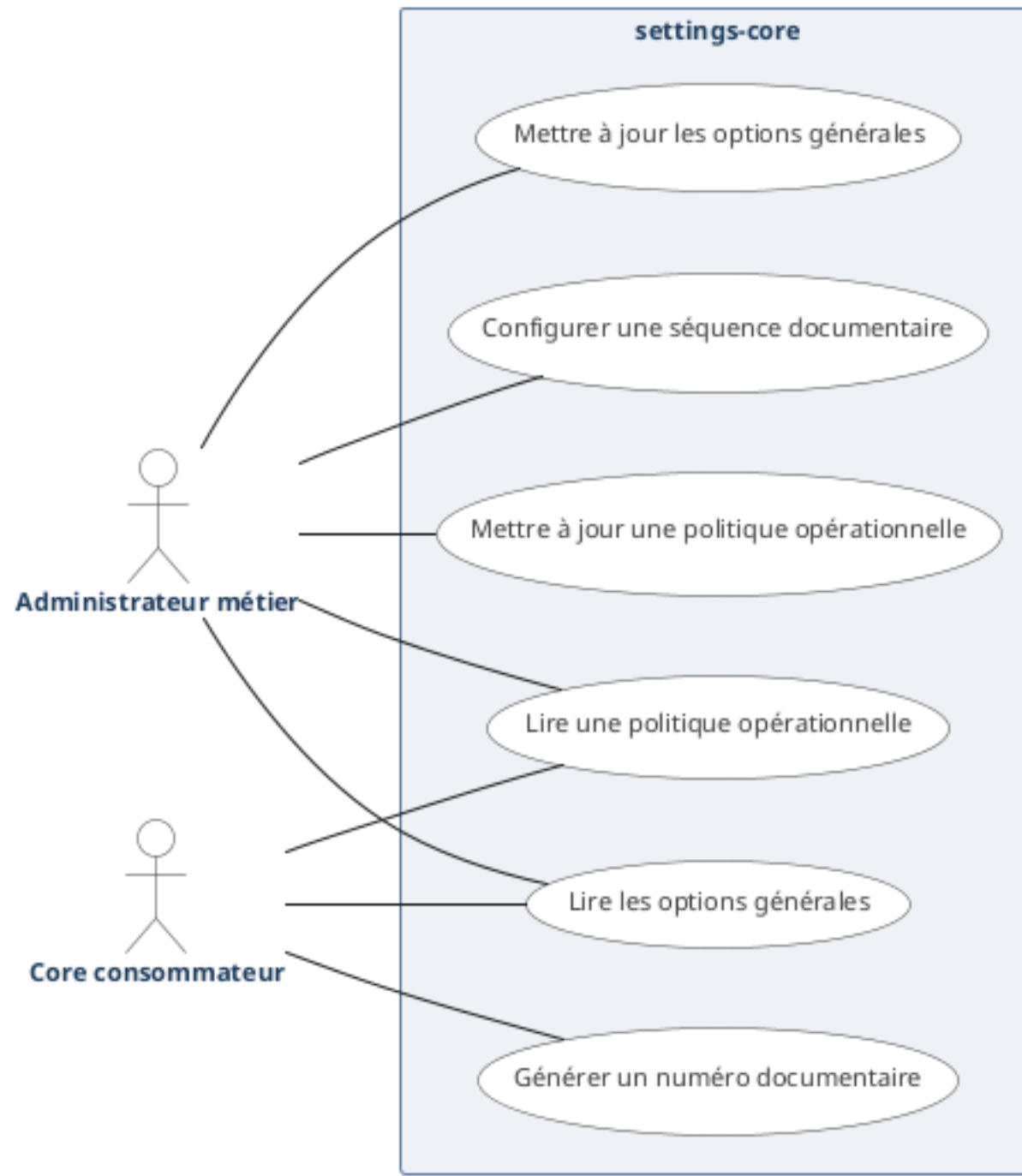
Diagramme de classes — settings-core



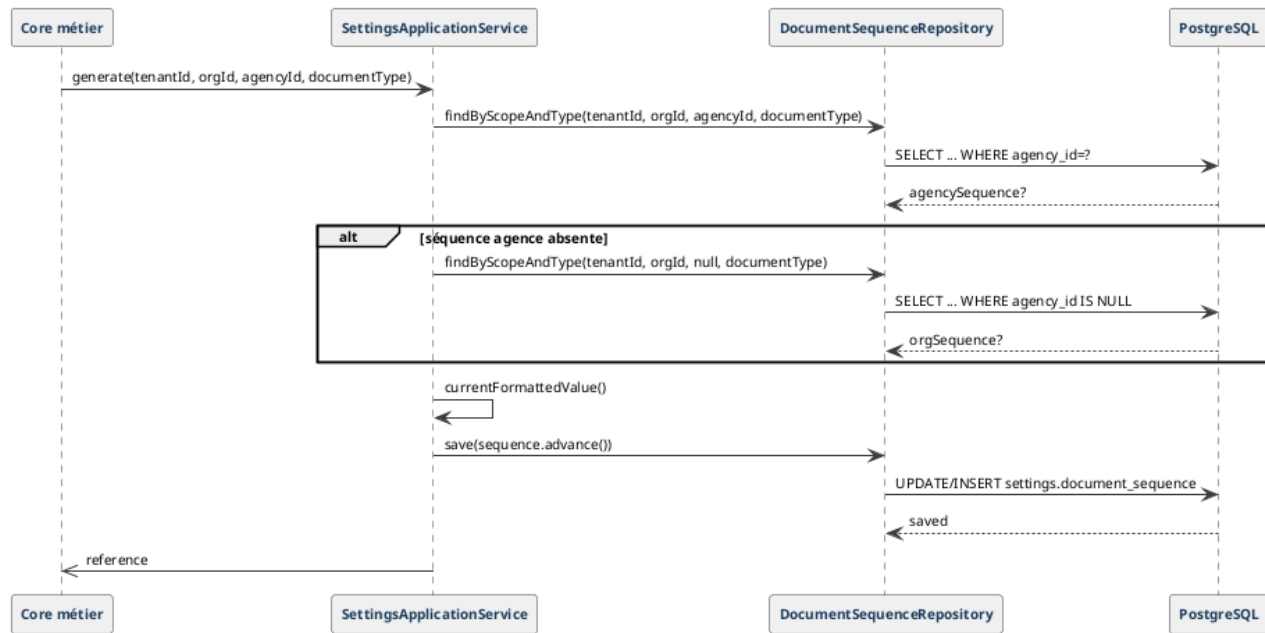
**Invariants du settings-core :**

- Une configuration `AppBusinessSettings` est unique par `(tenantId, organizationId, agencyId)`.
- Une séquence documentaire est unique par `(tenantId, organizationId, agencyId, documentType)`.
- `paddingWidth`  $\geq 1$  et `nextNumber`  $\geq 1$ .
- Une politique opérationnelle est unique par `(tenantId, organizationId, agencyId)`.
- La lecture agence peut retomber sur l'organisation, mais la ligne d'organisation n'est jamais fusionnée dynamiquement avec une seconde structure calculée intermédiaire.

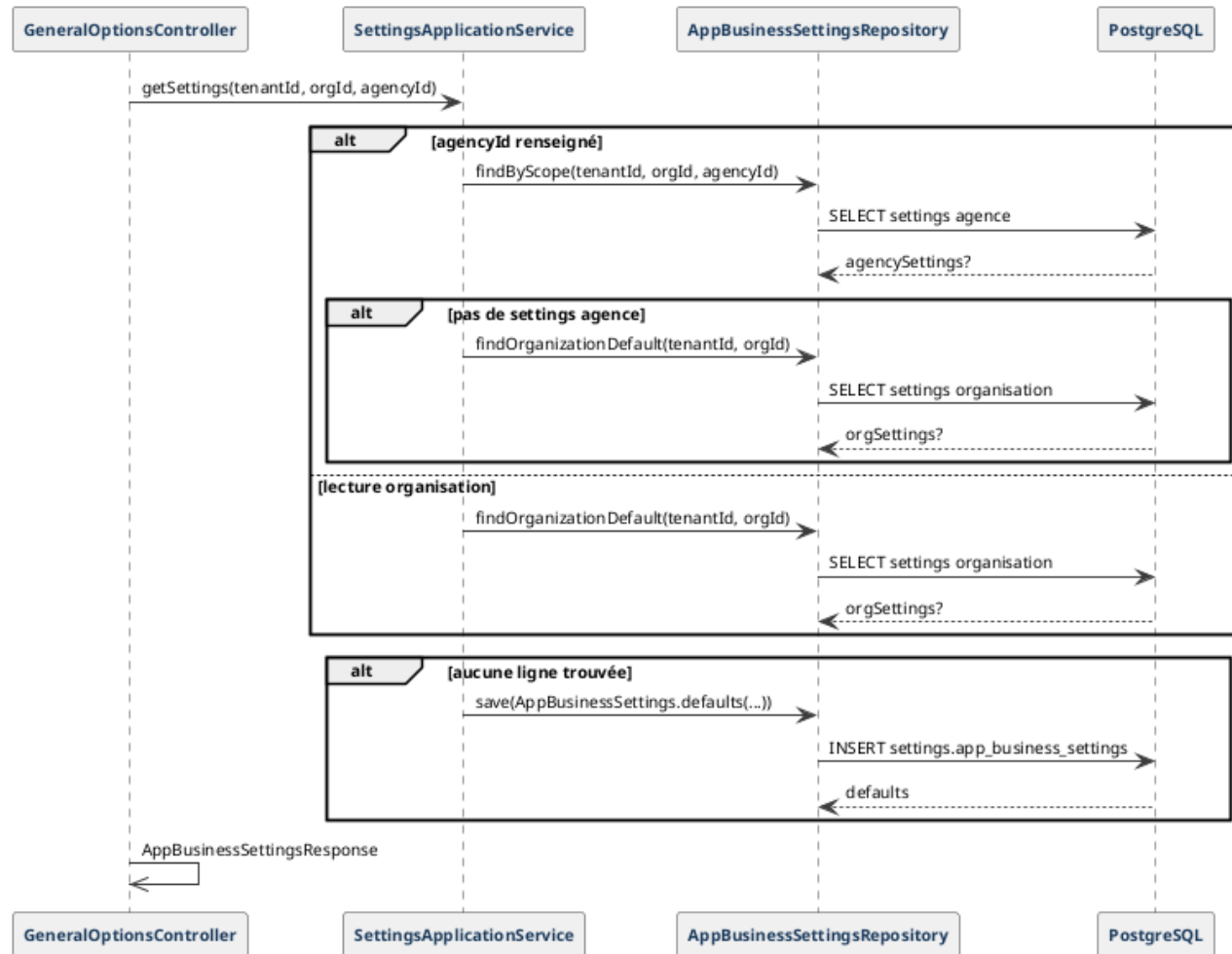
## Diagramme des cas d'utilisation — settings-core



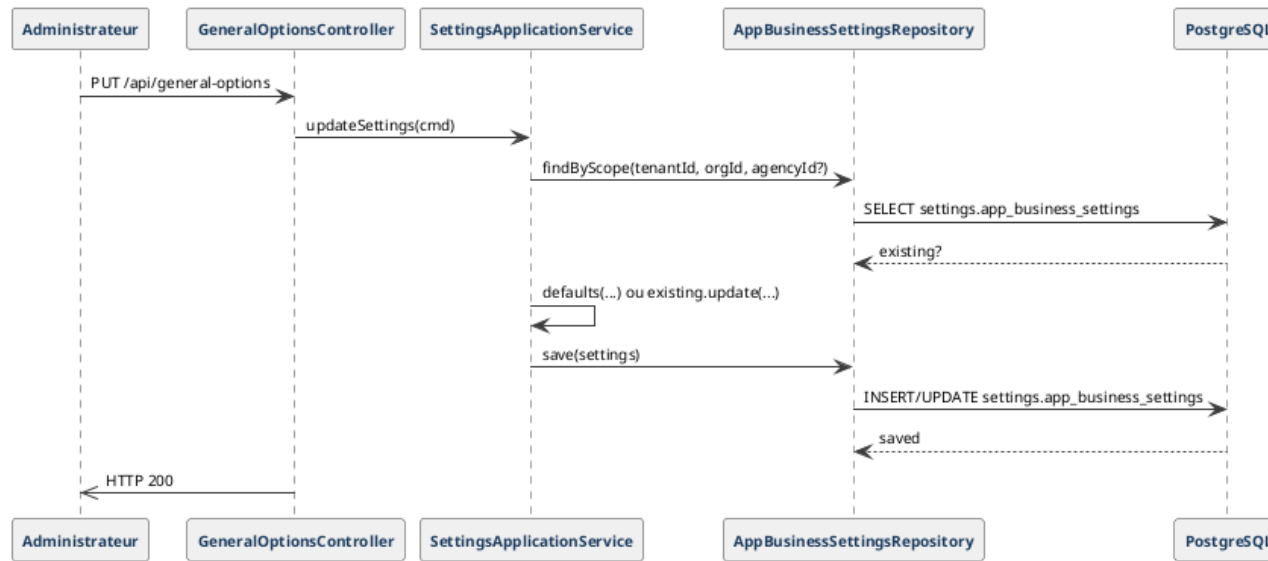
## DS-S-01 — Génération d'un numéro documentaire avec fallback agence → organisation



## DS-S-02 — Lecture des paramètres métier effectifs

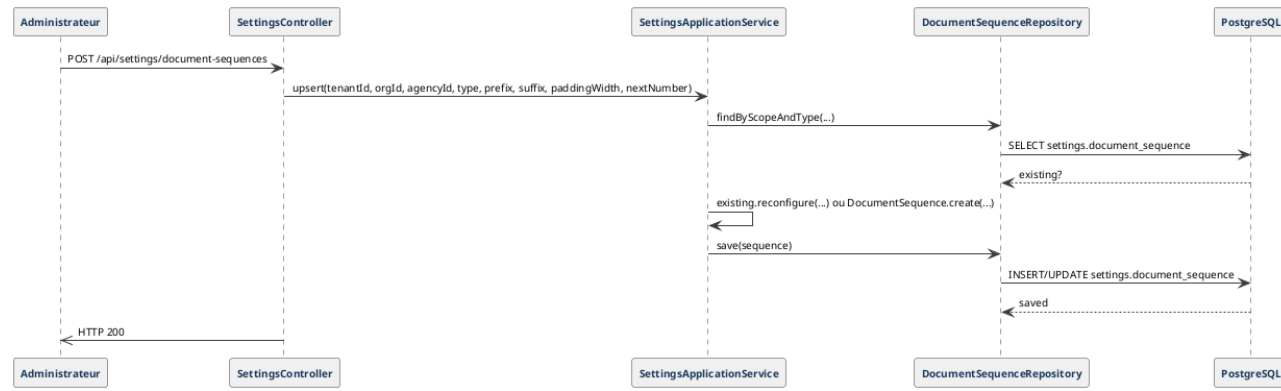


## DS-S-03 — Mise à jour des options générales d'une organisation

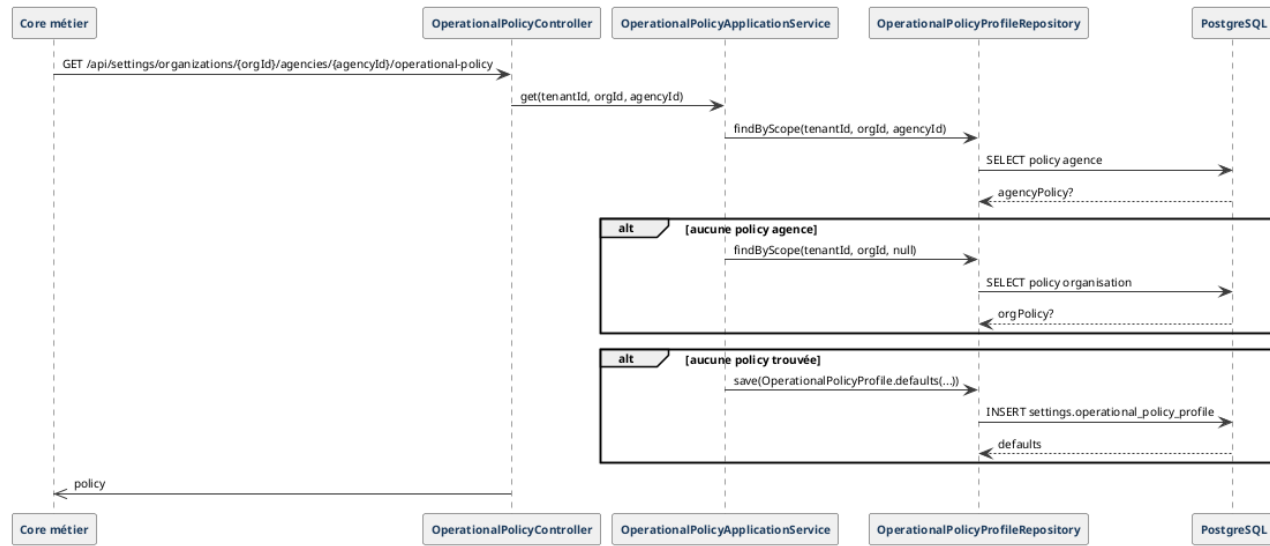




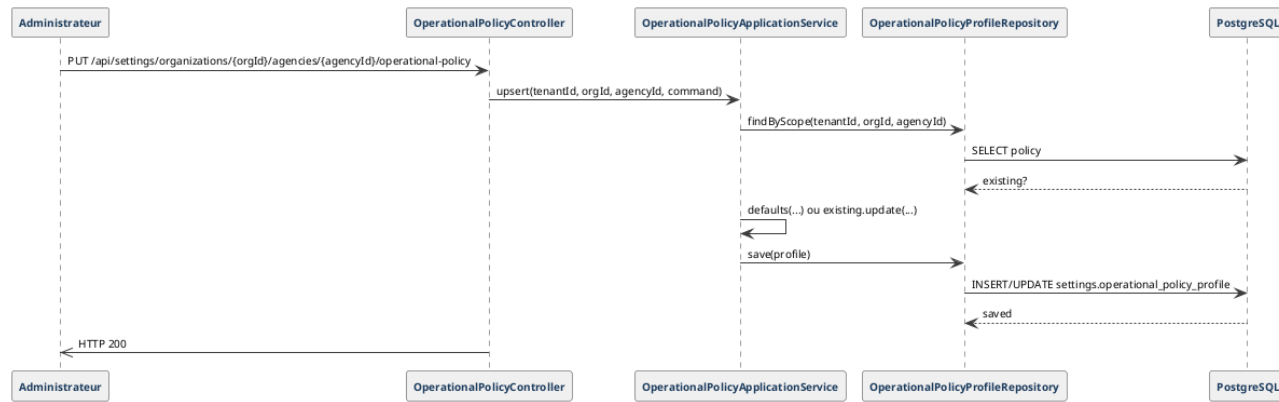
## DS-S-04 — Upsert d'une séquence documentaire d'agence



## DS-S-05 — Lecture de la politique opérationnelle d'agence avec repli organisation



## DS-S-06 — Mise à jour d'une politique opérationnelle d'agence



## 9 product-core — Catalogue Produits

---

### 9.1 Description du core

Le **product-core** est le référentiel de catalogue de la plateforme. Il gère la définition des produits, leur organisation en catégories hiérarchiques, les variantes (SKU), les lots de fabrication, la tarification datée, et les médias associés. Par conception, ce core ne gère **aucun stock** : les quantités disponibles relèvent exclusivement de l'inventory-core.

Le modèle supporte les cas d'usage industriels les plus courants : produits multi-variantes (taille, couleur, matière), fiches nutritionnelles pour les produits alimentaires, spécifications techniques, gestion multi-langue des descriptions.

### 9.2 Rôle principal

**Rôle principal** : Maintenir le catalogue des produits commercialisés, de leurs variantes ordonnables et de leur tarification, et fournir aux cores aval (ventes, stocks) des snapshots stables des données produit au moment des transactions.

### 9.3 Rôles secondaires

#### 1. Organisation en catégories hiérarchiques

Taxonomie arborescente de catégories avec support multi-langue (`CategoryI18n`). Les catégories peuvent être imbriquées à profondeur libre.

#### 2. Gestion des variantes (SKU)

Chaque produit peut avoir plusieurs variantes caractérisées par des attributs (taille, couleur). Chaque variante possède un code SKU unique et constitue l'unité orderable atomique.

#### 3. Traçabilité par lot (Batch)

Gestion des lots de fabrication avec numéro de lot, date de fabrication et date d'expiration. Essentiel pour les secteurs pharmaceutique et alimentaire.

#### 4. Tarification datée

Les prix sont versionnés avec une date d'effet. Le prix effectif d'une variante à une date donnée est le prix le plus récent dont la date d'effet est antérieure ou égale à cette date.

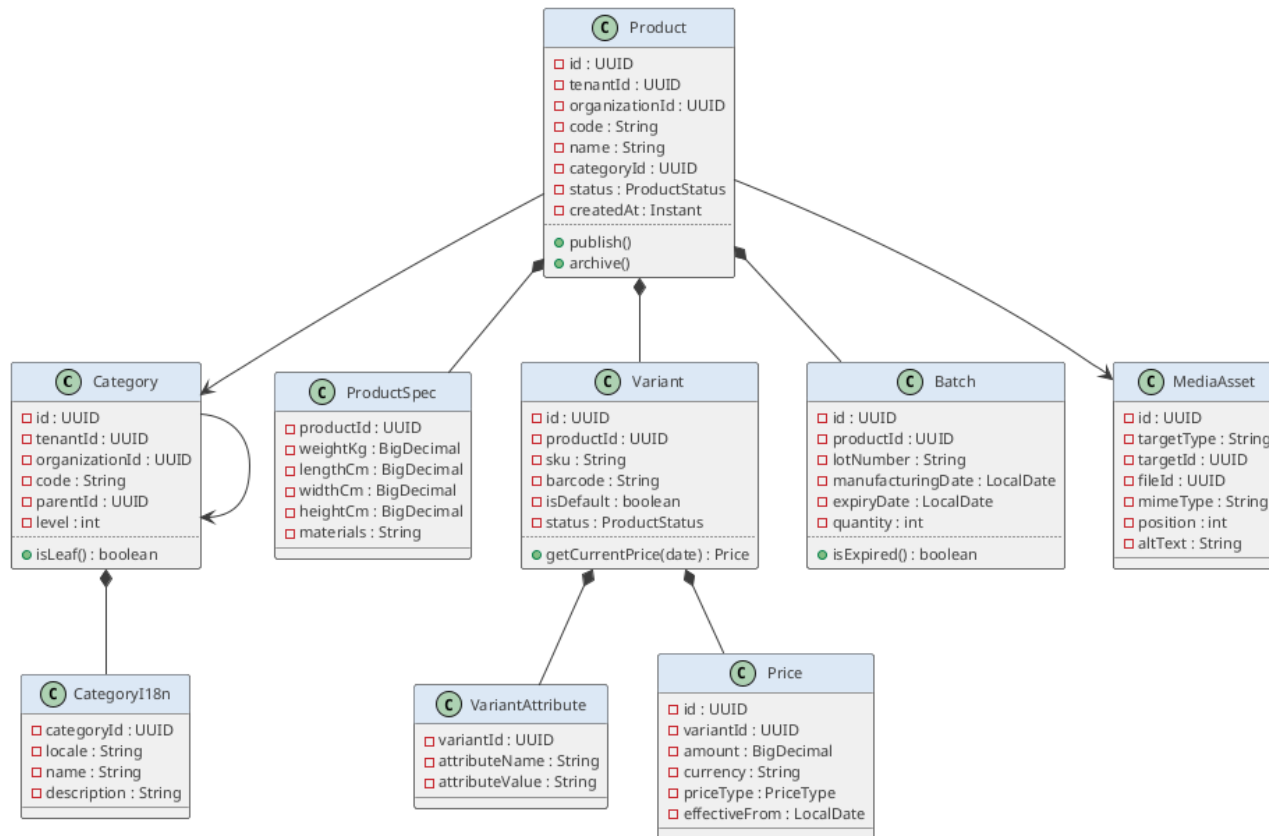
**5. Médias et assets**

Association d'images, fiches techniques, et autres documents à un produit ou une variante, avec gestion du type MIME et de la position d'affichage.

**6. Index de recherche Elasticsearch**

Projection des produits dans un index de recherche plein-texte pour les cas d'usage de catalogue avec filtres par famille, statut et attributs.

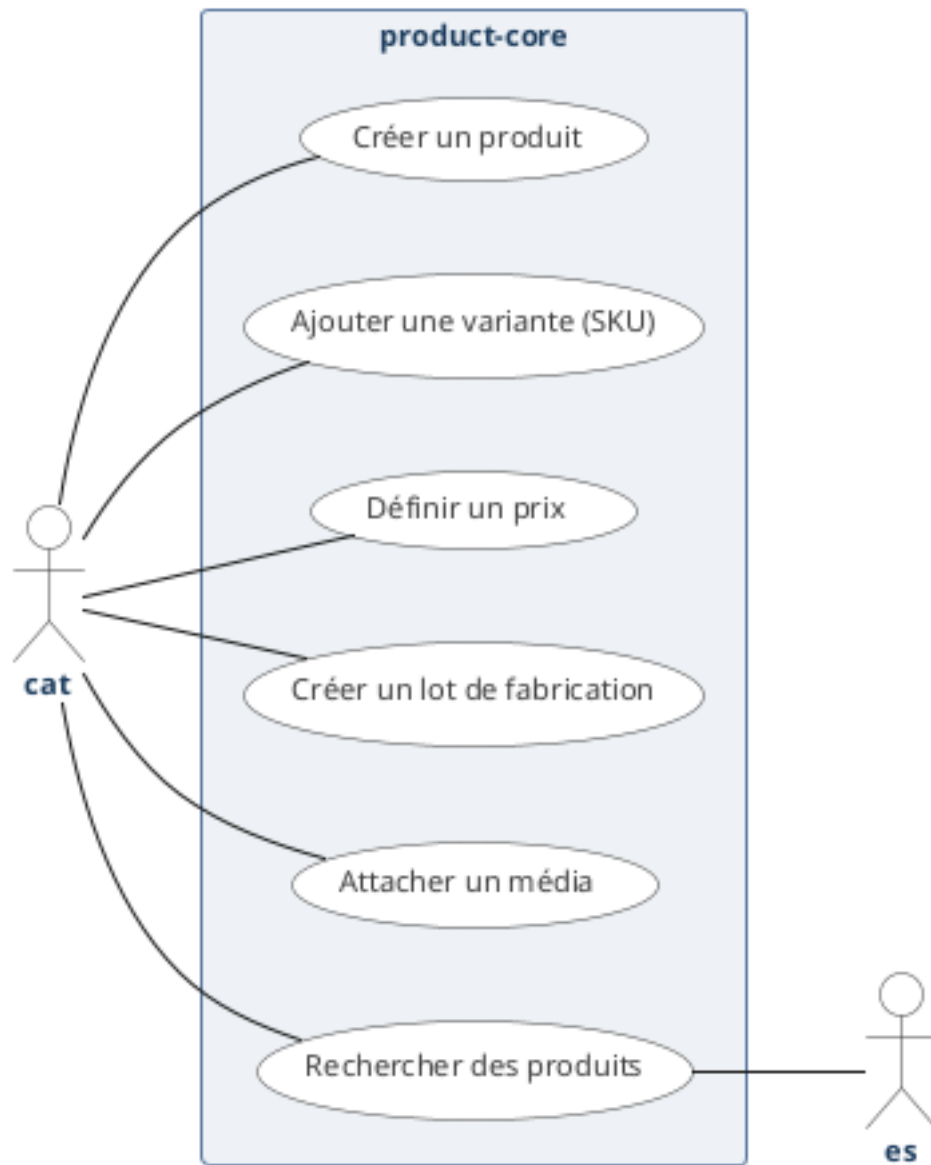
## Diagramme de classes — product-core



**Invariants du product-core :**

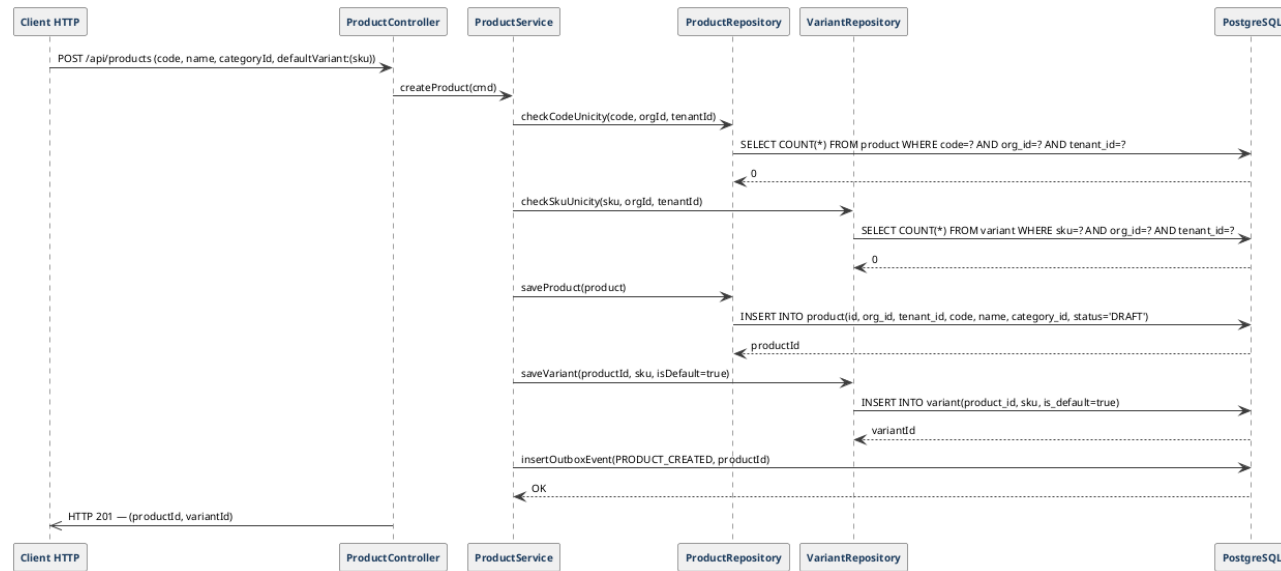
- Le code produit est unique par (tenantId, organizationId).
- Le SKU est unique par (tenantId, organizationId).
- Le numéro de lot est unique par produit.
- Chaque variante a exactement un prix effectif par type et par date d'évaluation.
- Le stock n'est **jamais** géré dans ce core.

## Diagramme des cas d'utilisation — product-core

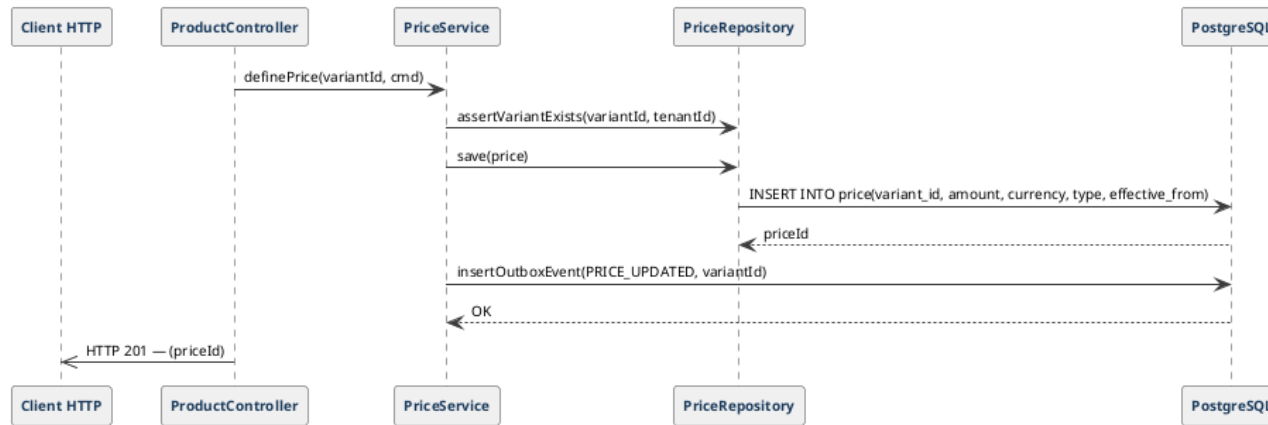




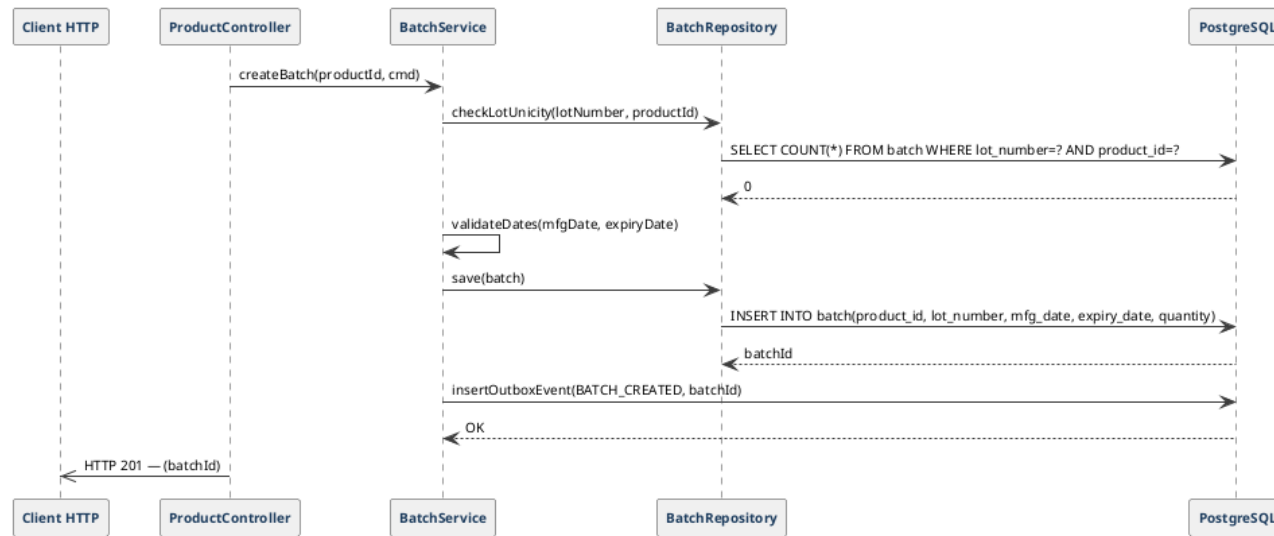
## DS-P-01 — Création d'un produit avec variante par défaut



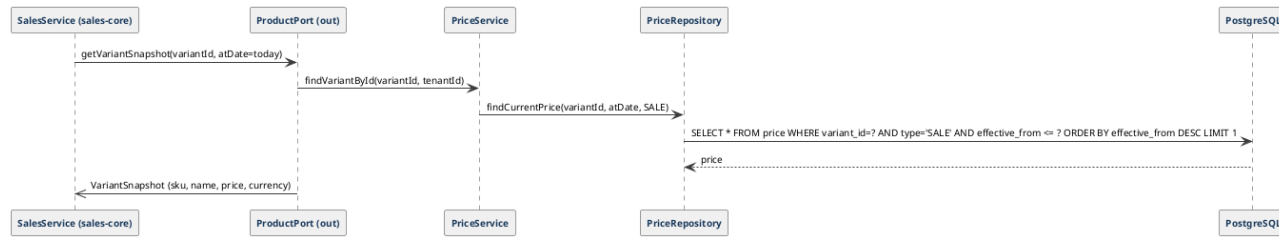
## DS-P-02 — Définition d'un prix pour une variante



## DS-P-03 — Création d'un lot de fabrication



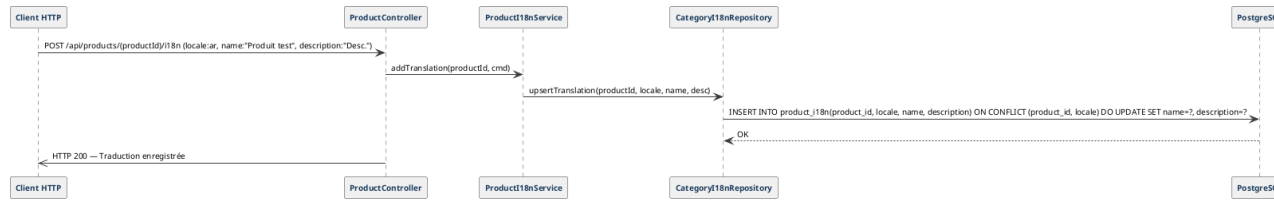
## DS-P-04 — Résolution du prix effectif d'une variante à une date



## DS-P-05 — Mise à jour de l'index Elasticsearch après modification produit



## DS-P-06 — Ajout d'une traduction multi-langue à un produit



# 10 inventory-core — Gestion des Stocks

---

*Kengfack Bernice*

## 10.1 Description du core

L'**inventory-core** est le responsable unique de la vérité métier du stock exploitable par produit et par agence. Dans l'implémentation actuelle, il ne maintient pas un agrégat persistant `stock_level` autonome ; le solde est recalculé à la demande à partir de trois sources canoniques : les **mouvements de stock**, les **transformations de produits** et les **transferts inter-entrepôts complétés**.

Le core expose les opérations d'enregistrement et de validation des mouvements, les sessions d'inventaire physique, les transformations et les transferts. Il reçoit aussi des sollicitations directes d'autres cores : par exemple, **sales-core** appelle explicitement le use case `DispatchSalesOrderUseCase` lors de la confirmation d'une commande, afin de créer les mouvements `OUTBOUND` correspondants. Le dispatch n'est donc plus documenté comme un consumer Kafka entrant.

Le core produit également des vues transverses de plus haut niveau : **operational-site**, **generalized-inventory** et **service-workspaces**, qui composent en lecture les données de **organization-core**, **resource-core**, **file-core** et **settings-core**.

## 10.2 Rôle Principal

Le rôle principal de **inventory-core** est la **gestion des stocks**. Il assure :

1. Le **traçage de tout mouvement de stock** : chaque entrée, sortie ou ajustement est enregistré avec un numéro de référence unique, un type, un document source éventuel et un statut.
2. Le **calcul du solde disponible en temps réel** pour chaque combinaison (organisation, agence, produit) en agrégeant les mouvements, transformations et transferts complétés.
3. La **garantie de cohérence métier des opérations** : références uniques, quantités strictement positives, validation des appartenances

agence/produit/organisation et contrôle des transitions d'état des transferts et sessions.

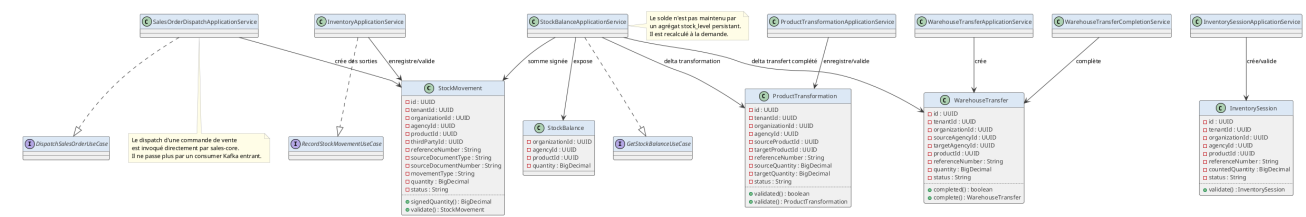
### 10.3 Rôles Secondaires

| Rôle                                   |                     | Description                                                                                                                                                                                                                                                                                        |
|----------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Transferts entrepôts</b>            | <b>inter-</b>       | Création et complétion de transferts de produits entre agences de type <code>WAREHOUSE</code> d'une même organisation. Un transfert suit le cycle <code>REQUESTED</code> → <code>COMPLETED</code> . Son effet sur le solde n'est pris en compte qu'à l'état <code>COMPLETED</code> .               |
| <b>Transformations de produits</b>     | <b>de</b>           | Enregistrement d'une conversion d'un produit source vers un produit cible avec quantités d'entrée et de sortie. Dans le modèle actuel, la transformation est persistée comme fait métier autonome et son impact est intégré au calcul du solde consolidé.                                          |
| <b>Sessions d'inventaire</b>           |                     | Enregistrement des comptages physiques de produits par agence. Une session capture une quantité comptée et peut ensuite être validée ; l'implémentation actuelle ne génère pas encore automatiquement un mouvement d'ajustement lors de cette validation.                                          |
| <b>Dispatch de commandes de vente</b>  | <b>de com-</b>      | Traitement direct d'une commande confirmée provenant de <code>sales-core</code> par appel explicite à <code>DispatchSalesOrderUseCase</code> . Chaque ligne produit un mouvement <code>OUTBOUND</code> après vérification de la disponibilité courante.                                            |
| <b>Publication d'événements métier</b> | <b>d'événements</b> | Émission d'événements tels que <code>STOCK_MOVEMENT_RECORDED</code> , <code>WAREHOUSE_TRANSFER_CREATED</code> , <code>WAREHOUSE_TRANSFER_COMPLETED</code> , <code>PRODUCT_TRANSFORMATION_RECORDED</code> ou <code>SALES_ORDER_STOCK_DISPATCHED</code> via le <code>BusinessEventPublisher</code> . |
| <b>Numérotation automatique</b>        | <b>auto-</b>        | Génération de numéros de référence uniques pour les mouvements, sessions, transformations et transferts en déléguant à <code>settings-core</code> .                                                                                                                                                |



| Rôle                                  | Description                                                                                                                                                                                                                                                                    |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vues opérationnelles composées</b> | Production d'une vue <code>OperationalSite</code> par agence ou entrepôt, agréant horaires, points d'intérêt, hiérarchie physique, portefeuille d'actifs, documents et résumé d'inventaire.                                                                                    |
| <b>Inventaire généralisé</b>          | Consolidation d'une lecture transverse combinant produits, mouvements, sessions, transformations, transferts, actifs matériels, espaces physiques et documents d'une organisation ou d'un site.                                                                                |
| <b>Workspaces de service</b>          | Calcul d'une <i>readiness</i> opérationnelle par service consommateur ( <code>BILLING</code> , <code>BANKING</code> , <code>ACCOUNTING</code> , <code>CASHIER</code> ) à partir des abonnements, agences, entrepôts, actifs, layout physique, catalogue et socle documentaire. |

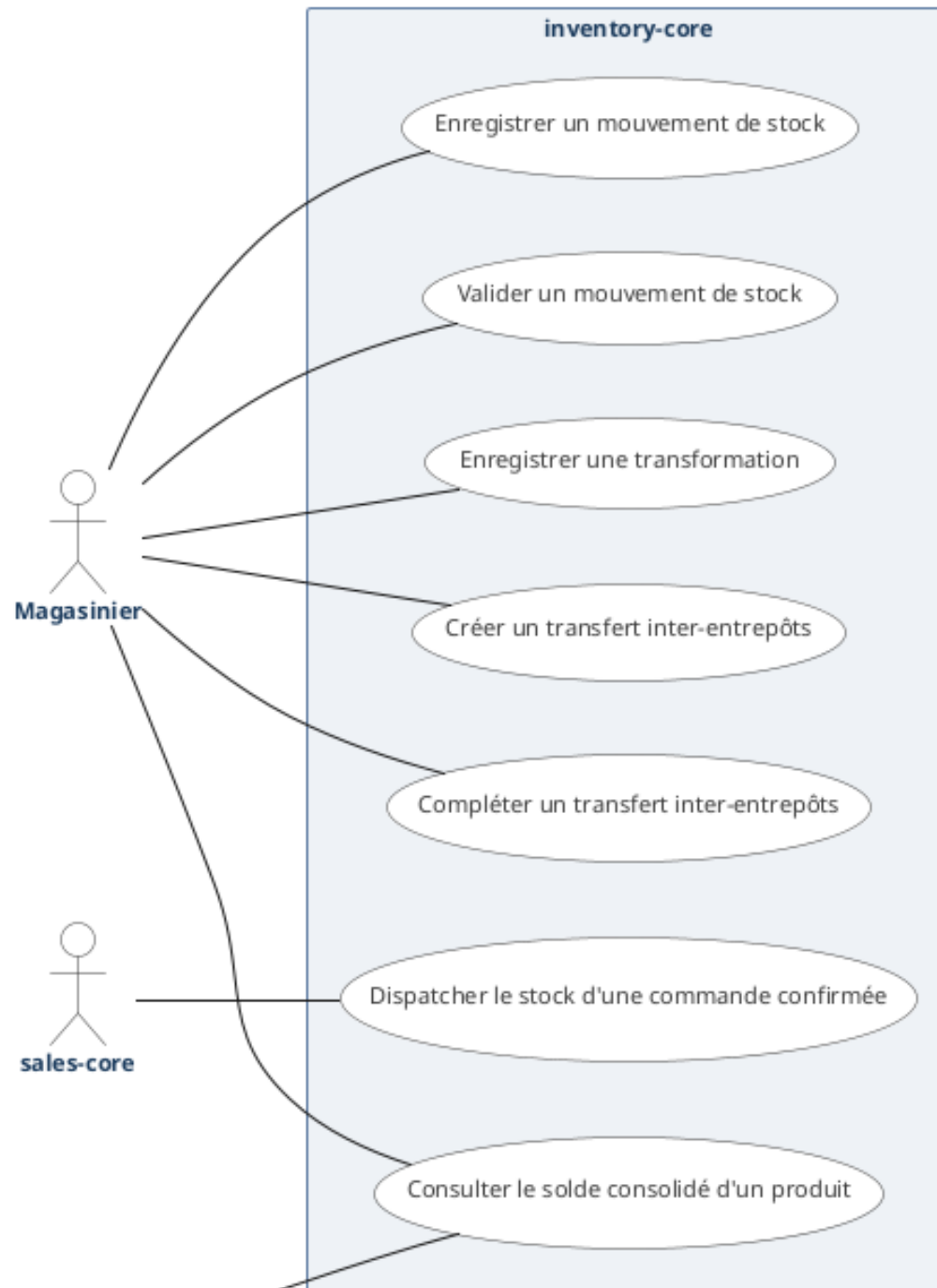
Diagramme de classe métier — inventory-core



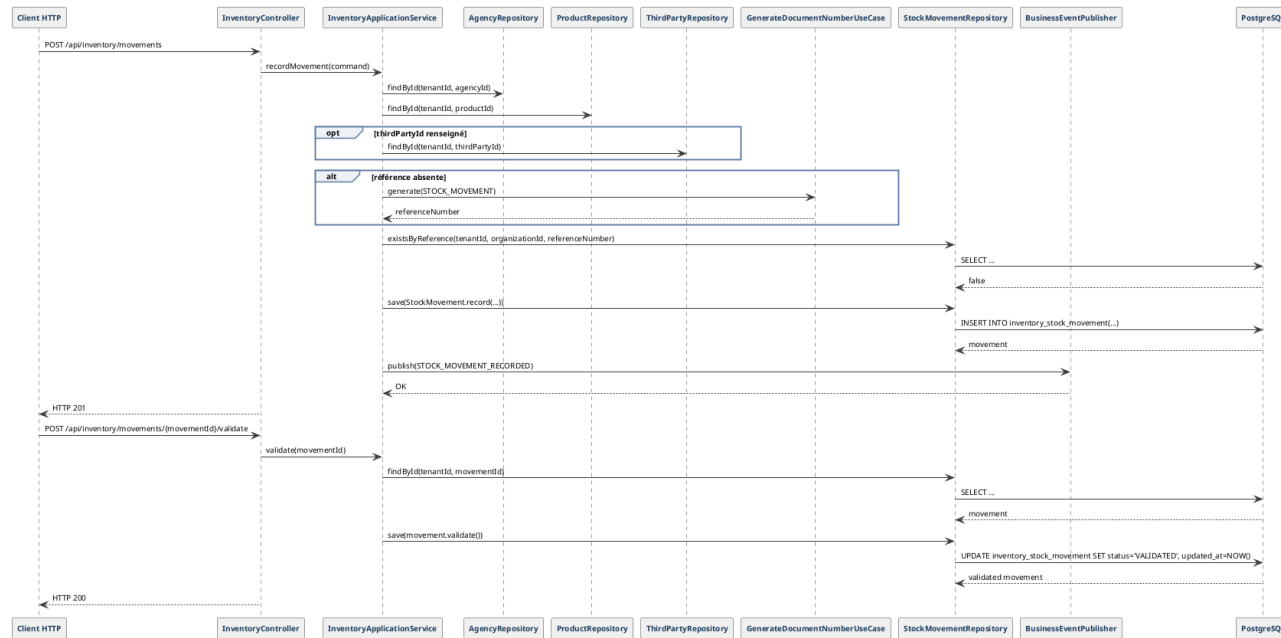
**Invariants du inventory-core :**

- `StockMovement.referenceNumber` est unique par `(tenantId, organizationId)`.
- Une quantité de mouvement, de transformation, de transfert ou de comptage doit être strictement positive.
- Un mouvement validé est immuable et conserve son document source.
- Une `InventorySession` n'accepte que les statuts `DRAFT` et `VALIDATED`.
- Une `ProductTransformation` n'accepte que les statuts `DRAFT` et `VALIDATED`, et les produits source et cible doivent être distincts.
- Un `WarehouseTransfer` n'accepte que les statuts `REQUESTED` et `COMPLETED`, avec agences source et cible distinctes.
- Le solde consulté pour un produit est dérivé de la somme des mouvements signés, du delta des transformations et des transferts complétés.

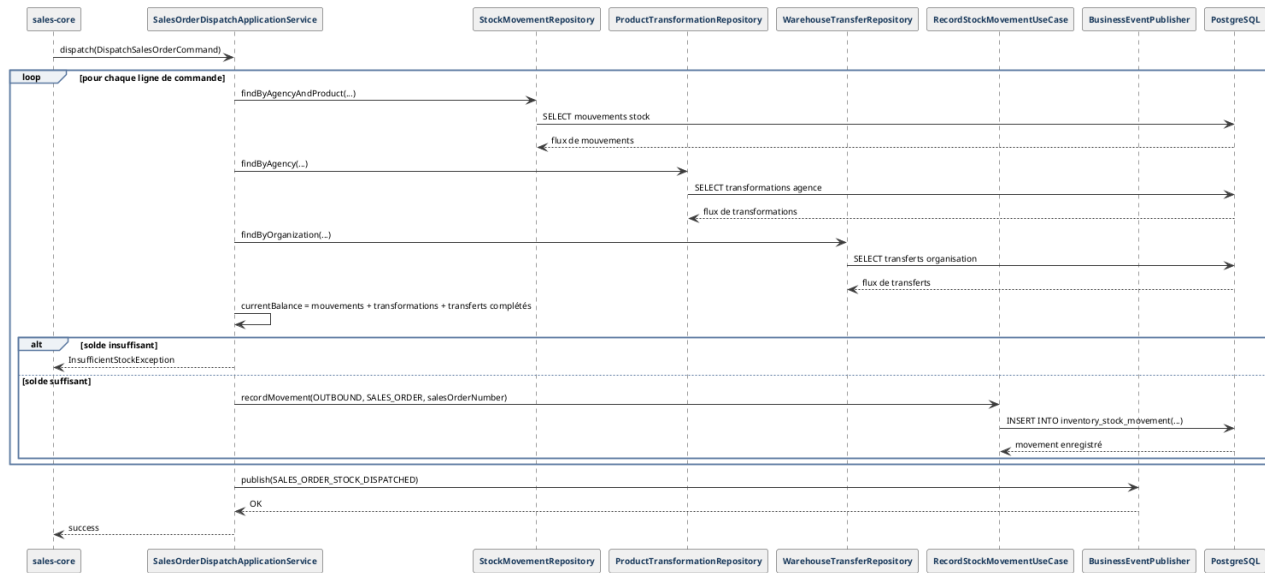
## Diagramme des cas d'utilisation — inventory-core



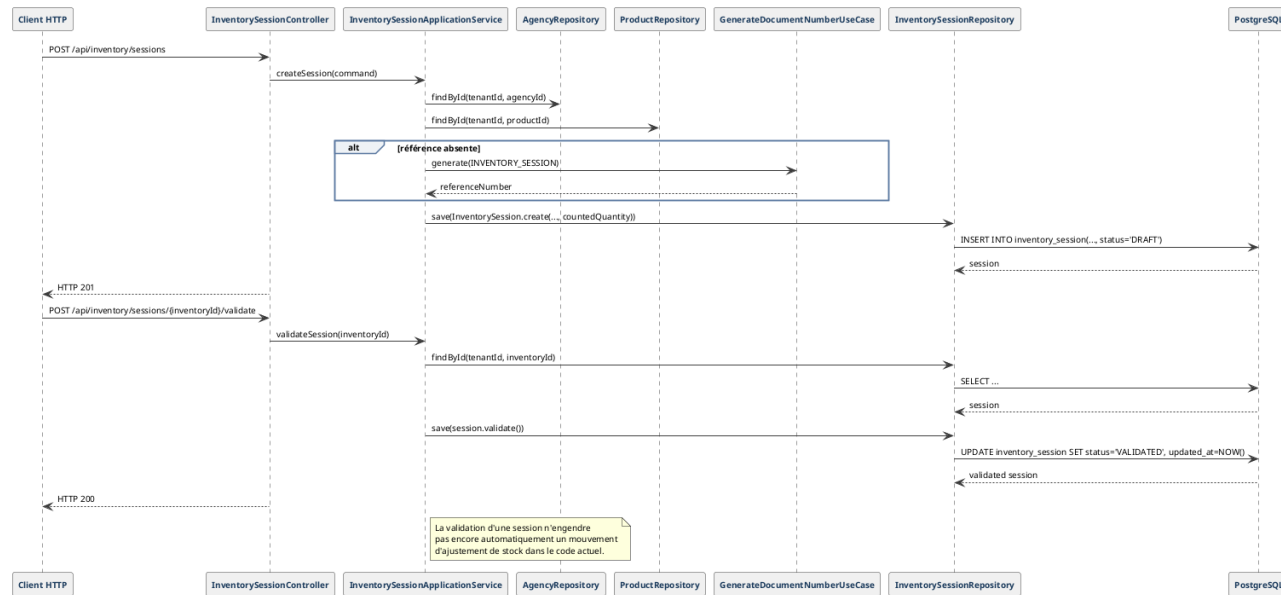
## DS-I-01 — Enregistrer puis valider un mouvement de stock



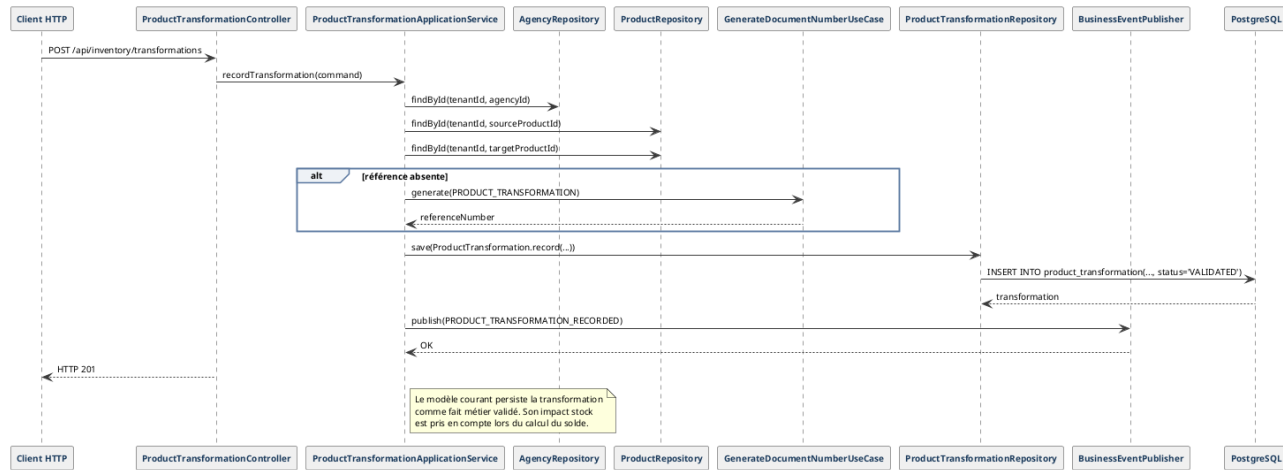
## DS-I-02 — Dispatcher directement le stock d'une commande confirmée



## DS-I-03 — Créer puis valider une session d'inventaire physique

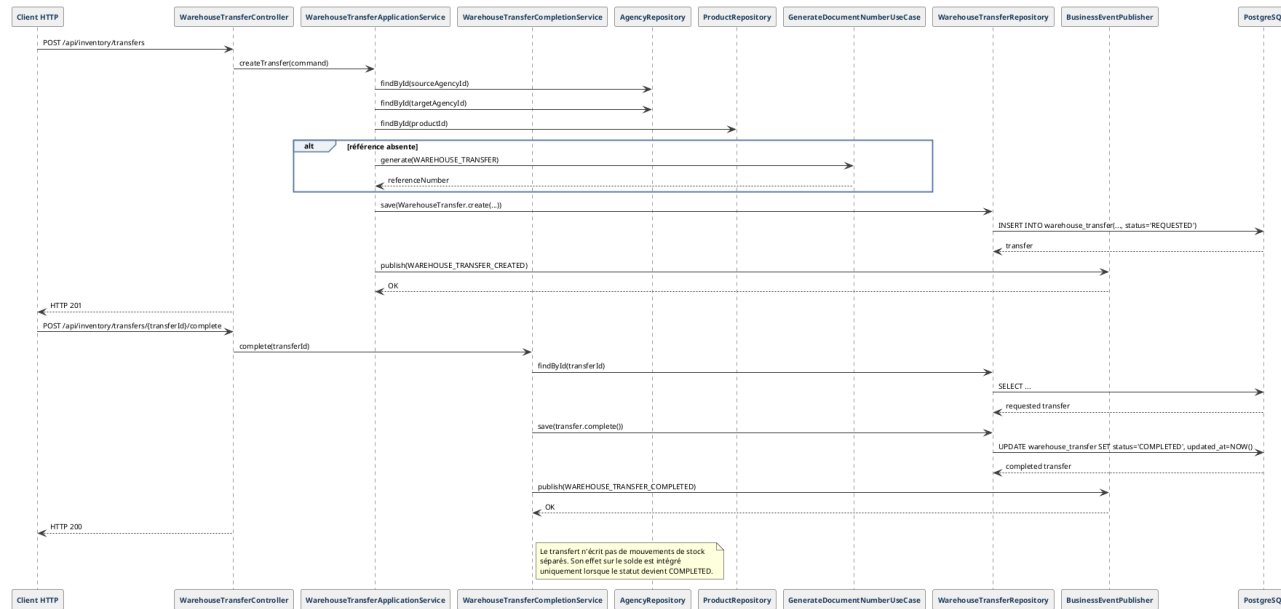


## DS-I-04 — Enregistrer une transformation de produits

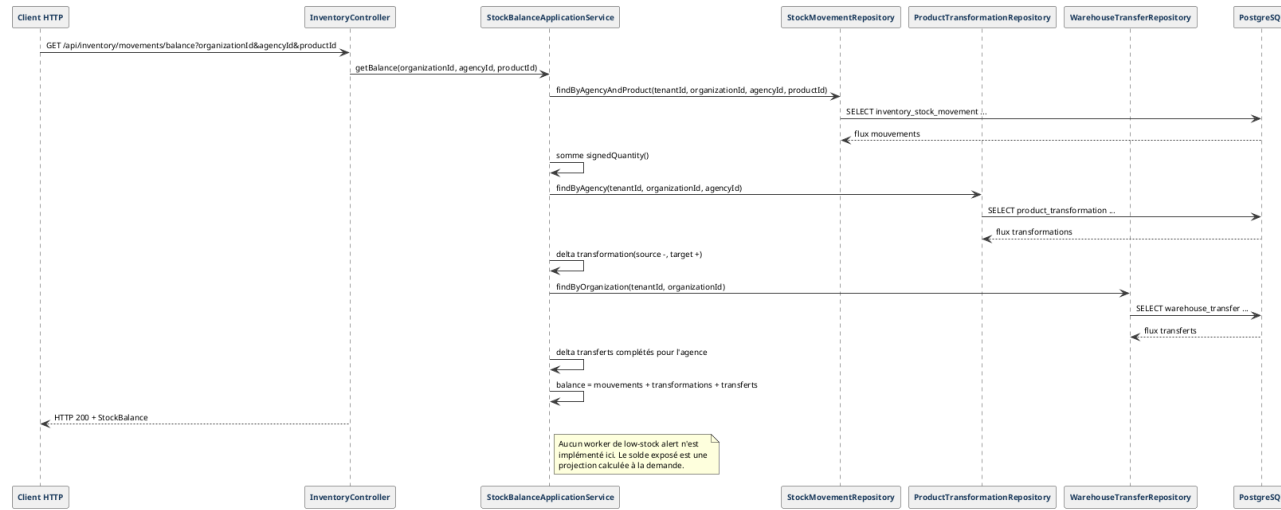




## DS-I-05 — Créer puis compléter un transfert inter-entrepôts



## DS-I-06 — Calculer le solde consolidé d'un produit



# 11 resource-core — Ressources Matérielles

---

## 11.1 Description du core

Le **resource-core** gère les actifs matériels internes de la plateforme : équipements, véhicules, dispositifs informatiques, outillage. Chaque ressource possède un cycle de vie complet — opérationnelle, en maintenance, mise au rebut — et peut être affectée à un acteur, une agence, une organisation ou un espace physique.

Ce core fournit également un historique de localisation GPS (utile pour les flottes de véhicules), la gestion des interfaces réseau (adresses MAC et IP), et le journal de maintenance permettant de suivre les interventions techniques. Il est aussi consommé comme socle de patrimoine opérationnel via les vues **asset-portfolio**, **operational-site** et **generalized-inventory**.

## 11.2 Rôle principal

**Rôle principal :** Gérer le cycle de vie complet des actifs matériels internes (création, affectation, maintenance, mise au rebut), maintenir leur localisation et leurs informations réseau, et assurer la traçabilité de chaque affectation dans un contexte multi-organisation.

## 11.3 Rôles secondaires

### 1. Affectation avec unicité d'affectation active

Une ressource ne peut avoir qu'une seule affectation active à la fois. L'affectation peut être à un acteur, une agence, une organisation ou un **PhysicalSpace**. Toutes les affectations historiques sont conservées pour la traçabilité.

### 2. Patrimoine physique de site

Une ressource peut être réservée ou affectée à un espace physique hiérarchique (bâtiment, étage, salle, poste, siège). Le core devient ainsi le support du parc matériel des agences, entrepôts, caisses et postes.

**3. Cycle de maintenance**

Transition d'état contrôlée vers `IN_MAINTENANCE`, empêchant toute nouvelle affectation standard. À la fin de la maintenance, la ressource repasse à `OPERATIONAL`.

**4. Traçabilité GPS**

Enregistrement horodaté des positions géographiques permettant de reconstituer l'historique de déplacement d'un actif (flottes de véhicules, équipements mobiles).

**5. Interfaces réseau**

Association d'adresses MAC et IP à une ressource, avec notion d'interface principale — utile pour les inventaires réseau.

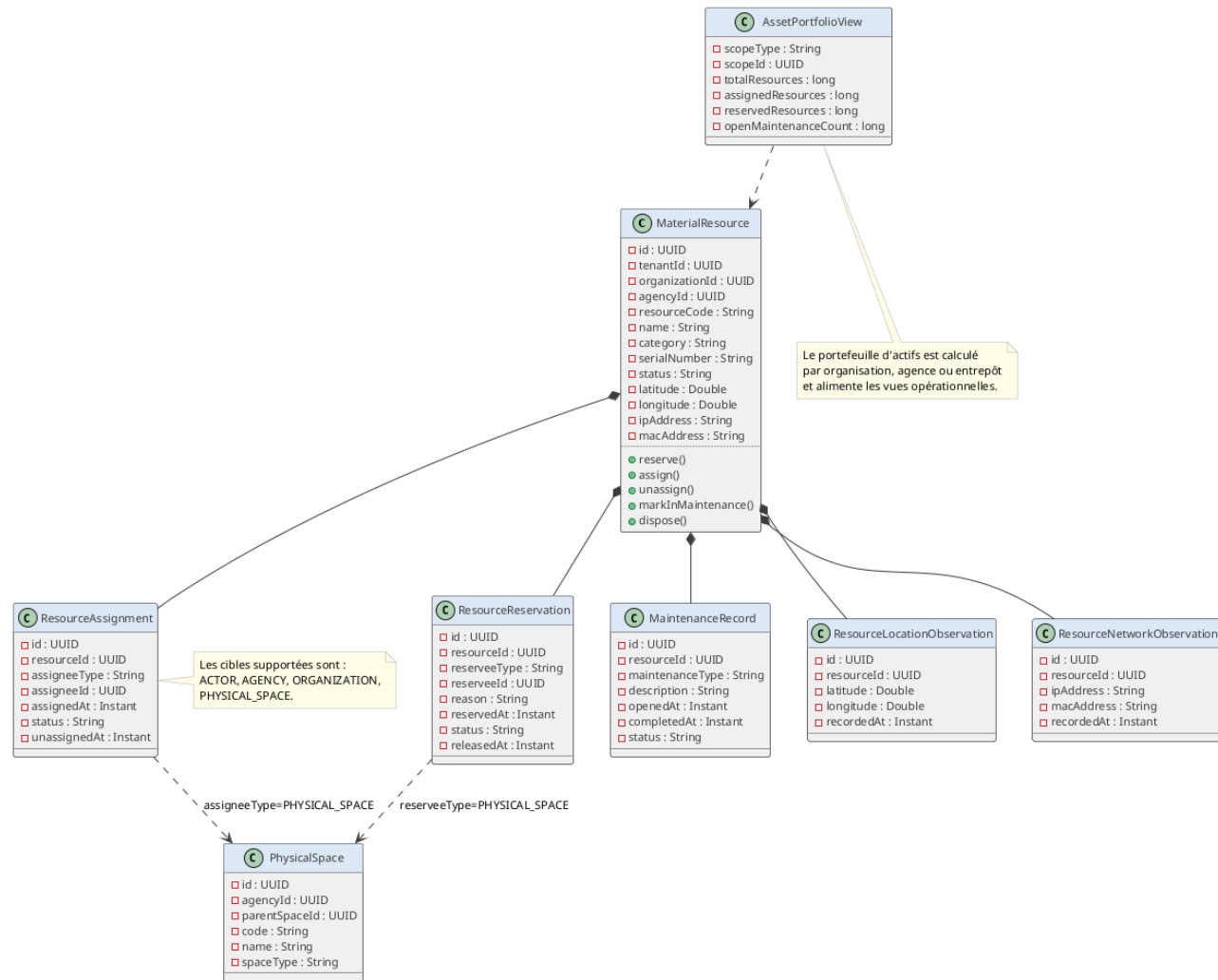
**6. Recherche par critères**

Recherche de ressources par catégorie, statut, agence d'affectation ou terme libre, avec projection Elasticsearch optionnelle.

**7. Mise au rebut irréversible**

L'état `DISPOSED` est terminal. Aucune nouvelle affectation ni maintenance ne peut être enregistrée pour une ressource mise au rebut.

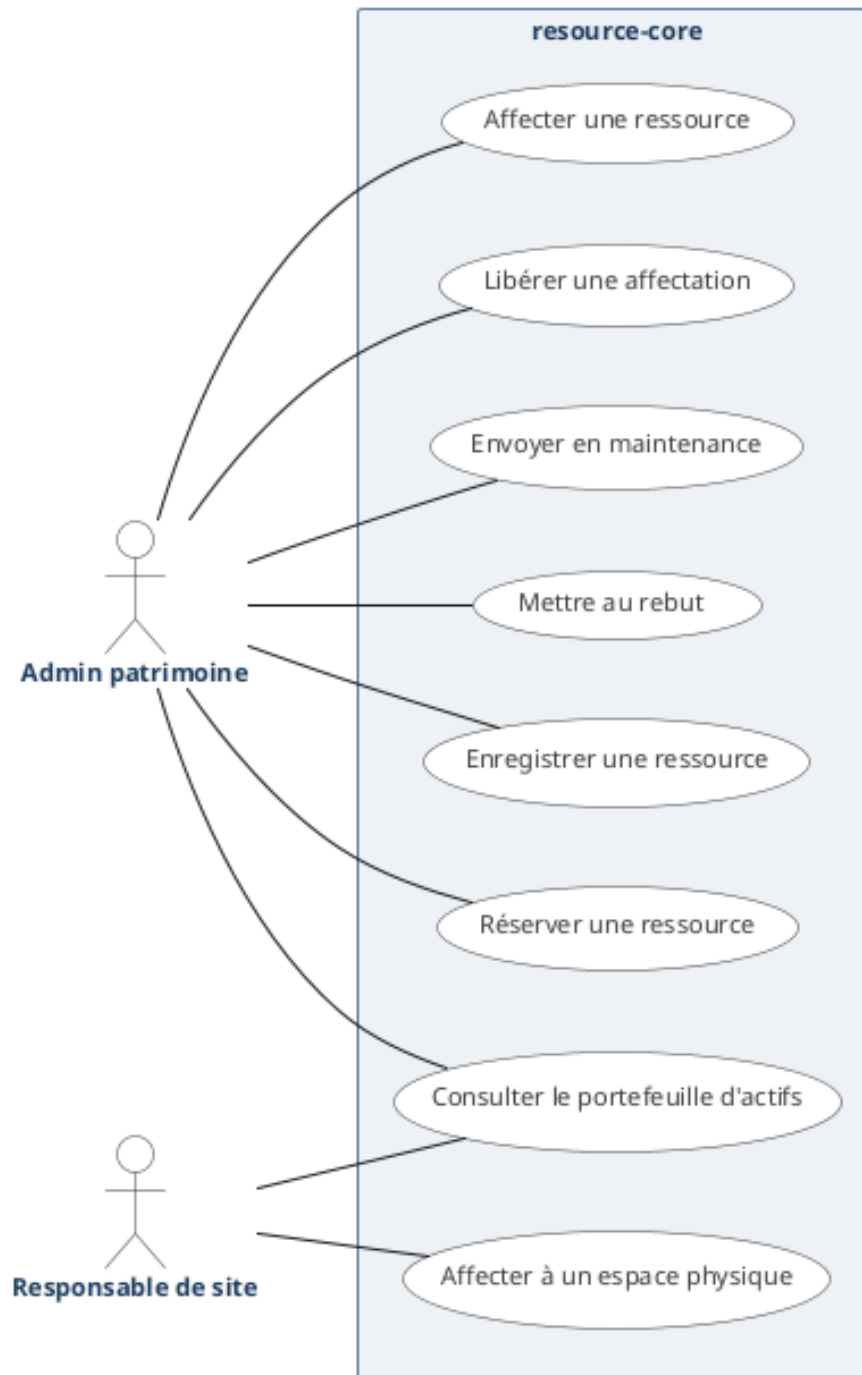
## Diagramme de classes — resource-core



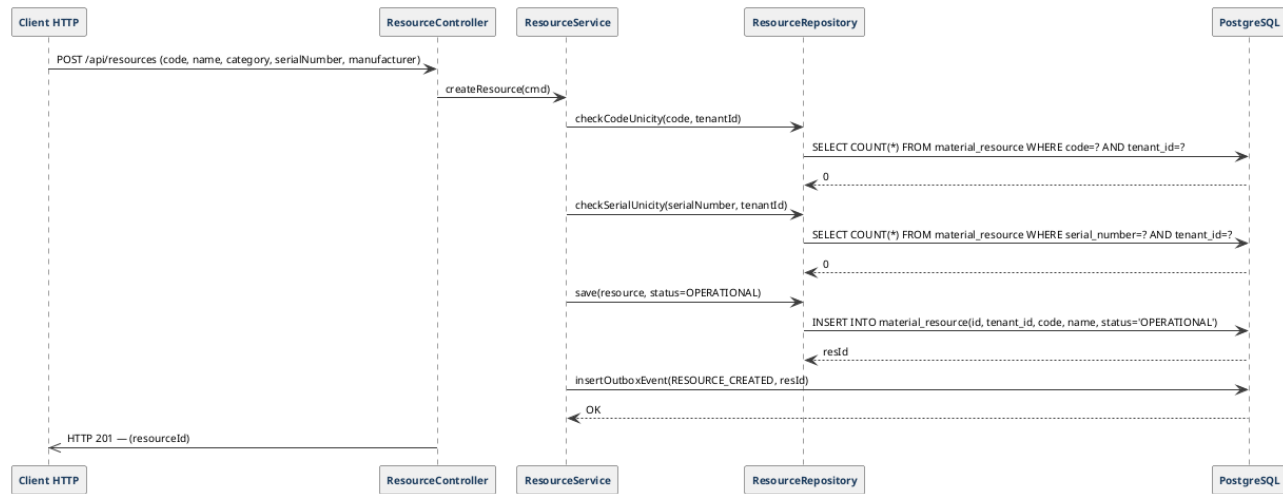
**Invariants du resource-core :**

- Le code d'une ressource est unique par tenant.
- Une seule affectation active par ressource à tout moment.
- Une ressource `DISPOSED` ne peut recevoir de nouvelles affectations.
- Une ressource `IN_MAINTENANCE` ne peut être affectée standard.
- Les cibles d'affectation et de réservation sont limitées à `ACTOR`, `AGENCY`, `ORGANIZATION` et `PHYSICAL_SPACE`.

## Diagramme des cas d'utilisation — resource-core

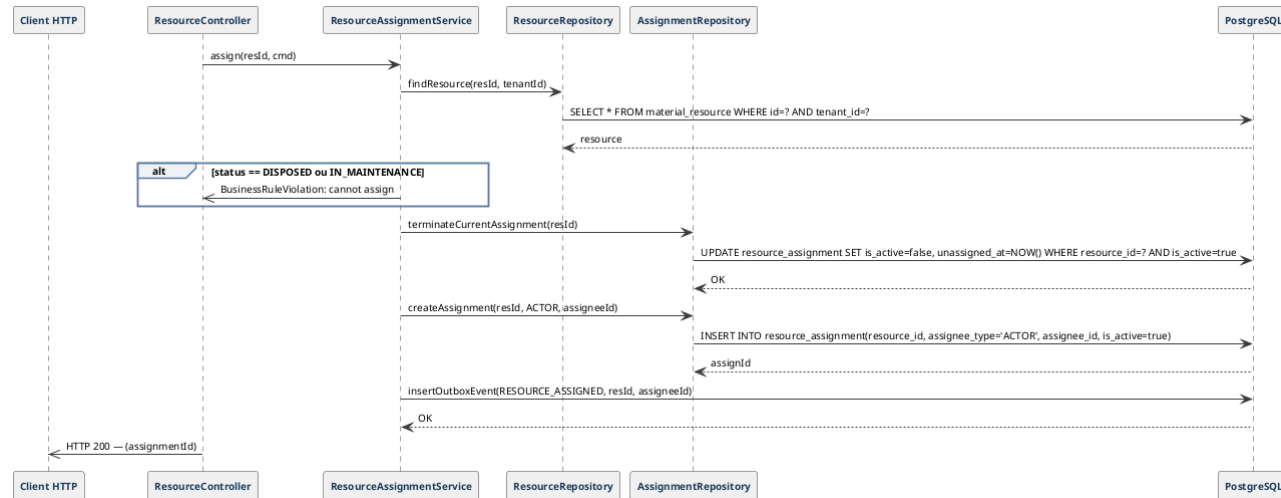


## DS-RE-01 — Enregistrement d'une ressource matérielle

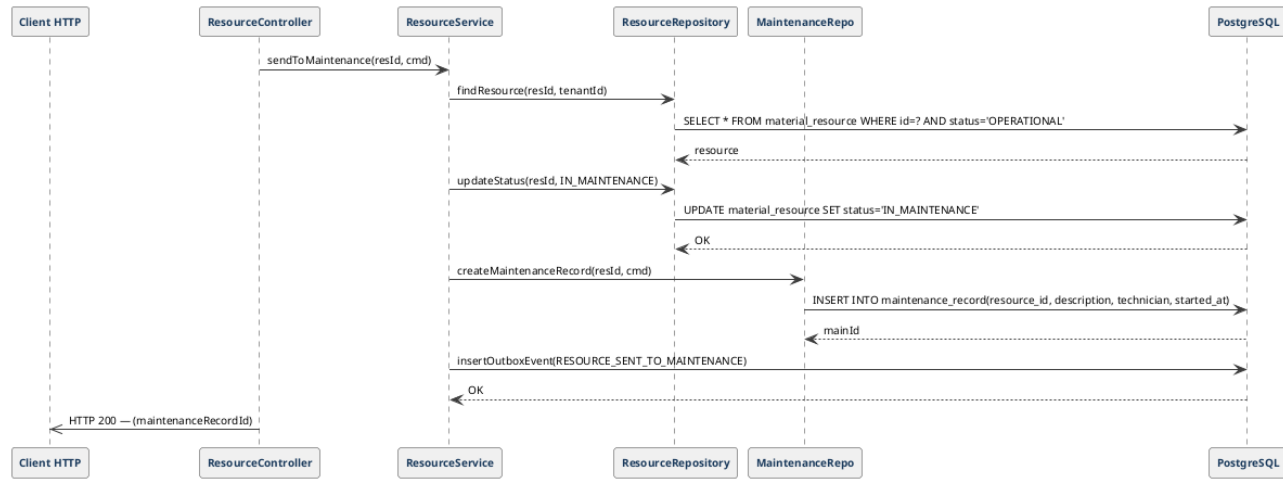




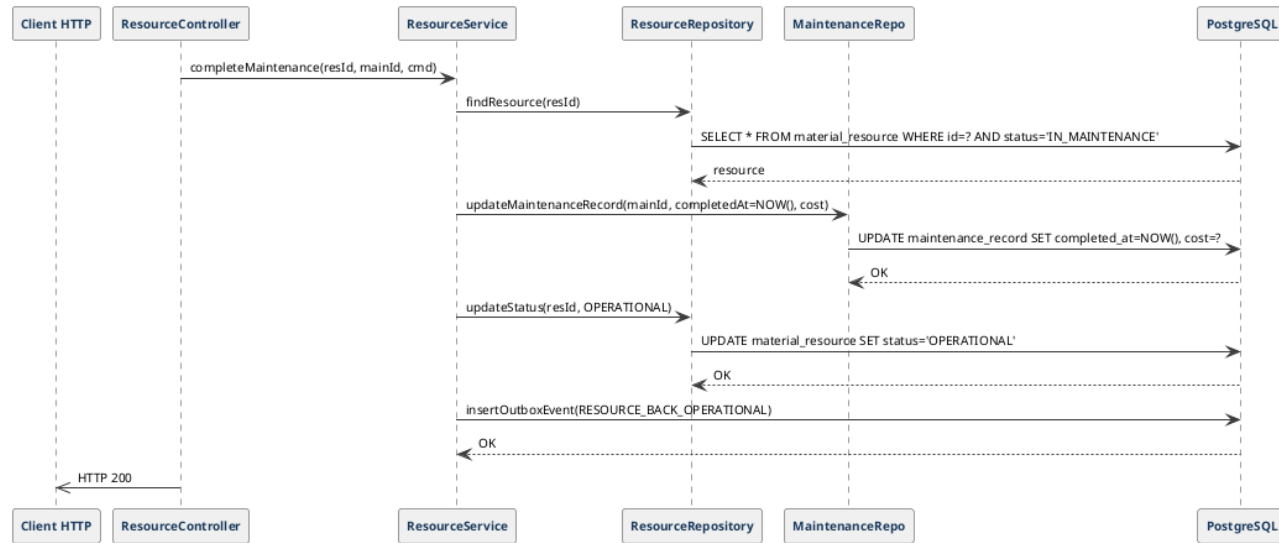
## DS-RE-02 — Affectation d'une ressource à un acteur



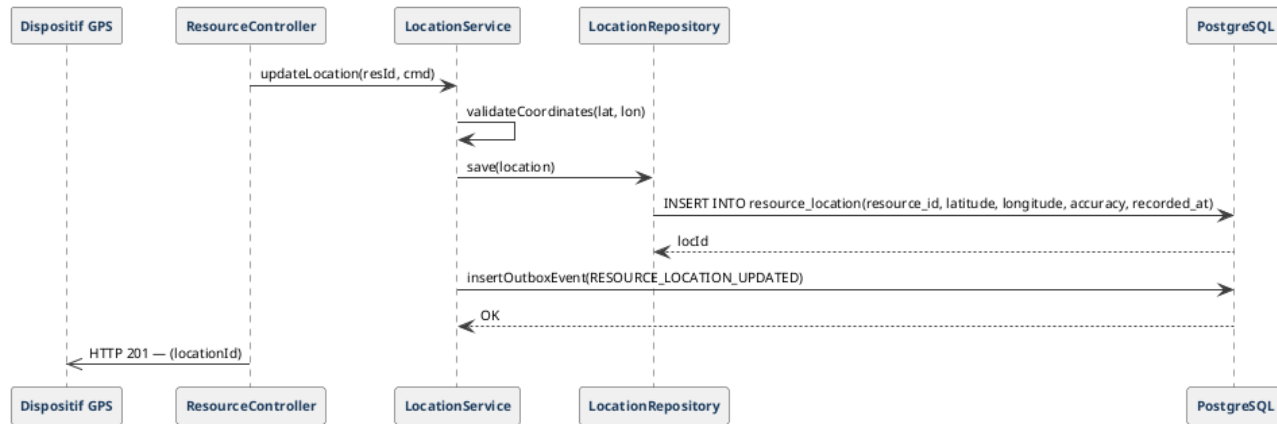
## DS-RE-03 — Envoi d'une ressource en maintenance



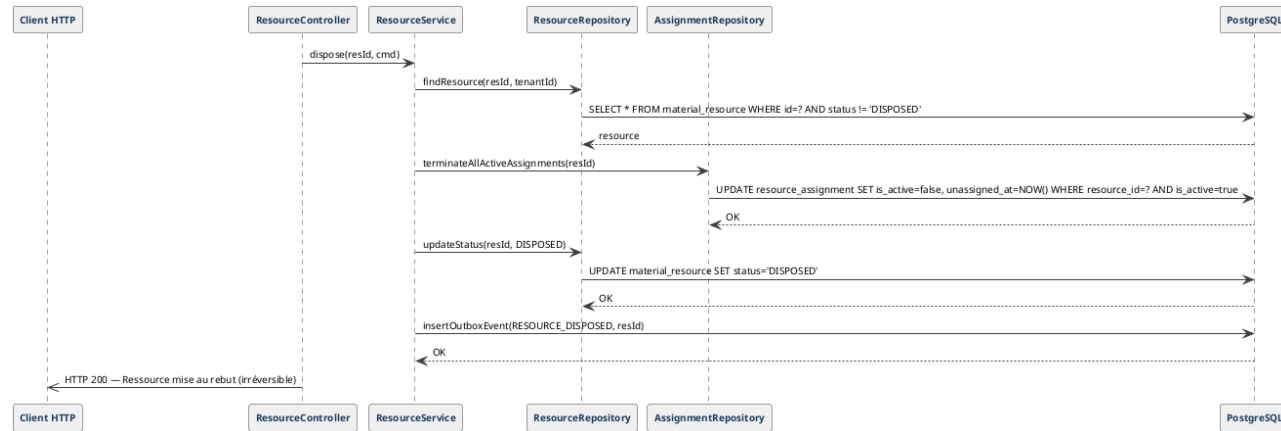
## DS-RE-04 — Retour de maintenance vers l'état opérationnel



## DS-RE-05 — Mise à jour de la localisation GPS d'une ressource



## DS-RE-06 — Mise au rebut irréversible d'une ressource



**Exploitation transverse** Le resource-core expose maintenant des lectures par cible métier (ACTOR, AGENCY, ORGANIZATION, PHYSICAL\_SPACE) et alimente les vues composées du noyau sur le patrimoine, les sites opérationnels et l’inventaire généralisé.

## 12 sales-core

# Commandes de Vente

---

### 12.1 Description du core

Le **sales-core** gère aujourd'hui le cycle de vie canonique des commandes de vente depuis le brouillon jusqu'à la confirmation. Il crée et met à jour les commandes, valide les références métier (agence, client, produits), publie l'événement de création, et surtout déclenche **directement** le **dispatch** de stock au moment de la confirmation via le gateway **SalesOrderStockDispatchGateway**. La confirmation n'est donc pas implémentée comme une chaîne asynchrone où l'**inventory-core** consommerait un événement pour décider ensuite de l'allocation; le dispatch de stock fait partie du flux de confirmation lui-même.

Le **sales-core** ne crée pas lui-même de facture et ne maintient plus dans son propre domaine de transition dédiée de préparation ou de clôture de facturation. Une fois la commande confirmée, l'**accounting-core** peut, si l'appelant le décide explicitement, récupérer un snapshot confirmé via le provider **ConfirmedSalesOrderProvider** et créer une facture canonique à partir de cette commande.

### 12.2 Rôle principal

**Rôle principal** : Gérer les commandes de vente canoniques, sécuriser leur confirmation par un dispatch de stock direct et explicite, et fournir une base confirmée exploitable ensuite par les autres services, notamment l'**accounting-core** pour la facturation explicite.

### 12.3 Rôles secondaires

#### 1. Création et mise à jour de brouillons

Les commandes sont créées en **DRAFT**. La mise à jour n'est permise que tant que la commande reste dans cet état.

## 2. Validation métier des références

Le core vérifie la cohérence des références agence, client et produits avant de persister la commande.

## 3. Numérotation documentaire

Le numéro de commande est généré par le `settings-core` au moment de la création si l'appelant n'en fournit pas un explicitement.

## 4. Confirmation avec dispatch direct du stock

La confirmation charge la commande, appelle le gateway de dispatch vers l'`inventory-core`, puis seulement après passe la commande à `CONFIRMED` et publie l'événement `SALES_ORDER_CONFIRMED`.

## 5. Remontée explicite des insuffisances de stock

Une insuffisance de stock côté `inventory` est remontée sous forme d'exception métier `InsufficientStockForSalesOrderException` vers le flux de confirmation.

## 6. Annulation limitée aux brouillons

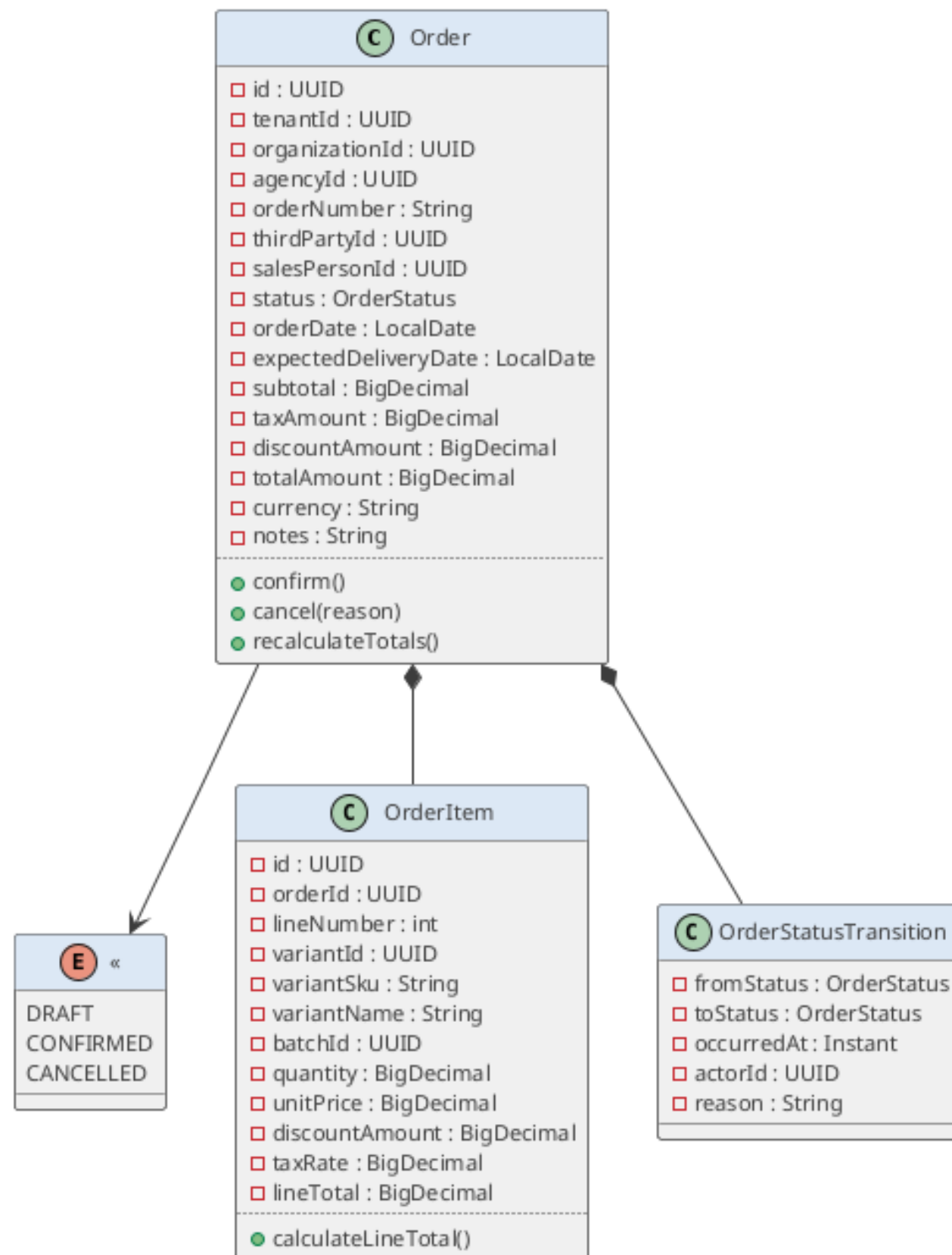
Le domaine courant n'autorise l'annulation que depuis `DRAFT`. Il n'existe pas dans le code actuel de scénario canonique d'annulation d'une commande déjà confirmée dans ce core.

## 7. Handoff explicite vers la comptabilité

Le `sales-core` ne pousse pas lui-même une transition métier `READY_FOR_INVOICING`. Le handoff vers la comptabilité se fait indirectement : l'`accounting-core` lit une commande `CONFIRMED` via le provider `ConfirmedSalesOrderProvider` quand on appelle explicitement `/api/accounting/invoices/from-orders/orderId`.



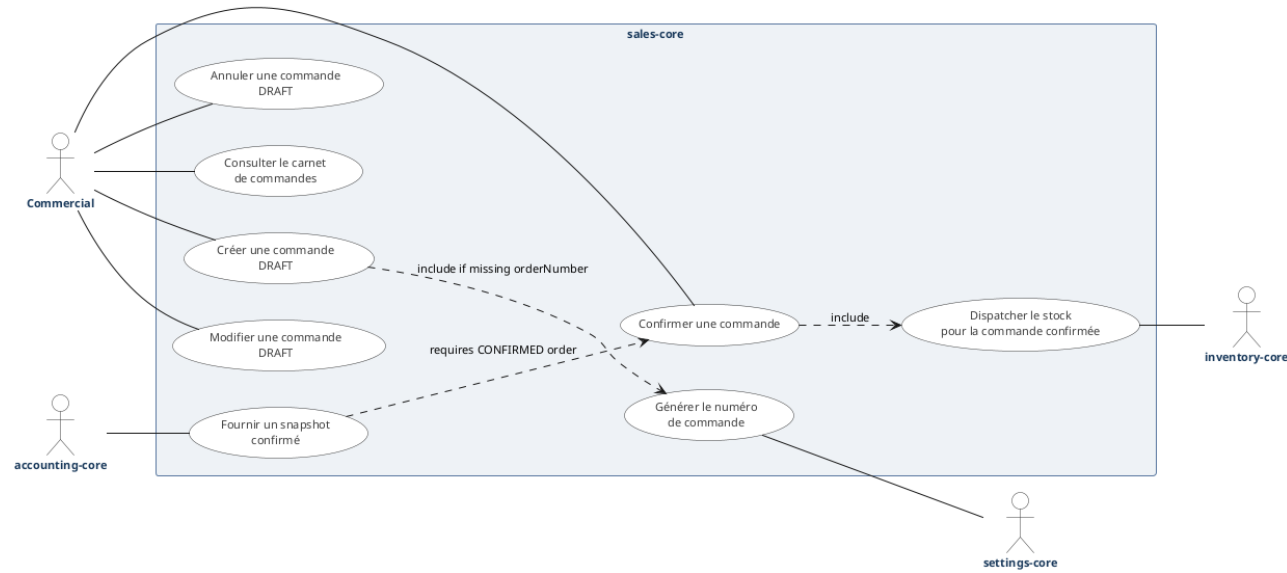
## Diagramme de classes — sales-core



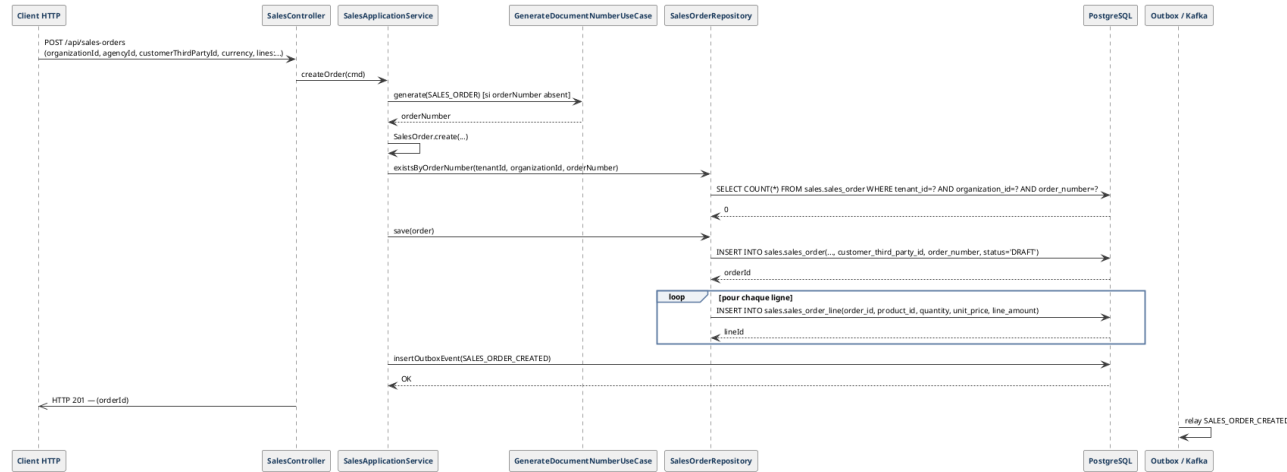
**Invariants du sales-core :**

- Le numéro de commande est unique par (tenantId, organizationId).
- Une commande **CONFIRMED** ou **CANCELLED** est immuable.
- Le numéro de ligne est unique au sein d'une commande.
- `totalAmount` = `subtotalAmount` dans le domaine canonique courant.
- La confirmation requiert le succès du dispatch de stock sur chaque ligne.
- L'annulation canonique n'est autorisée que depuis **DRAFT**.

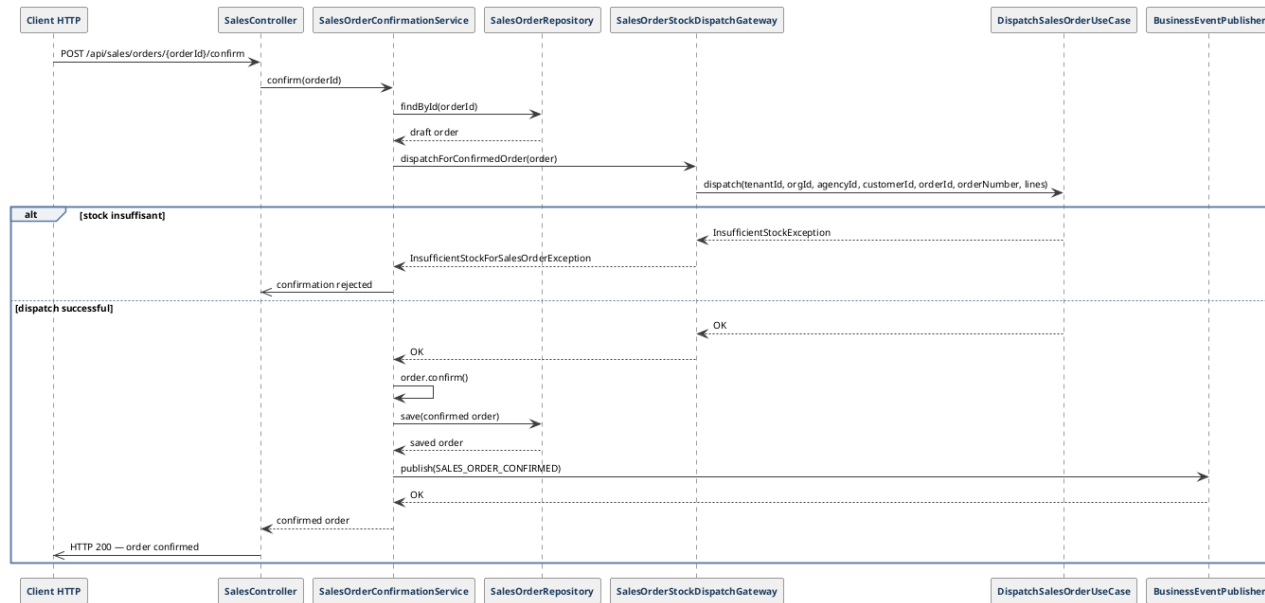
## Diagramme des cas d'utilisation — sales-core



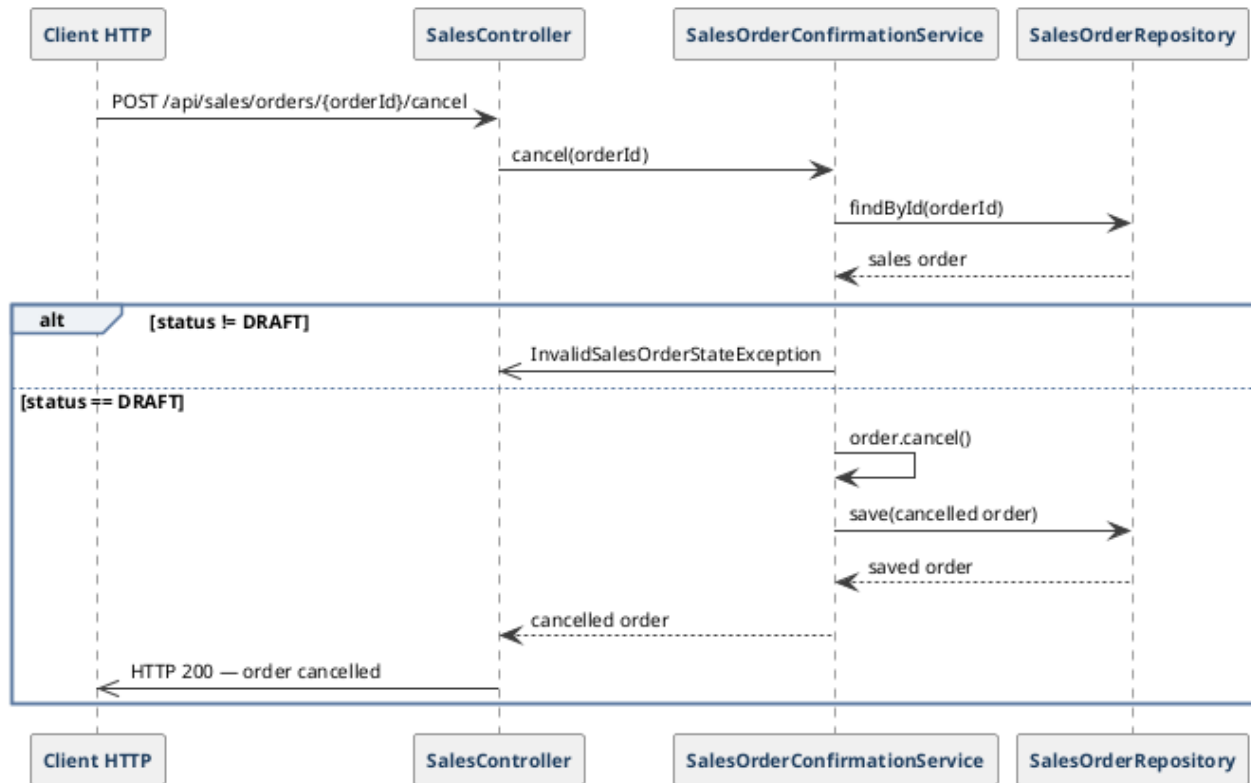
## DS-SA-01 — Création d'une commande en brouillon



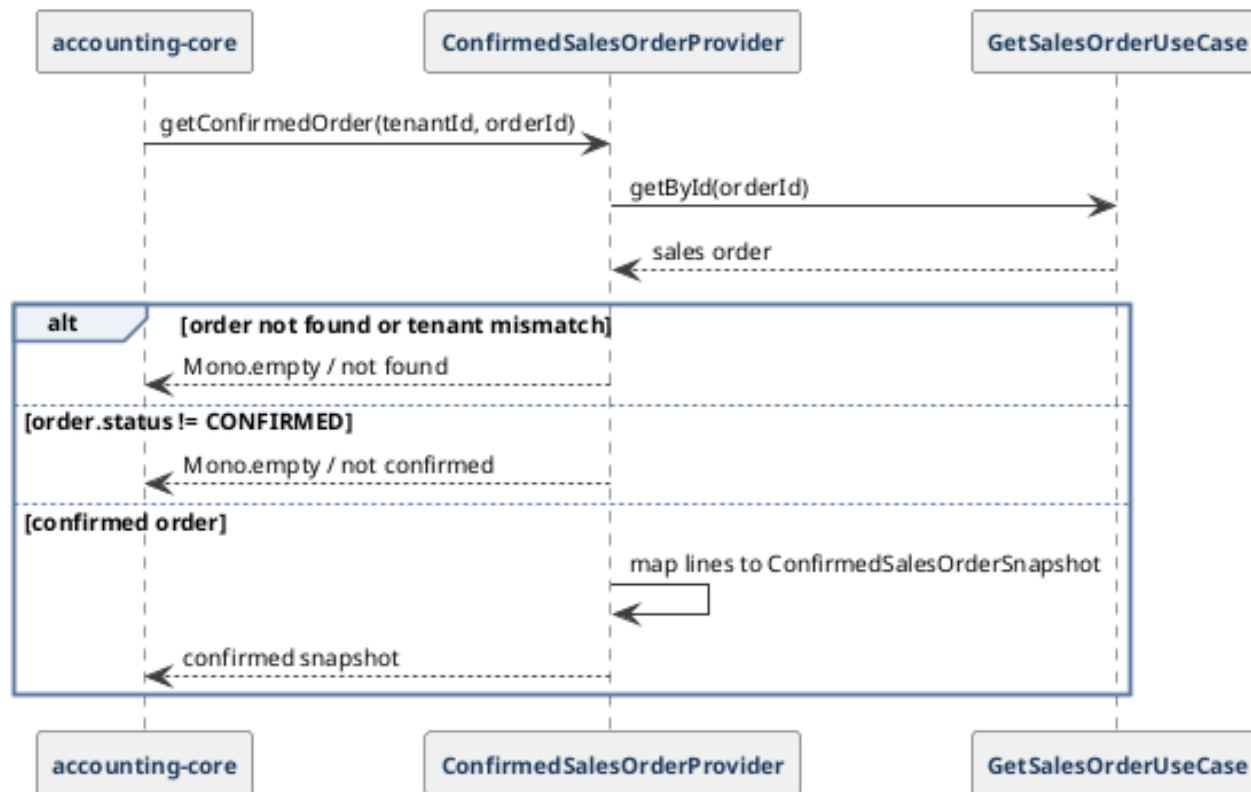
## DS-SA-02 — Confirmation avec dispatch direct du stock



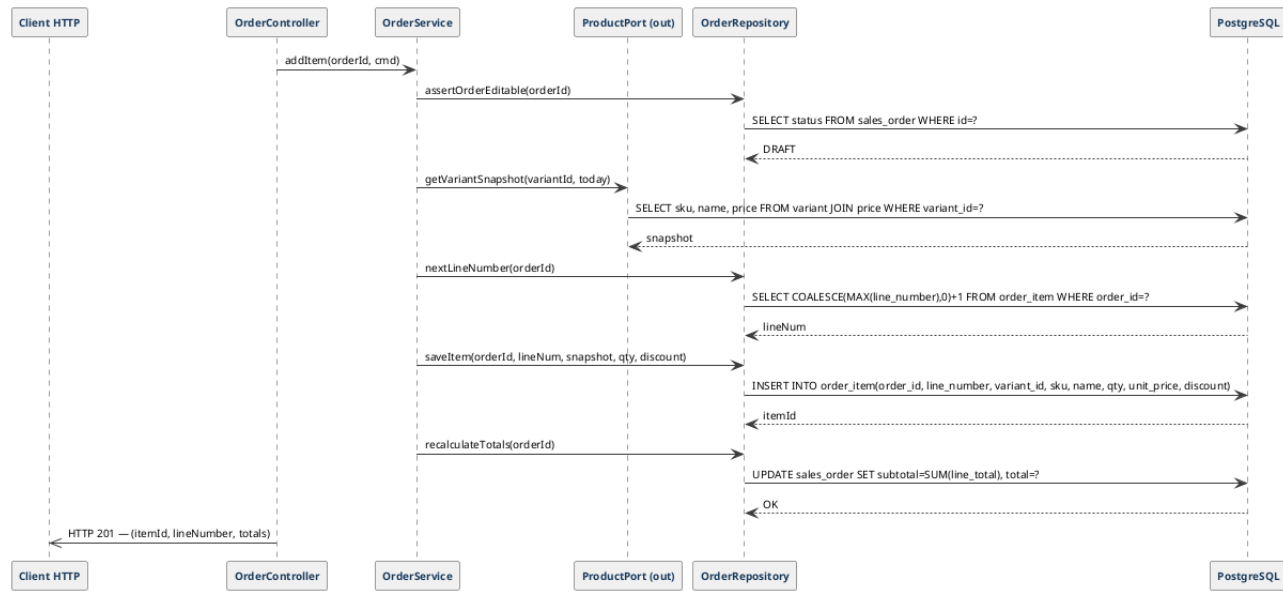
## DS-SA-03 — Annulation d'une commande restée en brouillon



## DS-SA-04 — Mise à disposition d'un snapshot confirmé pour facturation explicite

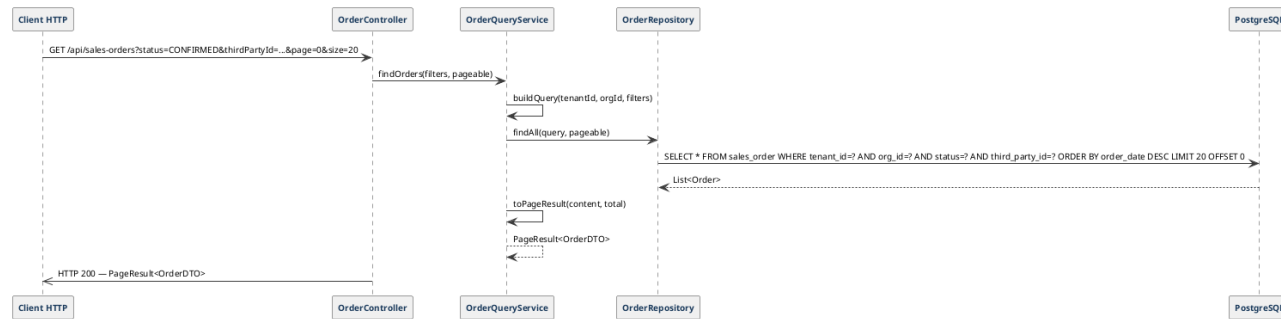


## DS-SA-05 — Ajout d'une ligne article à une commande en brouillon





## DS-SA-06 — Consultation paginée et filtrée du carnet de commandes



## 13 accounting-core — Comptabilité

---

### 13.1 Description du core

L'**accounting-core** porte aujourd'hui deux couches complémentaires. La première est le **noyau canonique** exposé sous `/api/accounting` : il gère la création, la mise à jour, la suppression conditionnelle, la validation et la consultation des factures, ainsi que la lecture des journaux comptables par défaut et des postes ouverts clients/fournisseurs. La seconde est le **bookkeeping étendu** exposé sous `/api/accounting-service` : il maintient les référentiels comptables d'organisation, les écritures et brouillards manuels, les projections **invoice-accounting**, les postings spécialisés de caisse, de relevés bancaires, de rapprochement bancaire et de mouvements de stock, ainsi que les audits et l'aperçu de clôture.

Le noyau canonique ne s'appuie plus sur un consumer Kafka interne pour créer les factures ; la génération depuis une commande de vente est déclenchée explicitement par `POST /api/accounting/invoices/from-orders/orderId` à partir d'un snapshot confirmé fourni par le sales-core. De même, la validation d'une facture et l'application d'un règlement sont des actions explicites. La coordination avec le treasury-core et le cashier-core se fait par appels de use cases et publication d'événements métier, pas par une chaîne opaque d'automatismes.

### 13.2 Rôle principal

**Rôle principal** : Maintenir la facture canonique et ses postes ouverts, fournir le référentiel et les opérations de bookkeeping d'une organisation, et servir de point de convergence explicite entre ventes, trésorerie, caisse, banque et stock lorsque ces services choisissent de synchroniser leurs effets comptables.

### 13.3 Rôles secondaires

#### 1. Facture canonique

Le noyau canonique crée, met à jour, supprime sous condition et valide des factures d'organisation. Une facture peut être créée manuellement ou à partir d'un snapshot de commande confirmée fourni par le sales-core.

## 2. Validation explicite des factures

POST `/api/accounting/invoices/invoiceId/post` fait passer une facture du statut DRAFT à POSTED et publie l'événement métier INVOICE\_POSTED. Le code courant ne génère pas à ce point des écritures comptables détaillées dans le noyau canonique.

## 3. Règlement explicite des factures

Le treasury-core ou un autre service autorisé peut appliquer un règlement explicite via le use case `ApplyInvoiceSettlementUseCase`. Le résultat est persistant, mis à jour sur la facture, puis propagé par l'événement INVOICE\_SETTLEMENT\_APPLIED.

## 4. Postes ouverts et journaux par défaut

Le core expose la lecture des journaux comptables par défaut et des postes ouverts clients/fournisseurs. Les journaux du noyau servent surtout de vue canonique légère, tandis que la tenue détaillée vit dans le bookkeeping étendu.

## 5. Référentiel comptable d'organisation

Le bookkeeping étendu gère paramètres, devises, taux de change, taxes, plan comptable OHADA, comptes, journaux et opérations d'une organisation.

## 6. Écritures manuelles et brouillards

`/api/accounting-service` permet de créer des écritures manuelles, de les valider, de les annuler, de générer des écritures depuis une opération, et de gérer des brouillards (draft entries) qui peuvent ensuite être postés en écritures.

## 7. Projection comptable des factures

La ressource `invoice-accounting` ne remplace pas la facture canonique ; elle matérialise une projection enrichie, persistée dans l'extension, qui rattache une facture canonique à un tiers et à un compte comptable exploitable pour le reporting et les traitements comptables.

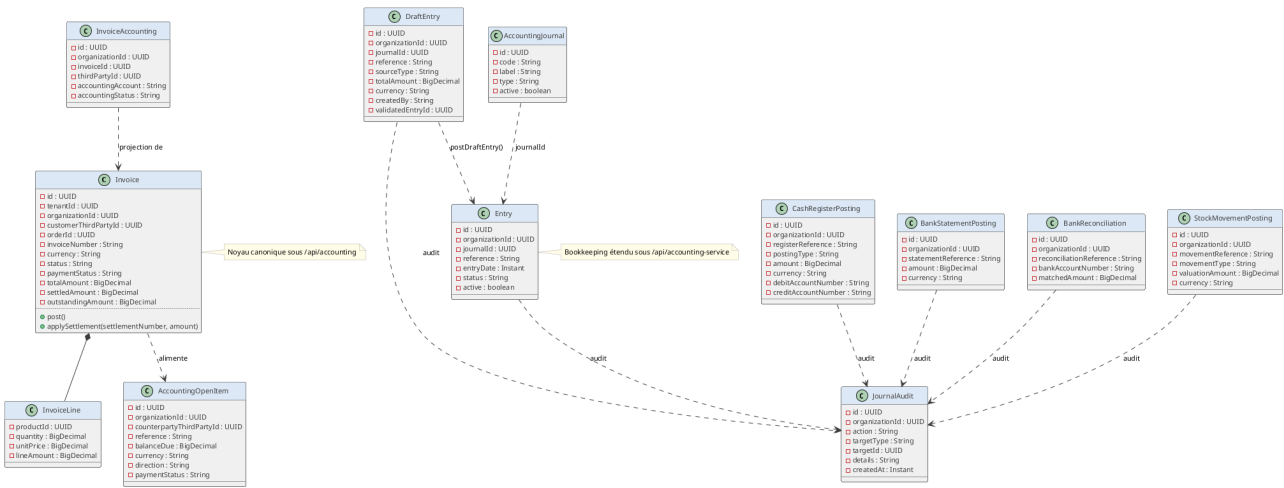
## 8. Postings spécialisés inter-services

Le bookkeeping étendu porte les postings spécialisés provenant de la caisse, des relevés bancaires, des rapprochements bancaires et des mouvements de stock. Ces flux sont explicitement créés et audités ; ils permettent à d'autres services de matérialiser leur impact comptable quand l'organisation y est éligible.

## 9. Reporting et clôture

Le core charge les données de référence de reporting et expose un aperçu de clôture via `/api/accounting-service/workflows/closing/preview`.

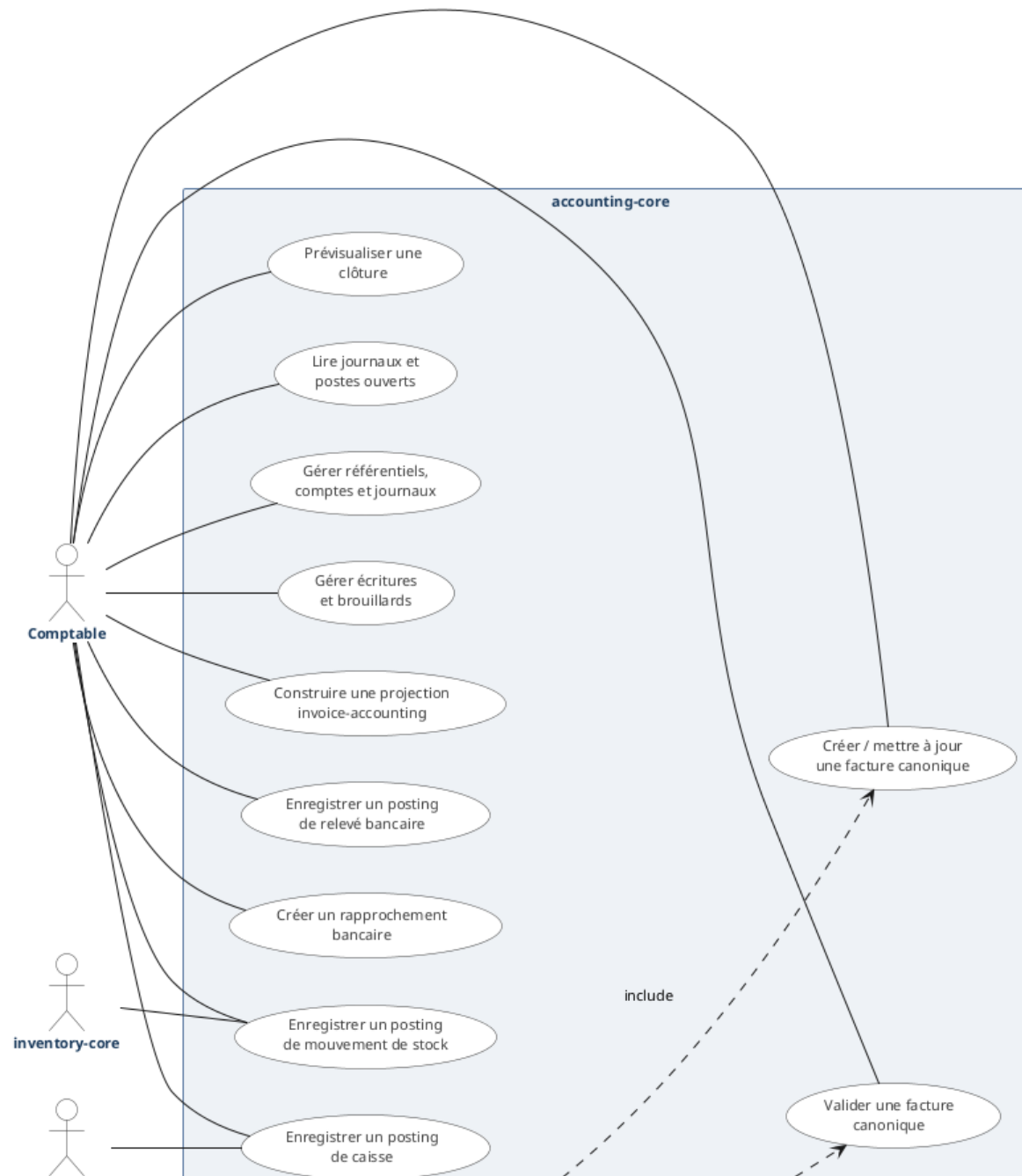
Diagramme de classes — accounting-core



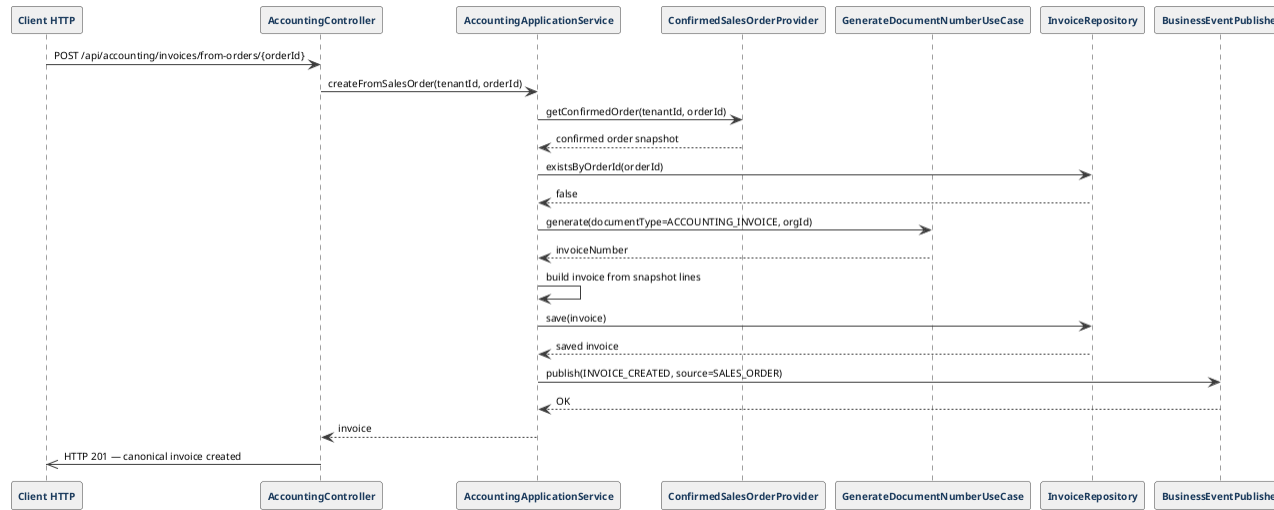
**Contraintes opérationnelles observables dans le code courant :**

- La facture canonique est toujours portée par un `tenantId` et un `organizationId`.
- Une facture canonique ne peut être supprimée que si son statut est `DRAFT`.
- La validation d'une facture canonique et l'application d'un règlement sont des actions explicites distinctes.
- Les objets du bookkeeping étendu sont tous bornés à l'organisation courante résolue par le contexte de requête.
- Un posting de caisse dérive ses comptes débit/crédit à partir du type d'opération et du compte de caisse résolu ou auto-généré.
- Les brouillards et écritures du bookkeeping sont gouvernés par des statuts et par l'audit ; le backend actuel ne centralise pas une garde unique imposant *débit = crédit* sur tous les endpoints `/api/accounting-service/entries`.

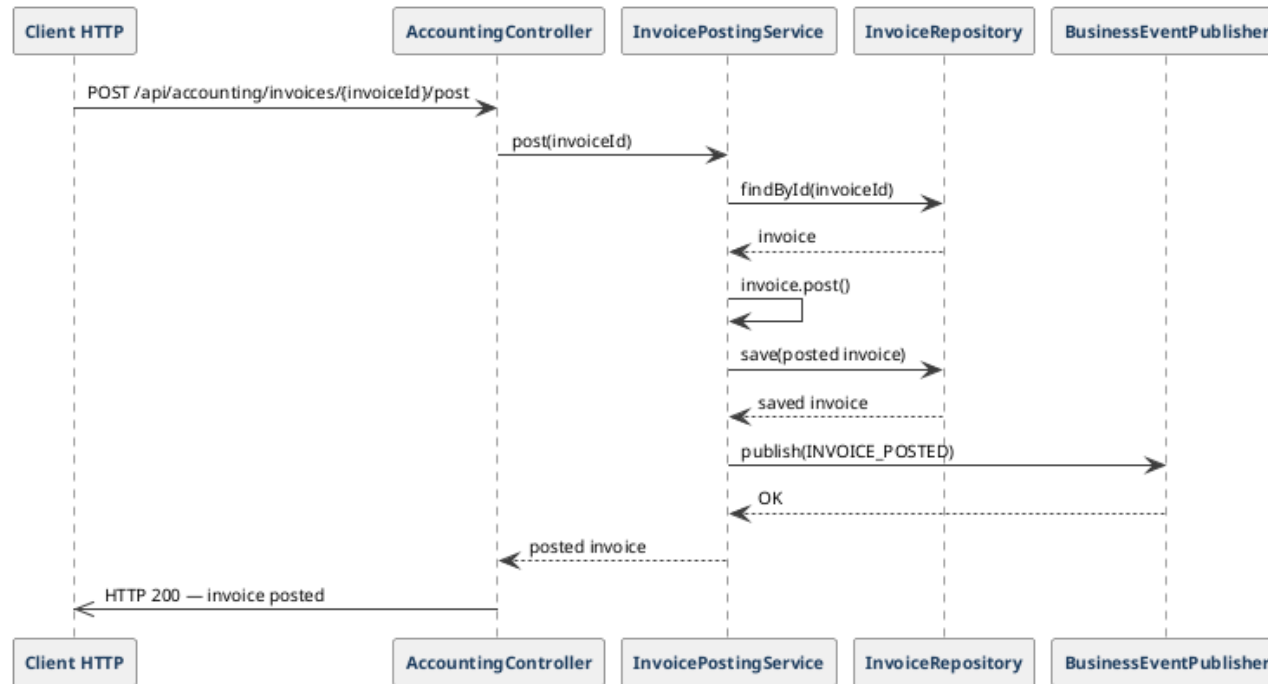
## Diagramme des cas d'utilisation — accounting-core



## DS-AC-01 — Créer une facture canonique à partir d'une commande confirmée

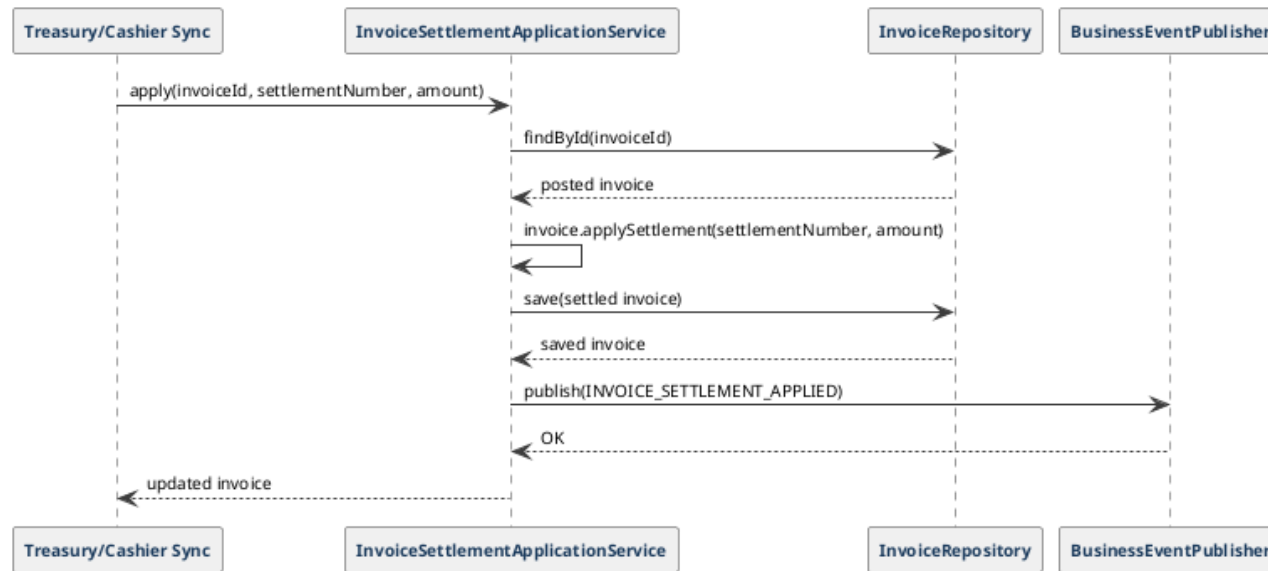


## DS-AC-02 — Valider explicitement une facture canonique

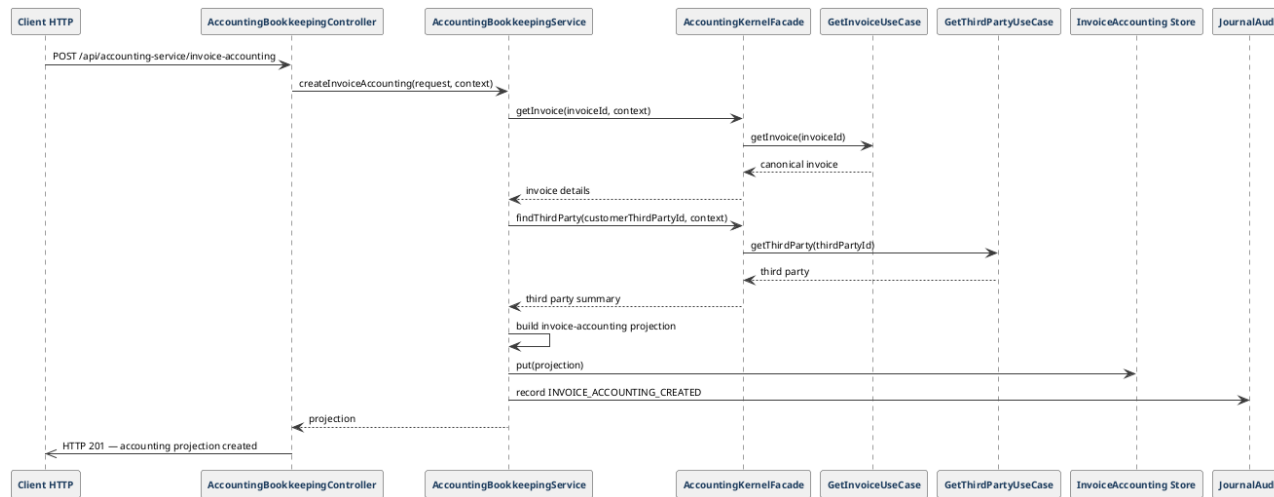




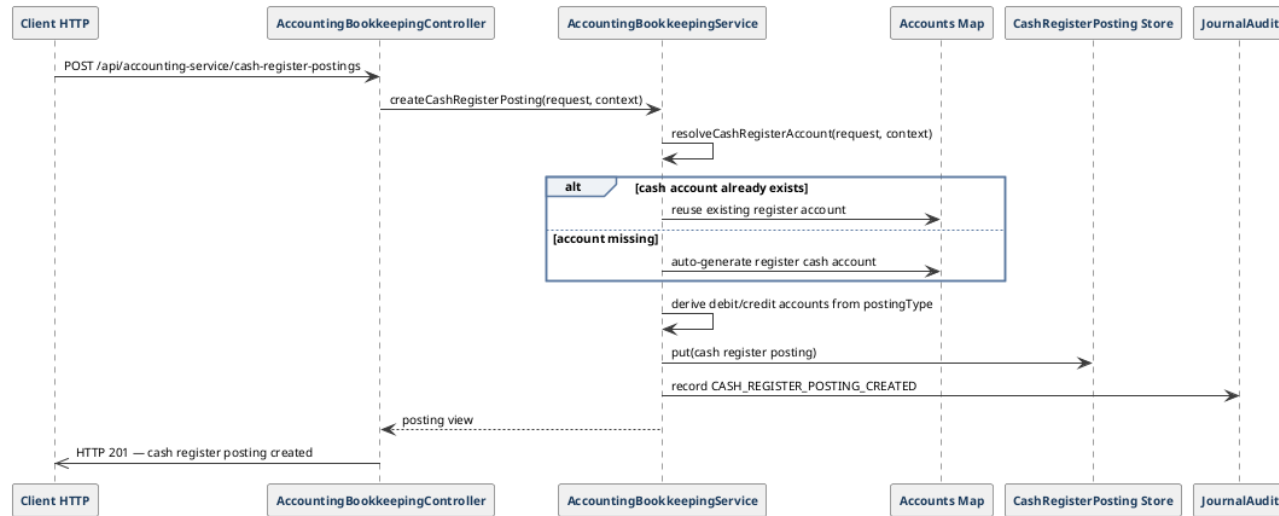
## DS-AC-03 — Appliquer explicitement un règlement sur une facture postée



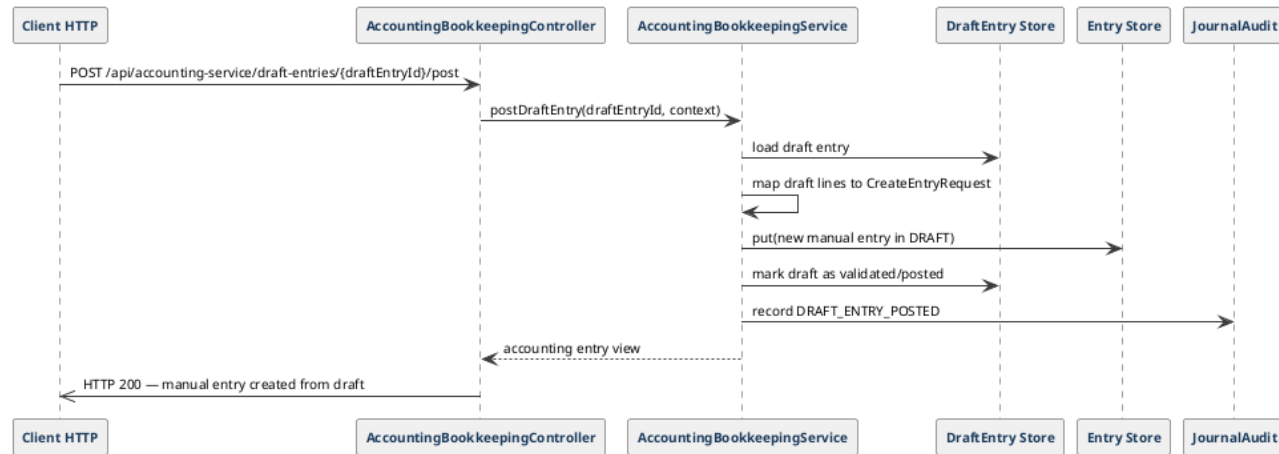
## DS-AC-04 — Construire une projection invoice-accounting depuis une facture canonique



## DS-AC-05 — Enregistrer un posting comptable de caisse



## DS-AC-06 — Poster un brouillard comptable en écriture manuelle



# 14 billing-core — Facturation commerciale legacy

---

## 14.1 Description du core

Le **billing-core** porte la surface historique de facturation commerciale encore exposée par le backend. Il gère des *documents commerciaux* organisationnels (**bon d'achat**, **bon de commande**, **bon de livraison**, **bon de réception**, **facture fournisseur**, **facture proforma**, **note de crédit**) ainsi qu'un registre simple de paiements legacy.

Le module enrichit ces documents avec des résumés produits et tiers issus du **product-core** et du **tp-core**, mais il ne tient pas encore le grand livre comptable. En revanche, le backend courant lui donne désormais des **ponts explicites** vers la chaîne canonique : un document legacy éligible peut être synchronisé vers une facture canonique de **accounting-core**, importé en bill lié dans **cashier-core**, ou réglé explicitement via **treasury-core**. Le billing-core reste donc un sous-système legacy de pièces commerciales et de paiements persistés, mais il peut maintenant orchestrer des handoffs contrôlés vers les services canoniques quand l'organisation a souscrit ces services et dispose encore du quota nécessaire. Aucune propagation automatique n'est imposée : chaque articulation inter-service reste volontaire et tracée.

## 14.2 Rôle principal

**Rôle principal** : fournir la surface HTTP legacy de gestion des pièces commerciales et des paiements associés, à l'échelle d'une organisation, avec enrichissement par les référentiels produits et tiers.

## 14.3 Rôles secondaires

### 1. Gestion polymorphe des pièces commerciales

Le même service applicatif manipule plusieurs types de documents commerciaux et résout leur type à partir du chemin HTTP legacy exposé.

### 2. Numérotation locale des documents

Lorsqu'aucun numéro n'est fourni, le core génère un numéro séquentiel simple

par `organizationId` et par type de document, avec des préfixes BA, PO, BR, BL, PF, SF et CN.

### 3. Gestion des lignes de document

Chaque document porte des lignes ordonnées (`lineIndex`, `productId`, `quantity`, `unitPrice`) dont le montant est recalculé à la lecture.

### 4. Enrichissement produit et contrepartie

Les vues retournées exposent un résumé produit et un résumé de contrepartie, résolus depuis le `product-core` et le `tp-core`, sans dupliquer ces référentiels.

### 5. Transitions métier limitées

Le cycle d'état des pièces legacy reste volontairement simple : DRAFT puis, pour certains documents comme le bon de livraison, FULFILLED.

### 6. Registre de paiements legacy

Les paiements sont persistés avec lien éventuel vers une facture legacy client ou fournisseur, une contrepartie, une référence, un montant, une devise et un horodatage.

### 7. Synchronisation explicite vers la facture canonique

Certains documents legacy, notamment la facture proforma et le bon de livraison, peuvent être synchronisés explicitement vers `accounting-core`. Cette synchronisation crée ou relie une facture canonique, avec option de validation immédiate, puis mémorise l'identifiant comptable sur le document billing.

### 8. Synchronisation explicite vers la caisse

Lorsqu'un document est déjà relié à une facture canonique postée, le `billing-core` peut demander son import dans `cashier-core` comme bill lié. Le règlement en caisse devient alors une opération de `cashier-core`, mais reste pilotée depuis la surface billing legacy si l'appelant le souhaite.

### 9. Règlements explicites par caisse ou banque

Le backend courant expose des opérations explicites `/payments/cashier` et `/payments/bank`. La première encaisse via `cashier-core` avec option de synchronisation du settlement comptable ; la seconde enregistre un règlement bancaire via `treasury-core`, qui applique ensuite le règlement à la facture canonique.

### 10. Compatibilité d'API historique

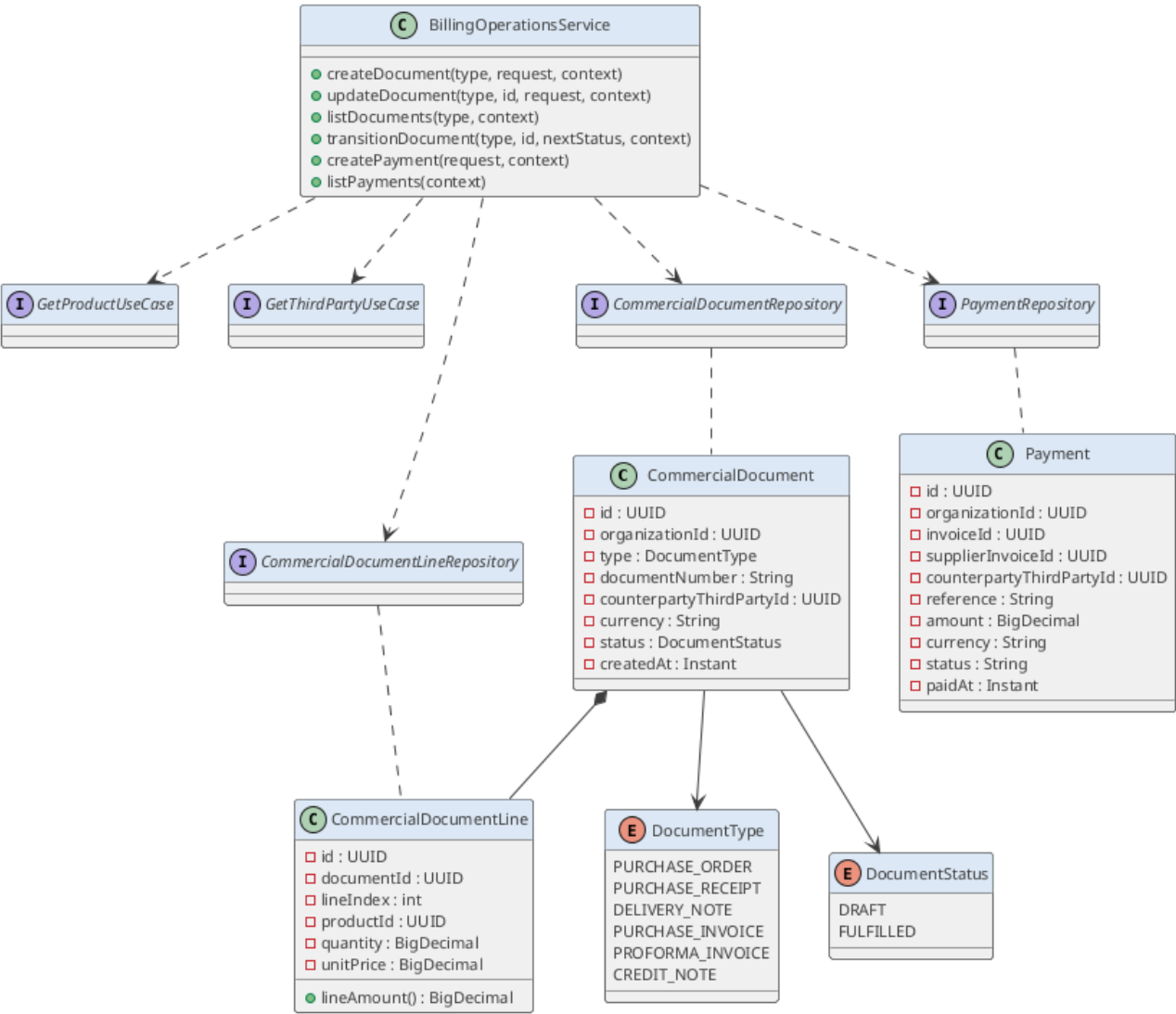
Le core maintient les routes legacy déjà utilisées côté front ou intégrations internes, notamment `/api/bons-achat`, `/api/factures-proforma` et `/api/paiement`.

### 11. Absence de propagation implicite

Le `billing-core` ne pousse jamais automatiquement un paiement vers la comptabilité, la banque ou la caisse. Toute articulation avec `accounting-core`,

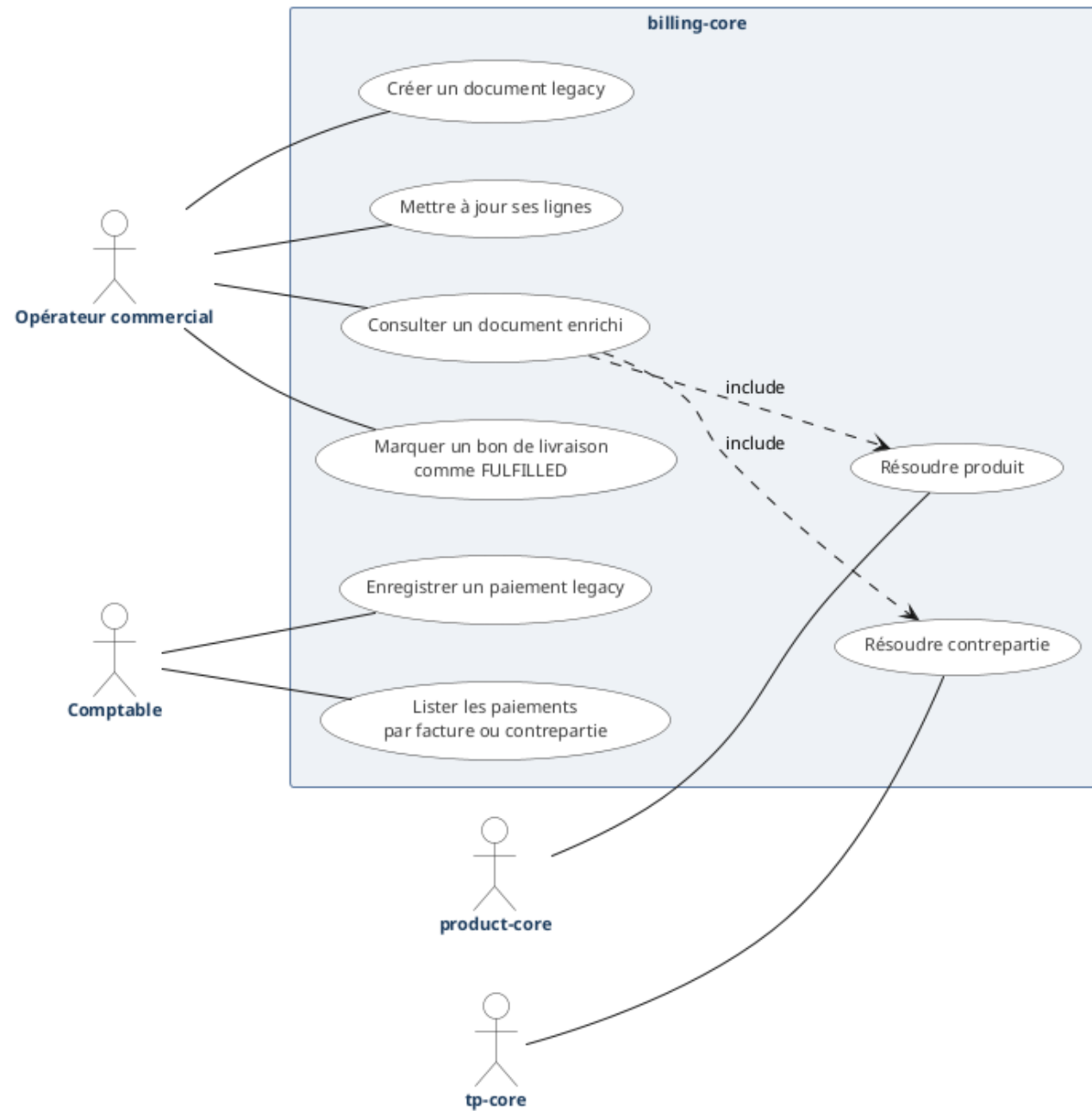
`treasury-core` ou `cashier-core` exige une route dédiée et un appel explicite.

Diagramme de classes — billing-core

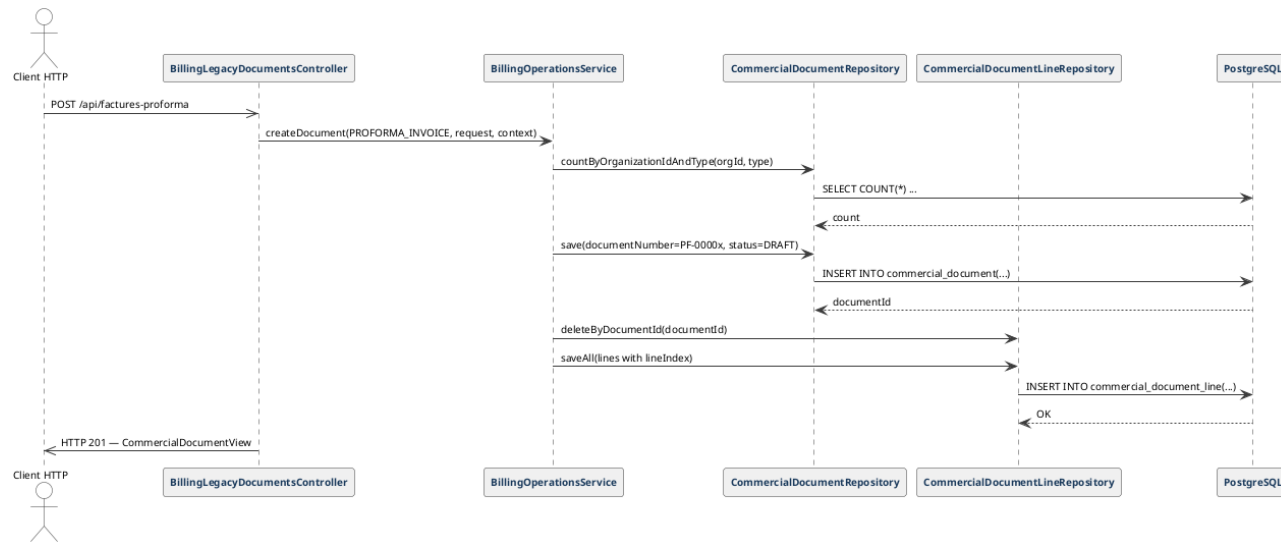




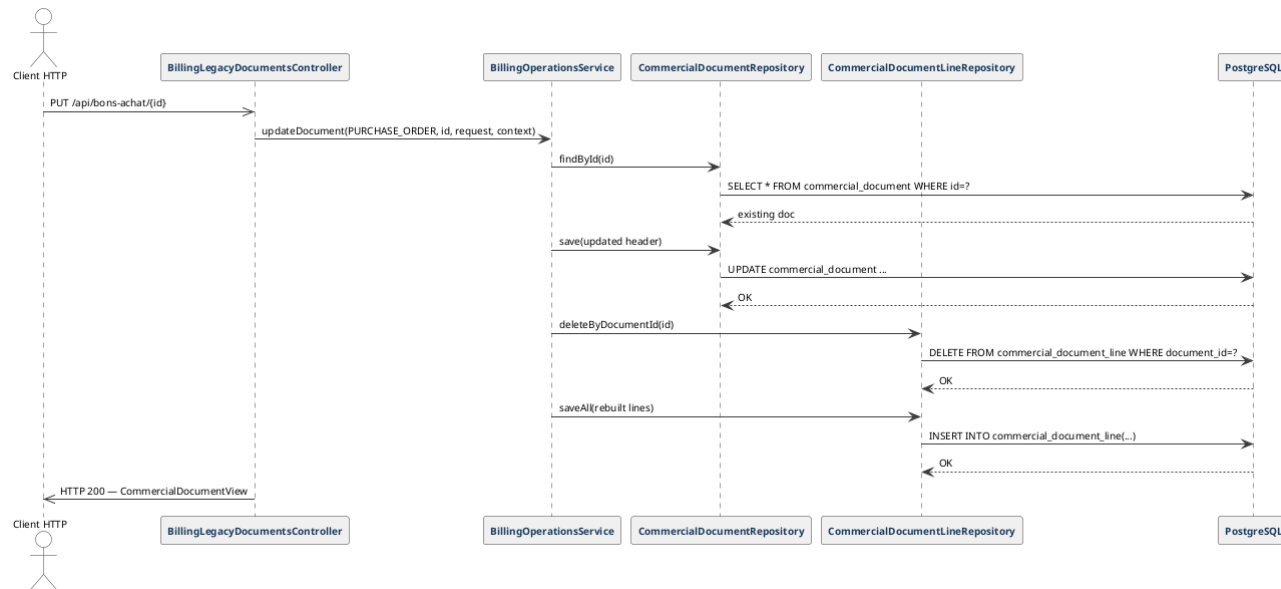
## Diagramme des cas d'utilisation — billing-core



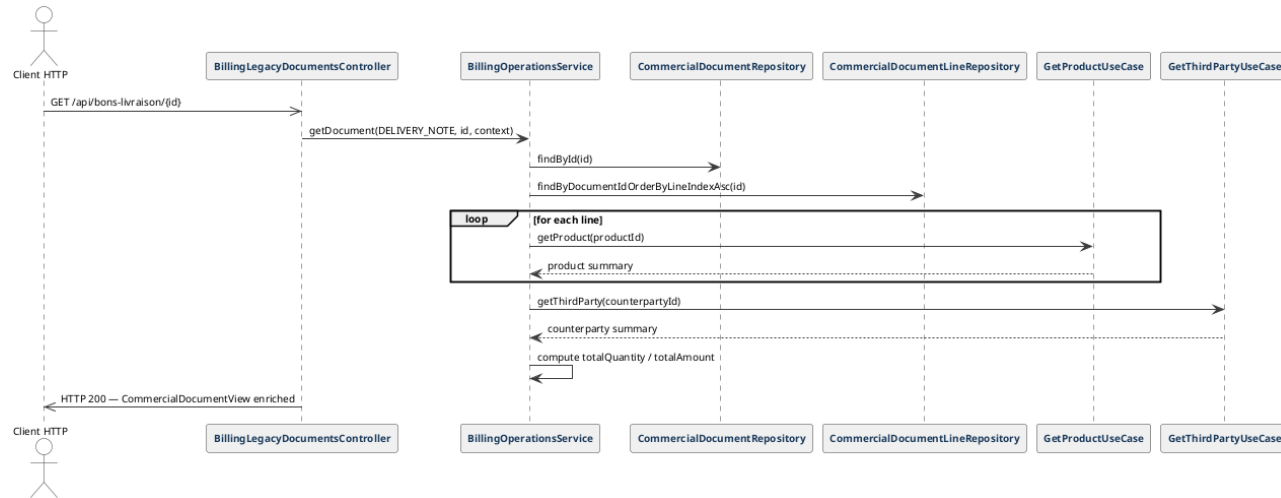
## DS-BI-01 — Création d'un document commercial legacy



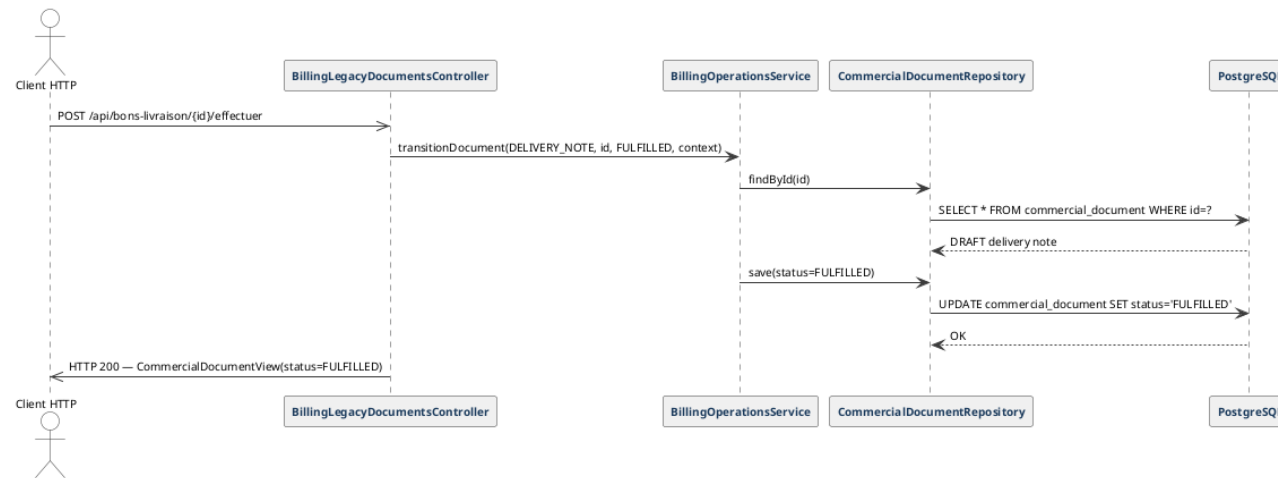
## DS-BI-02 — Mise à jour d'un document avec remplacement de ses lignes



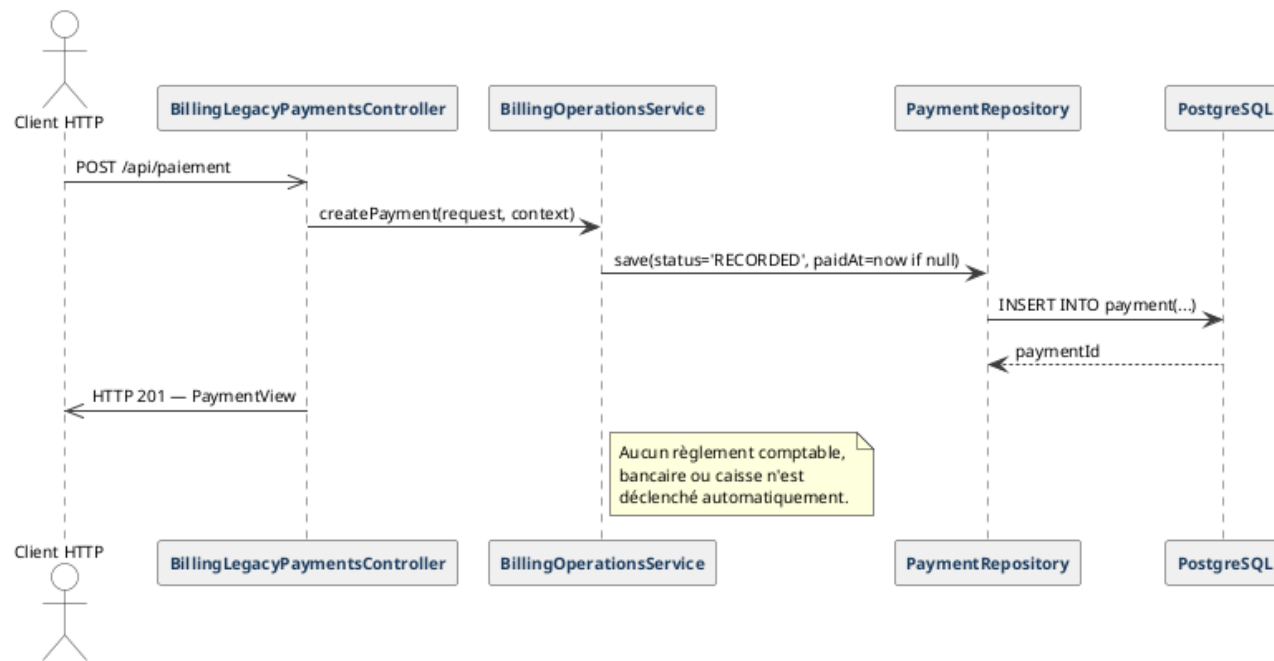
## DS-BI-03 — Consultation enrichie d'un document commercial



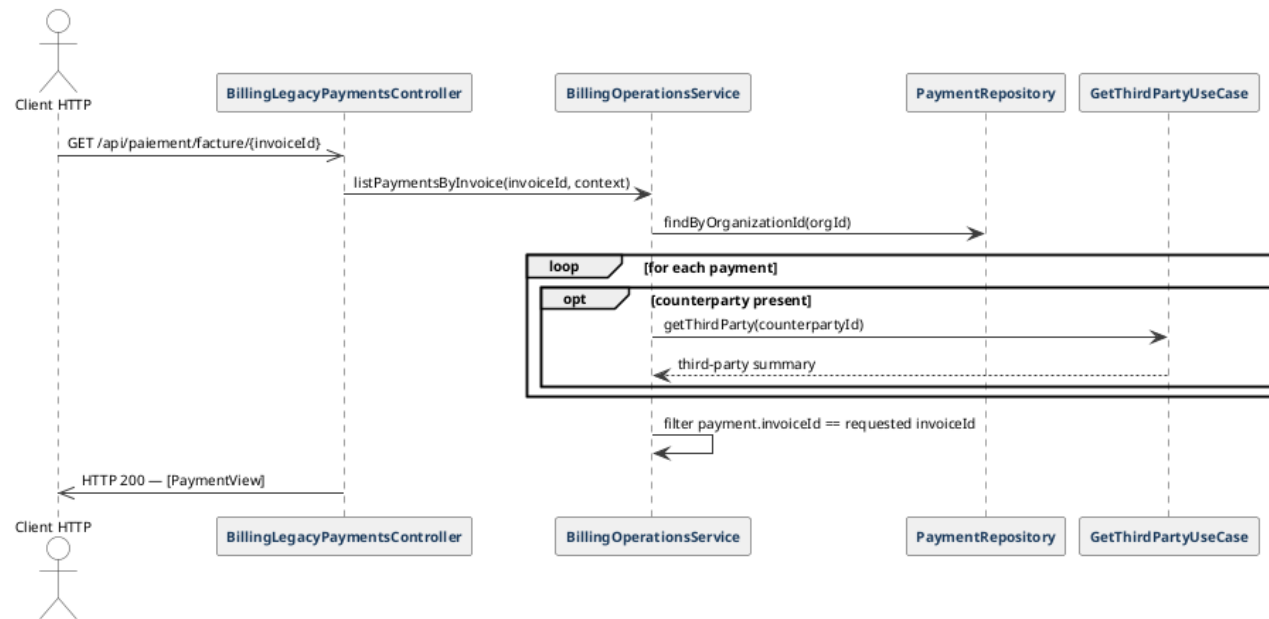
## DS-BI-04 — Transition explicite d'un bon de livraison vers FULFILLED



## DS-BI-05 — Enregistrement d'un paiement legacy



## DS-BI-06 — Consultation des paiements filtrés par facture legacy



**Invariants du billing-core :**

- Chaque document commercial est borné à une seule organisation.
- Le type de document est déterminé par la route legacy appelée et reste cohérent pendant toute la vie du document.
- L'ordre des lignes est stable à l'intérieur d'un document via `lineIndex`.
- Le montant d'une ligne est toujours recalculé comme `quantity * unitPrice`.
- Un paiement billing legacy ne vaut que pour l'organisation courante et n'implique aucun règlement comptable ou bancaire implicite.
- Une synchronisation inter-service depuis billing n'est autorisée que si le service cible est effectif pour l'organisation et que son quota le permet.
- Un paiement `/payments/cashier` ou `/payments/bank` opère sur un document déjà relié à son contexte canonique cible; le pont n'est jamais inféré implicitement.

## 14.4 Surface API effectivement exposée

Le backend courant expose notamment :

- GET/POST/PUT/DELETE sur les documents legacy : `/api/bons-achat`, `/api/bon-commande`, `/api/bons-livraison`, `/api/v1/facturation/bon-receptions`, `/api/facture-fournisseurs`, `/api/factures-proforma`, `/api/v1/facturation/note-cred`
- POST `/api/bons-livraison/{id}/effectuer` pour marquer un bon de livraison comme `FULFILLED`
- POST `.../{id}/sync/accounting-invoice` pour créer ou relier explicitement une facture canonique dans `accounting-core`
- POST `.../{id}/sync/cashier-bill` pour importer explicitement le document dans `cashier-core` comme bill lié
- POST `.../{id}/payments/cashier` pour encaisser via `cashier-core`, avec option de synchronisation du settlement comptable
- POST `.../{id}/payments/bank` pour enregistrer un règlement bancaire via `treasury-core`
- GET/POST/PUT/DELETE `/api/paiement`
- GET `/api/paiement/client/{clientId}`
- GET `/api/paiement/facture/{factureId}`

Cette surface est protégée par des permissions de lecture/écriture comptable, de caisse ou de settlement selon la route appelée, mais elle reste conceptuellement une API legacy de pièces commerciales qui délègue explicitement ses prolongements canoniques à d'autres cores.



## 15 treasury-core — Trésorerie

---

### 15.1 Description du core

Le **treasury-core** gère les flux de trésorerie et les opérations bancaires de l'organisation. Il maintient les comptes bancaires, enregistre les transactions, importe les relevés bancaires, gère les chèques, ouvre et clôture les rapprochements, et porte les règlements explicites de factures par compte bancaire.

Dans le backend courant, la coordination avec l'**accounting-core** ne repose pas sur une propagation implicite à base d'événements entrants. Le cas d'usage canonique est un **règlement explicite** : le **treasury-core** vérifie le compte bancaire choisi, lit l'instantané de facture comptable postée, valide la devise et l'encours restant, puis appelle explicitement l'**accounting-core** pour appliquer le settlement. Le rapprochement bancaire intervient ensuite comme contrôle de cohérence entre transactions, relevés et livres comptables.

### 15.2 Rôle principal

**Rôle principal** : Gérer les comptes bancaires de l'organisation, enregistrer les transactions, les relevés, les chèques et les rapprochements, puis déclencher explicitement les règlements bancaires de factures vers la comptabilité.

### 15.3 Rôles secondaires

1. **Gestion des établissements et comptes bancaires**

Référentiel des banques partenaires (SWIFT, pays) et des comptes de l'organisation (IBAN, BIC, titulaire).

2. **Enregistrement des transactions**

Chaque entrée ou sortie de fonds (virement, prélèvement, paiement par chèque, règlement de facture) est enregistrée comme une **BankTransaction**.

3. **Import de relevés bancaires**

Les relevés sont portés comme agrégats explicites et exposés à la fois sur les routes canoniques `/api/treasury/statements` et sur les routes de compatibilité `/api/banking/statements`.

#### 4. Gestion des chèques

Le core gère l'enregistrement des chèques et leur passage explicite vers un état encaissé/compensé, avec conservation d'une surface legacy `/api/banking/checks`.

#### 5. Rapprochement bancaire

Association automatique ou manuelle des transactions enregistrées aux lignes de relevé. Un rapprochement clôturé devient immuable et constitue une pièce d'audit.

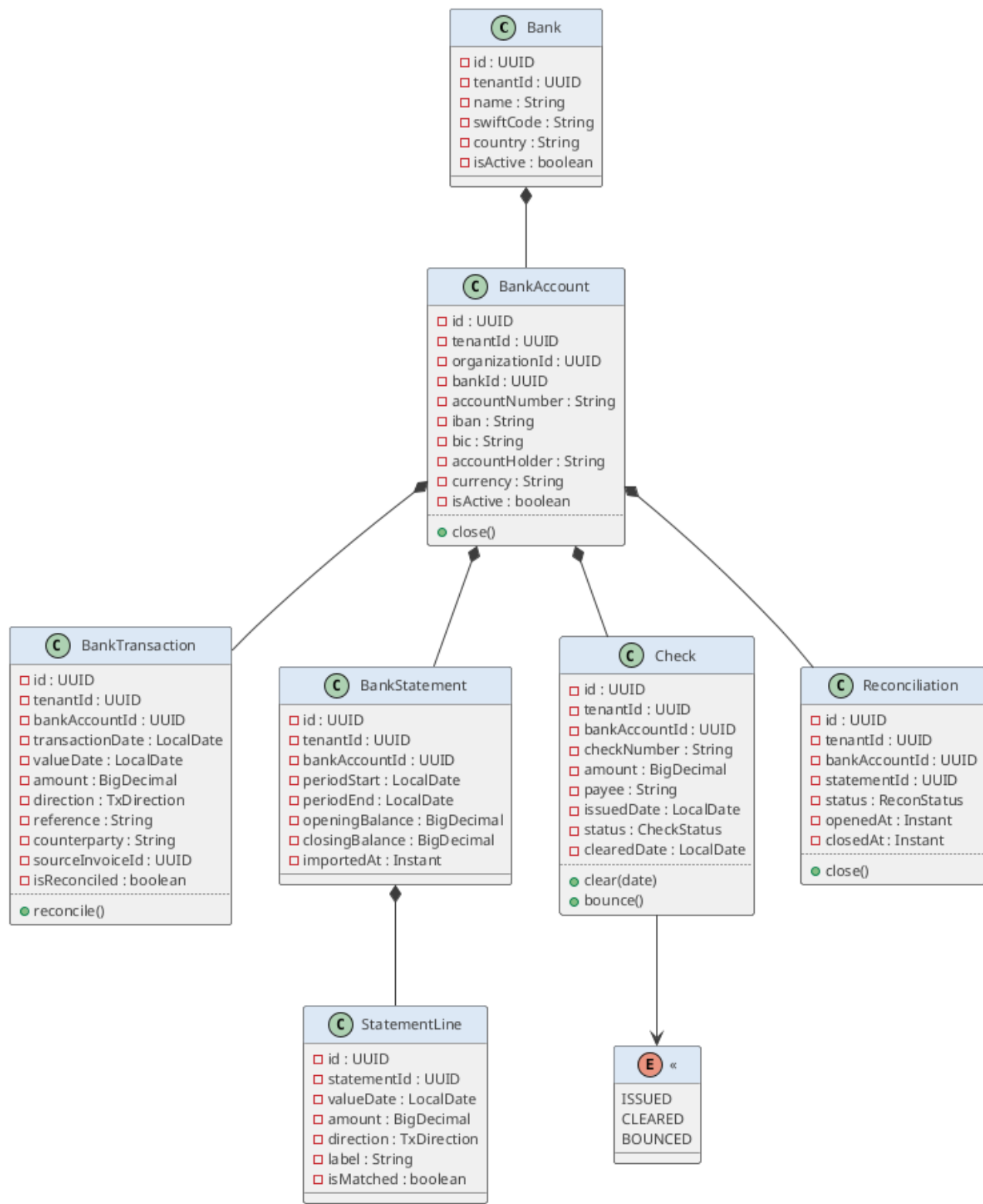
#### 6. Règlement bancaire explicite des factures

Le cas d'usage `invoice-settlement` vérifie l'appartenance du compte bancaire et de la facture au même tenant et à la même organisation, impose l'égalité de devise, refuse tout dépassement de l'encours restant et appelle explicitement `accounting-core` pour appliquer le règlement.

#### 7. Double surface canonique et legacy

Le backend expose à la fois une surface canonique `/api/treasury/**` et une surface de compatibilité `/api/banking/**`, complétée par des contrôleurs legacy de catalogues bancaires, lignes de relevé et audit.

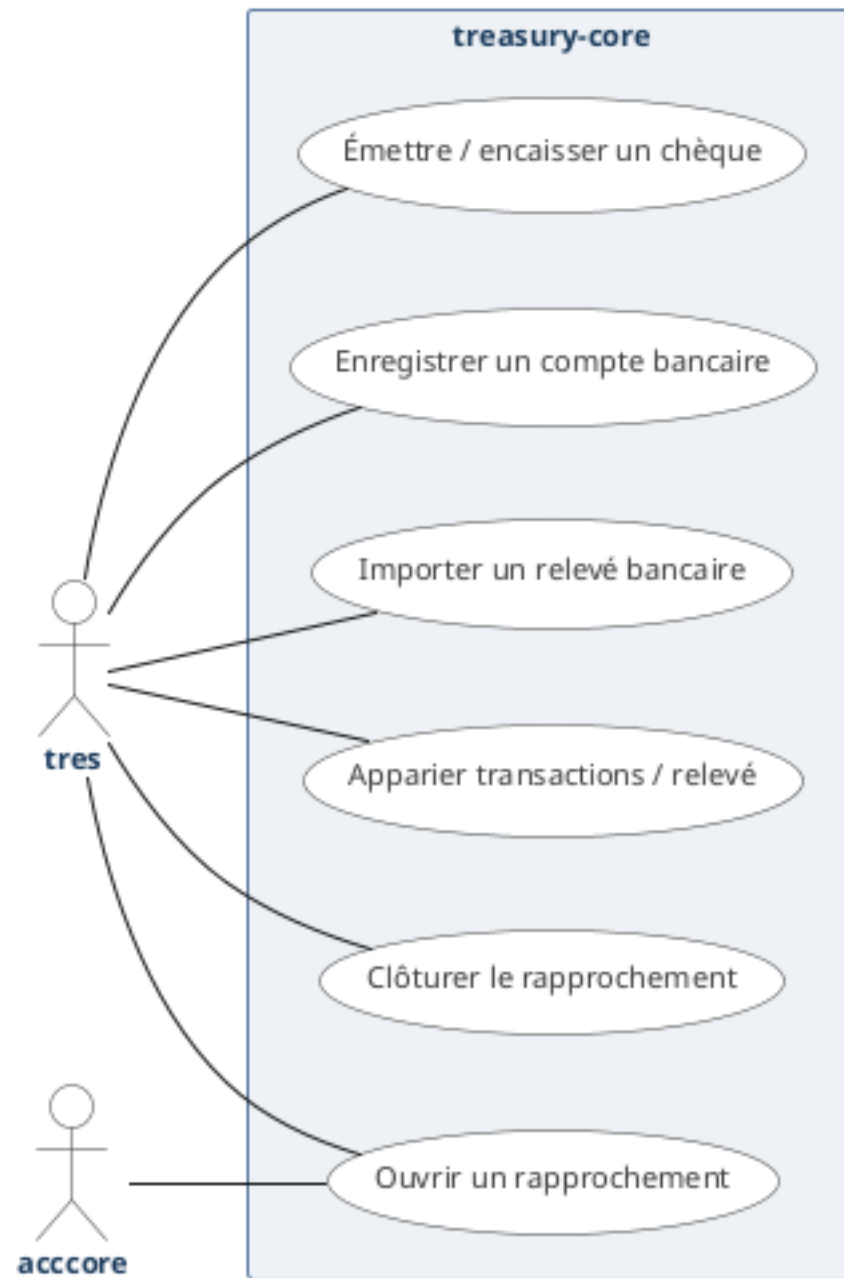
Diagramme de classes — treasury-core



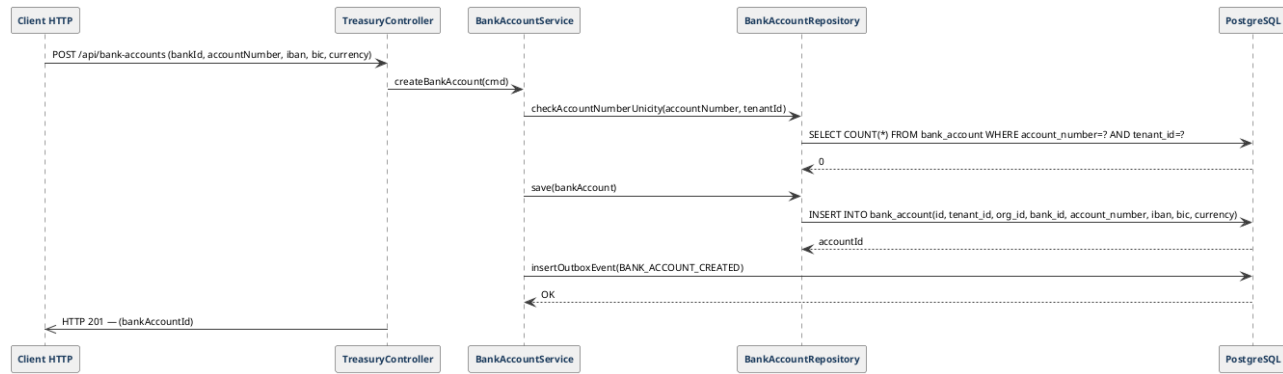
**Invariants du treasury-core :**

- Un compte bancaire et une facture réglée explicitement doivent appartenir au même tenant et à la même organisation.
- Un relevé est unique par (compteId, périodeStart, périodeEnd).
- Un chèque ne peut être encaissé (CLEARED) qu’une seule fois.
- Un rapprochement CLOSED est immuable.
- Un règlement de facture est refusé si son montant dépasse l’encours restant ou si la devise du compte bancaire ne correspond pas à celle de la facture.

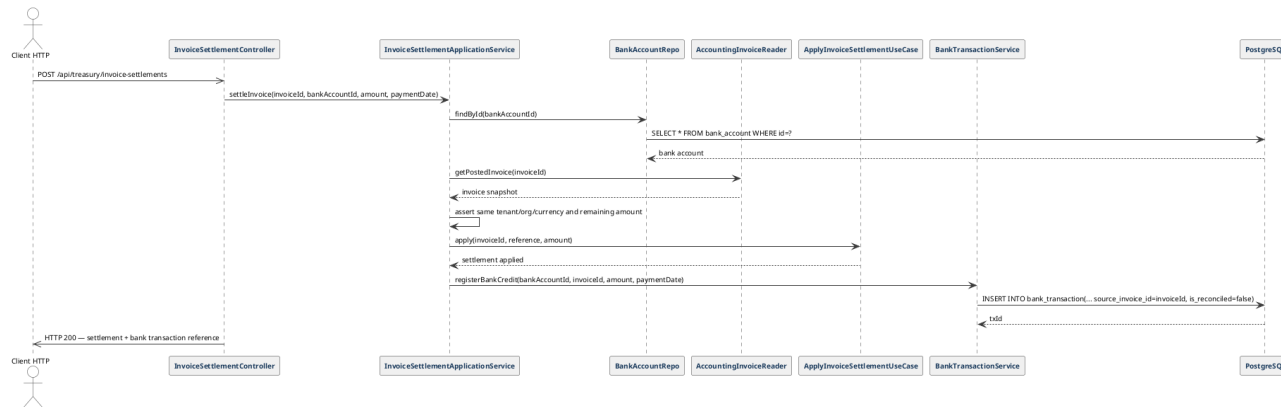
## Diagramme des cas d'utilisation — treasury-core



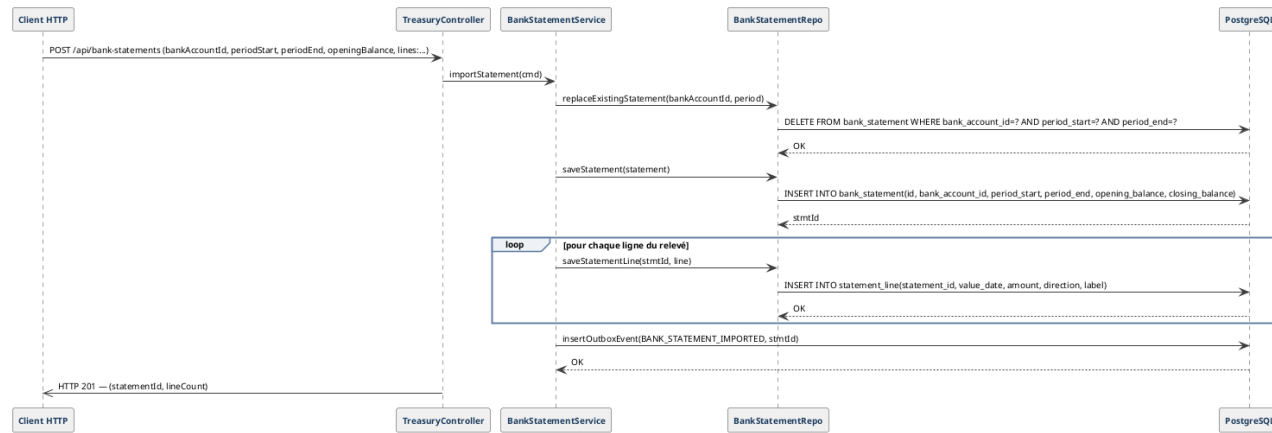
## DS-TR-01 — Enregistrement d'un compte bancaire



## DS-TR-02 — Enregistrement explicite d'un règlement bancaire de facture

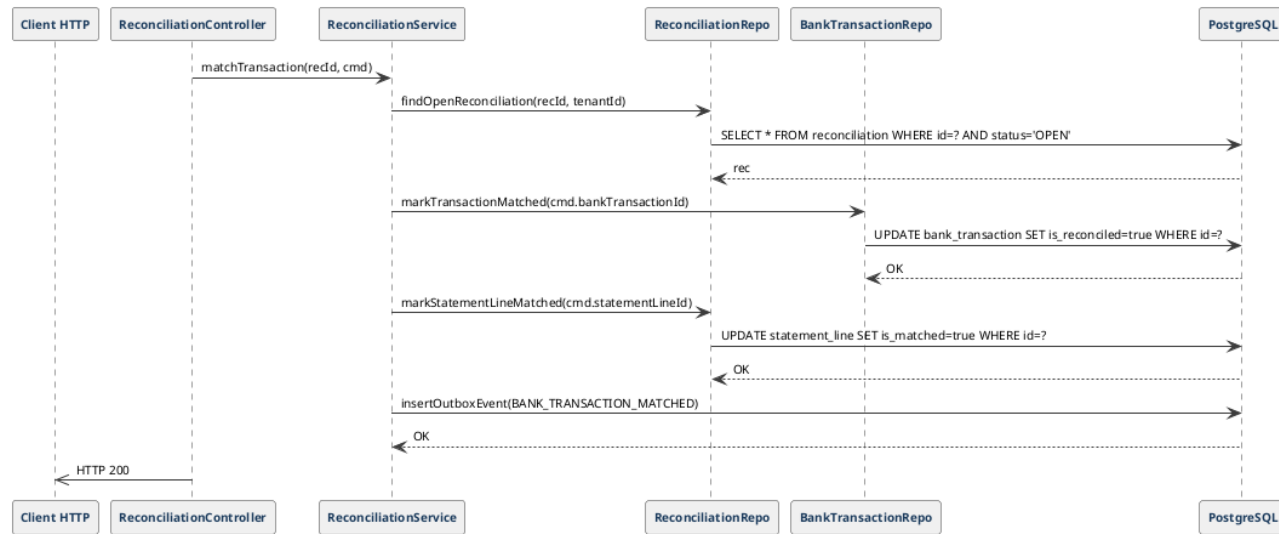


## DS-TR-03 — Import d'un relevé bancaire

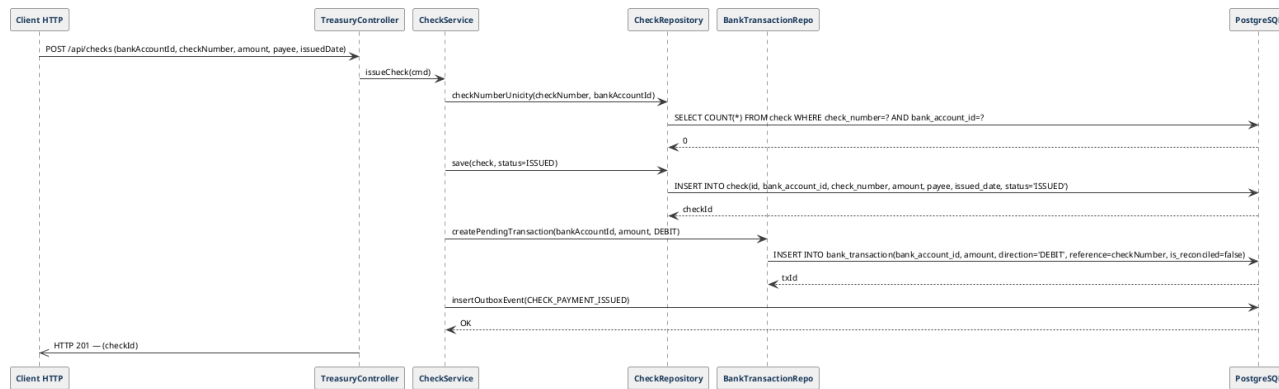




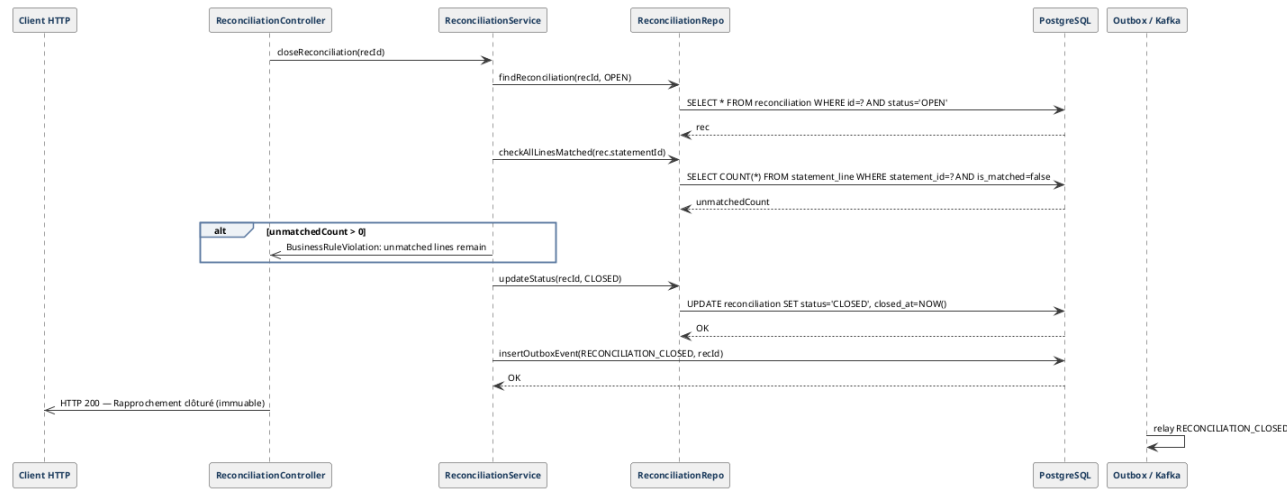
## DS-TR-04 — Appariement d'une transaction bancaire à une ligne de relevé



## DS-TR-05 — Émission d'un chèque avec création de la transaction en attente



## DS-TR-06 — Clôture définitive d'un rapprochement bancaire



## 16 cashier-core — Caisse opérationnelle

---

### 16.1 Description du core

Le **cashier-core** porte la gestion opérationnelle de caisse. Il maintient les caisses (*cash registers*), les profils de caissiers, les affectations, les sessions d'ouverture/fermeture, les comptes wallet, les demandes d'approvisionnement, les bills de caisse, les mouvements, les rapprochements, les notifications, les documents, les tableaux de bord et les rapports de contrôle.

Ce core interagit déjà avec l'**accounting-core** sur deux axes différents. D'une part, les mouvements de caisse importants produisent des postings comptables spécialisés (ouverture de session, paiements, écarts de clôture). D'autre part, un bill de caisse peut maintenant être **lié** explicitement à une facture comptable postée : la facture est importée dans la caisse comme bill lié, le paiement reste une opération de caisse autonome, puis la synchronisation vers la comptabilité est déclenchée explicitement et reste soumise à l'abonnement et au quota du service cible.

### 16.2 Rôle principal

**Rôle principal** : piloter l'encaissement opérationnel d'une organisation, depuis l'ouverture des sessions de caisse jusqu'aux bills, mouvements, demandes de fonds, audits et rapports, tout en gardant les synchronisations inter-services explicites et contrôlées.

### 16.3 Rôles secondaires

#### 1. Gestion des caisses et profils de caissier

Le core maintient les registres de caisse, les profils de caissiers, les affectations et les variantes de profils (**CASHIER**, **ORGANIZATION\_ADMIN**, **AGENCY\_ADMIN**, etc.).

#### 2. Sessions de caisse

Une session porte un montant d'ouverture, un statut, une devise, une caisse et

un caissier. L'ouverture et la fermeture déclenchent des contrôles opérationnels et des postings comptables dédiés.

### 3. Comptes wallet et transferts

Les comptes de caisse portent leur solde, leur devise et leur type. Le core expose des opérations de transfert, retrait et transfert pair à pair.

### 4. Bills et encaissements

Les bills de caisse peuvent être créés localement, listés, payés et suivis avec leur montant encaissé et leur statut courant.

### 5. Import explicite de factures comptables

Une facture comptable postée peut être importée explicitement comme bill lié. Le bill conserve alors l'identité du service source, du type de document lié, de l'identifiant du document et du montant déjà synchronisé.

### 6. Synchronisation explicite vers un service lié

Après encaissement, le bill lié peut être synchronisé explicitement vers son service cible. Dans l'état courant du backend, ce flux est implémenté pour ACCOUNTING / INVOICE et passe par la garde transverse du kernel, avec contrôle d'abonnement et de quota du service cible.

### 7. Mouvements, rapprochements et audit

Le core journalise les mouvements de caisse, propose des rapprochements, porte une piste d'audit dédiée et expose des opérations de revue et de justification.

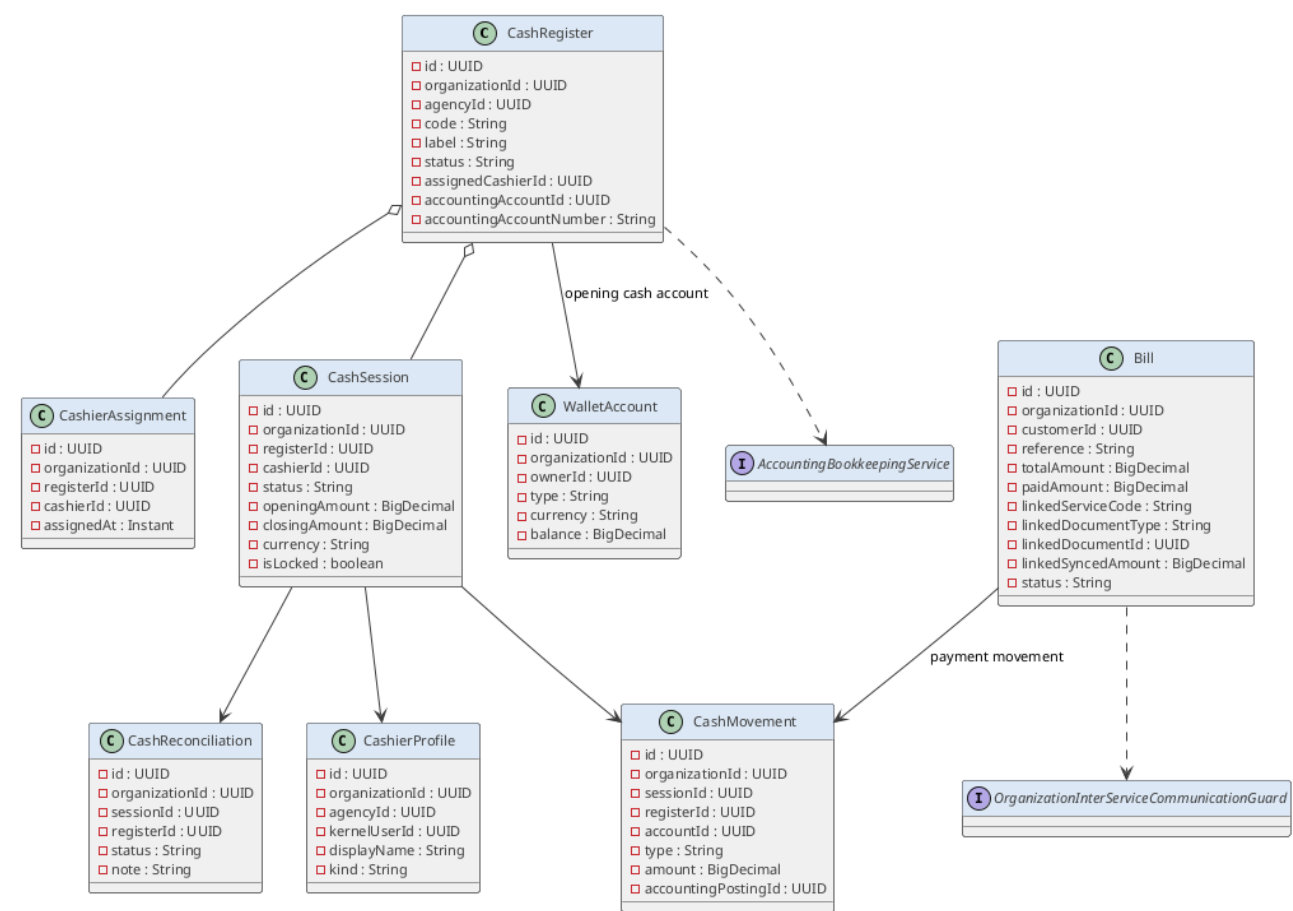
### 8. Notifications, documents et reporting

Notifications de contrôle, documents de caisse, dashboard agrégé, rapports transactionnels, rapports de session et rapports d'audit sont exposés comme lectures opérationnelles.

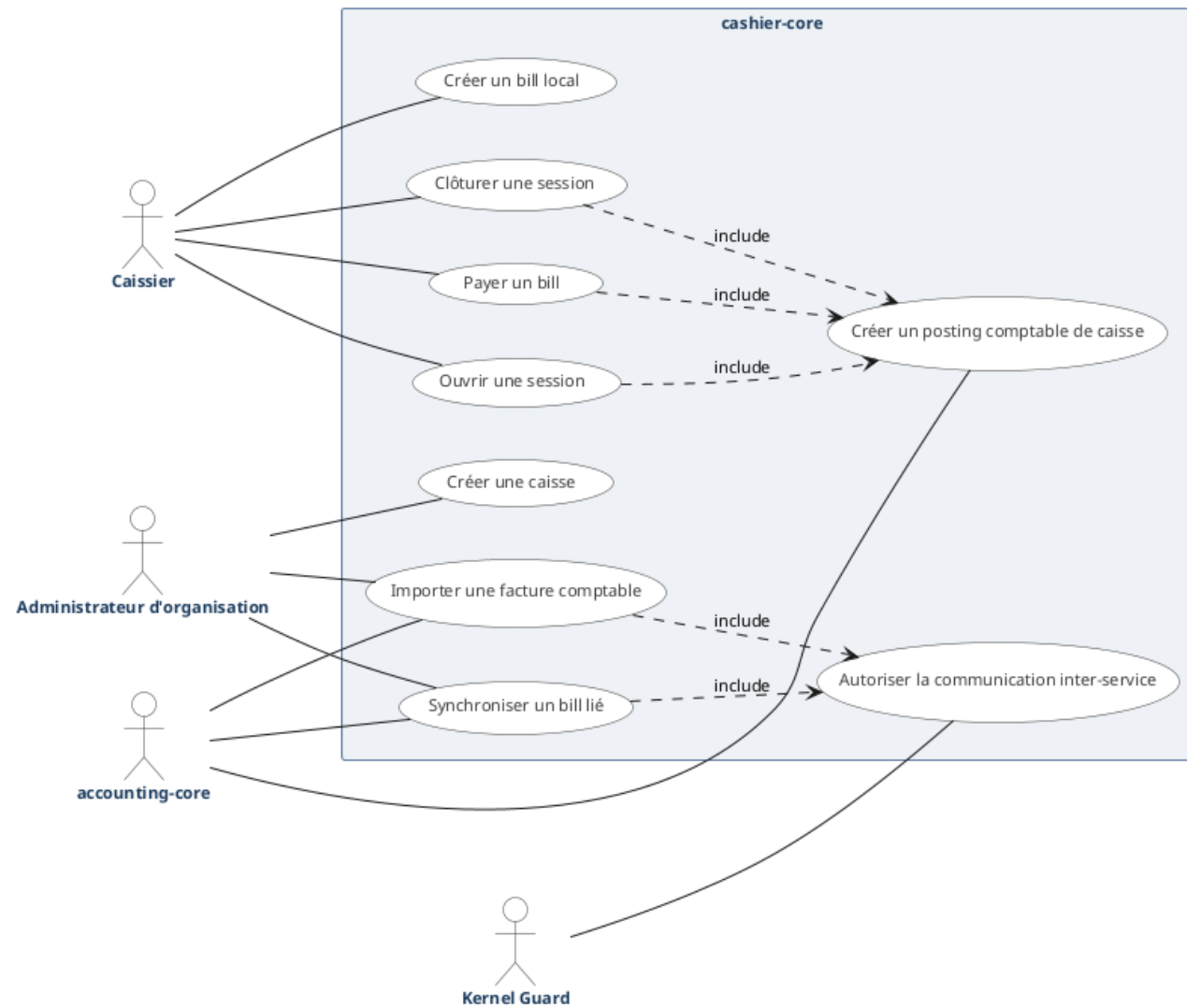
### 9. Surface API mixte admin / cashier

Le backend maintient à la fois des routes administratives (/api/admin/\*\*) et des routes métier caisse (/api/cashier/\*\*, /api/bills, /api/sessions, /api/reports/\*\*), ce qui permet d'exposer le même socle à des usages d'administration et d'exploitation.

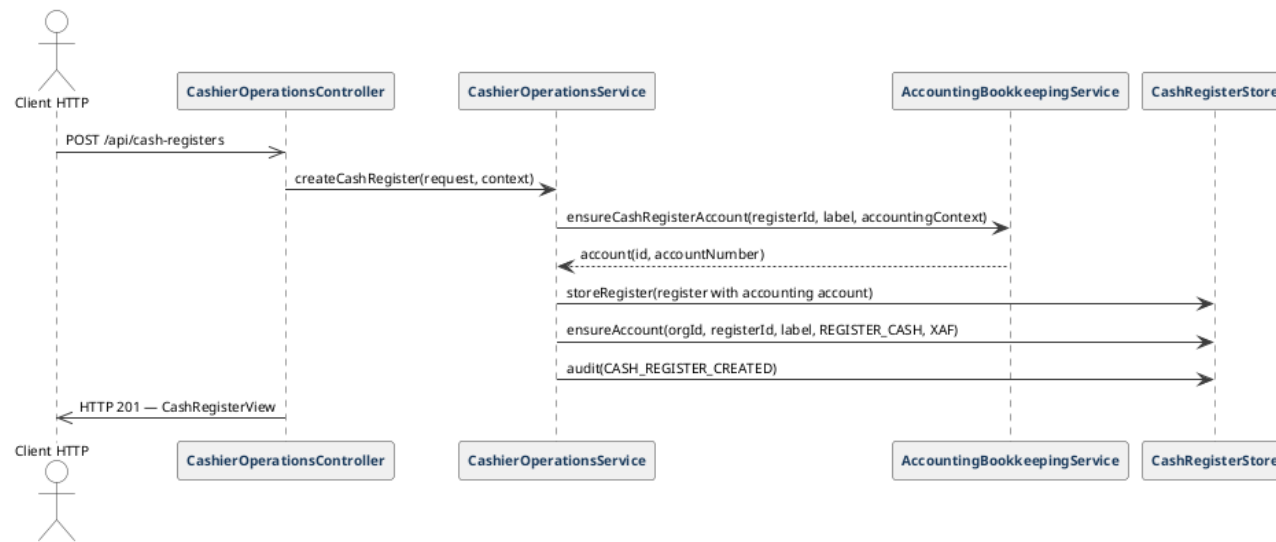
Diagramme de classes — cashier-core



## Diagramme des cas d'utilisation — cashier-core

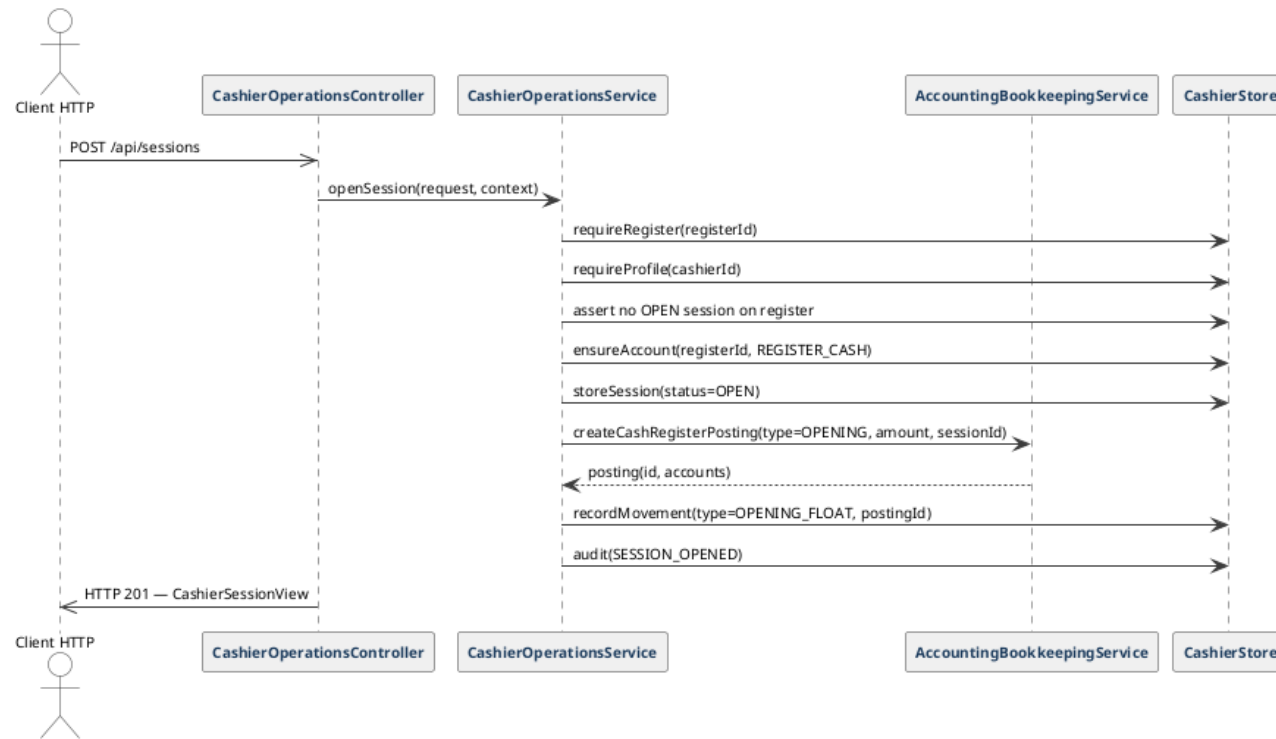


## DS-CA-01 — Création d'une caisse avec compte comptable et wallet d'ouverture

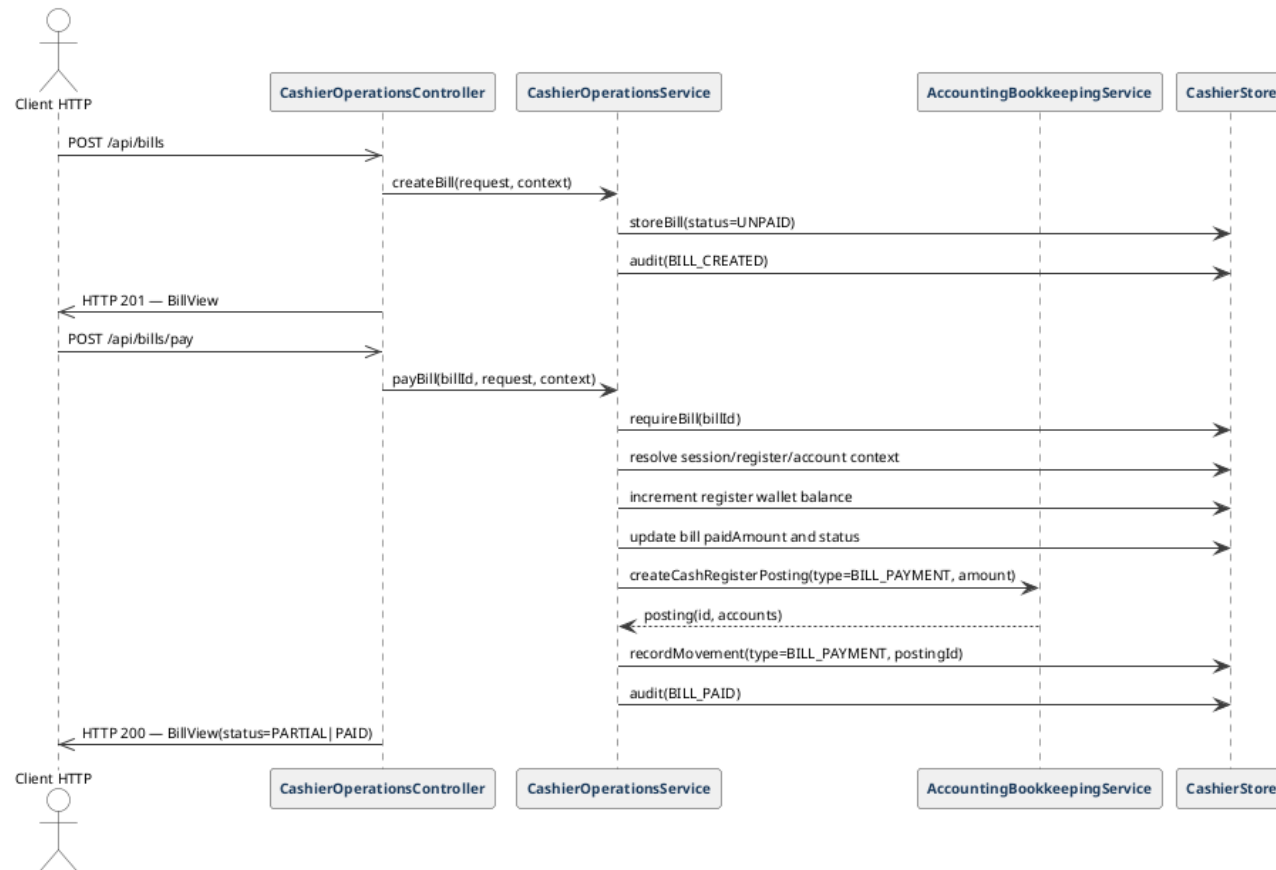




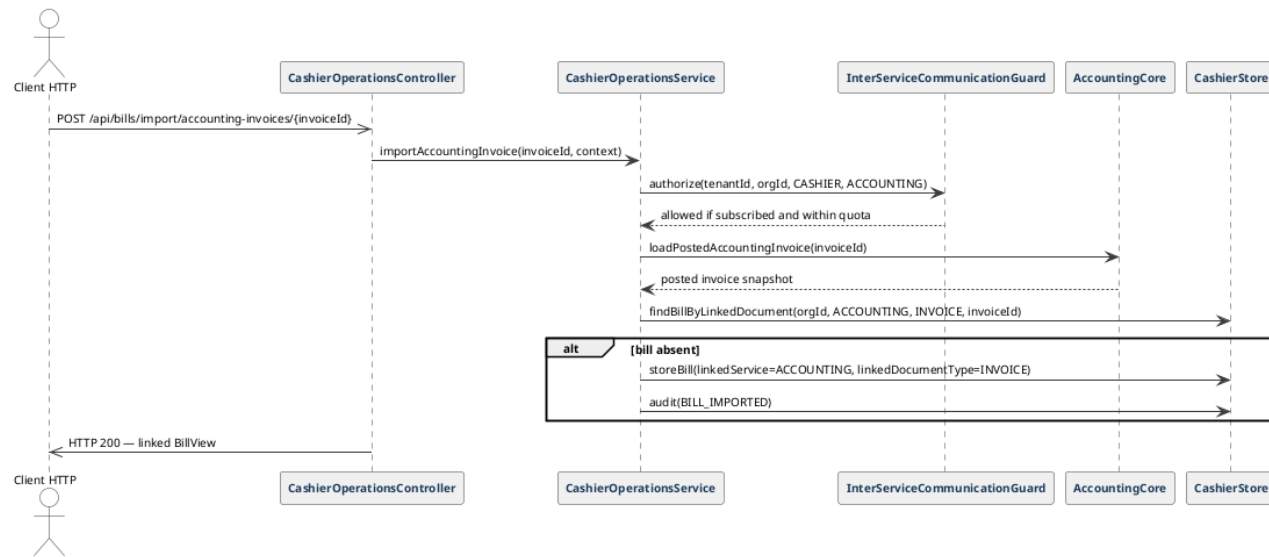
## DS-CA-02 — Ouverture d'une session de caisse avec posting comptable



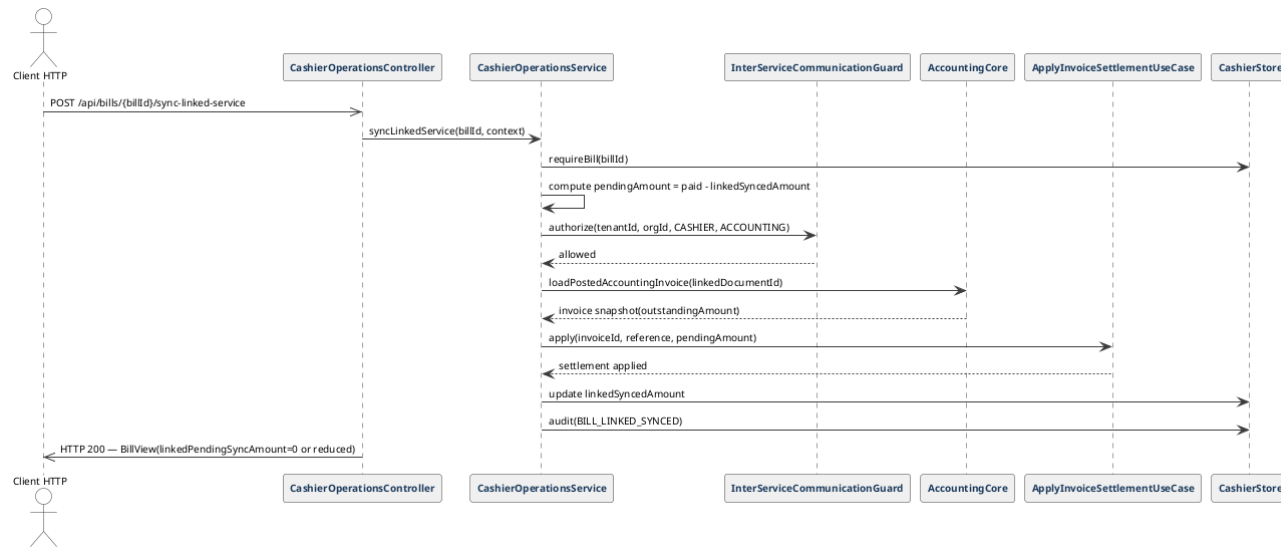
## DS-CA-03 — Encaissement d'un bill local avec mouvement et posting de caisse



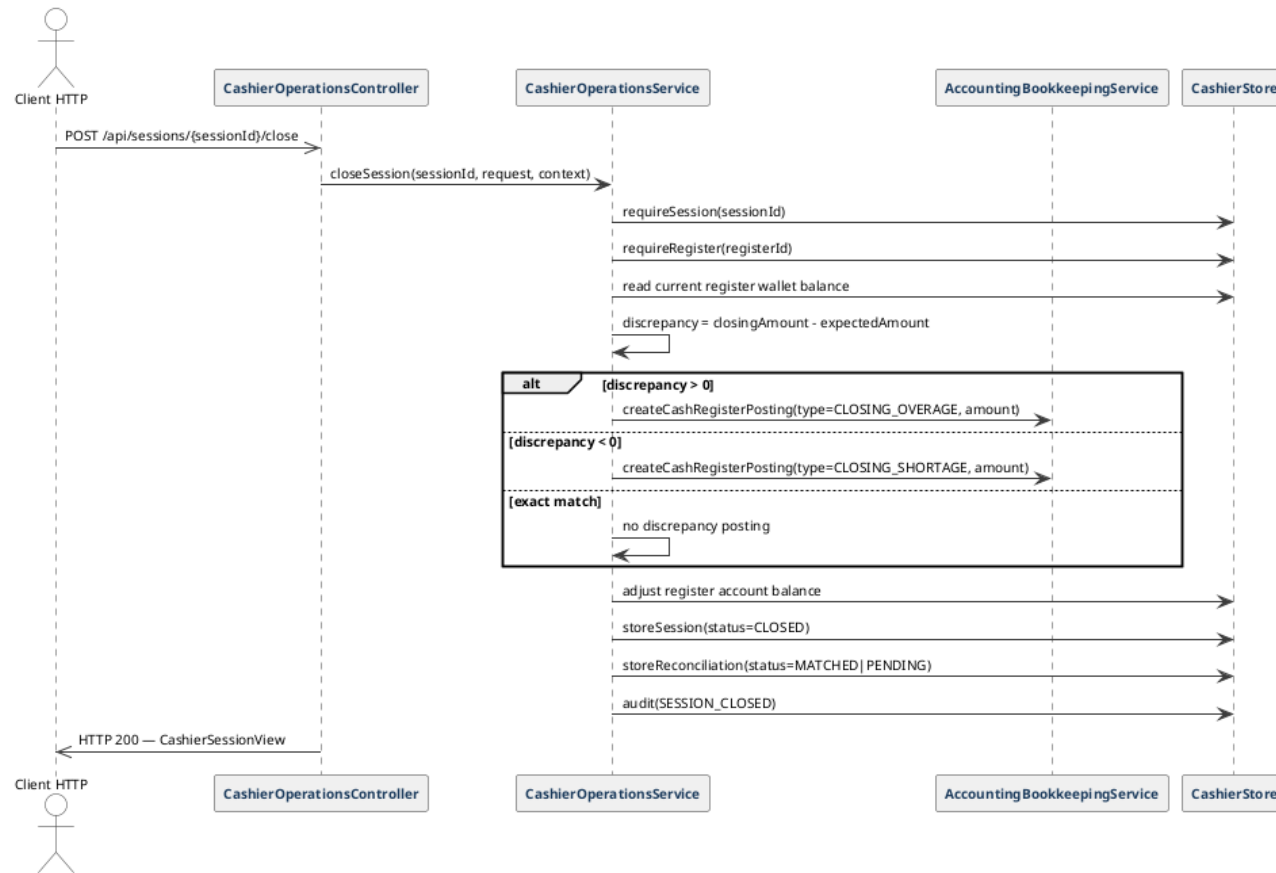
## DS-CA-04 — Import explicite d'une facture comptable comme bill lié



## DS-CA-05 — Synchronisation explicite d'un bill lié vers la comptabilité



## DS-CA-06 — Clôture d'une session avec traitement des écarts



**Invariants du cashier-core :**

- Une seule session OPEN peut exister simultanément pour une même caisse dans une même organisation.
- Tout mouvement, bill, session ou rapprochement reste borné à l'organisation courante.
- Un bill lié ne peut être synchronisé que s'il référence explicitement un service, un type de document et un identifiant de document.
- Le montant synchronisé d'un bill lié ne peut jamais dépasser son montant payé.
- La propagation d'un bill lié vers un autre service n'est jamais implicite : elle exige un appel explicite et peut être refusée si le service cible n'est pas effectif ou si son quota est dépassé.

## 16.4 Surface API effectivement exposée

Les familles d'API notables du backend courant sont :

- caisses : `/api/cash-registers/**`
- caissiers et profils : `/api/cashiers/**`
- sessions : `/api/sessions/**`, `/api/cashier/sessions`
- comptes et transferts : `/api/admin/accounts`, `/api/cashier/accounts`,  
`/api/accounts/transfer`, `/api/accounts/withdraw`, `/api/accounts/transfer-p2p`
- bills : `/api/bills`, `/api/cashier/bills`, `/api/bills/pay`
- lien comptable explicite : `/api/bills/import/accounting-invoices/{invoiceId}`  
et `/api/bills/{billId}/sync-linked-service`
- mouvements et transactions : `/api/cashier/movements`, `/api/transactions`
- rapprochements, audit, notifications et rapports : `/api/reconciliations/**`,  
`/api/audit`, `/api/notifications`, `/api/reports/**`

Cette surface fait du cashier-core un core métier complet, distinct de la trésorerie bancaire et désormais documenté comme tel.

# 17 administration-core — Gouvernance et Administration

---

## 17.1 Description du core

L'**administration-core** est le module de gouvernance transversale de la plateforme. Il regroupe les fonctions d'administration système qui ne relèvent pas d'un domaine métier précis : approbation des entités soumises à validation (*BusinessActors*, *Organisations*, *Agences*), gestion des options de plateforme, consultation de la piste d'audit, et orchestration des rôles et permissions à l'échelle du périmètre administré.

Ce core opère avec les permissions les plus élevées et expose des routes protégées par des rôles d'administration. Il coordonne avec les autres cores via des ports sortants et des événements, sans implémenter de logique métier opérationnelle.

## 17.2 Rôle principal

**Rôle principal** : Orchestrer la gouvernance des *BusinessActors*, des *Organizations* et des *Agencies*, configurer les politiques plateforme, et fournir la surface d'administration centralisée pour les rôles, l'audit et les paramètres globaux.

## 17.3 Rôles secondaires

### 1. Approbation des entités soumises à validation

Flux de gouvernance pour les *BusinessActors* (soumission, approbation/rejet), les *Organisations* et les *Agences* lorsque les options de plateforme exigent une validation manuelle.

### 2. Options de plateforme

Paramètres globaux du tenant contrôlant les comportements de self-service, notamment :

— `allowOrganizationSelfServiceCreation`

- requireBusinessActorApproval
- allowRoleCloning

### 3. Consultation de la piste d'audit

Interface de lecture de l'AdminAuditEntry, exposée aux administrateurs avec filtres par acteur, type d'action, date, organisation et cible.

### 4. Gestion centralisée des rôles et utilisateurs

Surface d'API unifiée pour les opérations d'administration des rôles (création, clonage, affectation) réexposées depuis le roles-core avec vérifications de politique supplémentaires.

### 5. Supervision des métriques opérationnelles

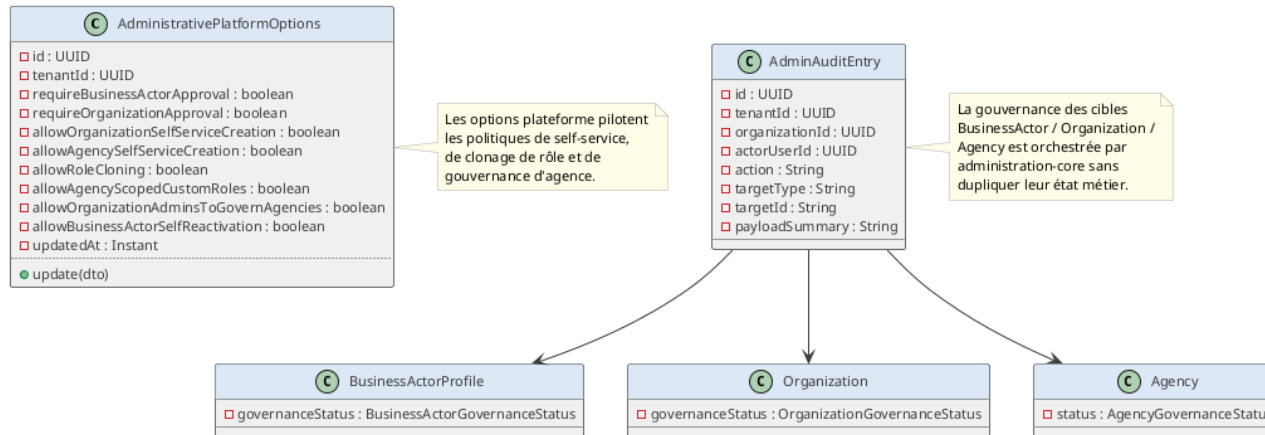
Accès aux indicateurs Actuator (santé, métriques, état de l'outbox) via le port de management sécurisé.

### 6. Gouvernance multi-scope

Distinction explicite entre administration générale, administration d'organisation et administration d'agence, avec contrôle fin des permissions de gouvernance.



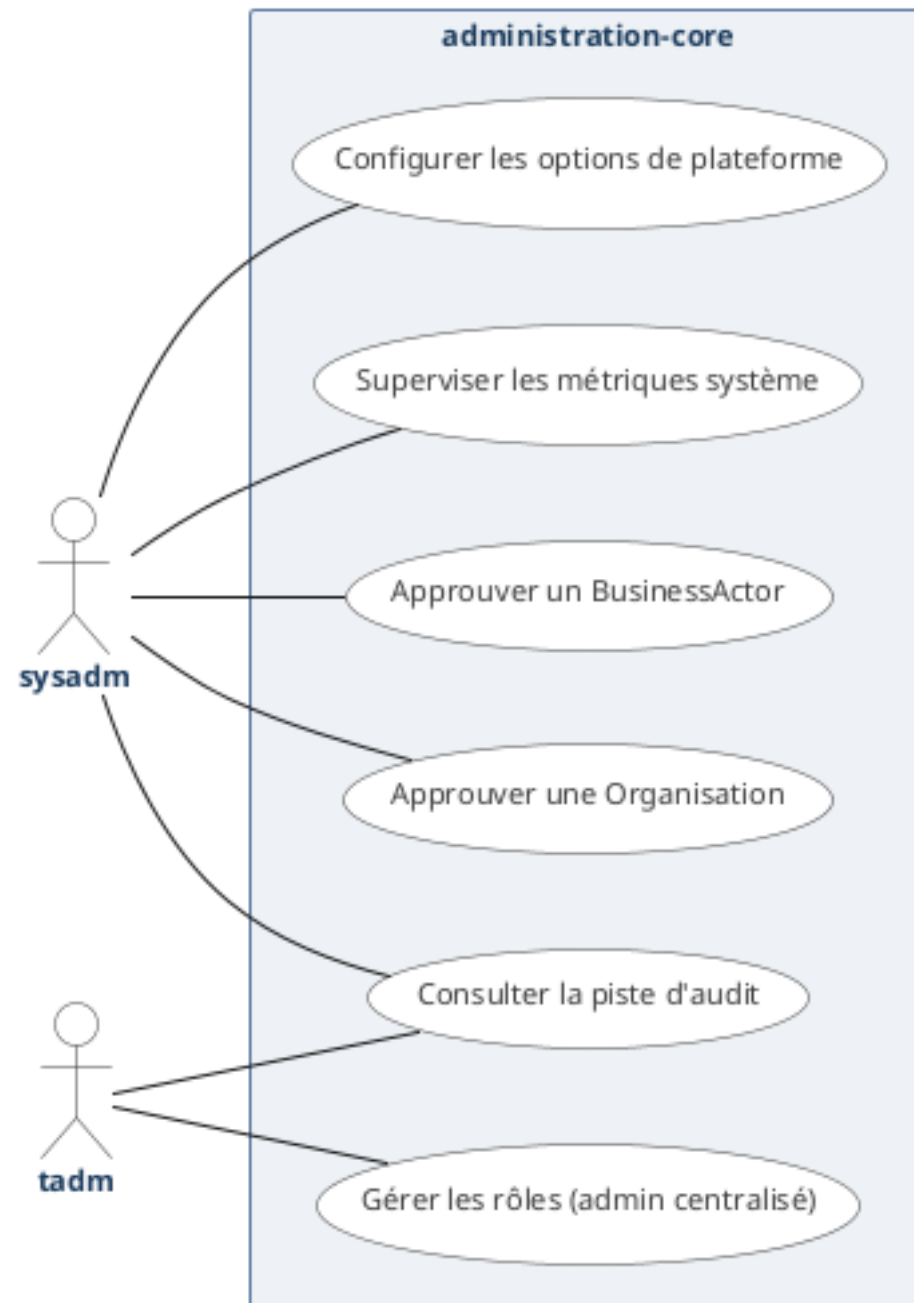
## Diagramme de classes — administration-core



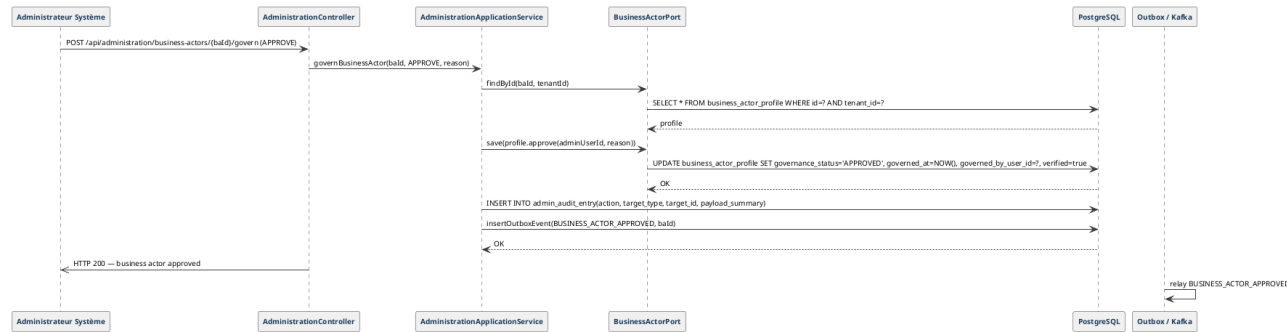
**Invariants du administration-core :**

- Les `AdministrativePlatformOptions` sont uniques par tenant.
- Une décision de gouvernance appliquée est immuable dans sa piste d’audit.
- La modification des options de plateforme est tracée dans l’`AuditTrail`.
- L’`AdminAuditEntry` est en lecture seule après écriture.

## Diagramme des cas d'utilisation — administration-core



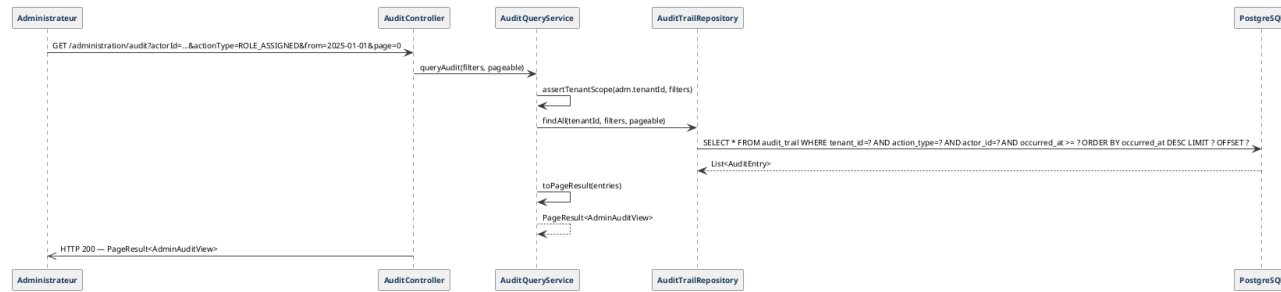
## DS-AD-01 — Approbation d'un BusinessActor



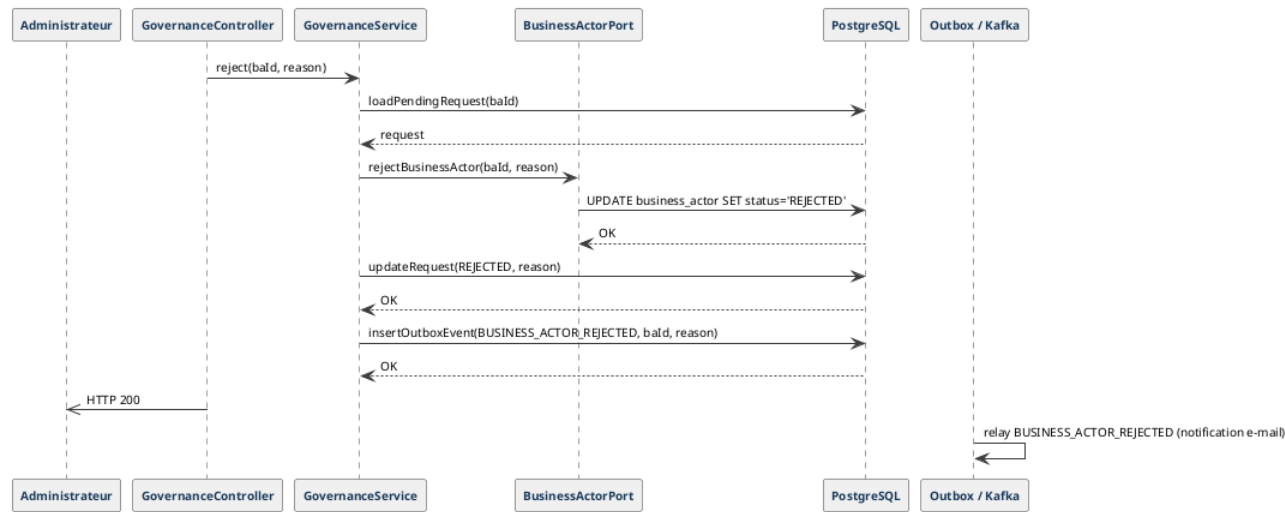
## DS-AD-02 — Mise à jour des options de plateforme avec audit



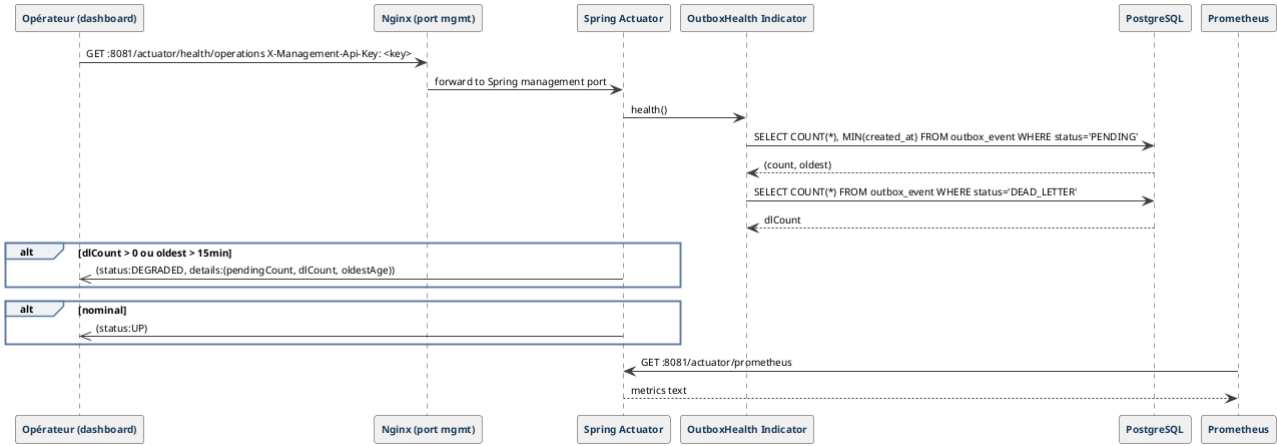
## DS-AD-03 — Consultation filtrée de la piste d’audit



## DS-AD-04 — Rejet d'une demande de gouvernance avec notification

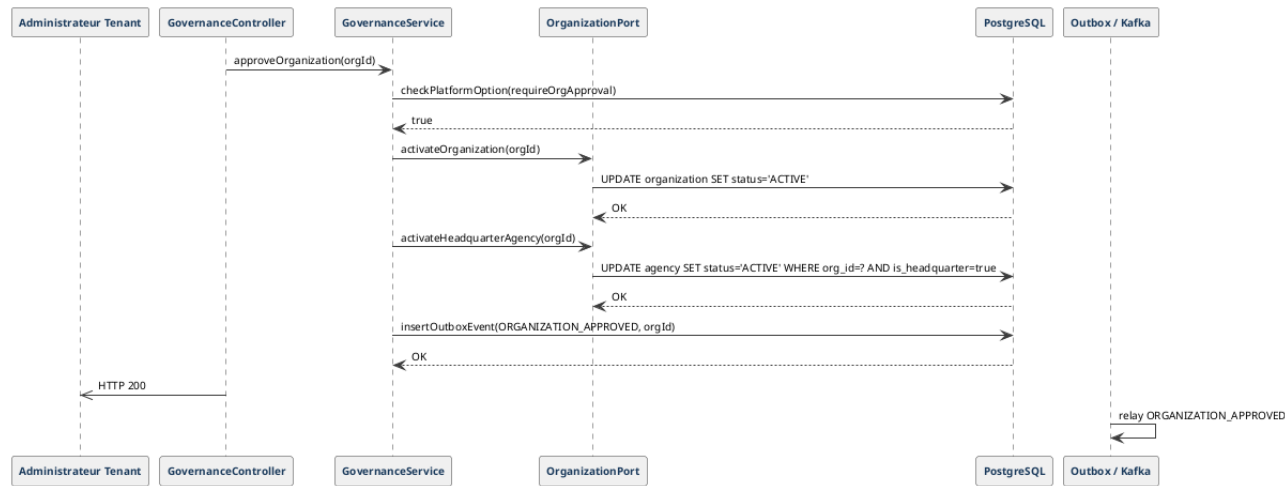


# DS-AD-05 — Supervision de l'état du système via le port de management





## DS-AD-06 — Approbation d'une organisation soumise à validation



# 18 file-core — Gestion des Fichiers

---

## 18.1 Description du core

Le **file-core** n'est plus seulement un mini service d'upload binaire. Dans l'état courant du backend, il regroupe trois capacités distinctes mais cohérentes : le stockage de fichiers bruts (**StoredFile**), le hub documentaire de rattachement métier (**DocumentLink**) et la gouvernance documentaire (**DocumentGovernancePolicy**, **DocumentReview**).

Le stockage de base permet de téléverser un fichier, de normaliser son nom, de contrôler son type MIME déclaré contre une liste autorisée configurable, de vérifier sa taille maximale et de l'écrire sur disque local sous une racine configurable. Les métadonnées persistées restent volontairement simples : organisation, utilisateur ayant téléversé, nom, type de contenu, taille et chemin de stockage relatif.

Le **DocumentHub** ajoute ensuite une couche fonctionnelle de liaison entre un fichier stocké et une cible métier quelconque (**targetType/targetId**), avec catégorie documentaire et label. Le module expose des vues par cible et par organisation, ainsi qu'un aperçu agrégé du hub documentaire d'une organisation.

Le sous-système de **DocumentGovernance** complète enfin ce hub avec des politiques documentaires par organisation ou agence : document obligatoire, approbation requise, durée d'expiration et responsabilité de revue. Les revues produisent des **DocumentReview** persistées, et la lecture du statut d'une cible combine les liaisons documentaires, la politique applicable et la dernière revue connue.

## 18.2 Rôle principal

**Rôle principal** : Fournir un service documentaire unifié couvrant le stockage brut des fichiers, leur rattachement aux entités métier et la gouvernance documentaire par organisation ou agence.

## 18.3 Rôles secondaires

### 1. Stockage binaire local et isolé par tenant

Les binaires sont écrits via **LocalDiskFileBinaryStorage** sous une racine locale configurable. Le chemin final est validé pour empêcher une sortie de la racine de stockage.

## 2. Validation d'entrée à l'upload

Le service vérifie le nom de fichier, le type de contenu déclaré et la taille maximale configurable. La validation actuelle repose sur le `contentType` reçu et non sur une analyse *magic bytes* ou un antivirus intégré.

## 3. Métadonnées de fichier simples

`StoredFile` conserve l'organisation, l'utilisateur source, le nom, le type MIME, la taille et le chemin de stockage. Le modèle courant ne porte ni checksum, ni flag de suppression logique, ni URL présignée.

## 4. Hub documentaire transversal

`DocumentLink` rattache un fichier à une cible métier via `targetType`, `targetId`, `documentCategory` et un label optionnel.

## 5. Gouvernance documentaire

`DocumentGovernancePolicy` définit les exigences de gouvernance par type de cible et catégorie documentaire, au niveau organisation ou agence.

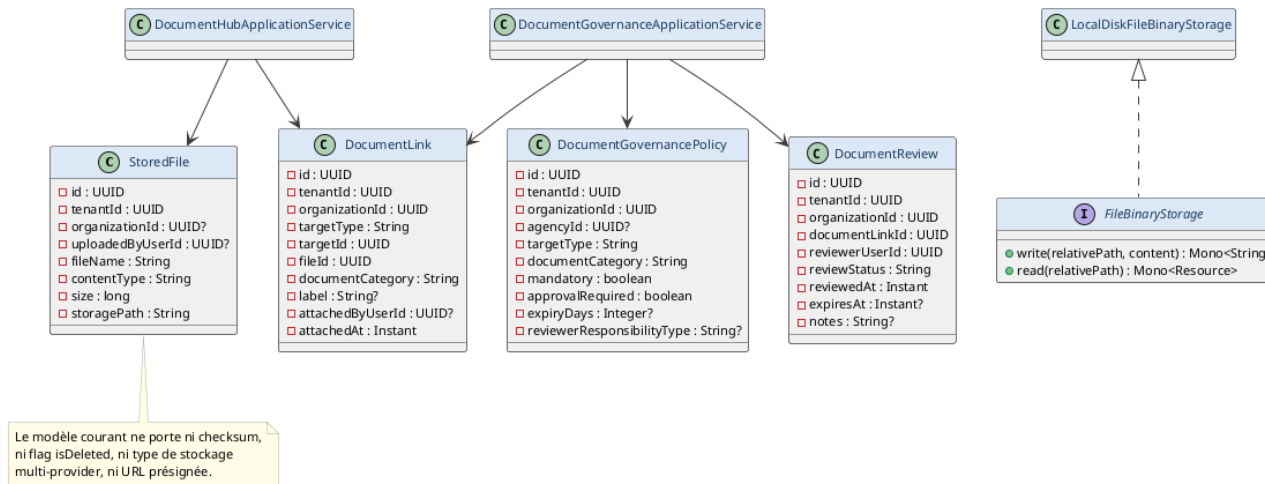
## 6. Revues documentaires

`DocumentReview` trace une décision de revue pour un document lié, avec statut, date de revue, expiration éventuelle et notes.

## 7. Vues d'ensemble documentaires

Le core expose un aperçu agrégé des documents d'une organisation, ainsi qu'un aperçu spécifique de la gouvernance documentaire.

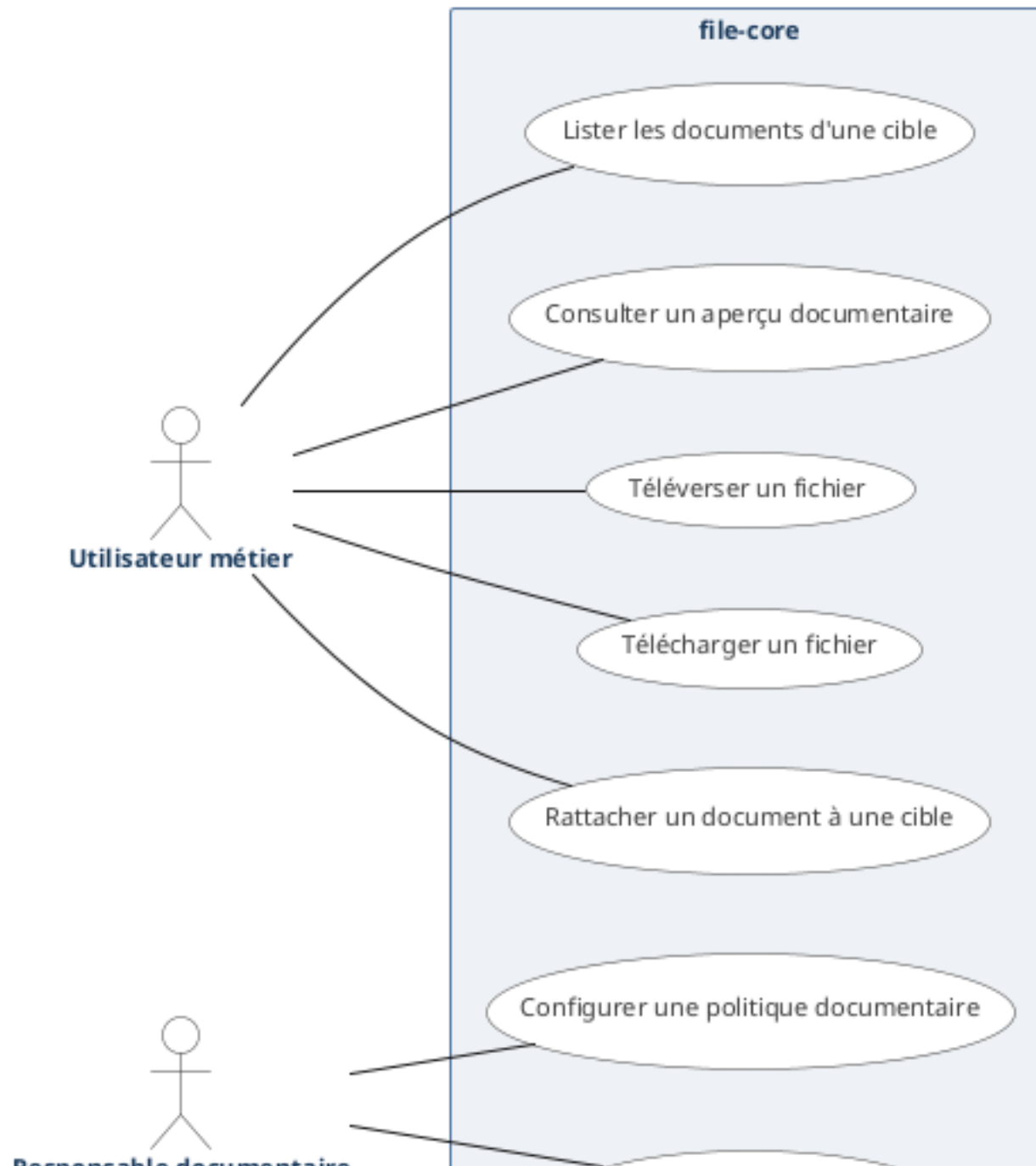
## Diagramme de classes — file-core



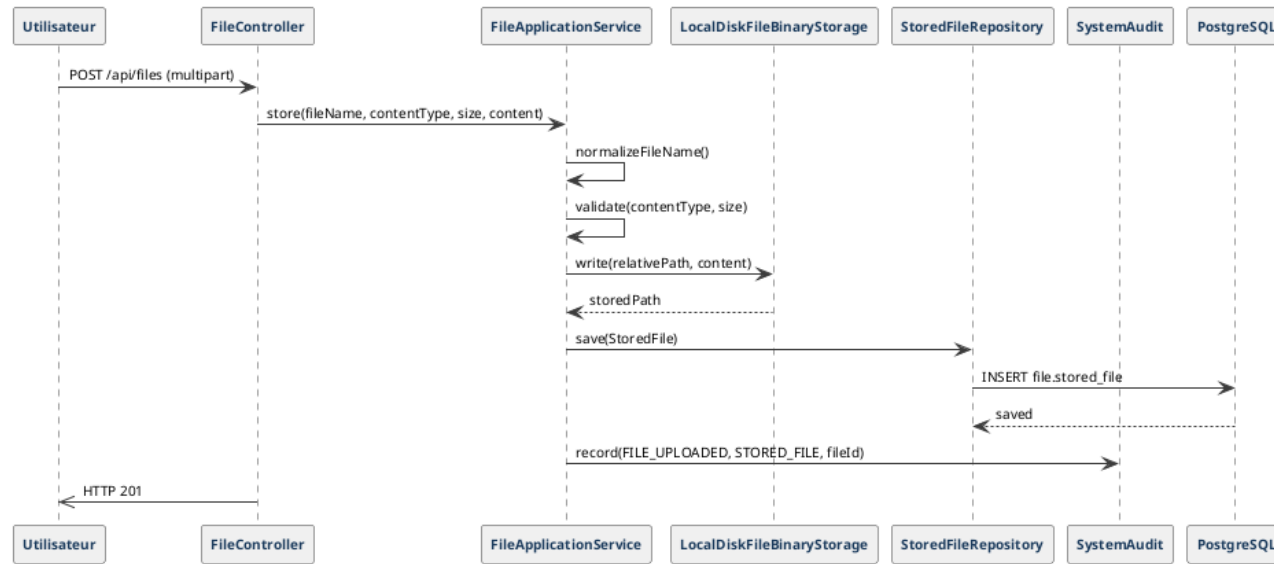
**Invariants du file-core :**

- Le chemin de stockage reste confiné à la racine locale configurée.
- Un `DocumentLink` appartient toujours à une organisation donnée.
- Une `DocumentGovernancePolicy` est unique par (`tenantId`, `organizationId`, `agencyId`, `targetType`, `documentCategory`).
- Une `DocumentReview` référence toujours un `DocumentLink` existant du même tenant.
- Le backend courant ne fournit pas de suppression logique native ni de génération d'URL présignée.

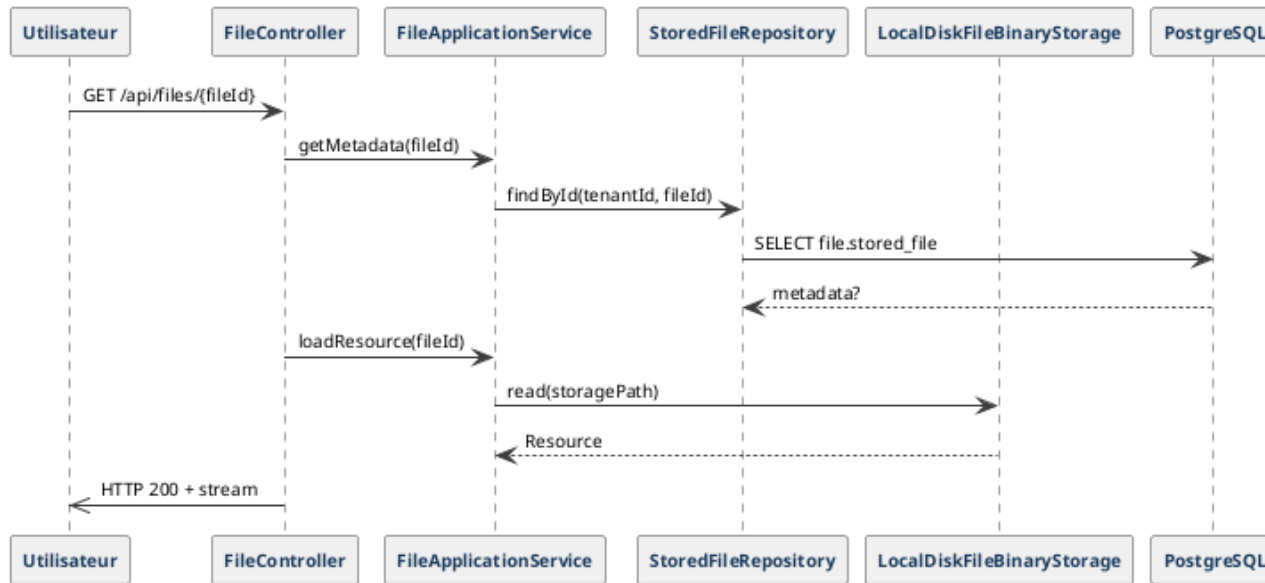
## Diagramme des cas d'utilisation — file-core



## DS-F-01 — Téléversement d'un fichier et audit système

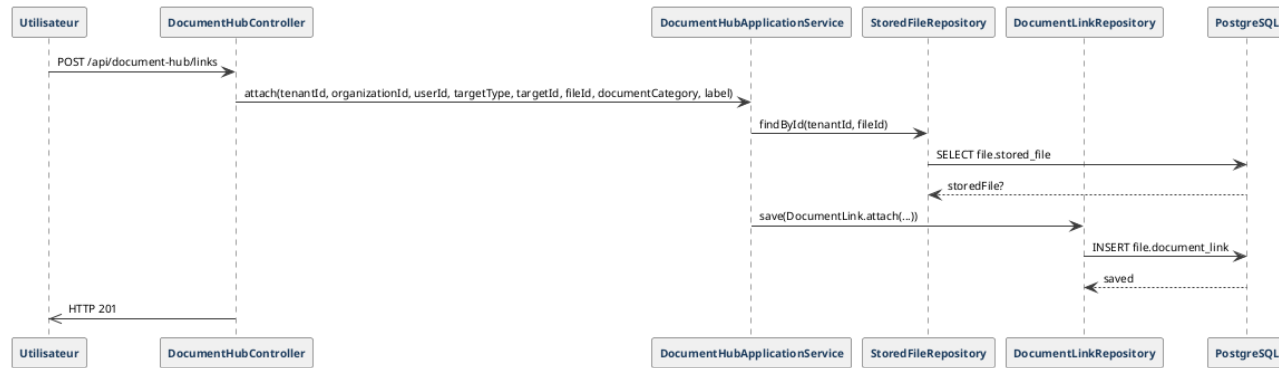


## DS-F-02 — Téléchargement d'un fichier stocké

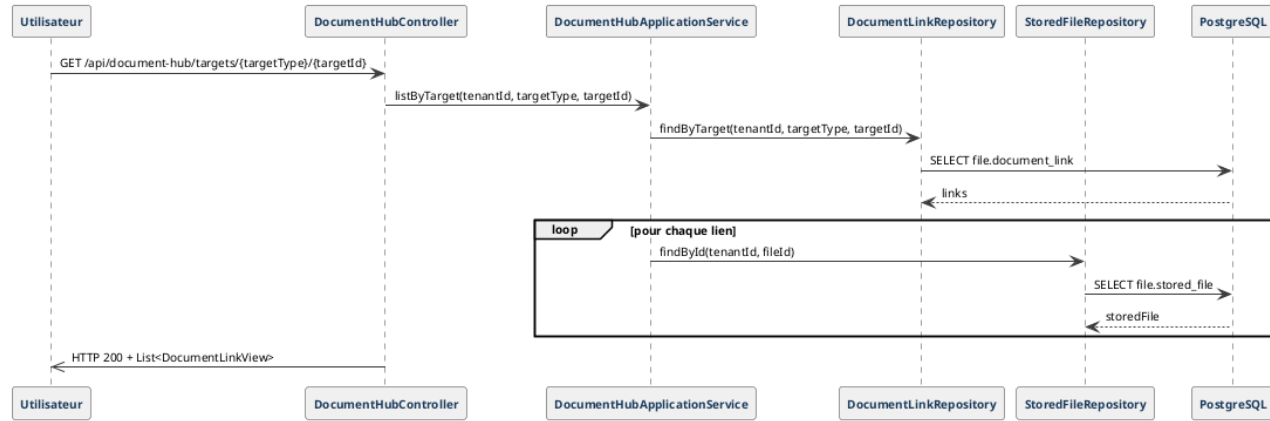




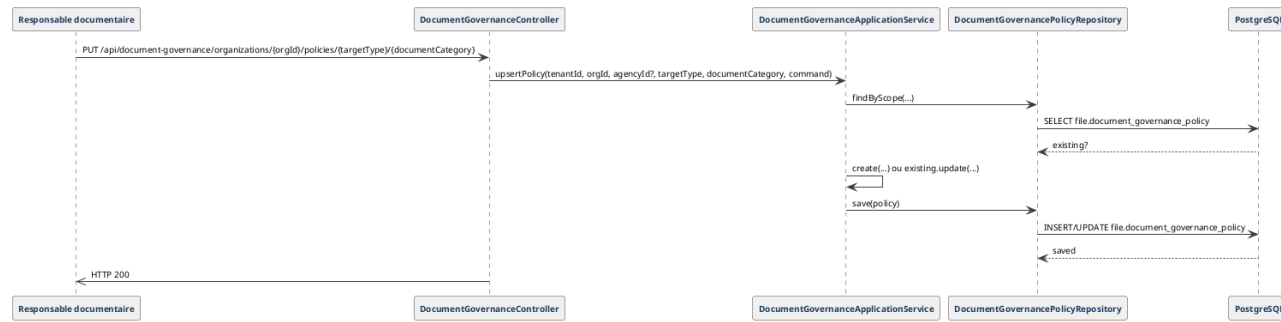
## DS-F-03 — Rattachement d'un document à une cible métier



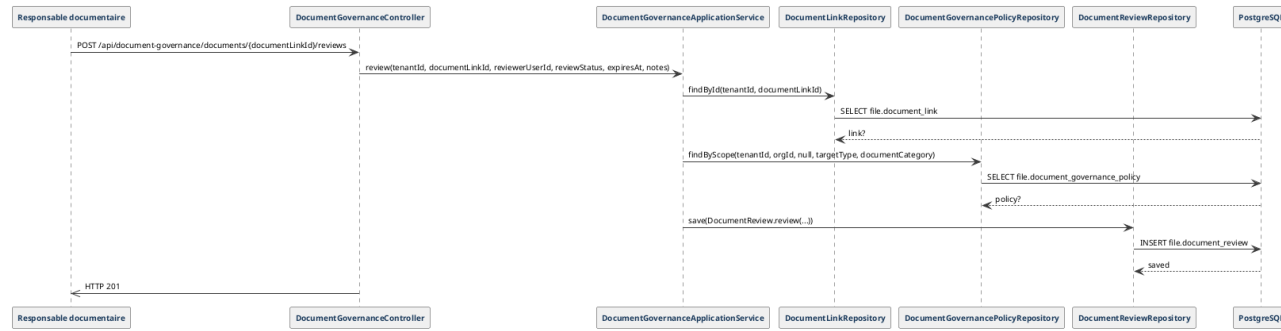
## DS-F-04 — Consultation des documents d'une cible



## DS-F-05 — Mise à jour d'une politique de gouvernance documentaire



## DS-F-06 — Revue d'un document lié avec expiration éventuelle



## 19 hrm-core

---

### 19.1 Description du core

Le **hrm-core** porte le domaine ressources humaines et paie du backend. Il couvre le cycle collaborateur complet : création d'employé, contrats, ayants droit, soldes et demandes de congés, avances sur salaire, temps, paie, recrutement, onboarding, formation, compétences, évaluations, missions, frais professionnels, médical/HSE, déclarations sociales et KPI RH.

Le module est exposé sous `/api/v1/hrm/**`. Il suit les mêmes règles transverses que les autres cores : isolation par `tenantId`, contexte organisationnel, sécurité par permissions RBAC et persistance R2DBC.

### 19.2 Rôle principal

**Rôle principal** : fournir un sous-système RH complet, exploitable par organisation, permettant de gérer les dossiers collaborateurs, les processus RH, la paie et les obligations sociales sans sortir du modèle multi-tenant du kernel.

### 19.3 Sous-domaines couverts

#### 1. Employés, contrats et ayants droit

Les endpoints `/api/v1/hrm/employees` gèrent la création, consultation, mise à jour, suspension, réactivation et terminaison des employés. Les contrats et ayants droit sont rattachés au dossier employé via `/contracts` et `/dependents`.

#### 2. Congés et soldes

`/api/v1/hrm/leaves` porte la soumission, l'approbation, le rejet, l'annulation et la consultation des demandes. Les soldes de congés restent attachés à l'employé et doivent refléter les consommations approuvées.

#### 3. Paie et avances

`/api/v1/hrm/payroll/run` lance un calcul de paie pour une période. Les runs peuvent être validés, consultés, et leurs entrées exposent les bulletins. Les avances sur salaire sont gérées par `/api/v1/hrm/loan-advances`.

#### 4. Temps et présence

`/api/v1/hrm/timesheets` gère la création, la soumission, la validation et la consultation des feuilles de temps par employé ou par période.

#### 5. Recrutement et onboarding

`/api/v1/hrm/job-offers`, `/applications`, `/interviews` et `/onboarding-tasks` couvrent les offres, candidatures, entretiens, embauche et tâches d'intégration.

#### 6. Formation, compétences et GPEC

`/api/v1/hrm/trainings`, `/training-budgets` et `/skills` couvrent le plan de formation, les inscriptions, les budgets engagés/réalisés et les compétences attachées aux employés.

#### 7. Évaluations et performance

`/api/v1/hrm/reviews` porte les revues, objectifs, soumission, acknowledgement et finalisation des évaluations.

#### 8. Frais, missions et mobilité

`/api/v1/hrm/expenses` gère les notes de frais et lignes de frais.  
`/api/v1/hrm/mission-orders` porte les ordres de mission, approbation, démarrage, clôture ou annulation.

#### 9. Médical, HSE, déclarations et KPI

`/api/v1/hrm/medical` gère visites et certificats médicaux.  
`/api/v1/hrm/declarations` porte les déclarations sociales. `/api/v1/hrm/kpi` expose les instantanés de pilotage RH.

## 19.4 Intégrations inter-cores

- **actor-core** : résolution des acteurs rattachés aux employés.
- **file-core** : rattachement de documents RH, justificatifs, certificats et pièces liées aux dossiers.
- **settings-core** : lecture de paramètres RH, règles de congés et options de gouvernance.
- **kernel-core** : application du contexte tenant, sécurité, audit, quotas et outbox.

### Invariants du hrn-core :

- Un employé appartient toujours à un tenant et une organisation.
- Une paie validée ne doit plus être recalculée implicitement.
- Une demande de congé approuvée consomme un solde disponible.
- Les processus de recrutement et onboarding avancent par transitions explicites, jamais par mutation libre de statut.

- Les remboursements de frais et avances doivent conserver leurs états d'approbation avant paiement ou remboursement.
- Les données médicales et sociales restent rattachées au scope organisationnel et au dossier employé concerné.

## 20 blockchain-core

---

### 20.1 Description du core

Le **blockchain-core** est un module de registre interne append-only. Il ne s'agit pas d'un simple stockage mémoire : le backend expose un domaine complet avec wallets, signatures, transactions, blocs, preuve de travail, racine de Merkle, validation de chaîne et persistance R2DBC.

Le core est accessible sous `/api/v1/blockchain`. Il est aussi branché sur l'outbox métier du kernel via un **BusinessEventDeliverySink** : les événements métier publiés par les autres cores peuvent être ancrés dans la chaîne interne pour obtenir une preuve d'intégrité vérifiable.

### 20.2 Rôle principal

**Rôle principal** : fournir une chaîne interne multi-tenant et organisation-scoped pour ancrer des transactions métier, des documents et des événements outbox, avec validation cryptographique et consultation auditable.

### 20.3 Capacités exposées

#### 1. Wallets

POST `/api/v1/blockchain/wallets` génère une paire de clés et persiste le wallet public. GET `/wallets` liste les wallets actifs d'une organisation.

#### 2. Signature et payload canonique

POST `/transactions/signing-payload` construit le payload à signer. POST `/crypto/sign` signe un payload avec une clé privée. La transaction finale vérifie la signature avant acceptation.

#### 3. Transactions métier

POST `/transactions` soumet une transaction signée avec `transactionType`, `sourceService`, `sourceReference`, `payload`, `hash de payload`, `clé publique` et



signature. `GET /transactions` liste les transactions d'une chaîne organisationnelle.

#### 4. Ancrage documentaire

`POST /anchors` crée une transaction d'ancrage à partir d'un hash documentaire, du service source et de la référence source.

#### 5. Minage et blocs

`POST /mine` regroupe les transactions pendantes, calcule la racine de Merkle, recherche un nonce selon la difficulté configurée et produit un bloc chaîné au hash précédent. `GET /blocks` et `GET /blocks/{blockId}/transactions` exposent la lecture.

#### 6. Validation de chaîne

`GET /validate` reconstruit la cohérence des hauteurs, hashes, previous hashes et racines Merkle pour détecter toute rupture.

## 20.4 Persistance et modèle

La migration `V66_blockchain_core.sql` crée les tables `blockchain_wallet`, `blockchain_block` et `blockchain_transaction`. Les index garantissent l'unicité des wallets par fingerprint, des blocs par hauteur/hash et des transactions par hash dans le scope `tenantId + organizationId + chainCode`.

Le modèle domaine repose sur :

- `BlockchainWallet` : identité cryptographique publique ;
- `BlockchainTransaction` : payload signé, hash, statut `PENDING` ou `MINED`, rattachement bloc ;
- `BlockchainBlock` : hauteur, hash précédent, Merkle root, nonce, difficulté, compteur de transactions ;
- `ChainValidationReport` : résultat de validation de chaîne.

## 20.5 Ancrage transversal du projet

Le `BlockchainOutboxEventSink` consomme les événements métier issus de `kernel.outbox_event`. Il calcule un hash canonique de l'événement, crée une transaction blockchain de type événement métier, puis mine un bloc selon la stratégie configurée. Cela permet d'ancrer les actions importantes des autres cores sans coupler leurs domaines à la logique blockchain.

Cette intégration ne remplace pas l'audit kernel : elle ajoute une preuve d'intégrité et de non-réécriture sur les événements publiés.

## 20.6 Sécurité

Chaque endpoint est protégé par une permission explicite : `blockchain:wallet:create`, `blockchain:wallet:read`, `blockchain:transaction:sign`, `blockchain:transaction:create`, `blockchain:transaction:read`, `blockchain:anchor:create`, `blockchain:block:mine`, `blockchain:block:read` et `blockchain:chain:validate`.

### Invariants du blockchain-core :

- Une chaîne est toujours scoped par `tenantId`, `organizationId` et `chainCode`.
- Une transaction ne peut être minée que si sa signature et son hash sont valides.
- Un bloc non-genesis doit pointer vers le hash du dernier bloc connu.
- La racine de Merkle du bloc doit être recalculable à partir des transactions minées.
- Une transaction minée ne retourne pas à l'état pending.
- La validation doit détecter toute rupture de hauteur, hash précédent, hash de bloc ou Merkle root.

## 20.7 Flow Postman de référence

Le flow exploitable est versionné dans `iwm-backend/docs/postman/blockchain-core-flow.postman`. Il couvre la création de wallet, la préparation du payload de signature, la signature, la soumission de transaction, l'ancrage documentaire, le minage, la consultation des blocs/transactions et la validation de chaîne.

# 21 Synthèse opérationnelle

---

## 21.1 Role dans la lecture projet

Ce chapitre relie la lecture fonctionnelle et technique précédente aux documents d'exploitation présents dans `iwm-backend/docs`. Il ne remplace pas ces fichiers : ils restent les sources de détail pour les commandes, les checklists, les scripts et les runbooks. Son rôle est de fixer, dans le cahier général, les décisions de production, de déploiement et de gouvernance qui encadrent les cœurs métier.

Le backend est considéré comme un socle modulaire exploitable et prêt pour la suite du développement professionnel. Il est également apte à entrer en durcissement préproduction. En revanche, le déploiement production sans restriction reste conditionné par l'industrialisation des secrets, du déploiement ciblé, des tests de charge et des procédures de résilience.

## 21.2 Corpus documentaire backend

Les documents de référence du backend sont organisés autour de cinq familles :

- **Gel et parité** : `socle-freeze.md`, `legacy-parity-freeze.md`, `ksm-v0-1-parity-matrix.md` et `post-ksm-v0-1-roadmap.md`. Ces documents établissent que la parité utile avec KSM\_V0.1 est atteinte et que les évolutions futures doivent enrichir le socle sans recopier le legacy.
- **Runtime et déploiement** : `deployment-runtime.md`, `docker-industrialization.md`, `local-infrastructure.md`, `deploy-kernel-cashier-stack.md` et `preprod-prod-checklist.md`. Ils décrivent les profils locaux, préproduction et production, les compose files, les secrets montés par fichiers et les validations runtime.
- **Exploitation et résilience** : `operations-readiness.md`, `incident-runbooks.md`, `resilience-drills.md`, `capacity-planning.md`, `performance-and-load.md` et `scaling-strategy.md`. Ils couvrent les endpoints management, les métriques, les alertes, les paliers de capacité et les drills critiques.

- **Gouvernance et integration** : `client-applications-design.md`, `organization-service-subscriptions.md`, `external-backend-integration.md`, `external-platform-jwt-integration.md`, `central-oidc-provider.md`, `platform-governance-design.md` et `plan-catalogs.md`. Ils fixent les contrats `ClientApplication`, les abonnements organisationnels, les quotas, le provider OIDC central et les integrations externes.
- **Roadmaps de durcissement** : `production-readiness-audit.md`, `hardening-roadmap.md`, `domain-enrichment-roadmap.md`, `administration-core-design.md` et `notification-core-design.md`. Ils listent les chantiers encore ouverts sans remettre en cause l'architecture modulaire.

## 21.3 Readiness production

Le verdict courant du document `production-readiness-audit.md` est le suivant :

- le socle est coherent, modulaire, teste et exploitable ;
- la poursuite du developpement professionnel est autorisee ;
- l'entree en durcissement preproduction est autorisee ;
- le deploiement production sans restriction n'est pas encore autorise.

Les capacites considerees pretes sont l'architecture hexagonale modulaire, le runtime PostgreSQL reactif, les migrations Liquibase, l'outbox persistante avec relay Kafka, les projections persistantes, Redis pour les quotas et le cache de permissions, Elasticsearch pour la recherche, l'observabilite Actuator / Prometheus / Grafana et une CI separee par familles de verification.

Les chantiers bloquants avant production sont :

1. industrialiser les secrets et les identites runtime ;
2. durcir les cas d'usage d'administration, de bootstrap et de securite ;
3. encadrer l'exploitation Kafka, les dead letters et les plans de rejeu ;
4. qualifier la charge HTTP, Kafka, Redis et Elasticsearch ;
5. verrouiller le packaging, les probes, les ressources et les limites de deploiement.

## 21.4 Contrat de deploiement

Le chemin de deploiement cible repose sur des profils explicites : `local`, `preprod` et `prod`. Les secrets ne doivent pas etre codes dans Git ; ils sont montes par fichiers et lus par l'entree container. Les scripts de validation runtime doivent etre executes avant le demarrage des stacks preproduction ou production.

Les variables de capacite ne sont pas implicites. Les pools R2DBC, la concurrence Kafka, les tailles de batch outbox, les quotas tenant, les quotas organisationnels et les limites Nginx sont des parametres d'environnement pilotes et qualifies par palier.

Le runtime local de reference est base sur Java 21, Spring Boot 4, PostgreSQL 18, Kafka en mode KRaft, Redis, Elasticsearch, Nginx, Prometheus et Grafana. Le scaling applicatif se fait derriere Nginx par replicas Docker ; il ne consiste pas a exposer directement plusieurs instances applicatives.

## 21.5 Observabilite et exploitation

La surface d'exploitation est fondee sur :

- `/actuator/health` pour le health public minimal ;
- `/actuator/health/operations` pour la lecture operationnelle ;
- `/actuator/metrics` et `/actuator/prometheus` pour les metriques ;
- `/api/observability/runtime` pour la vision runtime applicative ;
- `/api/observability/outbox/summary` et `/api/observability/projections/summary` pour l'etat outbox et projections par tenant.

Les endpoints management autres que `health` et `info` doivent etre proteges par `X-Management-API-Key`. En preproduction et production, le port management doit etre separe quand l'infrastructure le permet, et l'exposition Prometheus doit rester interne ou strictement controlee.

Les alertes recommandees concernent notamment les dead letters outbox, l'age du plus vieux pending, les echecs du relay Kafka, le silence des projections, le lag Kafka, les rejets de quotas tenant ou organisation et la degradation du health `operations`.

## 21.6 Capacite, charge et scaling

Le dimensionnement doit rester mesure par paliers. Les mesures de reference actuelles ont ete obtenues avec deux replicas applicatifs derriere Nginx, plus PostgreSQL, Kafka, Redis, Elasticsearch, Prometheus et Grafana.

Les resultats disponibles indiquent :

- une charge standard validee avec 0.00% d'echec HTTP et un p95 inferieur a 100ms sur les scenarios de base ;
- une panne PostgreSQL absorbee sans perte HTTP, avec pic de latence transitoire pendant la coupure ;
- une panne Kafka qui ne degrade pas la disponibilite immediate de l'API, Kafka etant dimensionne surtout pour le rattrapage d'outbox ;
- un restart simultane de deux replicas qui produit une indisponibilite transitoire, ce qui impose des redemarrages progressifs ;

- des scenarios spike et saturation plafonnes par les protections Nginx et quotas backend avant effondrement de latence.

Les paliers de depart recommandes sont :

- validation locale : `app=2`;
- preproduction critique : `app=3` minimum;
- production initiale : `app=4`;
- rolling restart obligatoire des que la continuite de service est un critere d'acceptation.

PostgreSQL reste la source de verite et le premier point de vigilance. Redis et Elasticsearch sont des accelerateurs; ils ne doivent pas devenir des stores primaires. Kafka absorbe les pics et decouple les projections, mais ne doit pas masquer un backlog outbox non traite.

## 21.7 Resilience et runbooks

Les drills de resilience a maintenir sont :

- rejeu massif de l'outbox;
- panne PostgreSQL;
- panne Kafka;
- redemarrage sous charge.

La lecture d'incident demarre par le health operationnel, puis par les metriques outbox, l'age du backlog, l'etat des projections, le topic Kafka principal et son dead-letter, Redis si les quotas ou le cache permissions sont actifs, et Elasticsearch si la recherche est active.

Les runbooks couvrent les saturations HTTP et 429, les quotas organisationnels, les backlogs outbox, les pannes PostgreSQL, les pannes Kafka et les degradations Elasticsearch. Ces procedures doivent rester des documents d'exploitation separes afin de pouvoir evoluer plus vite que le cahier d'analyse.

## 21.8 Gouvernance des integrations

Les integrations externes ne contournent pas le kernel. Un backend consommateur doit etre represente par une `ClientApplication`, appeler avec `X-Client-Id`, `X-API-Key`, `X-Tenant-Id`, `X-Organization-Id` quand le contexte organisationnel est requis, et un bearer utilisateur pour les operations sensibles.

L'ordre des controles runtime reste :

1. authentifier la `ClientApplication`;
2. verifier les services autorises;
3. verifier le tenant;

4. verifier l'abonnement organisationnel au service ;
5. appliquer les quotas tenant, client, service et organisation ;
6. valider les permissions utilisateur metier.

Le cas `cashier-service` illustre ce contrat : le service externe ne doit pas reutiliser durablement un client bootstrap. Il doit utiliser une `ClientApplication` dediee, typiquement `cashier-backend`, avec les services autorises et abonnements organisationnels correspondants.

## 21.9 Guide de connexion d'un backend externe

Pour connecter un backend externe au kernel, l'equipe integratrice doit traiter l'integration comme un contrat serveur-vers-serveur, pas comme un simple appel REST.

1. **Declarer le backend** : creer une `ClientApplication` avec un `clientId` stable, un secret propre et un perimetre `allowedServices` minimal.
  2. **Protger le secret** : stocker le secret dans l'environnement ou le gestionnaire de secrets du backend consommateur. Il ne doit jamais etre livre au frontend ni versionne dans Git.
  3. **Porter le contexte** : envoyer `X-Client-Id`, `X-API-Key`, `X-Tenant-Id`, le bearer utilisateur et `X-Organization-Id` quand le service est scope organisation.
  4. **Verifier les abonnements** : une organisation doit etre abonnee au service appele et a ses dependances. Par exemple `CASHIER` depend de `ACCOUNTING`.
  5. **Observer les quotas** : les refus 429 peuvent venir du quota backend `tenant + client + service` ou du quota metier `organization + service`.
  6. **Verifier les permissions** : meme si le backend est autorise, le JWT utilisateur doit porter les droits metier attendus dans le bon scope.
- `CLIENT_APPLICATION_SERVICE_NOT_ALLOWED` : le backend consommateur n'est pas autorise sur le service de la route.
  - `TENANT_REQUEST_QUOTA_EXCEEDED` : le quota plateforme du triplet `tenant + client + service` est atteint.
  - `ORGANIZATION_CONTEXT_REQUIRED` : l'appel vise un service metier organisationnel sans `X-Organization-Id`.
  - `ORGANIZATION_SERVICE_NOT_SUBSCRIBED` : l'organisation cible n'est pas abonnee au service demande.
  - `ORGANIZATION_SERVICE_QUOTA_EXCEEDED` : le quota fair-use de l'organisation pour ce service est atteint.
  - **Refus RBAC** : le token utilisateur est valide, mais ses permissions ne couvrent pas l'action.

Un frontend consommateur ne doit jamais appeler le kernel avec le secret d'une `ClientApplication`. Il appelle son backend. Le backend relaie ensuite vers le kernel avec son identite machine et le bearer utilisateur.

Les documents detaillés pour cette integration sont :

- `external-backend-integration.md`;
- `external-platform-jwt-integration.md`;
- `client-applications-design.md`;
- `organization-service-subscriptions.md`.

## 21.10 Roadmap encadree

Les roadmaps actuelles ouvrent trois axes :

- enrichir le metier au-dela du legacy : approbations plus fines, politiques de prix, remises avancees, rapprochement bancaire, maintenance planifiee et analytics commerciaux ;
- durcir le produit : secrets, politiques d'accès, alerting, runbooks, charge et resilience ;
- ameliorer l'experience equipe : tests de parite lisibles, contrats par module, garde-fous contre les logiques cross-module implicites.

Toute nouvelle evolution doit conserver la parite legacy, ne pas contourner Liquibase, Kafka, Redis ou Elasticsearch, passer par un port et un adapter pour les integrations externes, et mettre a jour les documents d'exploitation si elle modifie le runtime.



# Conclusion Générale

---

## Synthèse de l'architecture

Ce cahier d'analyse documente vingt cores fonctionnels ainsi que leur assemblage opérationnel au sein de la plateforme **RT-Comops Backend**. L'ensemble forme un **monolithe modulaire à architecture hexagonale**, conçu pour répondre aux exigences de maintenabilité, de sécurité et d'évolutivité d'un système multi-tenant industriel.

Chaque core respecte les mêmes conventions structurelles :

- une **couche domaine** indépendante de toute infrastructure, exposant ses contrats via des interfaces entrantes (*port/in*) et sortantes (*port/out*) ;
- des **adaptateurs** qui implémentent ces interfaces sans polluer la logique métier (contrôleurs WebFlux, repositories R2DBC, clients Kafka/Redis/Elasticsearch) ;
- un **cloisonnement strict** par tenant, garanti par l'injection du contexte tenant dans la chaîne réactive, la persistance systématique de `tenantId` et les contrôles de sécurité/quota associés, sans remettre en cause le modèle métier canonique centré sur `BusinessActor`, `Organization`, `Agency`, `Actor` et `ThirdParty`.

## Cohérence inter-cores

La communication entre cores repose sur trois mécanismes complémentaires, décrits en introduction et mis en œuvre de manière uniforme dans l'ensemble des modules :

1. **Appel synchrone par port/out** — utilisé pour les dépendances fonctionnelles directes (résolution d'un `PartyRef` par `tp-core`, vérification des droits par `roles-core`, lecture du catalogue par `inventory-core`). Le couplage reste minimal : seuls des identifiants ou des instantanés dénormalisés transitent entre cores, jamais d'objets domaine vivants.
2. **Outbox transactionnel + Kafka** — utilisé pour la propagation d'événements asynchrones (gouvernance des identités, création de compte, déclenchement de facturation, alertes de stock). L'écriture dans `kernel.outbox_event` au sein

de la même transaction que l'agrégat garantit l'atomicité ; le relais de fond assure la livraison vers le topic `iwm.events.business`. Les événements métier peuvent aussi être ancrés dans le `blockchain-core` afin d'ajouter une preuve d'intégrité `append-only`.

3. **Communication inter-services explicite et gardée** — utilisée lorsqu'un core métier déclenche volontairement une opération dans un autre core métier sans passer par une propagation automatique. Le kernel vérifie alors l'entitlement runtime du service cible et consomme son quota d'organisation avant d'autoriser l'opération synchrone.

## Couverture fonctionnelle

| Domaine                | Cores                                                      | Rôle clé                                                                                              |
|------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Transverse             | kernel-core, common-core                                   | Cycle de vie tenant, outbox, types partagés, adresses et contacts polymorphiques                      |
| Identité & Accès       | actor-core, auth-core, roles-core                          | Acteurs, comptes utilisateurs, OIDC central, JWT RS256, RBAC, OTP, MFA                                |
| Organisation           | organization-core, tp-core, settings-core                  | Structures, tiers, abonnements de services et paramétrage                                             |
| Opérations             | product-core, inventory-core, resource-core, sales-core    | Catalogue, catégories, prix datés, stocks, ressources, commandes                                      |
| Finance & Encaissement | accounting-core, treasury-core, billing-core, cashier-core | Comptabilité, règlements bancaires explicites, facturation commerciale legacy et encaissement caisse  |
| Ressources humaines    | hu- hrm-core                                               | Employés, contrats, congés, paie, recrutement, onboarding, temps, formation, frais, médical et KPI RH |
| Traçabilité renforcée  | blockchain-core                                            | Wallets, signatures, transactions, blocs, ancrage documentaire et ancrage des événements outbox       |
| Services               | administration-core, file-core                             | Gouvernance, fichiers et politiques d'exploitation                                                    |

## Invariants transverses

Tous les cores partagent un ensemble d'invariants non négociables, hérités de l'architecture globale :

- **Isolation tenant systématique** — toute entité persistée porte un `tenantId`. Aucun accès cross-tenant n'est possible par construction, y compris via les structures polymorphiques du common-core.
- **Hiérarchie physique native des sites** — les agences et entrepôts peuvent porter un arbre d'espaces physiques (`PhysicalSpace`) consommé par les modules d'actifs, de documents et d'inventaire. Le noyau sait donc représenter non seulement l'organisation logique, mais aussi la structure physique exploitable d'un site opérationnel.
- **Traçabilité immuable** — chaque opération significative produit une entrée dans `AuditTrail` (kernel-core) en insertion seule, sans possibilité de modification ni de suppression.
- **Ancrage blockchain interne** — les transactions, documents et événements outbox peuvent être ancrés dans une chaîne interne `tenantId + organizationId + chainCode`. La validation de chaîne contrôle les hashes, hauteurs, previous hashes et racines Merkle.
- **Cohérence événementielle garantie** — le patron outbox transactionnel élimine le risque de perte d'événement en cas d'échec entre l'écriture de l'agrégat et la publication Kafka.
- **Sécurité par défaut** — validation MIME sur contenu réel (file-core), authentification applicative par `ClientApplication` (kernel-core), restrictions explicites de services par `ClientApplication`, validation systématique des JWT RS256, exposition du JWKS, provider OIDC central, OTP, MFA, gouvernance explicite des BusinessActors, Organizations et Agencies.
- **Contrôle d'accès multicouche** — un appel métier n'est accepté qu'après validation du backend consommateur, validation de la capacité technique `tenant + client + service`, validation de l'abonnement de l'organisation au service concerné lorsque le module est abordable, validation du quota métier `organization + service`, puis vérification des permissions utilisateur dans le bon scope (`TENANT, ORGANIZATION, AGENCY`).
- **Protection de capacité orientée plateforme** — les quotas backend Redis sont portés par le triplet `tenantId + clientId + serviceCode`, ce qui permet d'observer et de contenir finement les surcharges par backend consommateur et par module fonctionnel.
- **Protection de capacité orientée organisation** — les abonnements de services portent leur propre quota `organizationId + serviceCode`. Cette

seconde couche protège le licensing, le fair-use et la répartition de capacité entre organisations d'un même tenant.

- **Communication métier non automatique** — lorsqu'un service veut collaborer avec un autre (par exemple caisse vers comptabilité), la propagation reste explicite. Le backend n'impose pas de synchronisation implicite ; il fournit une capacité contrôlée par abonnement et quota du service cible.
- **Services externes gardés par le kernel** — les backends spécialisés de `KSM_services` utilisent `ksm-kernel-client` pour consommer le kernel avec une `ClientApplication` dédiée. `cashier-service` doit par exemple s'exécuter avec `cashier-backend`, et non avec le client bootstrap permanent. Si aucun administrateur n'existe encore en base, un bootstrap admin initial est nécessaire avant de créer ces identités machine.
- **Couverture documentaire complète des cores actifs** — le présent cahier couvre désormais les cores `billing-core`, `cashier-core`, `hrm-core` et `blockchain-core` en chapitres dédiés, au même niveau que les autres modules métier du backend.
- **Dépôt allégé** — les artefacts générés Maven et Java (`target/`, `*.jar`, `*.class`) sont exclus du versionnement. Les branches distantes obsolètes qui portaient des artefacts historiques ont été supprimées afin de réduire les futurs fetch/pull.
- **Vues composées orientées exploitation** — au-delà des APIs de domaine, le backend expose des projections transverses `asset-portfolio`, `operational-site`, `generalized-inventory` et `service-workspaces`, qui composent organisation, actifs, documents, stocks et layout physique en lectures directement consommables.